

Spring Boot로 구현하는 OAuth 2.0 소셜 로그인

김 정현

(unicodaum@hanmail.net)

1. OAuth 2.0 개요

1) OAuth(Open Authorization)

인증(Authentication)은 시스템이나 서비스가 **사용자의 신원을 확인하는 과정**이다. 이를 통해 사용자의 신원을 식별하고 무분별한 접근을 차단함으로써 데이터 유출을 방지할 수 있다. 그러므로 인증은 보안의 핵심 요소로서 웹 서비스 개발 및 설계 시 중요한 고려사항이다.

신원 확인을 위한 다양한 인증 수단이 제공되는데 대표적인 방법으로는 **비밀번호 기반 인증**, **2단계 인증(Two-Factor Authentication, 2FA)**, **생체 인식**, **토큰 기반 인증**, 그리고 **인증서 기반 인증**이 있다. 그러나 이러한 인증 수단은 사용자가 애플리케이션에 직접적으로 인증 정보를 제공하는 방식이다. 이로 인해 애플리케이션이 사용자 인증(개인) 정보를 관리하게 되며, 개인 정보 유출이라는 보안 위험이 상존하게 된다.

특히 하나의 서비스에서 유출된 개인 정보는 다른 서비스에도 영향을 미칠 수 있다. 이러한 배경 속에서 **더욱 안전하고 효율적인 인증 및 권한 부여 메커니즘을 제공**하고자 **OAuth(Open Authorization**, 이하 OAuth)가 등장하게 되었다.

OAuth는 웹, 모바일 및 데스크톱 애플리케이션에서 널리 사용되는 **개방형 표준 권한부여 프로토콜(RFC 6749)**이다. **사용자의 인증(개인) 정보를 직접 공유하지 않고도 서비스 간에 안전한 인증과 권한 위임을 가능**하게 한다. 사용자의 로그인(인증, 개인) 정보를 직접 공유하지 않고도 제3자 애플리케이션이 다른 서비스의 사용자 데이터에 접근할 수 있도록 허용하는 **개방형 표준 권한 부여 프로토콜**이다.

The screenshot shows the official document page for RFC 6749, titled "The OAuth 2.0 Authorization Framework". The page is hosted on the IETF Datatracker website. Key elements visible include:

- Header:** Internet Engineering Task Force (IETF), Request for Comments: 6749, Obsoletes: 5849, Category: Standards Track, ISSN: 2070-1721. Editor: D. Hardt, Ed., Microsoft, October 2012.
- Title:** The OAuth 2.0 Authorization Framework (highlighted with a red box).
- Abstract:** The OAuth 2.0 authorization framework enables a third-party application to obtain limited access to an HTTP service, either on behalf of a resource owner by orchestrating an approval interaction between the resource owner and the HTTP service, or by allowing the third-party application to obtain access on its own behalf. This specification replaces and obsoletes the OAuth 1.0 protocol described in RFC 5849.
- Status of This Memo:** This is an Internet Standards Track document. This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 5741. Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at <http://www.rfc-editor.org/info/rfc6749>.
- Document type:** RFC - Proposed Standard, October 2012.
- Actions:** View errata, Report errata.
- Updated by:** RFC 8252, RFC 8414.
- Obsoletes:** RFC 5849.
- Was:** draft-ietf-oauth-v2 (c).
- Select version:** A table showing versions 23, 24, 25, 26, 27, and 28. Version 25 is selected, and the document is identified as RFC 6749.
- Compare versions:** A dropdown menu showing "draft-ietf-oauth-v2-31" and "RFC 6749".

구글, 카카오, 네이버 등 소셜 로그인을 통해 사용자 계정 정보를 노출하지 않고도 서비스 간 연동이 가능해지며, 주로 액세스 토큰을 발급받아 권한을 위임하게 된다.

기존의 중앙집중식 인증 모델의 한계를 극복하고, 각 서비스가 사용자의 민감한 인증 정보를 직접 저장하고 관리해야 하는 부담을 줄여준다. 결과적으로 OAuth는 보안성을 강화하면서도 사용자 경험을 개선하고, 서비스 간의 연동을 용이하게 만들어 현대 웹 생태계에서 새로운 인증과 인가(권한부여) 프레임워크의 기반이 되었다.

2) OAuth 버전

The image shows a typical web login form. At the top, there are two input fields: one for '이메일' (Email) and one for '비밀번호' (Password). To the right of the password field is a link that says '비밀번호를 잊었나요?' (Forgot your password?). Below these fields is a dark grey button labeled '로그인' (Login). Underneath the button is a link that says '계정이 있으신가요? 계정생성' (Do you have an account? Create account). At the bottom, there are four social login buttons, each with a logo and text: a red button with the Google 'G' logo and '구글로 로그인' (Login with Google), a blue button with the Facebook 'f' logo and '페이스북으로 로그인' (Login with Facebook), a yellow button with the Kakao logo and '카카오로 로그인' (Login with Kakao), and a green button with the Naver 'N' logo and '네이버로 로그인' (Login with Naver).

인터넷을 사용하면서 회원가입이나 로그인 시에 위의 그림과 같은 소셜 로그인 기능을 접해본 경험이 대부분 있을 것이다. **소셜 로그인**은 Facebook, Google, Naver, GitHub 등의 소셜 네트워킹 서비스 계정 정보를 활용하여 다른 애플리케이션이나 플랫폼에 간편하게 로그인하거나 회원가입 할 수 있는 인증 방식이다. **소셜 로그인 기능의 핵심 기반 기술이 바로 OAuth**이다.

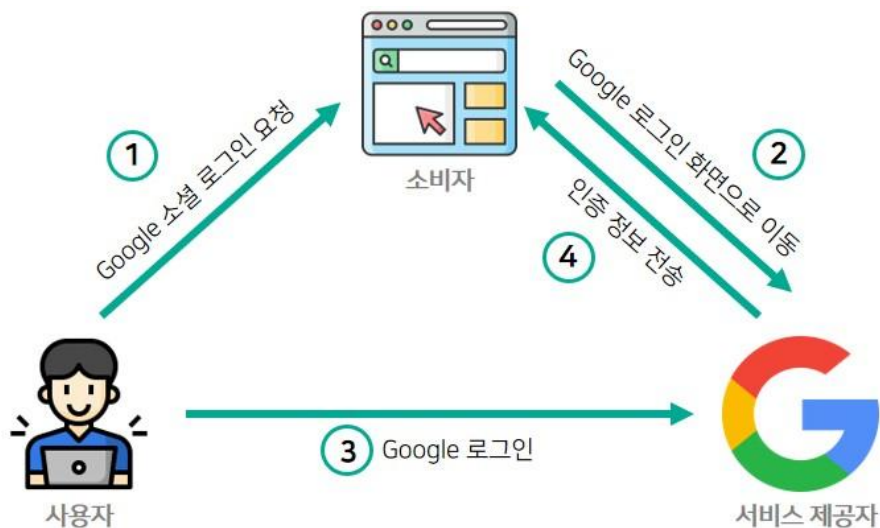
OAuth에는 1.0, 2.0, 2.1의 세 가지 주요 버전이 있으며, 현재 널리 사용되고 있는 버전은 **OAuth 2.0**이다. 각 버전 간의 주요 차이점은 **인증 방식과 범용성**에 있으며, 버전별 인증 프로세스는 다음과 같다.

An **open protocol** to allow **secure authorization** in a **simple** and **standard** method from web, mobile and desktop applications.

[Learn more about OAuth 2.0 »](#)



[OAuth 1.0]



- ① 사용자는 애플리케이션(소비자)을 사용하려고 접근한다.
- ② 애플리케이션은 서비스 제공자에게 접근 권한을 요청하기 위해 사용자를 소셜 네트워킹 페이지로 이동시킨다.
- ③ 사용자는 본인 인증을 수행하고, 애플리케이션이 서비스 제공자로부터 정보를 제공 받는 것을 승인한다.
- ④ 서비스 제공자는 애플리케이션에게 접근 권한을 부여하고, 애플리케이션은 이를 통해 사용자 정보에 접근할 수 있게 된다.

OAuth 1.0에서는 역할에 따라 **사용자(User)**, **소비자(Consumer)**, 그리고 **서비스 제공자(Service Provider)**로 구분 지으며 각각 다음 역할이 부여된다.

참여자	설명
사용자(User)	보호된 자원에 대한 접근 권한을 가진 최종 사용자(개인)로, 자신의 정보 (예: 개인정보, 데이터 등)에 대한 접근 권한을 소비자(Consumer)에게 부여한다.
소비자(Consumer)	OAuth 프로토콜을 사용하여 서비스 제공자(Service Provider)에게 접근하는 애플리케이션 및 서비스이다.
서비스 제공자(Service Provider)	사용자(User) 정보를 보유하는 애플리케이션으로 소비자(Consumer)에게 사용자의 정보를 호스팅한다.

OAuth 1.0은 **사용자**와 **사용자가 이용하려는 애플리케이션(소비자)** 그리고 **사용자 정보를 보유하고 있는 서비스 제공자**가 상호 작용을 통해 소비자에게 위임 접근을 제공한다. 여기까지 보면, OAuth 1.0은 단순하고 간편한 서비스처럼 보일 수 있지만, 실제로는 여러 복잡한 메커니즘과 한계점을 가지고 있었다.

첫째, OAuth 1.0은 모든 요청에 대해 암호화된 서명을 생성하고 검증하는 방식을 사용했다. 이는 보안성을 강화하는 수단이기도 했지만, **각 요청마다 복잡한 서명 과정을 필요로 하여 서버 측에서 검증 로직을 구현하는데 어려움이** 따랐다.

둘째, OAuth 1.0의 제한된 인증 방식으로 인해 모바일 애플리케이션과 같은 **다양한 환경에서 사용하기 어려웠고**, 이로 인해 다른 플랫폼에서의 적용이 쉽지 않아 확장성이 부족했다.

마지막으로, OAuth 1.0은 **인증 토큰(Access Token) 유효기간에 대한 명확한 스펙이 없어** 재 사용 가능성이라는 보안 위험이 잔존했고, 토큰을 무효화하려면 서비스 제공자의 애플리케이션에서 비밀번호를 직접 변경해야 했다.

이와 같은 기존 문제점을 개선하고 더 다양한 인증 방식과 간편한 토큰 관리 그리고 세밀한 권한 제어를 제공하기 위해 OAuth 2.0 표준이 개발되었다.

[OAuth 2.0]



- ① 사용자인 Resource Owner는 소셜 로그인을 통해 클라이언트가 제공하는 서비스에 접근하고자 하며, 이를 위한 권한 위임을 요청한다.
- ② 클라이언트(Client)는 Authorization Server에게 Resource Server가 보호하는 자원에 접근하기 위한 Access Token 발급을 요청한다. 이때, 사용자는 인증(로그인)을 위해 Authorization Server의 페이지로 이동한다.
- ③ 사용자는 클라이언트에 대한 권한 위임을 승인한다.
- ④ Authorization Server는 클라이언트에게 Access Token을 발급한다. 이 Access Token에는 만료기간이 설정되어 있다.
- ⑤ 클라이언트는 Resource Server에게 Access Token을 전달하면서, 사용자의 정보에 접근을 요청한다.
- ⑥ Resource Server는 Access Token의 유효성을 확인하기 위해 이를 발급한 Authorization Server에 검증을 요청한다.
- ⑦ Authorization Server가 Access Token의 유효성을 확인하면, Resource Server에게 인증 성공 메시지를 전송한다.
- ⑧ Resource Server는 클라이언트가 요청한 사용자 정보를 반환한다.

OAuth 2.0 표준에서는 역할에 따라 Client, Resource Owner, Resource Server 그리고 Authorization Server로 구분 지었다.

참여자	설명
Resource Owner	보호된 자원에 대한 접근 권한을 가진 최종 사용자(개인)로, 자신의 정보 (예: 개인정보, 데이터 등)에 대한 접근 권한을 Client에게 부여한다.
Client	사용자를 대신하여 권한을 부여 받아 보호된 사용자의 자원에 접근하려는 애플리케이션이다.
Resource Server	사용자의 보호된 자원을 타사 애플리케이션에 호스팅하는 서버이다.
Authorization Server	실질적인 인증과 인가를 수행하는 서버로 Client의 접근 자격을 확인하고 권한 부여를 위한 Access Token을 발급한다.

비교	OAuth 1.0	OAuth 2.0
참여자	· 사용자(User) · 소비자(Consumer) · 서비스 제공자(Service Provider)	· 자원 소유자(Resource Owner) · 클라이언트(Client) · 자원 서버(Resource Server) · 권한 서버(Authorization Server)
인증 절차	· 디지털 서명 사용	· HTTPS 프로토콜 사용
유효 기간	· 유효기간이 존재하지 않음	· 유효기간 부여 · 만료 시 재발급 토큰 사용
플랫폼	· 웹 기반 서비스	· 웹 애플리케이션 · 데스크톱 애플리케이션 · 모바일 애플리케이션
인증 종류	· 2-Legged OAuth · 3-Legged OAuth	· Authorization Code Grant · Implicit Grant · Resource Owner Password Credentials Grant · Client Credentials Grant · Device Code Grant · Refresh Token Grant

OAuth 2.0은 1.0 버전과 비교하여 여러 측면에서 크게 개선되었다. 가장 주목할 만한 변화는 **서버 구조의 분리**이다. OAuth 1.0에서는 인증, 권한 부여, 자원 제공이 단일 서버에서 통합 처리되었으나, 2.0에서는 이를 **인증 및 권한 부여 서버**와 **자원 서버**로 분리하여 시스템의 유연성과 확장성을 높였다.

OAuth 2.0은 Access Token의 유효 기간에 대한 명확한 기술 명세를 제시하여 토큰 재사용과 관련된 보안 취약점을 개선했다. 또한, HTTPS 프로토콜을 통한 암호화를 도입함으로써 OAuth 1.0에서 요구되던 복잡한 서명 과정을 제거했다. 이로 인해 기능 구현이 간소화되었고, 다양한 인증 수단을 지원하게 되어 범용성이 향상되었다.

결과적으로 OAuth 2.0은 1.0에 비해 **다양한 인증 방식, 간편한 구현, 강화된 보안**을 제공한다. 이러한 개선 사항들을 바탕으로 OAuth 2.0은 현대 애플리케이션 환경에서 널리 사용되는 개방형 인증 표준 프로토콜로 자리매김하게 되었다.

2. OAuth 2.0 Token 타입과 Client 타입

2.1 Token 타입

(1) Access Token

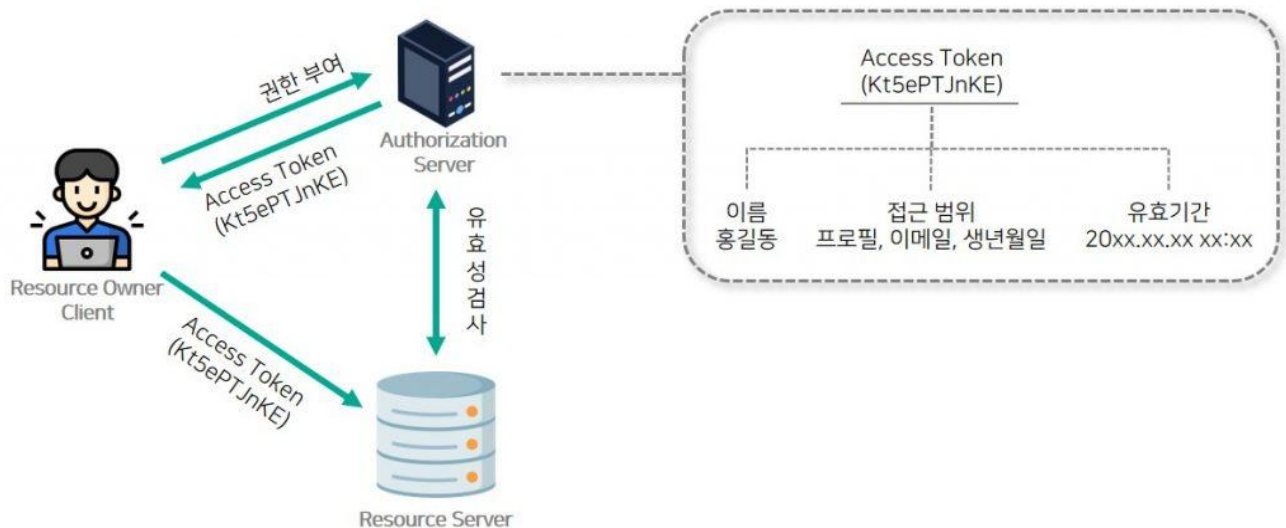
OAuth는 사용자의 비밀번호를 애플리케이션에 직접 제공하지 않고도 자원에 대한 접근 권한을 위임할 수 있는 인증 및 인가 방식이다. 이 과정에서 자원을 요청한 사용자가 적절한 권한을 가졌는지 검증하기 위해 자격증명 토큰이 사용된다.

OAuth 2.0에서는 Access Token, Refresh Token, ID Token, Authorization Code 네 가지 주요 토큰을 사용하며, 이들은 각각 고유한 역할을 수행하여 OAuth 2.0의 인증 및 인가 프로세스를 안전하고 효율적으로 관리하는데 기여한다.



Access Token은 클라이언트에서 사용자의 보호된 리소스에 접근하는데 사용되는 자격 증명이다. 즉, 인증을 위한 사용자의 아이디와 비밀번호를 일련의 문자열로 표현한 형태라고 할 수 있다. 일반적으로 JWT(JSON Web Token) 형식을 취하며, 유형에 따라 식별자 타입(Identifier Type)과 독립형 타입(Self-contained Type)으로 분류 된다.

[식별자 타입(Identifier Type)]



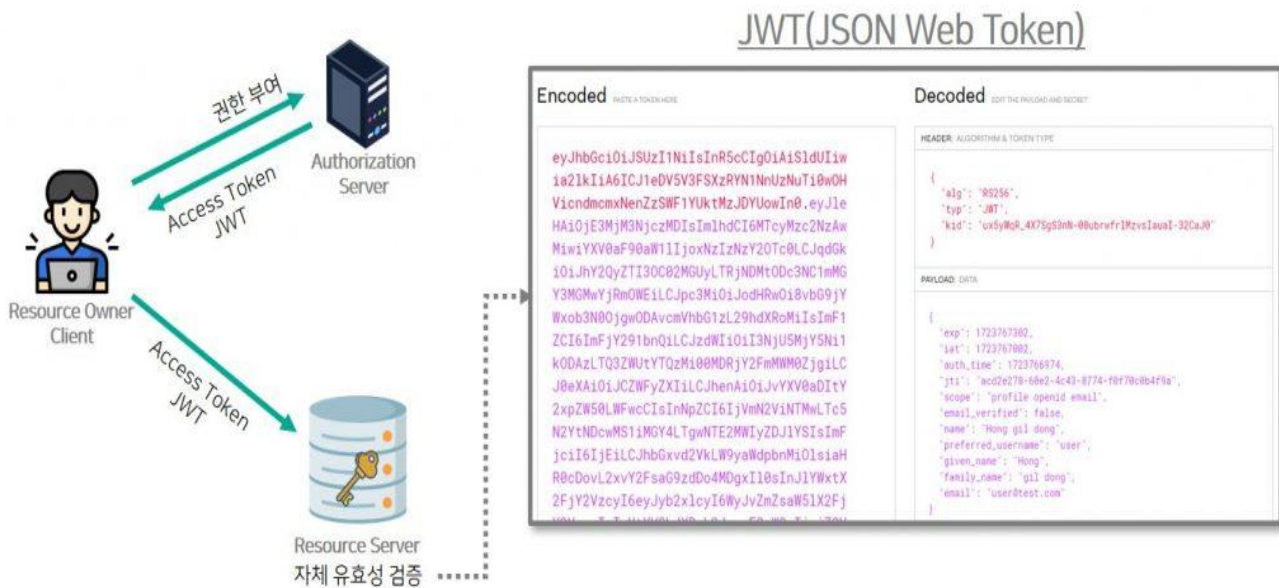
식별자 타입(Identifier Type) 자격증명 방식은 클라이언트(Client)가 사용자의 보호된 자원에 접근할 때, 인가 서버(Authorization Server)에 제출한 Access Token과 인가 서버가 보유한 Access Token의 일치 여부를 검증하는 방식이다.

사용자가 최초 권한 부여를 승인하면, 인가 서버는 사용자에게 관한 정보를 데이터 저장소에 저장하고, 이 정보에 대한 고유 식별자를 포함한 Access Token을 발행한다. 이후 클라이언트가 해당 토큰을 사용해 자원 서버(Resource Server)에 요청을 보내면, 자원 서버는 클라이언트가 요청한 자원을 반환하기 전에 인가 서버와 통신하여 토큰의 유효성을 검증한다.

식별자 타입은 독립형 타입과 달리 발행된 토큰 자체에 민감한 정보가 포함되지 않기 때문에, 토큰이 외부에 유출되더라도 사용자 정보나 권한 정보가 직접적으로 노출되지 않는다. 또한, 식별자 타입 토큰은 짧은 문자열로 구성된 경량화된 구조를 가지고 있어 URL 매개변수로 쉽게 전달될 수 있으며, 이로 인해 토큰 발급과 검증 과정에서의 처리 부담이 줄어든다.

다만, **자원 서버가 토큰의 유효성을 검증하기 위해서는 반드시 인가 서버와 통신하는 절차가 필요하다.** 이는 해당 자격증명 방식이 **인가 서버에 대한 높은 의존도**를 가지고 있음을 의미한다. 인가 서버의 네트워크 지연으로 인해 응답 시간이 증가하면 유효성 검증에 실패할 수 있으며, 인가 서버의 가용성이나 성능 문제가 발생할 경우, 전체 시스템의 영향을 미치게 된다. 또한, 각 토큰 요청마다 인가 서버와 통신해야 하므로 트래픽이 증가하게 되고 이는 서버의 부하를 증가시키는 요인이 된다. 그럼에도 불구하고, **토큰에 사용자의 중요한 정보가 포함되지 않기 때문에, 정보 유출에 민감한 시스템에서는 많이 사용되는 방식이다.**

[독립형 타입(Self-contained Type)]



독립형 타입(Self-contained Type) 자격증명 방식은 **토큰 자체에 필요한 모든 정보를 포함하고 있어, 리소스 서버가 Access Token의 유효성을 자체적으로 검증할 수 있는 방식이다.**

사용자가 권한 부여를 승인하면, 인가 서버는 사용자의 정보, 권한 정보, 토큰의 만료 시간 등 인증과 인가에 필요한 필수 정보를 JWT(JSON Web Token) 형태로 인코딩하여 발행한다. 클라이언트가 해당 토큰을 사용해 자원 서버에 요청을 보내면, 자원 서버는 토큰을 디코딩하여 자체적으로 유효성을 검증한다. 이 검증 과정은 일반적으로 공개키 방식을 사용하며, 사용되는 알고리즘은 서버의 사양에 따라 달라질 수 있다.

독립형 타입의 가장 큰 장점은 **독립적인 검증**이다. 식별자 타입과 달리, 자원 서버가 토큰의 유효성을 검증할 때 인가 서버와의 추가적인 통신을 할 필요가 없다. 이는 검증 과정을 빠르게 만들고, 추가적인 리소스 소모를 줄여준다. 특히, 요청 빈도가 높은 시스템에서 이러한 이점이 두드러진다. **독립형 타입에서 자원 서버는 인가 서버에 대한 의존도가 낮다.** 이로 인해 인가 서버에 가용성 문제가 발생하더라도 그 영향이 최소화된다.

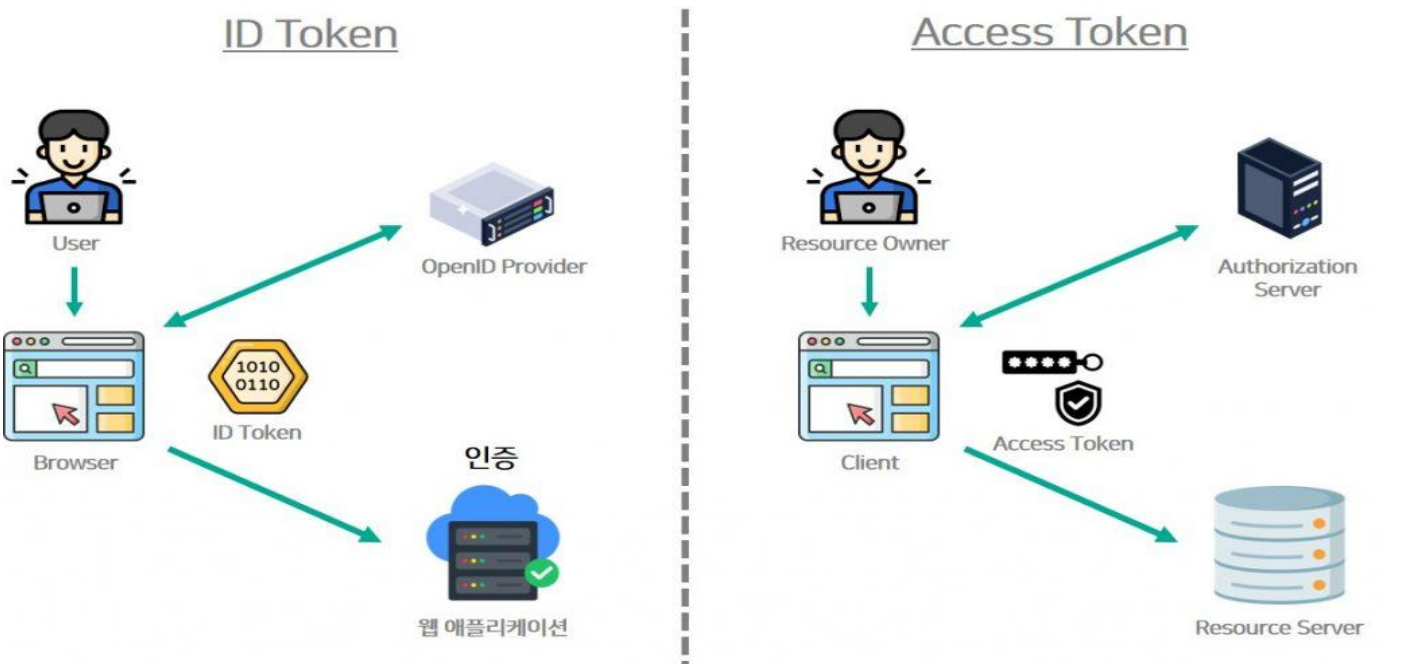
또한, Access Token은 상태 비저장(stateless) 방식으로 운영되므로 인가 서버가 토큰의 상태를 관리할 필요가 없다. 따라서 장애 복구나 서버 간 세션 공유가 불필요하다. 반면에, 토큰 검증을 위해 인가 서버와의 추가적인 통신 절차가 필요 없다는 것은, **한 번 발급된 토큰의 내용을 변경할 수 없다는 의미**이기도 하다. 따라서, 사용자의 권한이나 정보가 변경될 경우 새로운 토큰을 발급해야 한다. 이로 인해 권한 변경이나 정보 갱신에 어려움이 발생한다.

더구나, 식별자 타입과 달리 토큰 자체에 중요한 정보가 포함되어 있기 때문에, 토큰이 탈취될 경우 사용자의 민감한 정보도 함께 노출될 위험이 있다. 하지만, 이와 같은 단점에도 불구하고 독립형 타입은 확장성과 효율성 측면에서 많은 이점을 제공하기에 가장 널리 사용되는 방식이다.

(2) ID Token

소셜 로그인은 OAuth를 기반으로 구현된 기능 중 하나이며, 이 기능을 구현하는 핵심 기반 기술이 바로 OpenID Connect이다. OpenID Connect(이하 OIDC)는 OAuth 2.0을 확장하여 사용자 인증을 제공하는 프로토콜이다.

OIDC의 핵심 구성 요소 중 하나인 ID Token은 사용자 인증 정보를 담고 있는 토큰으로, 클라이언트는 이 ID Token을 통해 사용자의 인증 상태를 확인하게 된다.



ID Token과 Access Token은 겉으로 보기에는 사용자 정보를 담고 있으며, 검증 역할을 수행한다는 점에서 유사하지만, 그 역할과 목적이 명확히 구분된다.

비교	Access Token	ID Token
리소스 서버 접근 권한	O	X
사용자 정보 획득 권한	X	O

ID Token은 "누가" 인증 되었는지를 알려주는 반면, Access Token은 "무엇을" 할 수 있는지를 나타낸다. ID Token은 OpenID Provider가 발행하며, OpenID Provider는 OAuth 2.0 서버의 일종으로, 최

종 사용자를 인증하고 그 결과와 사용자 정보를 신뢰 당사자에게 제공하는 역할을 한다. 그리고 이 과정에서 사용자의 신분을 증명하는 용도로 사용하는 것이 바로 ID Token이다.

[OIDC(OpenID Connect)란?]

OpenID Connect(OIDC) 는 OAuth2.0 기반의 ‘사용자 인증(Authentication)’ 프로토콜이다. OAuth2가 원래 권한 부여(Authorization) 용이었다면, OIDC는 로그인(Login) + 사용자 정보 조회를 가능하게 만드는 확장 규격이다.

- OAuth2 → 리소스 접근 권한 부여
- OIDC → 로그인 인증 + 유저 정보 조회

OAuth2는 오직 클라이언트가 리소스 서버 접근 권한을 얻기 위한 프로토콜일 뿐 사용자 인증은 정의하지 않는다. 그래서 구글/네이버/카카오 로그인을 구현할 수 있도록 OAuth2 위에 사용자 인증 규격을 추가한 것이 바로 OIDC이다.

[OIDC가 제공하는 3가지 핵심 요소]

1) ID Token

- 사용자를 식별하는 토큰(= 로그인 정보)
- JWT 형식
- Authorization Server(= OpenID Provider, OP)가 서명하여 발급
- sub, email, name 등 유저 정보를 포함
- Access Token과는 역할이 다름

토큰	역할
Access Token	API 접근 권한
ID Token	로그인/사용자 인증 정보

2) UserInfo Endpoint

- GET /userinfo
- Access Token을 가지고 호출하는 API
- 사용자의 상세 프로필을 제공(name, email, birthdate, picture, ...)

3) 표준 스코프(Standard Scopes)

- OIDC가 제공하는 유저 정보 종류:

Scope	내용
openid	OIDC 로그인 필수 스코프
profile	이름, 나이, 프로필
email	이메일
phone	전화번호
address	주소

[OIDC 동작 흐름(Authorization Code Flow)]

1) 클라이언트가 AS로 리디렉션

`https://authserver.com/authorize?response_type=code&client_id=client
&redirect_uri=http://app/callback&scope=openid profile email`

2) 사용자 로그인

3) 사용자 동의 (프로필 제공 허용)

4) 클라이언트로 Authorization Code 전달

5) 클라이언트가 Authorization Code로 토큰 요청

`POST /token`

`grant_type=authorization_code`

`code=xxxx`

`redirect_uri=xxx`

6) 서버가 다음 두 가지를 반환

Access Token

ID Token

[OIDC 용어 정리]

용어	설명
Relying Party(RP)	클라이언트 앱
OpenID	Provider(OP) 인증 서버, Authorization Server
End-User	실제 사용자
ID Token	로그인 정보
Userinfo Endpoint	사용자 상세 프로필 제공 API

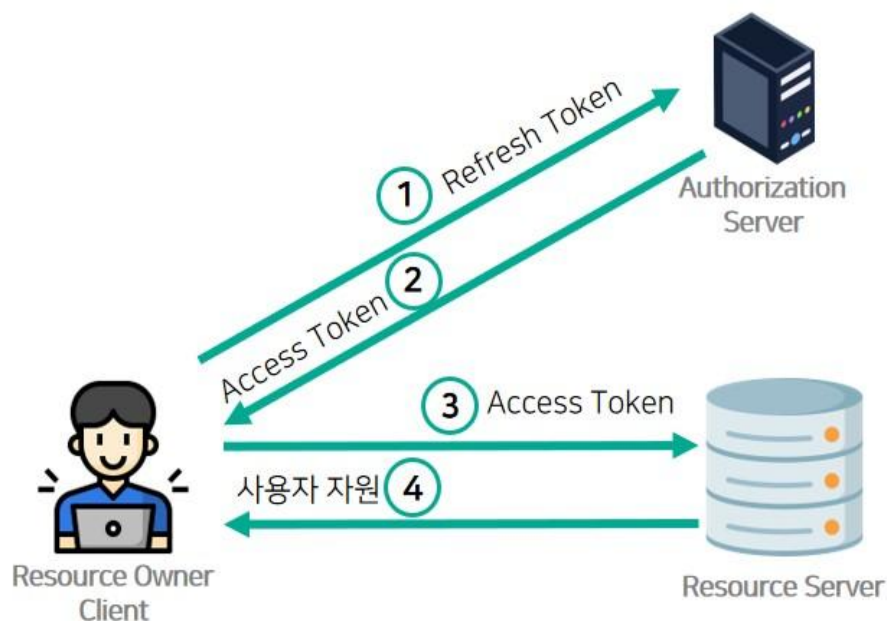
[OIDC와 OAuth2의 차이 요약]

항목	OAuth2	OIDC
목적	권한 부여(Authorization)	인증(Authentication)
누가 사용?	API 서버	로그인/회원 시스템
사용자 정보 제공	없음	ID Token, UserInfo 제공
토큰	Access Token	Access Token + ID Token
필수 Scope	없음	openid

[OIDC를 사용하면 얻는 장점]

- 글로벌 표준(구글, 애플, MS, 네이버, 카카오 모두 사용)
- JWT 기반 로그인 정보(ID Token)
- API 인증/인가 시스템과 자연스럽게 결합
- 모바일/SPA에서도 잘 동작
- 세션 없는 서버에서도 인증 가능 (stateless)

[Refresh Token]



Refresh Token은 Access Token이 만료되었을 때 클라이언트가 새로운 Access Token을 얻기 위해 사용하는 자격 증명이다. OAuth 1.0에서는 토큰의 유효 기간에 대한 명확한 규정이 없어 Access Token을 무한히 사용할 수 있었다. 그러나 **OAuth 2.0에서는 토큰 발행 시 유효 기간을 설정하여, 일정**

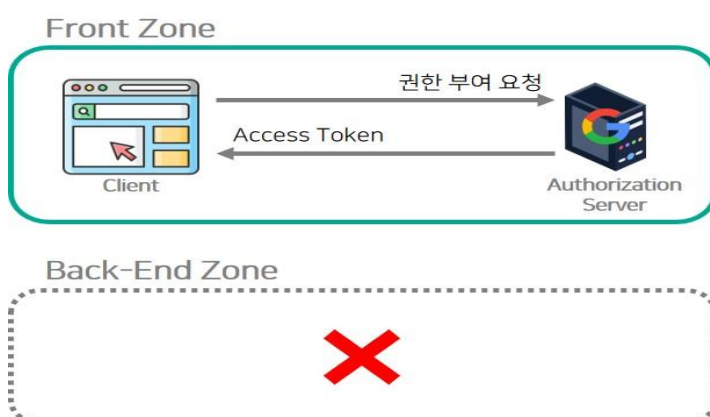
시간이 지나면 Access Token을 사용할 수 없게 되었다. 이는 토큰이 탈취되었을 경우, 공격자가 이를 오랫동안 사용할 수 없도록 제한하는데 도움이 되었지만, 장기간 실행되는 애플리케이션에는 적합하지 않다는 단점이 있었다. 이러한 문제를 해결하기 위해 도입된 것이 Refresh Token이다.

Refresh Token은 인가 서버에서 Access Token과 함께 발행되며, 클라이언트가 이를 관리한다. 클라이언트가 자원 서버로, 유효 기간이 만료된 Access Token을 보내면, 자원 서버는 클라이언트에게 유효 기간이 만료되었다는 메시지를 반환한다. 클라이언트는 이 메시지를 수신한 후 Refresh Token 을 인가 서버에 전달하여 새로운 Access Token을 재발급 받게 된다.

2.2 Client 타입

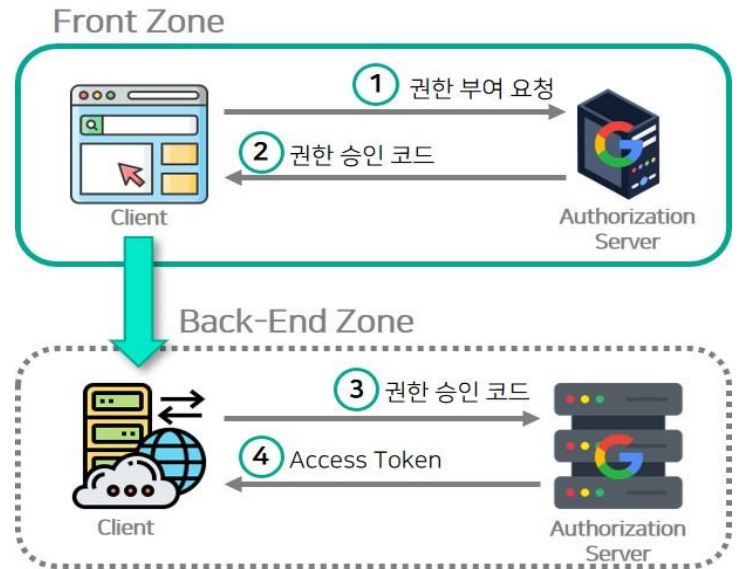
사용자(Resource Owner)는 권한 부여를 승인하는 핵심 주체이지만, 자원 서버나 인가 서버와 직접적으로 상호작용하지 않는다. 다음 워크 플로우에서와 같이 사용자를 대신하여 실제로 요청을 처리하고 서버들과 상호작용하는 주체는 클라이언트이다. 그리고 OAuth 2.0에서 클라이언트는 인증 및 권한 부여의 보안 요구 사항에 따라 공개 클라이언트(Public Client)와 기밀 클라이언트(Confidential Client)으로 나뉘게 된다. 클라이언트 유형을 공개 클라이언트와 기밀 클라이언트로 구분하는 주된 이유는 자격 증명의 기밀성 유지 여부에 있다.

공개 클라이언트와 기밀 클라이언트의 주요 차이점은 Access Token 획득 방식에 있다. 공개 클라이언트의 경우 인가 서버로부터 Access Token을 직접 전달받는 반면, 기밀 클라이언트는 먼저 권한 승인 코드(Authorization Code)를 받은 후 이를 이용해 Access Token으로 교환하는 과정을 거친다. 이 과정에서 승인 코드는 철저히 백엔드에서 처리되므로, 공개 클라이언트와 달리 사용자에게 노출될 위험이 없다. 또한, 권한 승인 코드가 탈취되더라도, 이를 검증하기 위한 보안 토큰이 없으면 Access Token이 발급되지 않기 때문에 보안성이 우수하다.



공개 클라이언트는 인가 서버가 Access Token을 발행할 때, 별도의 비밀 통신 채널을 사용하지 않고 프론트엔드에서 URL을 통해 사용자에게 직접 전달한다. 이로 인해 Access Token이 사용자 화면에 그대로 노출되며, 외부 사용자에게 의한 토큰 탈취 위험이 존재하게 된다. 즉, 공개 클라이언트는 클라이언트의 비밀 정보를 안전하게 보호할 수 없는 환경에서 운용되는 애플리케이션을 의미한다.

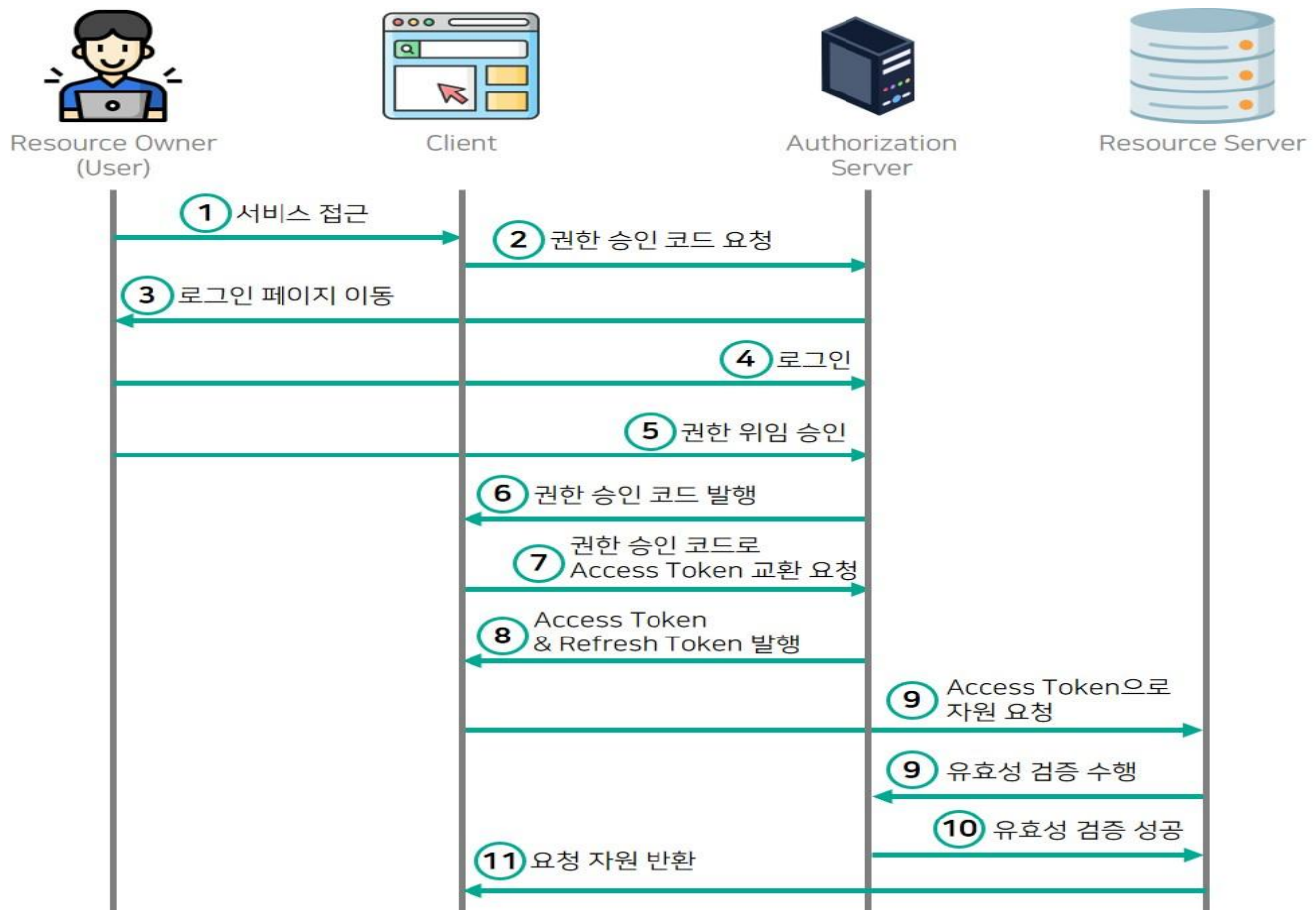
기밀 클라이언트는 클라이언트 비밀을 안전하게 저장하고 관리할 수 있는 환경에서 운용되는 애플리케이션으로, 주로 서버 측에서 실행되는 백엔드 서비스나 데스크톱 애플리케이션에서 사용된다.



3. OAuth 2.0 Grant 타입

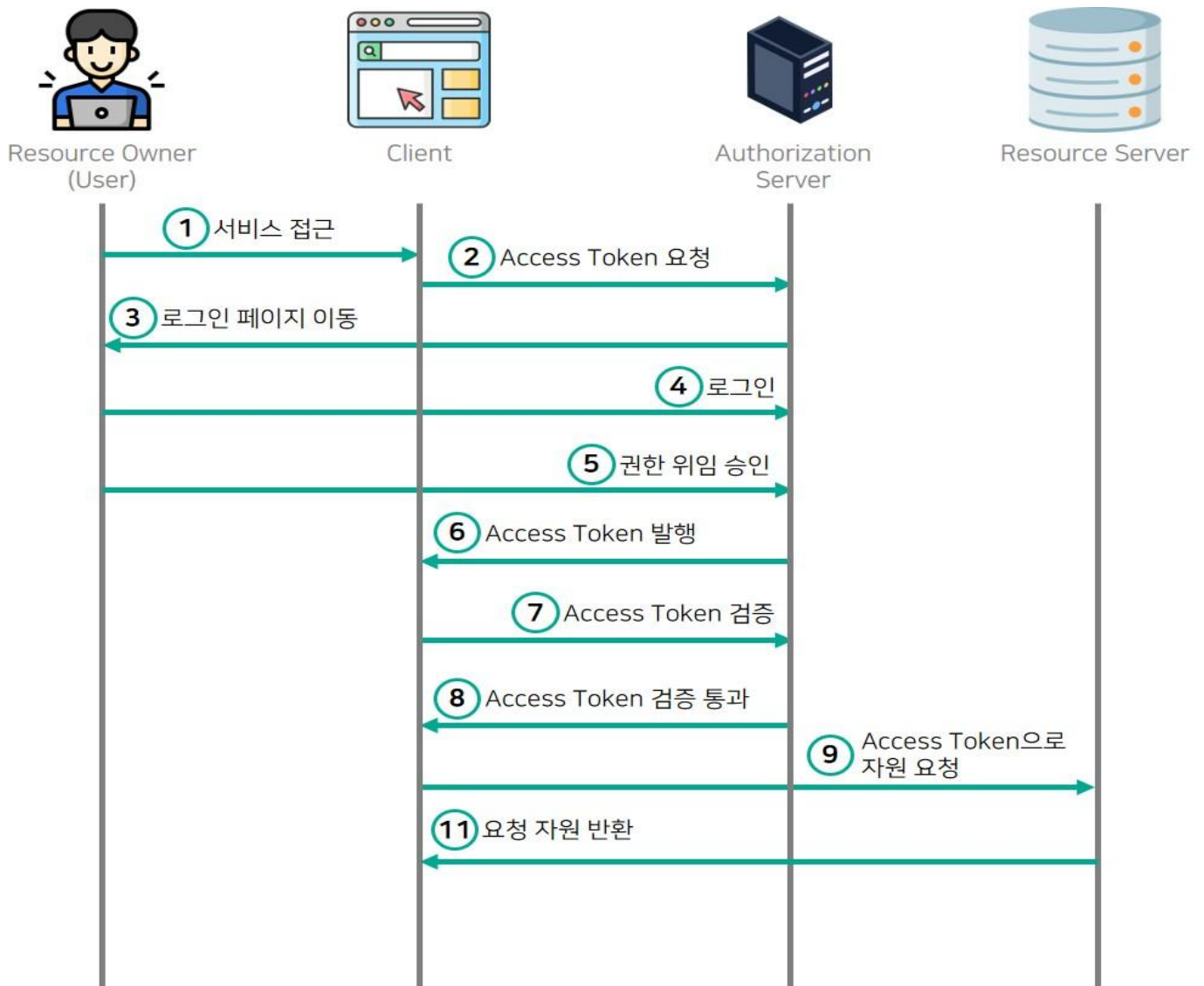
OAuth 2.0은 다양한 클라이언트 환경과 사용 시나리오에 적합한 인증 및 인가 방식을 제공하기 위해 6가지의 인증 수단을 제공한다.

3.1 Authorization Code Grant Type



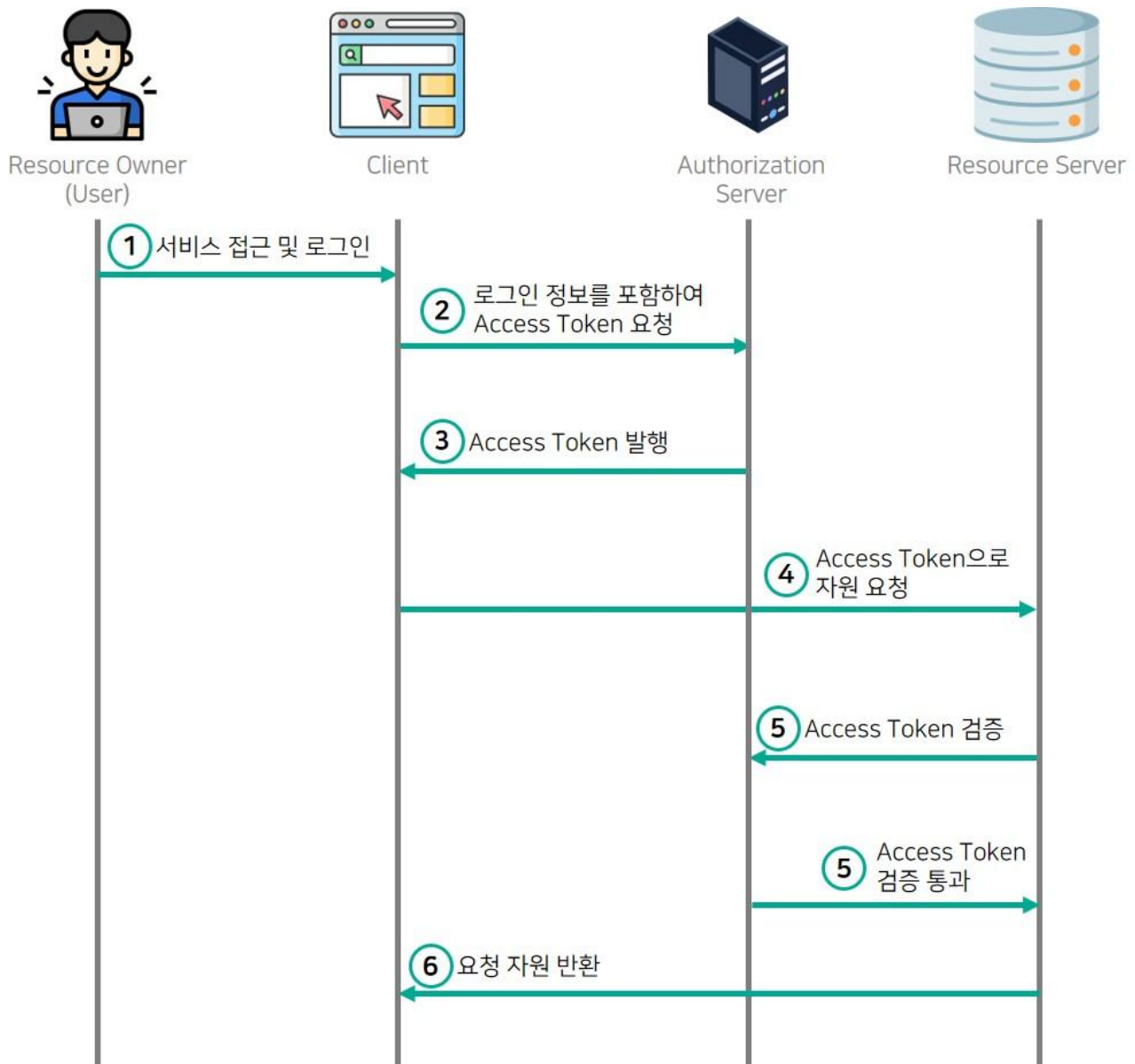
Authorization Code Grant Type은 **보안이 중요한 웹 애플리케이션과 서버 간 통신에 사용되는 대표적인 기밀 클라이언트 유형**이다. 이 방식에서는 사용자 인증과 인가가 클라이언트에서 직접 이루어지지 않고 인가 서버를 통해 처리되며, **클라이언트는 권한 승인 코드(Authorization Code)를 받아 이를 Access Token으로 교환**한다. 이로 인해 민감한 정보가 사용자 에이전트(브라우저)나 프론트엔드에 노출되지 않아 토큰 탈취의 위험성이 적다.

3.2 Implicit Grant 타입



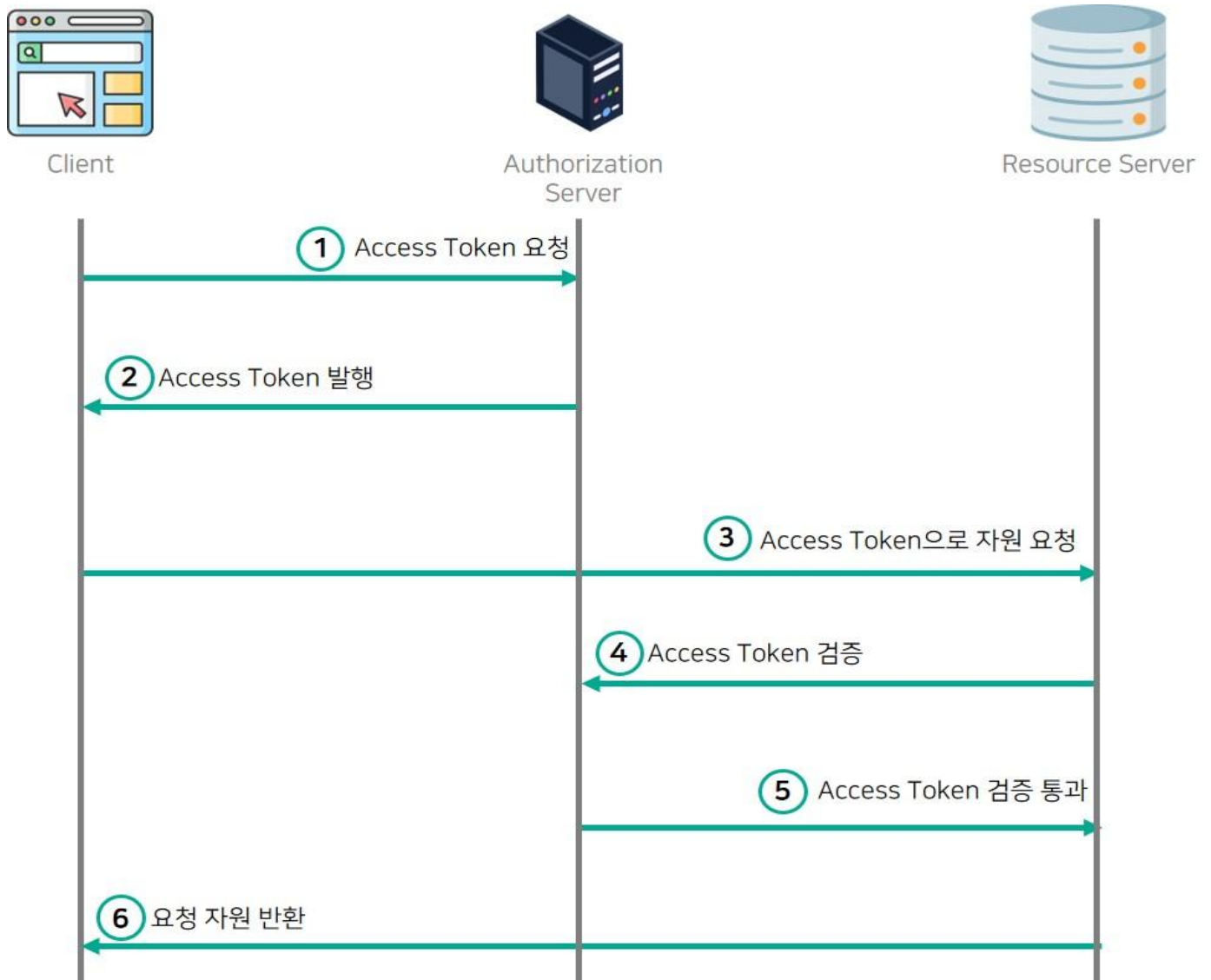
Implicit Grant Type은 대표적인 공개 클라이언트 유형으로, 주로 SPA(Single Page Application)와 같은 클라이언트 측 애플리케이션에서 사용된다. 이 방식에서는 Access Token이 권한 승인 코드 없이 직접 발행되므로, 클라이언트와 인가 서버 간의 통신이 단순해지는 장점이 있다. 그러나 Access Token이 URL 매개변수를 통해 브라우저에 직접 전달되기 때문에, 토큰 탈취의 위험성이 존재한다. 따라서, 가용성이 우선 시 되는 환경에서 사용하기에 적합하다.

3.3 Resource Owner Password Credentials Grant 타입



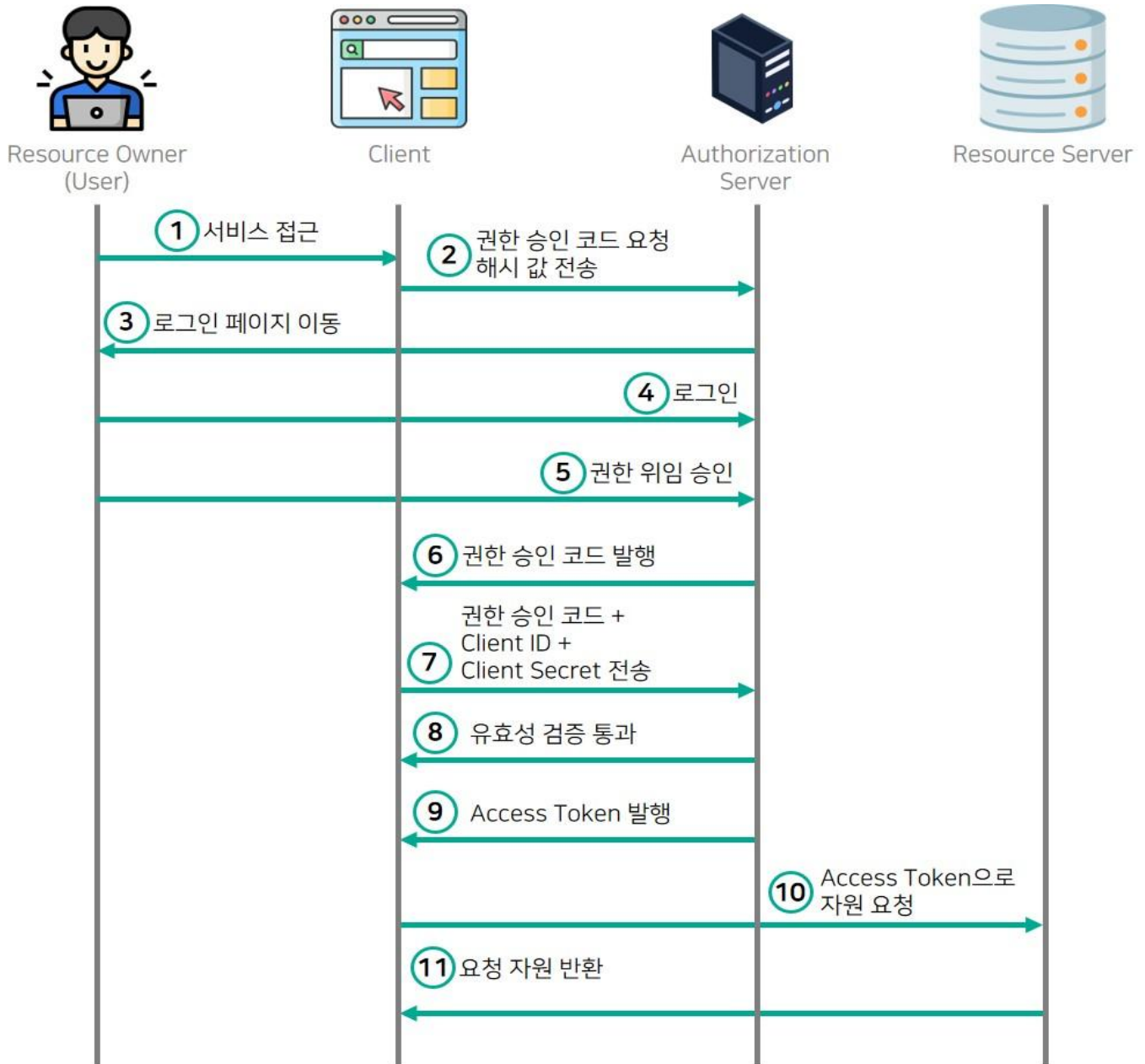
Resource Owner Password Credentials Grant Type은 사용자 계정 정보(아이디와 비밀번호)를 직접 사용하여 Access Token을 얻는 방식이다. 이 방식은 사용자의 계정 정보를 신뢰할 수 있는 애플리케이션에서 직접 수집(저장)할 수 있는 경우에만 사용된다. 따라서, 제3자 애플리케이션보다는 자사 서비스에서 제공하는 애플리케이션 환경에 운용하기 적합하다.

3.4 Client Credentials Grant 타입



Client Credentials Grant Type은 애플리케이션이 리소스 소유자이자 클라이언트의 역할을 동시에 수행하는 방식이다. 주로 자사에서 관리하는 애플리케이션 환경에서 서버 간 통신이 필요한 IoT 환경에서 사용된다. 이 방식은 클라이언트의 정보를 기반으로 하기 때문에 OAuth 2.0 인증 및 권한 부여 방식 중 가장 간단한 구성을 가진다.

3.5 PKCE-enhanced Authorization Code Grant Type



PKCE-enhanced Authorization Code Grant Type(이하 PKCE)은 CSRF(Cross-Site Request Forgery) 및 코드 교환 공격을 방지하기 위해 **Authorization Code Grant Type의 보안성을 향상시킨 확장 버전**이다. PKCE는 본래 취약한 모바일 애플리케이션 환경에서 Authorization Code Grant Type의 보안을 강화하기 위해 설계되었으나, 이후에는 모든 유형의 OAuth 2.0 클라이언트에서도 적용할 수 있도록 개선되었다.

이 방식은 특징은 **클라이언트가 권한 승인 코드를 Access Token으로 교환할 때, 초기 요청 시 전송한 해시 값과 Client Secret(code_verifier) 값이 일치하는 경우에만 Access Token을 발행**한다는 것이다. 이를 통해 교환 요청이 올바른 클라이언트로부터 온 것인지를 보장하며, 권한 승인 코드가 탈취되더라도, 공격자가 Client Secret 값을 알지 못하기 때문에, Access Token을 획득할 수 없다.

3.6 OAuth2를 사용하는 상황

- 제3자 애플리케이션에 권한 위임이 필요할 때

사용자가 자신의 ID와 비밀번호를 직접 제공하지 않고, 안전하게 신뢰할 수 있는 서비스(구글, 페이스북, 네이버 등)를 통해 다른 앱에 권한만 위임하고자 할 때 적합하다. 예를 들어, 소셜 로그인, API 접근 권한 위임 등에서 사용된다.

- 권한 범위(scope)를 제한하여 최소 권한 원칙 적용이 필요할 때

OAuth2는 읽기 전용, 작성 권한 등 특정 권한만 가진 액세스 토큰을 발급할 수 있어, 안전하게 필요한 권한만 부여하는 것이 가능하다.

- 플랫폼(웹, 모바일, 네이티브 등)과 무관한 통합 인증 및 권한 관리가 필요할 때

다양한 플랫폼에서 동일한 OAuth2 표준을 통해 통합 인증과 권한 제어를 손쉽게 구현할 수 있다.

- 사용자 자격 증명을 서버에 저장하거나 직접 다루지 않고 안전한 인증 관리가 필요할 때

비밀번호를 직접 다루지 않고 액세스 토큰으로 안전하게 인증 관리가 가능해 보안 사고를 줄일 수 있다.

- 큰 규모의 분산 시스템이나 API 중심 아키텍처에서 클라이언트가 여러 서버에 접근할 때

OAuth2를 통해 중앙 집중형 권한 관리를 하면서도 확장성과 유연성을 확보할 수 있다.

OAuth2는 사용자의 인증 정보를 직접 받지 않으면서도, 다양한 클라이언트가 안전하고 제한적인 권한으로 사용자 자원에 접근하도록 제어할 때 매우 유용한 표준 프로토콜이다.

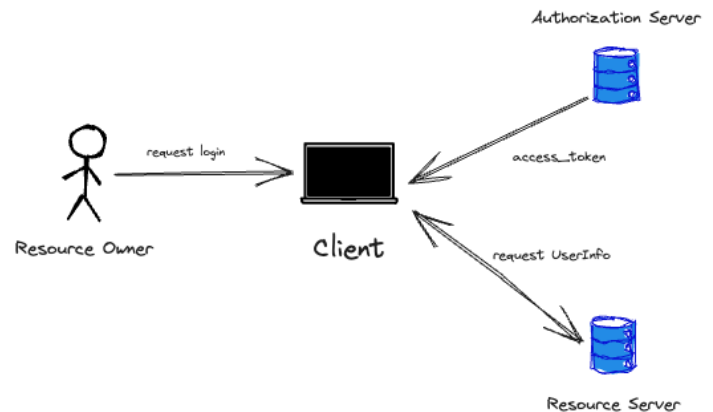
4. OAuth2 프로바이더 구현

4.1 OAuth2 프로바이더란

OAuth 2.0 프로바이더는 OAuth 2.0 프로토콜을 통해 보호된 리소스에 대한 접근 권한을 관리하고 제공하는 주체를 말한다. 일반적으로 OAuth 2.0 프로바이더는 두 가지 주요 역할을 포함한다.

- Authorization Server(인가 서버)
- Resource Server(리소스 서버)

(1) Authorization Server(인가 서버) 역할



- 사용자를 인증하고, 클라이언트 애플리케이션 (Third-party 앱)이 사용자 리소스에 접근할 수 있도록 권한을 부여한다.
- 액세스 토큰(access token)과 필요시 리프레시 토큰(refresh token)을 발급한다.
- 권한 부여 코드, 토큰 발급, 갱신 등의 과정을 처리하는 엔드포인트(예: /oauth/authorize, /oauth/token)를 제공한다.
- 사용자가 권한 부여를 승인하는 인터페이스(예: 로그인 폼, 권한 동의 페이지)를 제공한다.

(2) Resource Server(리소스 서버) 역할

- 보호된 리소스(데이터, API 등)를 호스팅한다.
- 클라이언트가 보낸 액세스 토큰을 검증하여 권한을 확인하고, 인증이 유효하면 리소스를 제공한다.
- 토큰의 유효성을 확인하기 위해 인가 서버와 통신하거나 토큰 자체를 검증할 수 있다.

[프로바이더 구성과 분리]

실제 OAuth 2.0 프로바이더 시스템은 Authorization Server와 Resource Server를 하나의 서비스로 운영할 수도 있고, 분리하여 독립된 서버로 운용할 수도 있다.

분리할 경우 보안과 확장성 측면에서 유리하며, Spring Security OAuth 같은 프레임워크는 이를 설정하고 구성하는 표준 방식을 제공한다.

[처리 흐름]

- 클라이언트는 사용자를 대신해 인가 서버에 인가 코드를 요청하고, 이를 통해 **액세스 토큰을 발급** 받는다.
- 클라이언트는 이 토큰으로 리소스 서버에 보호된 데이터를 요청하고, 리소스 서버는 토큰을 검증한 후 응답한다.

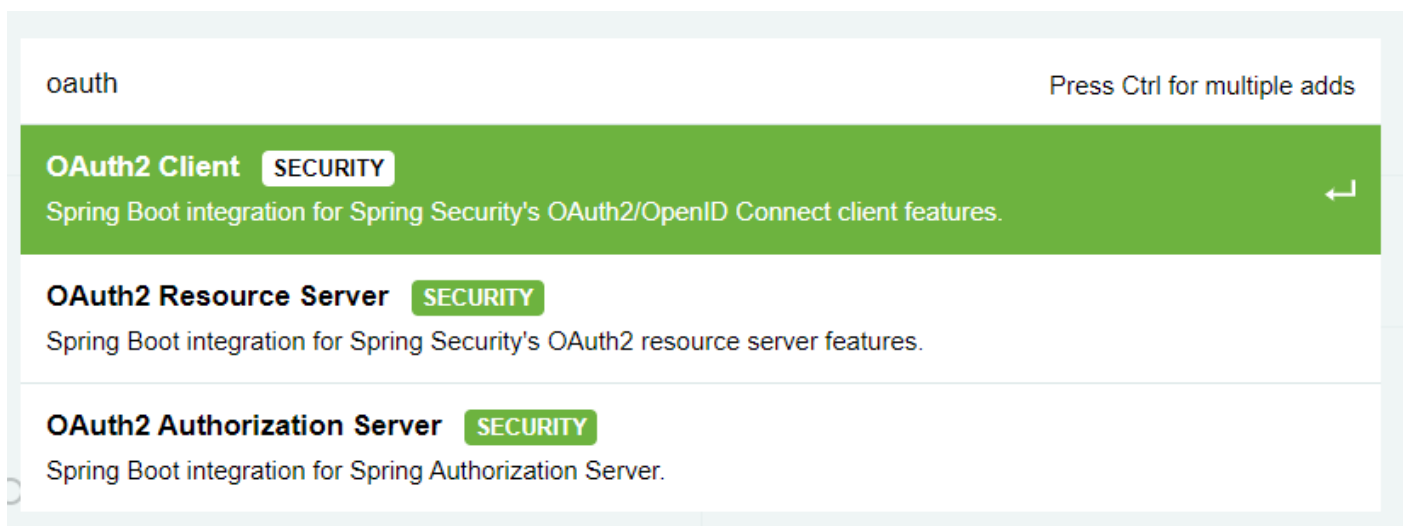
역할	주요기능
Authorization Server	인증, 권한 부여, 토큰 발급 및 관리
Resource Server	보호된 리소스 제공, 토큰 유효 검증

Spring Security OAuth2 기반의 Authorization Server와 Resource Server를 구현

Authorization Server(인가 서버)는 사용자 인증과 클라이언트 애플리케이션 검증, 클라이언트 등록, 권한 부여 코드 및 토큰 발급 흐름을 관리하며 액세스 토큰 발급을 담당한다.

Resource Server(자원 서버)는 보호된 API나 데이터를 호스팅하며, 클라이언트가 요청할 때 전달된 액세스 토큰을 검증하여 접근 권한을 부여한다.

Spring Security OAuth2는 이러한 역할을 구현하기 위한 프레임워크로, Authorization Server와 Resource Server 구성을 위한 다양한 설정과 컴포넌트를 제공한다.



Authorization Server의 토큰 발급 방식으로는 **Authorization Code**, Client Credentials, Password 등 다양한 그랜트 타입을 지원한다.

Resource Server는 수신된 요청에서 토큰을 추출해 검증하고, 토큰의 권한 범위(scope)에 따라 자원 접근을 제어하며, JWT 토큰을 활용하는 경우 자체적으로 서명 검증을 수행할 수 있다.

Spring Security에서는 SecurityFilterChain을 구성해 OAuth2 로그인, 토큰 발급, 인증 필터, 예외 처리 등을 설정하며, OAuth2AuthorizationServer 라이브러리를 통해 Authorization Server를 쉽게 구축할 수 있다.

또한 TokenStore 혹은 JWT를 활용한 토큰 관리, 클라이언트 정보 및 사용자 정보 서비스, 그리고 권한 부여 핸들러 등을 세밀하게 제어할 수 있다. 이러한 구성은 모듈화되고 확장성이 뛰어나 MSA 환경 등 다양한 아키텍처에 적합한 보안 시스템 구현을 가능하게 한다.

Spring Security OAuth2를 활용한 Authorization Server와 Resource Server 구현은 OAuth2 표준에 따른 권한 위임과 자원 보호를 효과적으로 수행하며, 강력한 보안 설정과 다양한 확장 옵션을 통해 실무에서 널리 적용 가능한 인증/인가 인프라를 구축하는 핵심 기술이다.

[Authorization Server 구현]

(1) SecurityFilterChain 빈 등록

HttpSecurity 객체를 받아 OAuth2AuthorizationServerConfigurer를 사용하여 OAuth2 권한을 부여하는 서버용 필터 체인을 구성한다. 이때 클라이언트 정보, 토큰 발급 정책, 인증 및 권한 부여 관련 설정을 적용한다.

(2) RegisteredClientRepository 빈 등록

RegisteredClientRepository 는 OAuth2 클라이언트를 관리하는 저장소로, 클라이언트 ID, 시크릿, 인증 방식, 리디렉션 URI, 권한 범위(scope), 권한 부여 타입(authorization code, client credentials 등)을 등록한다. 보통 메모리 또는 데이터베이스 기반으로 구현하며, 이 클라이언트 정보로 인증이 진행된다.

(3) AuthorizationServerSettings와 JWT 토큰 설정

Authorization Server의 엔드포인트 URI, 토큰 유효기간 등 환경 설정을 담당한다.

JWT(Json Web Token) 사용 시, 키 생성과 서명, JWT 디코더를 설정해 액세스 토큰 발급 및 검증에 사용한다.

(4) OAuth2AuthorizationService, OAuth2AuthorizationConsentService 빈 등록

발급된 토큰과 권한 동의 정보를 관리하는 서비스로, 주로 DB 연동하여 토큰의 저장과 만료, 권한 동의 상태를 유지한다.

(5) 인증 매니저(AuthenticationManager) 및 사용자 인증 처리

사용자의 인증을 처리하는 컴포넌트로, 로그인 정보 검증 및 인증 결과 전달을 담당한다.

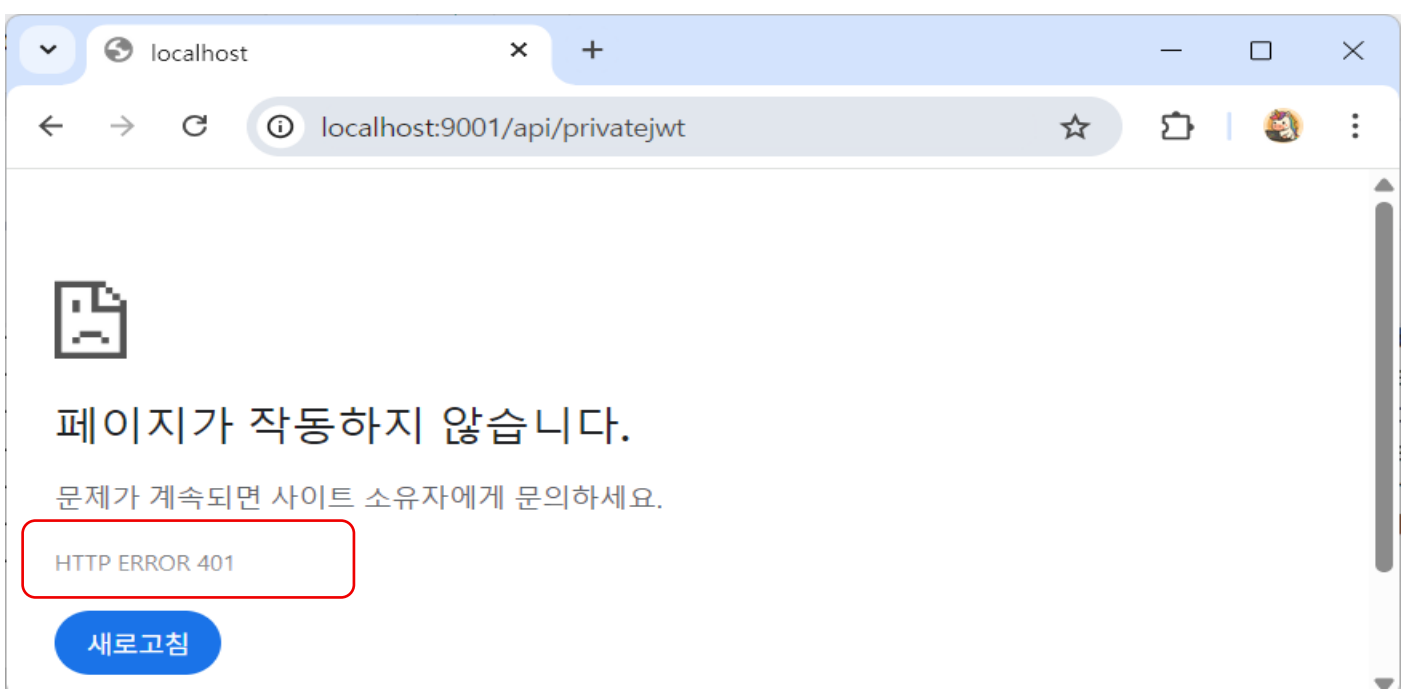
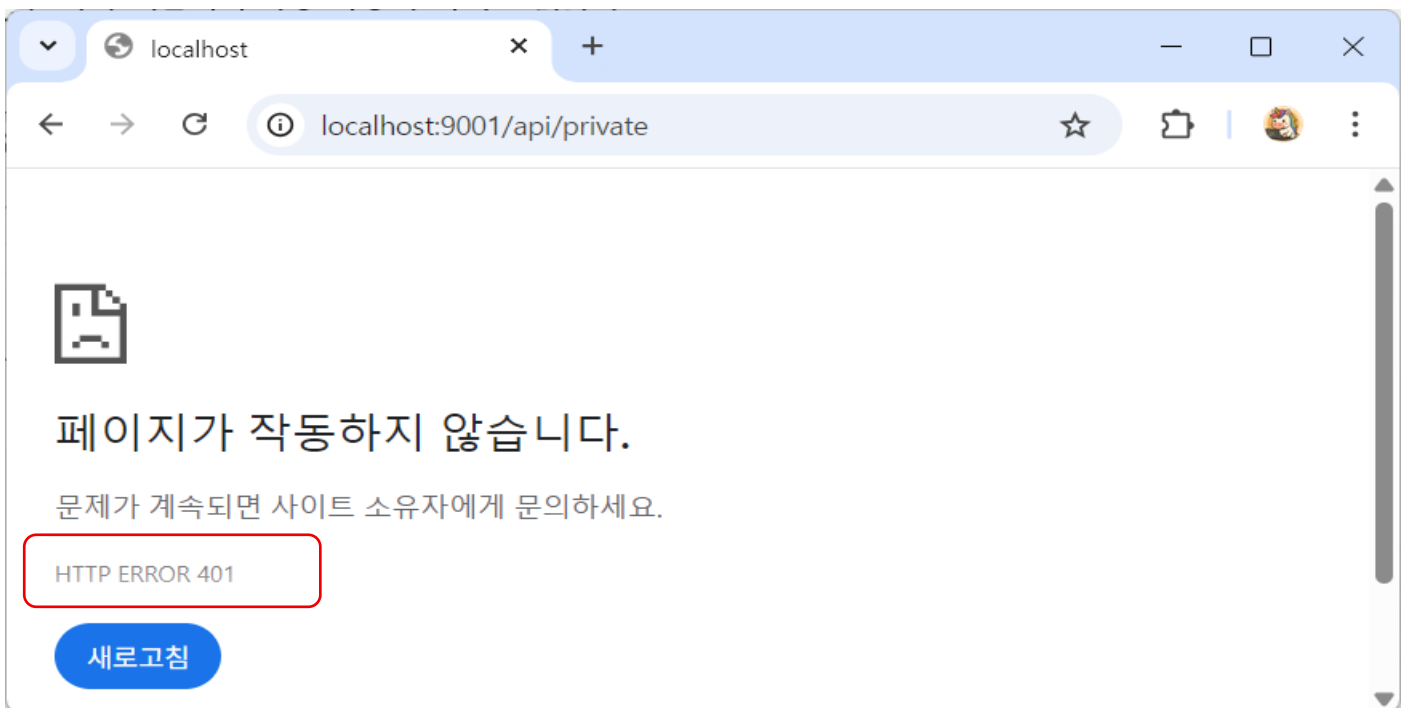
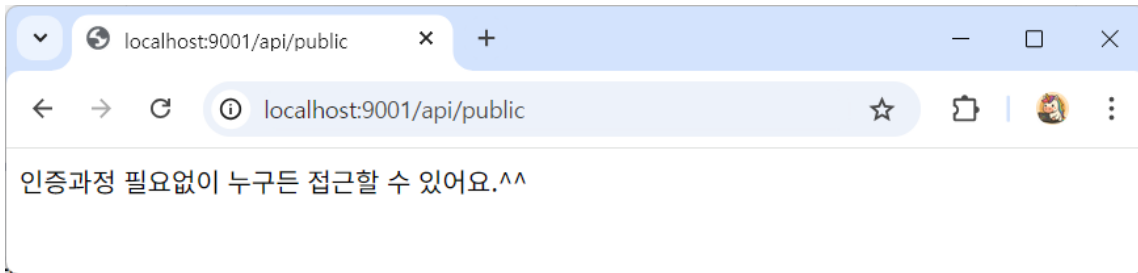
대개 Spring Security의 기본 인증 매니저를 활용하며 필요 시 커스텀 인증 전략도 구현 가능하다.

이 구조는 OAuth2 권한 부여 코드 흐름, 토큰 발급 및 검증 과정을 안전하고 확장 가능하게 구현하도록 설계되어 있으며, Spring Authorization Server 라이브러리가 제공하는 기본 기능과 확장성을 적극 활용한다. JWT를 통한 토큰 기반 인증과 권한 관리, 클라이언트 및 사용자 정보 저장소 설정이 핵심이며, 커스터마이징 가능한 콜백 및 예외 처리 기능도 포함된다.

RegisteredClient의 설정 예제는 Spring Security OAuth2 Authorization Server에서 클라이언트 등록 정보를 관리하는 핵심 요소이다.

- **clientId**: 클라이언트 식별자, 고유해야 하며, 클라이언트가 인증 요청 시 사용
- **clientSecret**: 클라이언트 비밀 값으로, 클라이언트 인증 시 사용
- **clientAuthenticationMethod**: 인증 방식을 지정
(**CLIENT_SECRET_BASIC**, CLIENT_SECRET_POST)
- **authorizationGrantType**: 허용하는 권한 부여 방식
(**AUTHORIZATION_CODE**, CLIENT_CREDENTIALS, PASSWORD, 등).
- **redirectUri**: 인가 코드 발급 후 리다이렉션될 URI, 등록 필수
- **scope**: 클라이언트가 요청하는 권한 범위
- **clientSettings**: 추가 설정, 예를 들어 사용자 동의 필요 여부 등

제공된 authorization_server(9000) 프로젝트와 resource_server(9001) 프로젝트를 인텔리제이로
открывает. 각각 실행시킨 다음 다음과 같이 요청한다.



http://localhost:9000/oauth2/authorize?response_type=code&client_id=oidc-client&redirect_uri=http://localhost:8080/login/oauth2/code/oidc-client&scope=openid%20profile%20address&state=xyz123

The screenshot shows a web browser window with the address bar at `localhost:9000/login`. The page displays a 'Please sign in' form with fields for 'Username' and 'Password', and a 'Sign in' button. The Network tab of the developer tools is open, showing a list of requests. Two requests are highlighted with a red box: `/oauth2/authorize` (Status: 302) and `/login` (Status: 200). The status bar at the bottom indicates 9 requests, 790 kB transferred, and 789 kB resources.

Path	Url	Meth...	Status	Type	Initiator	Size	Ti...
<code>/oauth2/authorize</code>	<code>http://localhost:9000/oauth2/authorize?response_type=code&client_id=oidc-client&redirect_uri=http://localhost:8080/login/oauth2/code/oidc-client&scope=openid%20profile%20address&state=xyz123</code>	GET	302	document	Other	0.3 kB	3...
<code>/login</code>	<code>http://localhost:9000/login</code>	GET	200	document	<code>/oauth2/authorize</code>	1.4 kB	2...
<code>/default-ui.css</code>	<code>http://localhost:9000/default-ui.css</code>	GET	200	stylesheet	<code>login:9</code>	3.4 kB	1...
<code>/content-all.css</code>	<code>http://localhost:9000/content-all.css</code>	GET	200	fetch	<code>content-all.js:1</code>	512 kB	3...
<code>/fonts/fonts.css</code>	<code>http://localhost:9000/fonts/fonts.css</code>	GET	200	fetch	<code>content-all.js:1</code>	6.7 kB	2...
<code>/fonts/DM-Sans-regular.woff2</code>	<code>http://localhost:9000/fonts/DM-Sans-regular.woff2</code>	GET	200	font	<code>login</code>	14.1 kB	2...
<code>/assets/logo-O35E636P.png</code>	<code>http://localhost:9000/assets/logo-O35E636P.png</code>	GET	200	png	<code>content-all.js:7</code>	29.1 kB	1...
<code>/i18n/ko.json</code>	<code>http://localhost:9000/i18n/ko.json</code>	GET	200	fetch	<code>content-all.js:4</code>	175 kB	2...

The screenshot shows the same web browser window. The Headers tab of the developer tools is open for the `/oauth2/authorize` request. The 'General' section shows the Request URL, Request Method (GET), Status Code (302 Found), Remote Address ([::1]:9000), and Referrer Policy (strict-origin-when-cross-origin). The 'Response Headers' section shows various headers including Cache-Control, Connection, Content-Length, Date, Expires, Keep-Alive, and Location (`http://localhost:9000/login`). The status bar at the bottom indicates 9 requests, 790 kB transferred, and 789 kB resources.

Path	Request Method	Status Code	Remote Address	Referrer Policy
<code>/oauth2/authorize</code>	GET	302 Found	[::1]:9000	strict-origin-when-cross-origin

Header	Value
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Connection	keep-alive
Content-Length	0
Date	Fri, 05 Dec 2025 20:59:51 GMT
Expires	0
Keep-Alive	timeout=60
Location	<code>http://localhost:9000/login</code>

Please sign in

Sign in

9 requests790 kB transferred789 kB resourcesFinish: 929 msDOMContentLoaded: 339 msLoad: 7

Consent required

oidc-client wants to access your account unico

The following permissions are requested by the above app.
Please review these and consent if you approve.

☐ address
☐ profile

Submit Consent

Cancel

9 requests788 kB transferred948 kB resourcesFinish: 815 msDOMContentLoaded: 317 msLoad: 6

Consent required

oidc-client wants to access your account unico

The following permissions are requested by the above app.

Please review these and consent if you approve.

- ☒ address
- ☒ profile

Submit Consent

Cancel

Your consent to provide access is required.

If you do not approve, click Cancel, in which case no information will be shared with the app.

9 requests 788 kB transferred

Path

- /login
- /oauth2/authorize
- /bootstrap/4.5.2/css/bootstrap...
- /content-all.css
- /fonts/fonts.css
- /fonts/DM-Sans-regular.woff2
- /assets/logo-O35E636P.png
- /i18n/ko.json
- /fonts/Noto-Sans-Regular.w...

Headers

General

Request URL: http://localhost:9000/login

Request Method: POST

Status Code: 302 Found

Remote Address: [::1]:9000

Referrer Policy: strict-origin-when-cross-origin

Response Headers

Cache-Control: no-cache, no-store, max-age=0, must-revalidate

Connection: keep-alive

Content-Length: 0

Date: Fri, 05 Dec 2025 21:13:29 GMT

Expires: 0

Keep-Alive: timeout=60

Location: http://localhost:9000/oauth2/authorize?response_type=code&client_id=oidc-client&redirect_uri=http://localhost:8080/login/oauth2/code/oidc-client&scope=openid%20profile%20address&state=xyz123&continue

Pragma: no-cache

localhost

localhost:8080/login/oauth2/code/oidc-client?code=kXK4XU4YefPVG5RgmZ_leniDidZ40nZgaTOz4dPQcuynr0Fi-PpH_25Jw2yhDRw2FcJk-NJM...

사이트에 연결할 수 없음

localhost에서 연결을 거부했습니다.

다음 방법을 시도해 보세요.

- 연결 확인
- 프록시 및 방화벽 확인

ERR_CONNECTION_REFUSED

새로고침

세부정보

16 requests 789 kB transferred

Path

- /oauth2/authorize
- /bootstrap/4.5.2/css/boot...
- /content-all.css
- /fonts/fonts.css
- /fonts/DM-Sans-regular.w...
- /assets/logo-O35E636P.png
- /i18n/ko.json
- /fonts/Noto-Sans-Regular...
- /oauth2/authorize
- /login/oauth2/code/oidc-...
- /login/oauth2/code/oidc-...
- /login/oauth2/code/oidc-...

Headers

General

Request URL: http://localhost:9000/login

Request Method: POST

Status Code: 302 Found

Remote Address: [::1]:9000

Referrer Policy: strict-origin-when-cross-origin

Response Headers

Cache-Control: no-cache, no-store, max-age=0, must-revalidate

Connection: keep-alive

Content-Length: 0

Date: Fri, 05 Dec 2025 21:13:29 GMT

Expires: 0

Keep-Alive: timeout=60

Location: http://localhost:9000/oauth2/authorize?

localhost

40nZgaTOz4dPQcuynr0Fi-PpH_25Jw2yhDRw2FcJk-NJMrvzA4H6qTOhs44BPi3ZzyE5FrhcbXSHsStungHUR3Ou9ODjgNwZDQld&state=xyz123

이 페이지에 대해 Google에 질문하기 - localhost

kXK4XU4YefPVG5RgmZ_leniDidZ40nZgaTOz4dPQcuynr0Fi-PpH_25Jw2yhDRw2FcJk-NJMrzvzA4H6qTOhs44BPi3ZzyE5FrhcbXSHsStungHUR3Ou9ODjgNwZDQld

[illegible]

```
C:\WINDOWS\system32\cmd.exe
C:\Users\unico>curl -X POST "http://localhost:9000/oauth2/token" -H "Authorization: Basic b2lkYy1jbGllbnQ6c2VjcmV0" -H "Content-Type: application/x-www-form-urlencoded" -d "grant_type=authorization_code&code=kKX4U4YefPVG5RgmZ_leniDidZ40nZgaT0z4dPQCuyrn0Fi-PpH_25Jw2yhDRw2FcJk-NJMrvzA4H6qTOhs44BPi3ZzyE5FrhcXSHsStungHUR30u90DjgNwZDQld&redirect_uri=http://localhost:8080/login/oauth2/code/oidc-client"
{"access_token": "eyJraWQ1OiI0LWZmYTBMZC01YzcxLTQxYWETOTBmMC00MjMwZDZmYzc4NTciLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJlbmljbyIsImF1ZCI6Im9pZGMtY2xpZW50IiwibmJmIjoxeNzY0OTY5Mzc0LClzY29wZSI6WyJhZGRyZGNzIiwib3BlbmkiOiwiichJvZmZlZSJsZSJdLCJpc3MiOiJodHRwOi8vbG9jYXRob3N0eXkwMDAiLCJleHAiOjE3NjQ5Njk2NzQsImhhdCI6MTc2NDk2OTM3NCwianRpIjoiaWVwInGZlMmEtY2Y0NC00NTJLLWEwZTctZDhlZWE4Y2VmOTc1In0.Dinhw2XoITdu3SyUcvhKuqrgeZ4wF8gQmd3akz3EymXdNFGzQjraIRZw5eDh0gigGGDWye5bpN776_2RLetXJgMprmlxfAbnH60bxfdeJiiijPhjdI8X098blRjU0hrSmuExGFmGmqyStcgYQ-IDZCKcfwx0htZztIWWrZazzqTj0KOJkjkrqg7kmsNmjiFgXGcbUmUlJju7DWBck4siBWLuxcCIgS2G7aikCnj7GcdOfLuMo0yveOHRu7JXDxnPxN4Q2LIIZCMCVh2e0dJmkwmXZsisNYo6oSsXW80Bd99HTRPMA434laLbgocjJ4t1M54TYHwIo2DgvJakcihM8w","refresh_token":"giOpGsXAjf5a7m3BDtkhBpeJE1Qo7aEI8uMEIDADV5wzlwgG4L4R0KKY0jWu-7hagccjLg1gnHwkNo9-JmzqS3qhLLL7TxZBG84RLXn9ovt48Vq8qtX3-aHBvojt1u","scope":"address openid profile","id_token":"eyJraWQ1OiI0LWZmYTBMZC01YzcxLTQxYWETOTBmMC00MjMwZDZmYzc4NTciLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJlbmljbyIsImF1ZCI6Im9pZGMtY2xpZW50IiwibmJmIjoib2lkYy1jbGllbnQiLCJhdXRoX3RpbWUiOiE3NjQ5Njk5Nzg5ImZcyI6Imh0dHA6Ly9sb2NhbgHvc3Q60TAwMCIsImV4cCI6MTc2NDk3MTE3NCwiaWF0IjoxeNzY0OTY5Mzc0LClzY2dGkiOiJlMmIyTAZYS04NzQxLTRjMmYwODJjOS0xN2I1M2MxNDc2YmIlLCJzaWQiOiJrWHNHRRkUYuk3V3gzZzBLAVdLaLPetFQZFyTX1BoRVFEskNdndmRvwXuWIn0.WGIgxizbPGP9sdexn8RL2FVB1x0_IrkTKKaGxQUXnfKCjQbBTNOVLjJGSKsp6-IecSRvuAOYfZBMSXIroJGvjtgYZBeWXDHJygnCirB8mKSTWp3XOMOB1pmkWgPXs84PBiuElmXpZsaAKhuCCZo3mz22nR-Iyy9I9de4RxQHMRUccG3dt50IfmaH2Xo4EsrrWwQnlrJxr__jMrqJpoA70R5Kq5GLaQL-EV9oR0rF0SNJJo0JOyNKooRAMutody42Qw9JF5fgHtdMcJbbikCd12JE095WgCL9k9L-n6Lq4I3k4H_xP36YZNJmLfAskZFATTFNOMTofteRxcHdjDISpg"},"token_type":"Bearer","expires_in":299}
```


curl -X GET "http://localhost:9001/api/private" -H "Authorization: Bearer 액세스토큰"

```
C:\Users\unico>
C:\Users\unico>curl -X GET "http://localhost:9001/api/private" -H "Authorization: Bearer eyJraWQiOiI4OWMzYTBMZC01YzcxLTQxYWVlOTBmMC00MjMwZDZmYzcxNTciLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1bmJbyIsImF1ZCI6Im9pZGMtY2xpZW50IiwibmJmIjozNzY0OTY5Mzc0LCJzY29wZSI6WyJhZGRyZXNzIiwib3BlbmkiOiwiYXNjaWZlZCJpc3MiOiJodHRwOi8vbG9jYXRob3N0OjkwMDAiLCJleHAiOiE3NjQ5Njk2NzQsImhhdCI6MTc2NDk2OTM3NCwianRpIjoieWVlNGZlMmEtY2Y0NC00NTJlLWVlZTctZDhlZWE4Y2VmOTc1In0.Dinhw2XoITdu3SyUcvhKuqrqeZ4wF8gQmd3akz3EymXdNFGzQjraIRZW5eDh0gigGGDwyE5bpN776_2RLETxJgMprmlxfAbnH60bxfEjIiljPhj4dI8X098blRjU0hrSmuExGFmGMqyStcgyQ-IDZCKCfxwOhTZztIWWrZazzqTj0K0JjkjRqg7kmsNMijFgXGcbUmUilJju7DWBCK4siBWLuxcCIgS2G7aikCnj7GcD0fLuM0yveOHRu7JXdXnPxn4Q2LIZCMcVh2e0dJmkwmXZsisNYo6oSsXW80Bd99HTrPM434laLbgocjJ4t1M54TYHwIo2DgvJakcihM8w"
인증을 거치고 인가 과정을 통해서 받은 액세스 토큰이 있어야 접근할 수 있어요!!
C:\Users\unico>
```

curl -X GET "http://localhost:9001/api/privatejwt" -H "Authorization: Bearer 액세스토큰"

```
C:\Users\unico>curl -X GET "http://localhost:9001/api/privatejwt" -H "Authorization: Bearer eyJraWQiOiI4OWMzYTBMZC01YzcxLTQxYWVlOTBmMC00MjMwZDZmYzcxNTciLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1bmJbyIsImF1ZCI6Im9pZGMtY2xpZW50IiwibmJmIjozNzY0OTY5Mzc0LCJzY29wZSI6WyJhZGRyZXNzIiwib3BlbmkiOiwiYXNjaWZlZCJpc3MiOiJodHRwOi8vbG9jYXRob3N0OjkwMDAiLCJleHAiOiE3NjQ5Njk2NzQsImhhdCI6MTc2NDk2OTM3NCwianRpIjoieWVlNGZlMmEtY2Y0NC00NTJlLWVlZTctZDhlZWE4Y2VmOTc1In0.Dinhw2XoITdu3SyUcvhKuqrqeZ4wF8gQmd3akz3EymXdNFGzQjraIRZW5eDh0gigGGDwyE5bpN776_2RLETxJgMprmlxfAbnH60bxfEjIiljPhj4dI8X098blRjU0hrSmuExGFmGMqyStcgyQ-IDZCKCfxwOhTZztIWWrZazzqTj0K0JjkjRqg7kmsNMijFgXGcbUmUilJju7DWBCK4siBWLuxcCIgS2G7aikCnj7GcD0fLuM0yveOHRu7JXdXnPxn4Q2LIZCMcVh2e0dJmkwmXZsisNYo6oSsXW80Bd99HTrPM434laLbgocjJ4t1M54TYHwIo2DgvJakcihM8w"
반가워요~unico사용자님..., 접근할 수 있는 리소스(데이터 또는 기능)의 범위 : [address, openid, profile]
C:\Users\unico>
```

Getting Started :: Spring Auth...

docs.spring.io/spring-authorization-server/reference/getting-started.html

spring

Why Spring ▾ Learn ▾ Projects ▾ Academy ▾ Solutions ▾ Community ▾ ⚙

Spring Authorization Server

1.5.3

Q Search CTRL + k

Overview

Getting Help

Getting Started

Configuration Model

Core Model / Components

Protocol Endpoints

How-to Guides

Spring Authorization Server / Getting Started

Defining Required Components

If you want to customize the default configuration (regardless of whether you're using Spring Boot), you can define the minimum required components as a `@Bean` in a Spring `@Configuration`.

These components can be defined as follows:

SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    @Order(1)
    public SecurityFilterChain authorizationServerSecurityFilterChain(HttpSecurity http)
        throws Exception {
        OAuth2AuthorizationServerConfigurer authorizationServerConfigurer =
            OAuth2AuthorizationServerConfigurer.authorizationServer();

        http
            .securityMatcher(authorizationServerConfigurer.getEndpointsMatcher())
            .with(authorizationServerConfigurer, (authorizationServer) ->
                authorizationServer
                    .side(customizer.withDefaults()) // Enable OpenID Connect 1.0
```

Getting Started

System Requirements

Installing Spring Authorization Server

Developing Your First Application

Defining Required Components

Edit this Page

GitHub Project

Stack Overflow

[AuthorizationServerConfig 소스]

메서드	역할
authorizationServerSecurityFilterChain 로그인 페이지 연결	Authorization Server 전용 보안 설정, OIDC 활성화,
defaultSecurityFilterChain	일반 Spring Security 보안 설정 (formLogin 포함)
userDetailsService	테스트용 메모리 사용자 생성
registeredClientRepository	OAuth2/OIDC Client 등록
jwtSource	RSA 키 생성 및 JWK 제공
generateRsaKey	RSA 키쌍 생성 유틸
jwtDecoder JWT	서명검증용 디코더
authorizationServerSettings	Authorization Server 엔드포인트 기본 설정

@Bean

@Order(1)

```
public SecurityFilterChain authorizationServerSecurityFilterChain(HttpSecurity http)
    throws Exception {
    // Spring Authorization Server 전용 설정 객체 생성
    OAuth2AuthorizationServerConfigurer authorizationServerConfigurer =
        OAuth2AuthorizationServerConfigurer.authorizationServer();
    http
        // 이 필터체인은 Authorization Server 엔드포인트에만 적용
        .securityMatcher(authorizationServerConfigurer.getEndpointsMatcher())

        // OAuth2 Authorization Server 설정을 적용하며 OIDC 기능 활성화
        .with(authorizationServerConfigurer, (authorizationServer) ->
            authorizationServer
                .oidc(Customizer.withDefaults()) // OpenID Connect 1.0 활성화
        )
        // 모든 AS 엔드포인트는 인증된 사용자만 접근 가능
        .authorizeHttpRequests((authorize) ->
            authorize.anyRequest().authenticated()
        )
        // 인증되지 않은 접근 시 로그인 페이지로 리디렉션 처리
        .exceptionHandling((exceptions) -> exceptions
    }
```

```

        .defaultAuthenticationEntryPointFor(
            new LoginUrlAuthenticationEntryPoint("/login"),
            new MediaTypeRequestMatcher(MediaType.TEXT_HTML)
        )
    );

    return http.build();
}

```

@Bean

@Order(2)

```

public SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http
        // 일반 요청은 모두 인증 필요
        .authorizeHttpRequests((authorize) -> authorize
            .anyRequest().authenticated()
        )
        // 기본 Spring Security formLogin 활성화 (/login 자동 생성)
        .formLogin(Customizer.withDefaults());
    return http.build();
}

```

@Bean

```

public UserDetailsService userDetailsService() {
    // 메모리에 저장되는 테스트용 유저 생성
    UserDetails userDetails = User.withDefaultPasswordEncoder()
        .username("unico")
        .password("1234")
        .roles("USER")
        .build();
    return new InMemoryUserDetailsManager(userDetails);
}

```

@Bean

```

public RegisteredClientRepository registeredClientRepository() {
    // Authorization Server가 허용할 OAuth2/OIDC 클라이언트 정보 등록
    RegisteredClient oidcClient = RegisteredClient.withId(UUID.randomUUID().toString())
        .clientId("oidc-client")

```

```

        .clientSecret("{noop}secret") // client_secret_basic 방식에서 사용되는 secret
        // 클라이언트 인증 방식: Basic Auth (Authorization: Basic ...)
        .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        // OAuth2 Authorization Code Grant + Refresh Token Grant 허용
        .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
        .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
        // 인증 후 Authorization Code를 전달받을 클라이언트 redirect URI
        .redirectUri("http://localhost:8080/login/oauth2/code/oidc-client")
        // OIDC 로그아웃 후 리디렉션될 URI
        .postLogoutRedirectUri("http://localhost:8080/")
        // 클라이언트가 접근 가능한 scope 선언 (OIDC 필수: openid)
        .scope(OidcScopes.OPENID)
        .scope(OidcScopes.PROFILE)
        .scope(OidcScopes.ADDRESS)
        // 사용자 동의 화면 활성화 (Consent 화면 표시)
        .clientSettings(ClientSettings.builder()
            .requireAuthorizationConsent(true)
            .build()
        )
        .build();
    return new InMemoryRegisteredClientRepository(oidcClient);
}

```

@Bean

```

public JWKSSource<SecurityContext> jwkSource() {
    // Access Token / ID Token 등에 서명하기 위한 RSA 키 생성
    KeyPair keyPair = generateRsaKey();
    RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
    RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
    // RSA 키를 JWK 형태로 구성 (public/private 포함)
    RSAKey rsaKey = new RSAKey.Builder(publicKey)
        .privateKey(privateKey)
        .keyID(UUID.randomUUID().toString())
        .build();
    // JWK Set 생성 (Authorization Server는 /oauth2/jwks 로 공개키 제공)
    JWKS jwkSet = new JWKS(rsaKey);
    return new ImmutableJWKS(jwkSet);
}

```

```
private static KeyPair generateRsaKey() {
    // 2048비트 RSA 키 생성 (토큰 서명용)
    try {
        KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("RSA");
        keyPairGenerator.initialize(2048);
        return keyPairGenerator.generateKeyPair();
    } catch (Exception ex) {
        throw new IllegalStateException(ex);
    }
}
```

```
@Bean
public JwtDecoder jwtDecoder(JWKSSource<SecurityContext> jwkSource) {
    // 생성된 JWKSSource를 기반으로 JWT 서명 검증기 생성
    return OAuth2AuthorizationServerConfiguration.jwtDecoder(jwkSource);
}
```

```
@Bean
public AuthorizationServerSettings authorizationServerSettings() {
    // Authorization Server 기본 엔드포인트 설정을 그대로 사용
    // (/oauth2/authorize, /oauth2/token, /oauth2/jwks 등)
    return AuthorizationServerSettings.builder().build();
}
```

[Resource Server 소스]

```
server:
  port: 9001
```

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000
```

JWT 발급자(Issuer)의 위치(URI)를 지정
issuer-uri는
JWT를 발급한 Authorization Server의 Issuer(iss) 를 의미

Resource Server는 issuer로부터 메타데이터(OpenID Connect Discovery document) 를 자동 조회
Spring Security는 issuer-uri를 기반으로 아래 URL을 자동 호출한다:

<http://localhost:9000/.well-known/openid-configuration>

이 문서 안에는 다음 정보가 들어 있다:

jwks_uri (공개키 제공 URL)

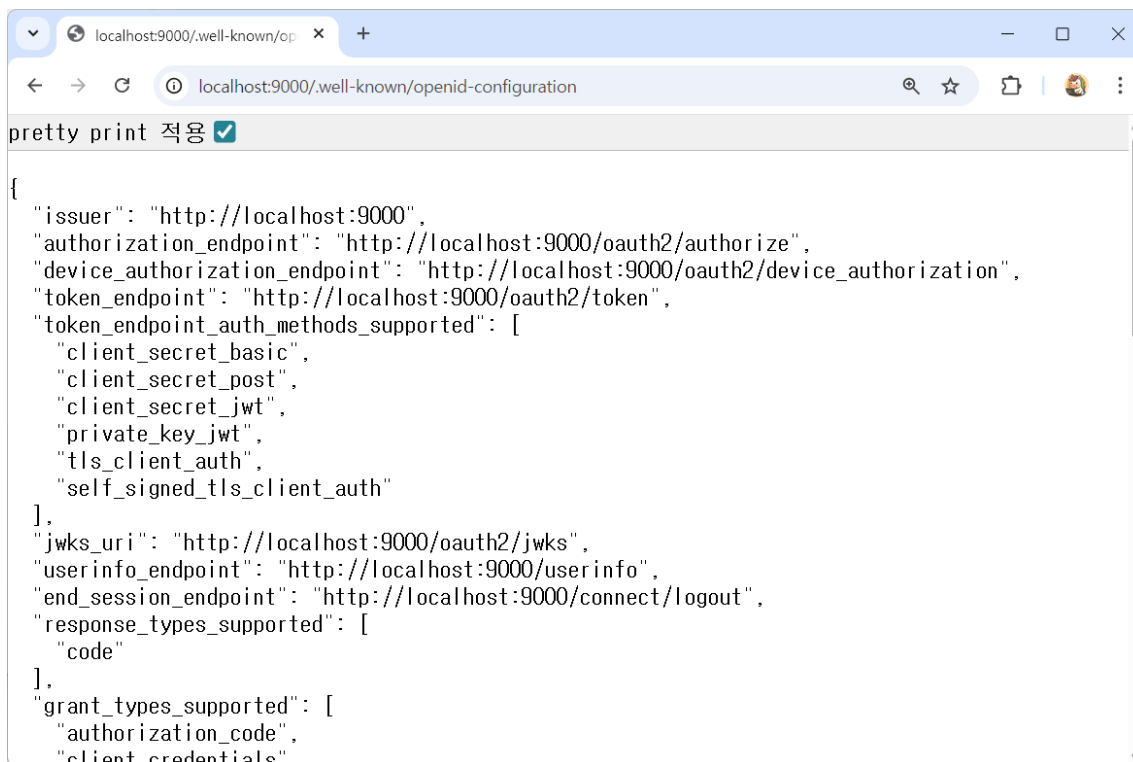
authorization_endpoint

token_endpoint

issuer

userinfo_endpoint 등

즉, Resource Server는 issuer만 알면 나머지는 다 자동으로 가져온다.



@Configuration

@EnableWebSecurity

public class SecurityConfig {

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

http

// HTTP 요청에 대한 접근 제어 규칙 정의

.authorizeHttpRequests((authorize) -> authorize

// "/api/public" 경로는 인증 없이 접근 허용 (누구나 사용 가능)

.requestMatchers("/api/public").permitAll()

```

// 나머지 모든 요청은 인증 필요
.anyRequest().authenticated()
)
// OAuth2 Resource Server 기능 활성화
// Authorization Server가 발급한 JWT Access Token을 검증하여 인증 처리
.oauth2ResourceServer((oauth2) -> oauth2
    // JWT 기반 인증 활성화
    .jwt(Customizer.withDefaults())
);
// SecurityFilterChain 구성 완료 후 리턴
return http.build();
}
}

```

@RestController

public class ApiController {

```

@GetMapping("/api/public")
public String publicEndpoint() {
    System.out.println("***public***");
    return "인증과정 필요없이 누구든 접근할 수 있어요.^";
}

```

```

@GetMapping("/api/private")
public String privateEndpoint() {
    System.out.println("***private***");
    return "인증을 거치고 인가 과정을 통해서 받은 액세스 토큰이 있어야 접근할 수 있어요!!";
}

```

```

@GetMapping("/api/privatejwt")
public String privateEndpoint(@AuthenticationPrincipal Jwt jwt) {
    System.out.println("***privatejwt***");
    System.out.println("전달받은 JWT 객체 : "+jwt);
    return "반가워요~~" + jwt.getSubject() + "사용자님..., 접근할 수 있는 리소스(데이터 또는 기능)의 범위 : " +
    jwt.getClaimAsStringList("scope");
}
}

```

5. 소셜 로그인

5.1 소셜 로그인의 정의

소셜 로그인은 **구글**, 페이스북, **네이버**, **카카오** 등과 같은 **소셜 네트워크 서비스의 계정 정보를 이용해 다른 웹사이트나 앱에 간편하게 로그인할 수 있게 해주는 기능**이다.

[소셜 로그인의 특징]

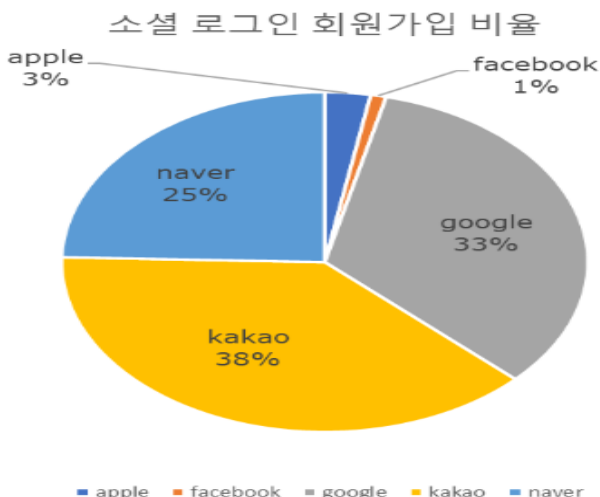
- 사용자는 별도의 회원 가입이나 복잡한 비밀번호 설정 없이 기존 소셜 계정으로 손쉽게 로그인할 수 있다.
- 복잡한 로그인 절차를 간소화하고, 비밀번호 관리의 부담을 줄여준다.
- 서비스 제공자는 소셜 로그인을 통해 사용자를 쉽게 확인하고, 다양한 사용자 데이터를 활용해 개인화된 서비스를 제공할 수 있다.

[작동 원리]

- 사용자가 "소셜 로그인" 버튼(예: 구글로 로그인, 네이버로 로그인 등)을 클릭한다.
- 소셜 네트워크 서비스에서 인증을 거치고, 인증이 완료되면 사용자가 원래의 사이트나 앱에 접근할 수 있도록 권한이 부여된다.
- 주요 기술로는 **OAuth 2.0**, **OpenID Connect** 같은 표준 프로토콜이 사용된다.

[소셜 로그인의 장점]

- 로그인 및 회원가입 절차가 단순하고 빠르다.
- 여러 웹사이트나 앱마다 별도의 계정을 만들 필요 없다.
- 서비스를 신속하게 이용할 수 있어 이용률과 전환율을 높이는 데 도움이 된다.



로그인

이메일과 비밀번호를 입력하세요.

이메일

비밀번호 비밀번호를 잊었나요?

로그인

계정이 있으신가요? 계정생성

G f N

5.2 소셜 로그인 동작 방식

(1) OAuth 2.0 프로토콜 기반

소셜 로그인은 주로 OAuth 2.0 프로토콜을 사용한다. OAuth 2.0은 인증(Authentication)과 인가(Authorization)를 위한 개방형 표준이다.

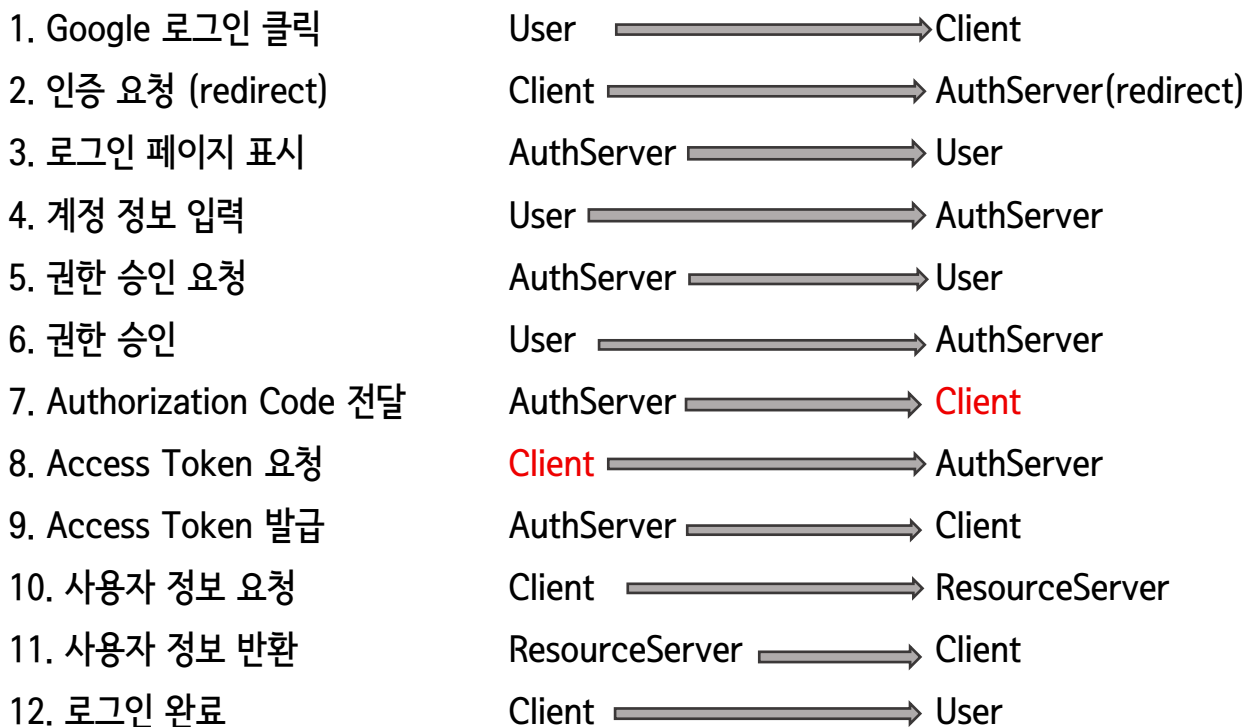
주요 구성 요소:

- Resource Owner: 사용자 (리소스 소유자)
- Client: 회원 정보를 제공받아서 서비스를 제공하는 애플리케이션
- Resource Server: 사용자 정보를 가지고 있는 서버 (Google, Kakao, Naver 등)
- Authorization Server: 인가를 담당하는 서버 (보통 Resource Server와 동일)

(2) Authorization Code Grant 플로우

가장 일반적으로 사용되는 Grant Type 이다.

User(사용자), Client(클라이언트 앱), AuthServer(인가 서버), ResourceServer(리소스 서버)



[단계별 상세 설명]

1단계: 인증 요청

GET [https://accounts.google.com/oauth/authorize?](https://accounts.google.com/oauth/authorize?client_id=YOUR_CLIENT_ID&redirect_uri=https://yourapp.com/callback&scope=openid%20email%20profile&response_type=code&state=random_string_for_security)

client_id=YOUR_CLIENT_ID&

redirect_uri=https://yourapp.com/callback&

scope=openid email profile&

response_type=code&

state=random_string_for_security

[파라미터 설명]

- client_id: 애플리케이션 식별자
- redirect_uri: 인증 완료 후 돌아올 URL
- scope: 요청할 권한 범위
- response_type=code: Authorization Code 방식 사용
- state: CSRF 공격 방지용 랜덤 값

2단계: 사용자 인증 및 권한 승인

사용자가 소셜 플랫폼에 로그인

애플리케이션이 요청한 권한에 대한 승인/거부 결정

3단계: Authorization Code 받기

GET [https://yourapp.com/callback?](https://yourapp.com/callback?code=AUTHORIZATION_CODE&state=random_string_for_security)

code=AUTHORIZATION_CODE&

state=random_string_for_security

4단계: Access Token 교환

POST <https://oauth2.googleapis.com/token>

Content-Type: application/x-www-form-urlencoded

client_id=YOUR_CLIENT_ID&

client_secret=YOUR_CLIENT_SECRET&

code=AUTHORIZATION_CODE&

grant_type=authorization_code&

redirect_uri=https://yourapp.com/callback

응답:

```
{
  "access_token": "ya29.a0AfH6SMC...",
  "expires_in": 3599,
  "token_type": "Bearer",
  "scope": "openid email profile",
  "id_token": "eyJhbGciOiJSUzI1NiIs..."
}
```

5단계: 사용자 정보 조회

GET <https://www.googleapis.com/oauth2/v2/userinfo>
Authorization: Bearer ya29.a0AfH6SMC...

응답:

```
{
  "id": "123456789",
  "email": "user@example.com",
  "name": "홍길동",
  "picture": "https://lh3.googleusercontent.com/...",
  "locale": "ko"
}
```

(3) 보안 고려사항

[CSRF 공격 방지]

Cross-Site Request Forgery(교차 사이트 요청 위조)

- state 파라미터 사용

```
const state = crypto.randomBytes(32).toString('hex');
sessionStorage.setItem('oauth_state', state);
```

```
const authUrl = `https://accounts.google.com/oauth/authorize?
  client_id=${CLIENT_ID}&
  redirect_uri=${REDIRECT_URI}&
```

```
scope=openid email profile&
response_type=code&
state=${state}`;
```

[Authorization Code 탈취 방지]

- **HTTPS 필수** 사용
- Authorization Code는 일회성으로 즉시 무효화
- 짧은 만료 시간 설정 (보통 10분)

[Access Token 보안]

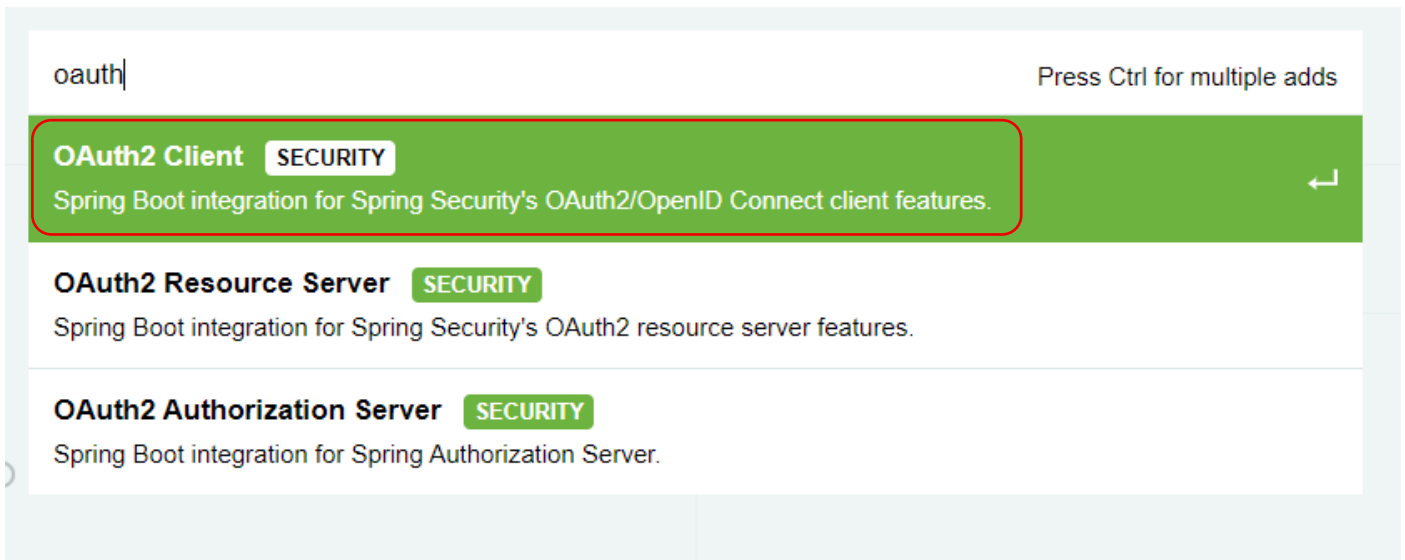
- 토큰 저장 시 localStorage에 저장하지 않는다. (XSS 공격 위험)
- **HttpOnly 쿠키 사용을 권장**한다.

```
res.cookie('access_token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 3600000 // 1시간
});
```

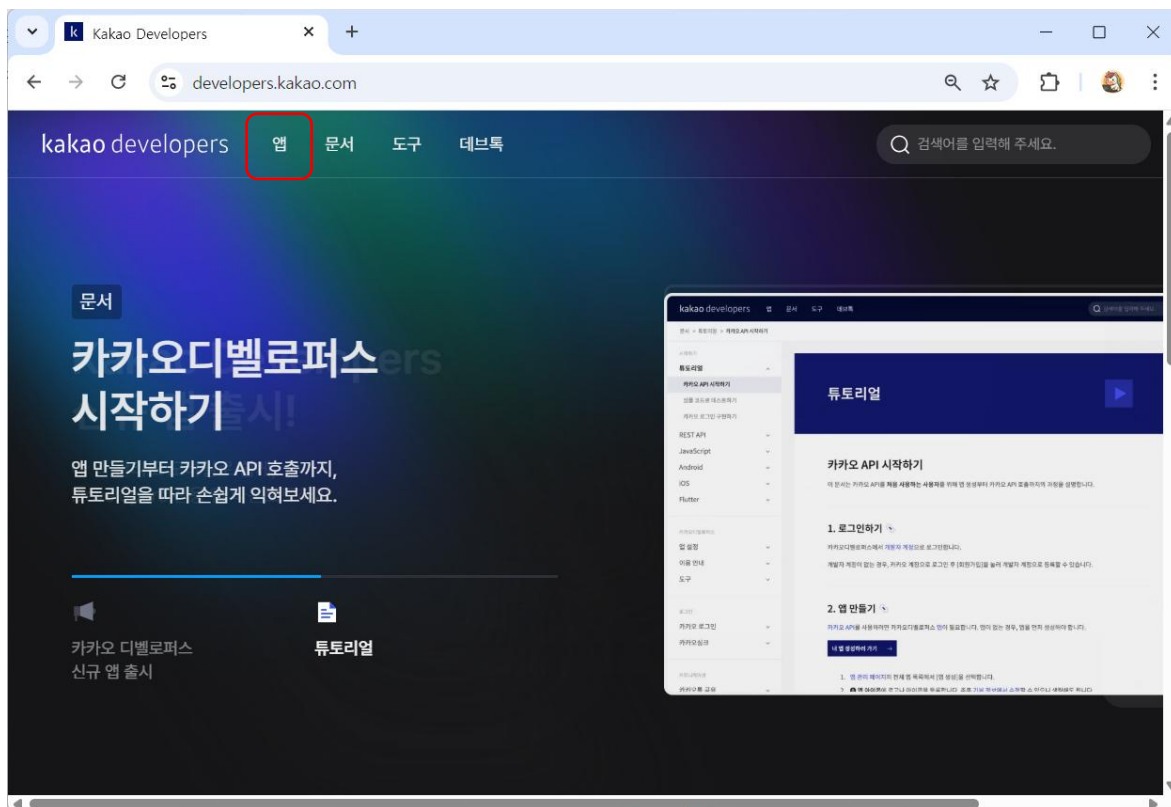
[Redirect URI 검증]

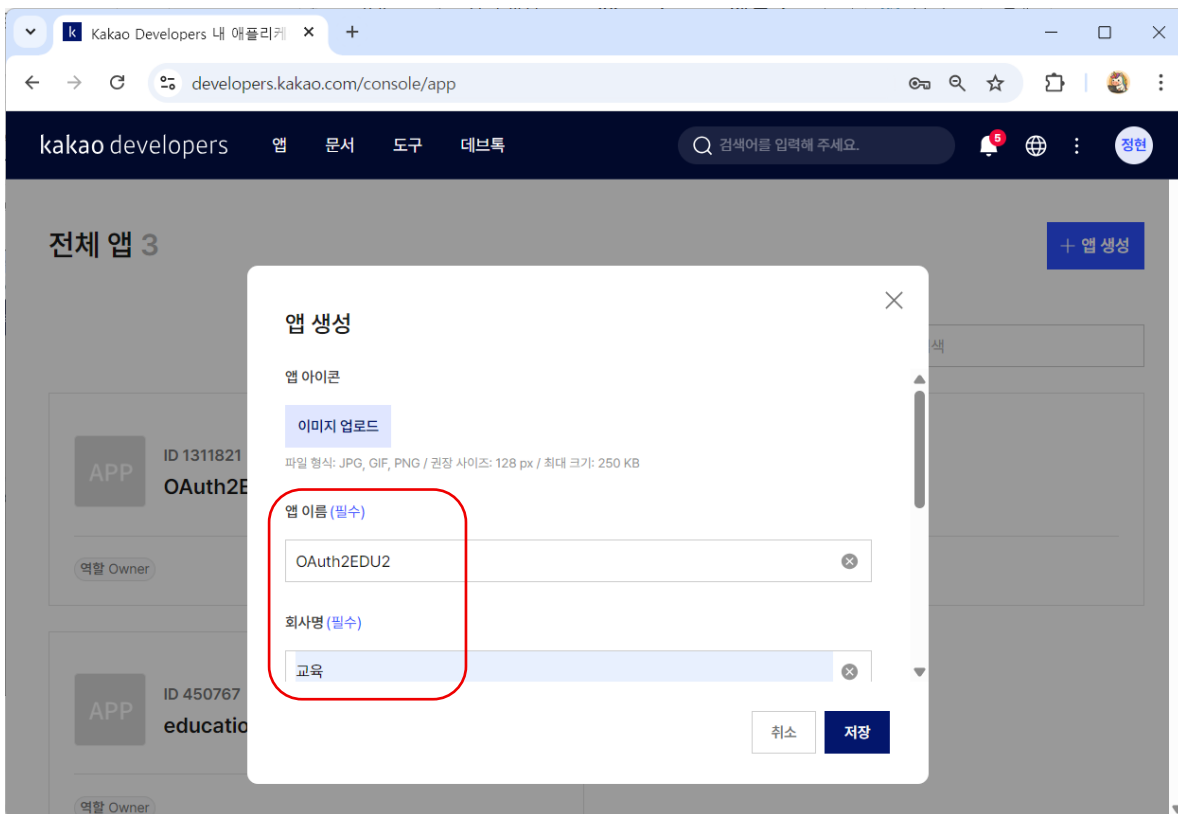
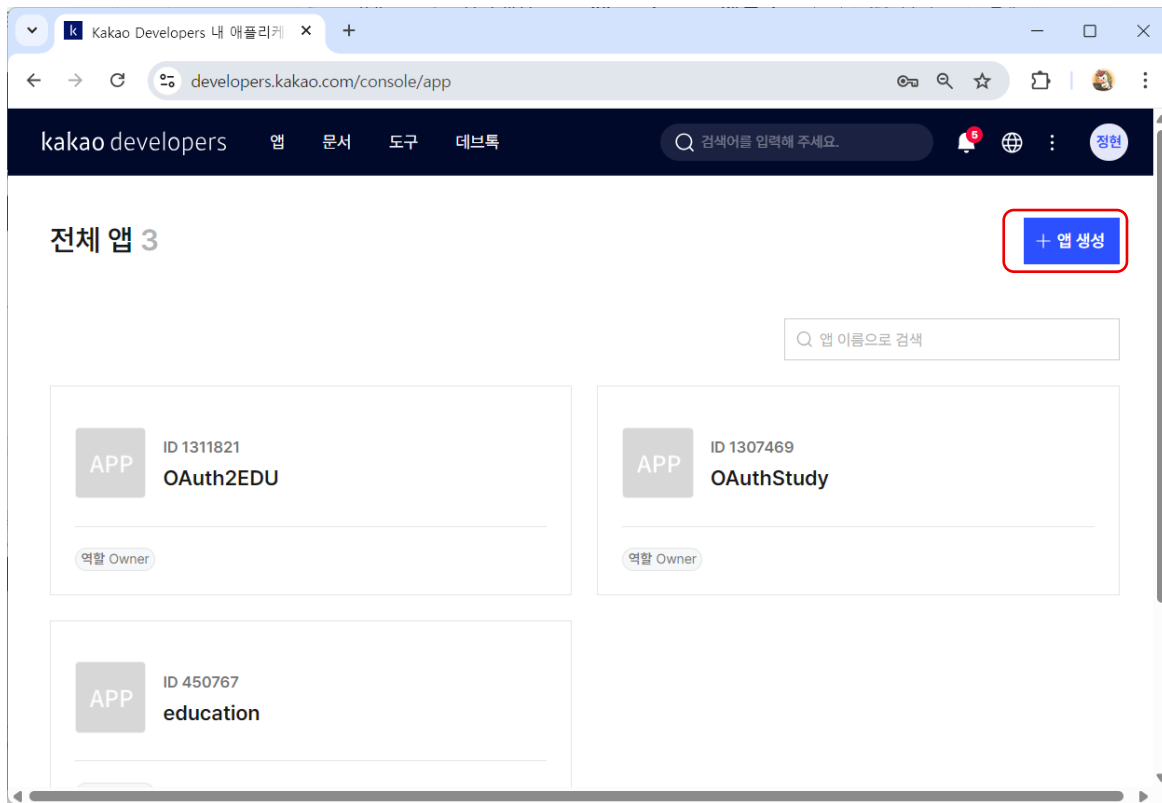
```
const allowedRedirectUris = [
  'https://yourapp.com/callback',
  'https://staging.yourapp.com/callback'
];

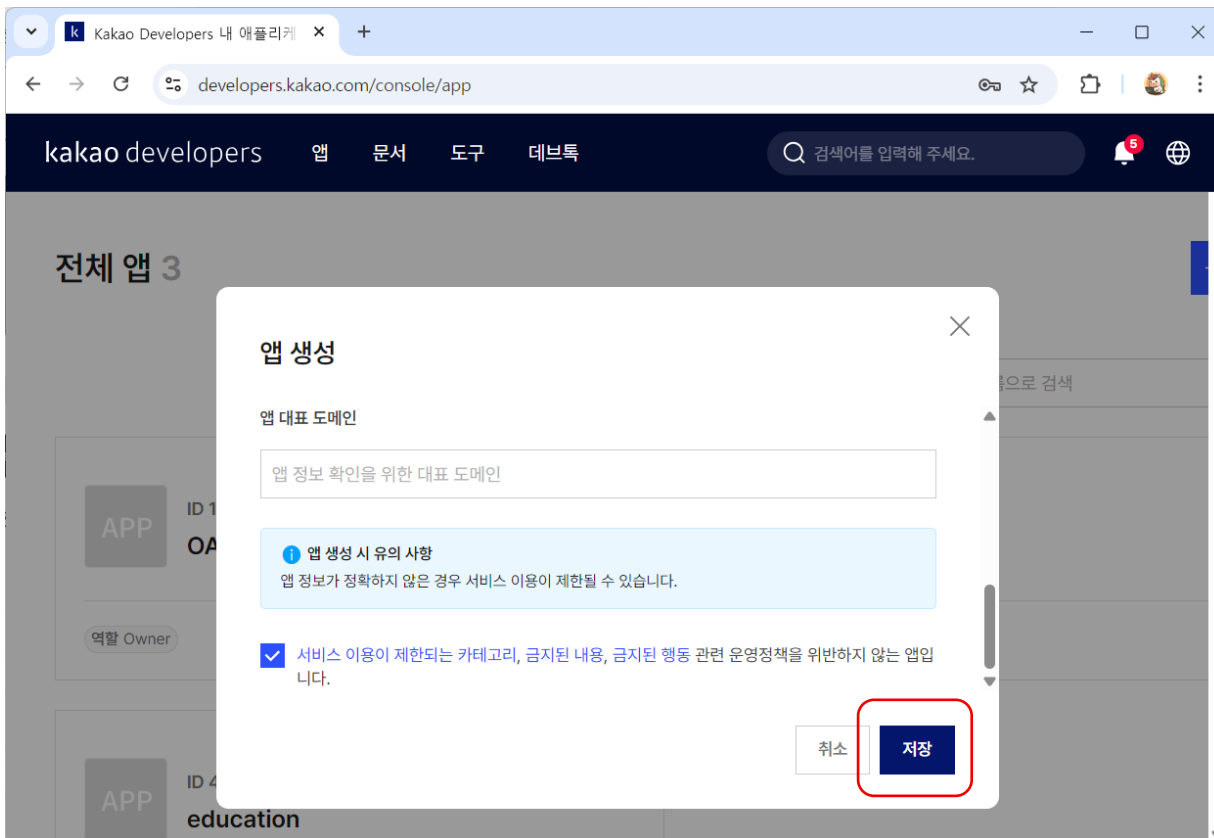
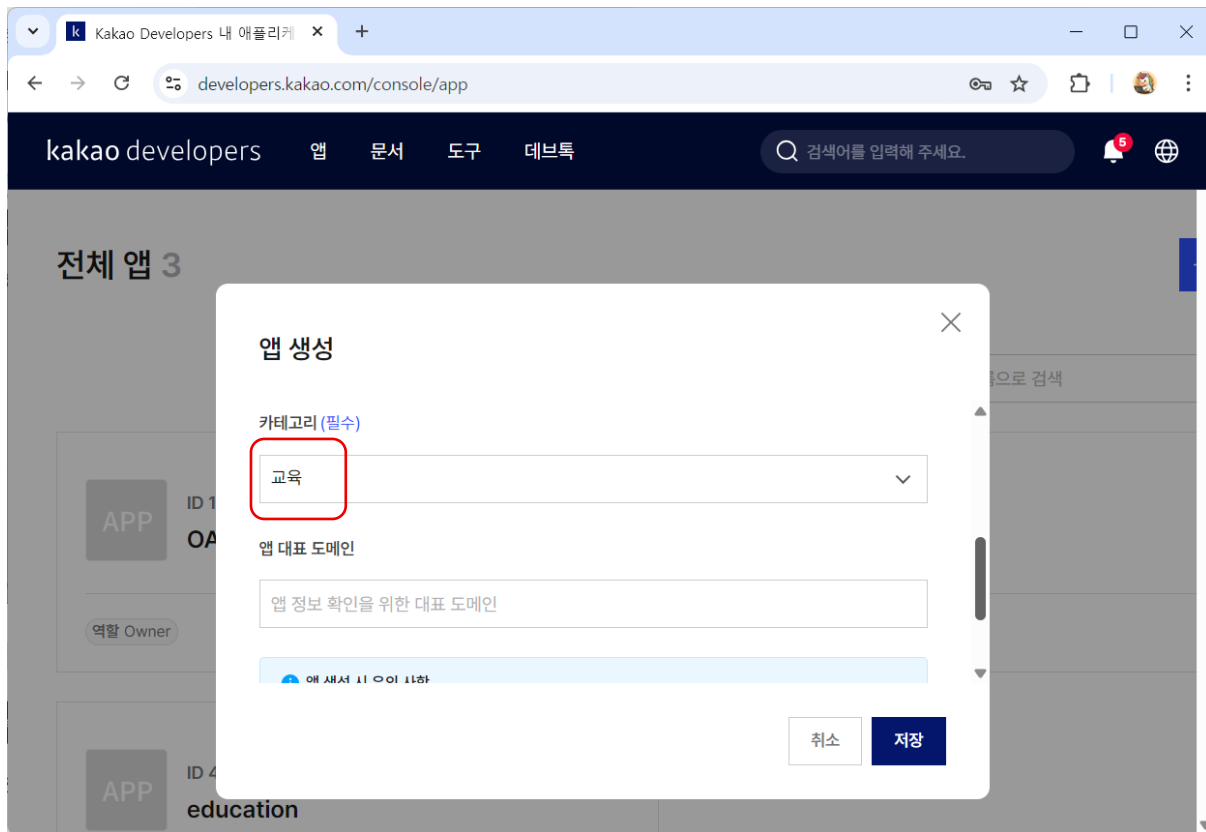
if (!allowedRedirectUris.includes(receivedRedirectUri)) {
  throw new Error('Invalid redirect URI');
}
```

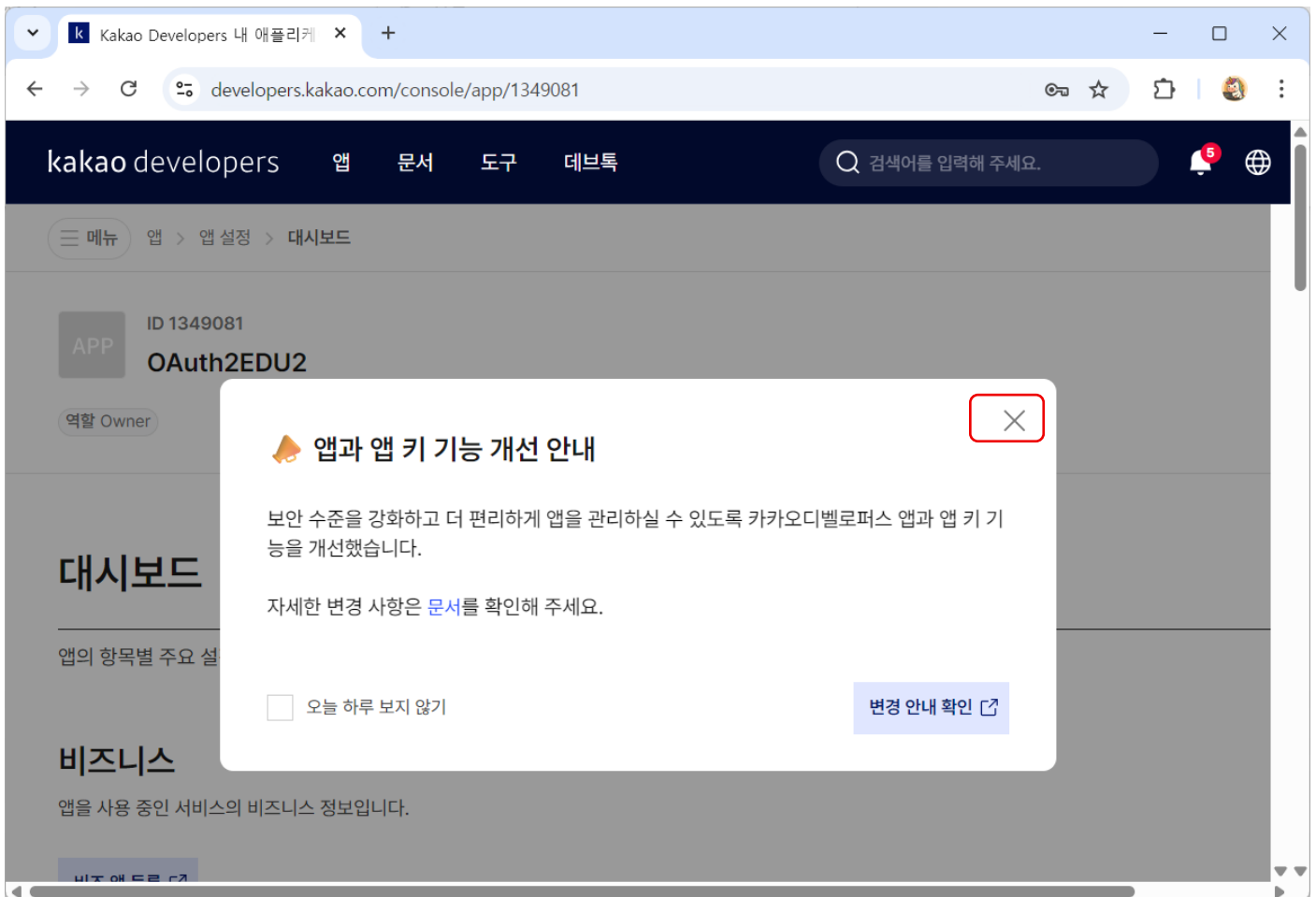
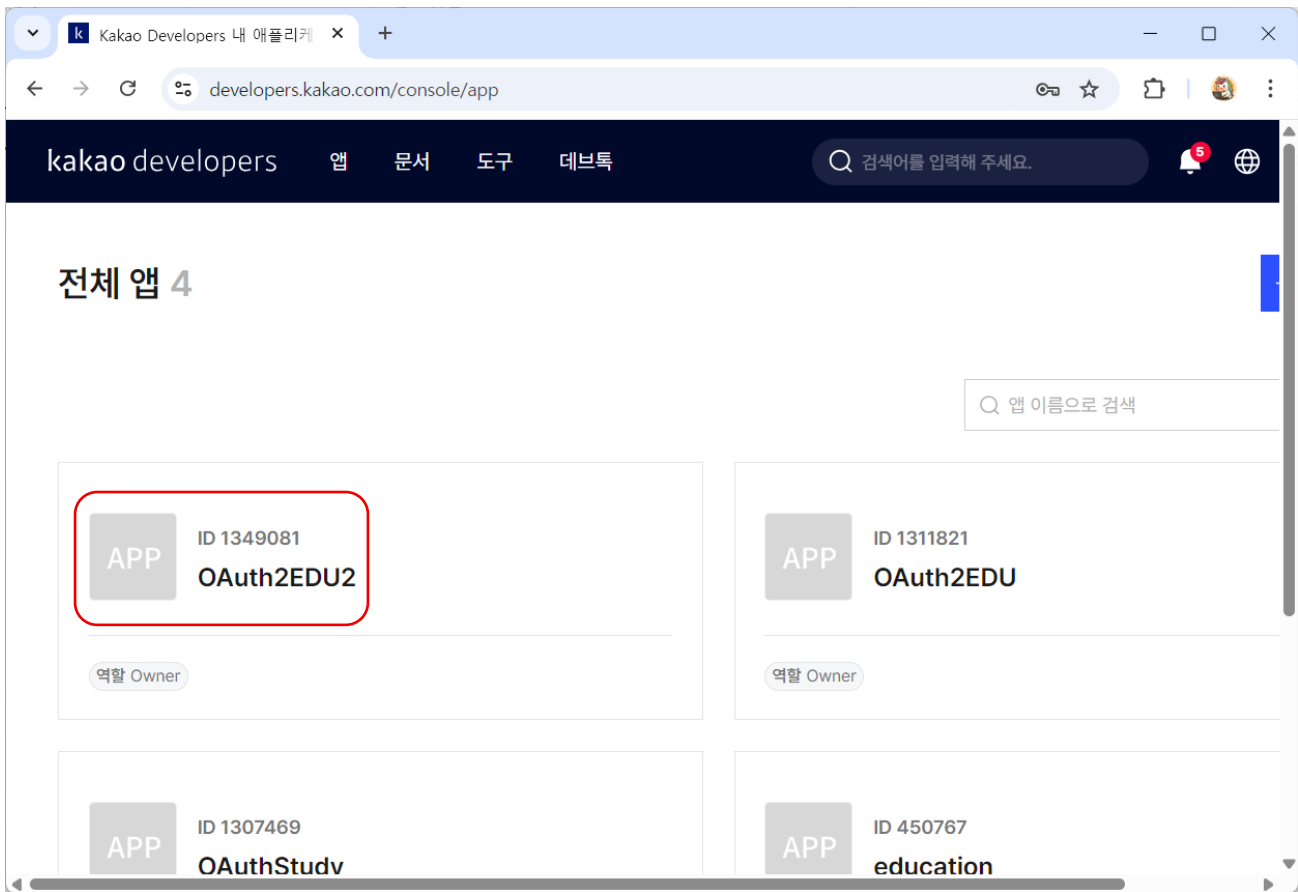


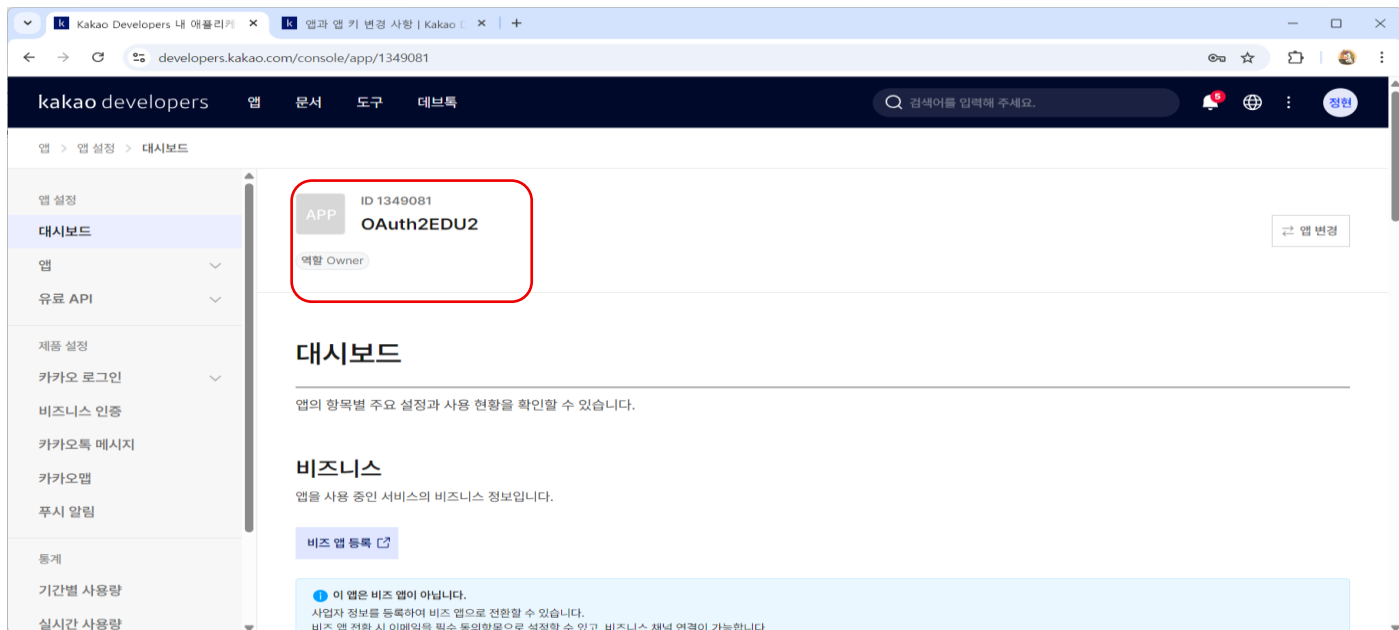
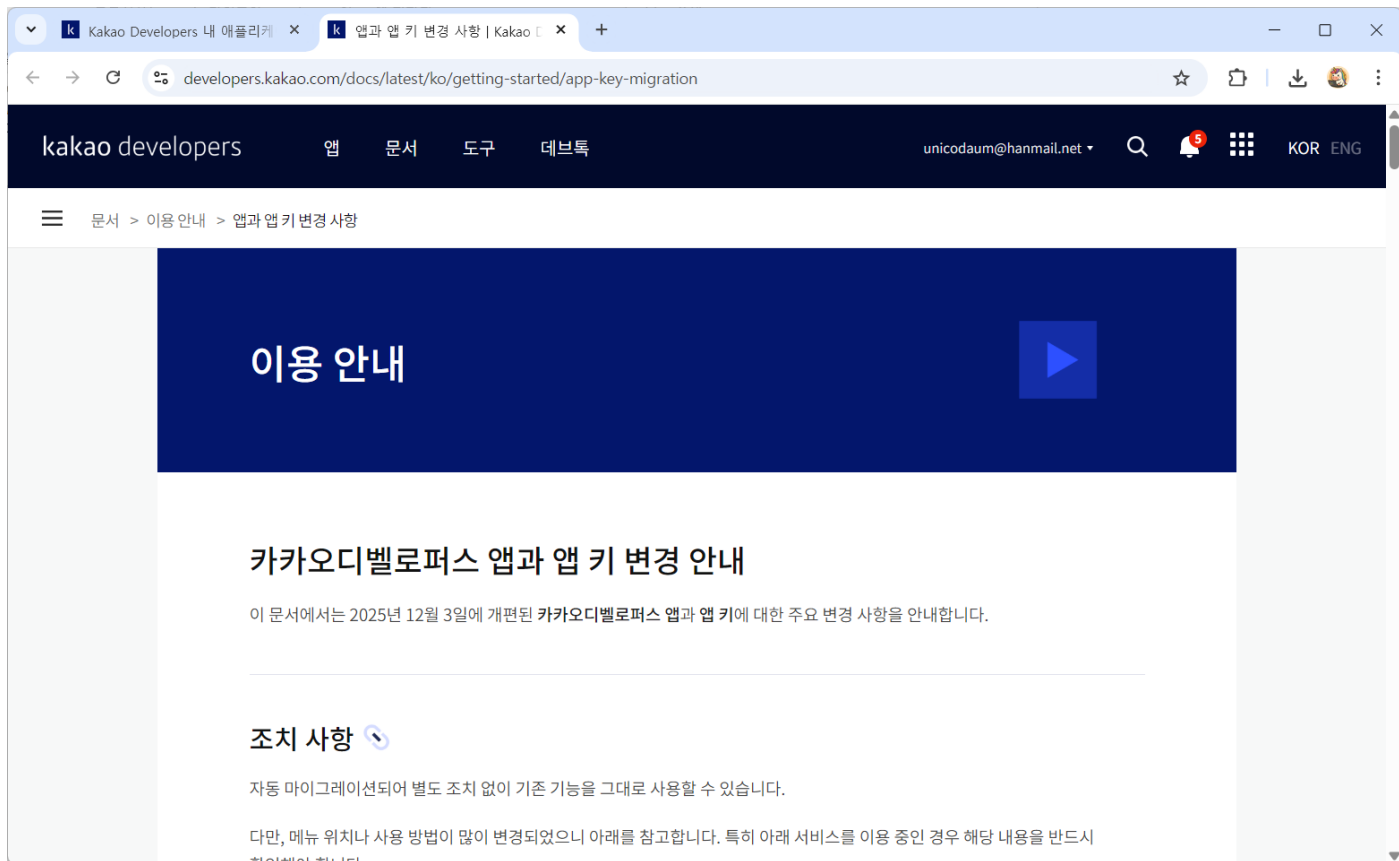
[카카오 개발자 사이트에서 카카오톡 로그인 설정 과정]

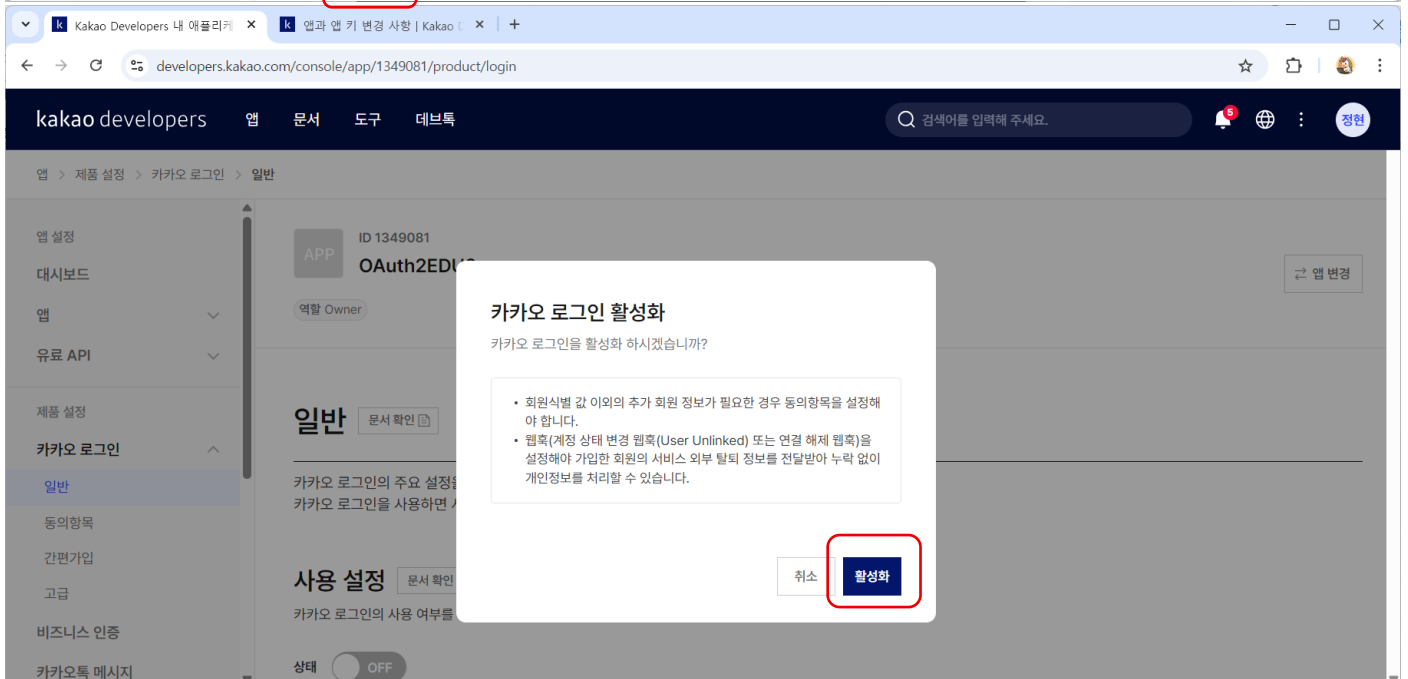
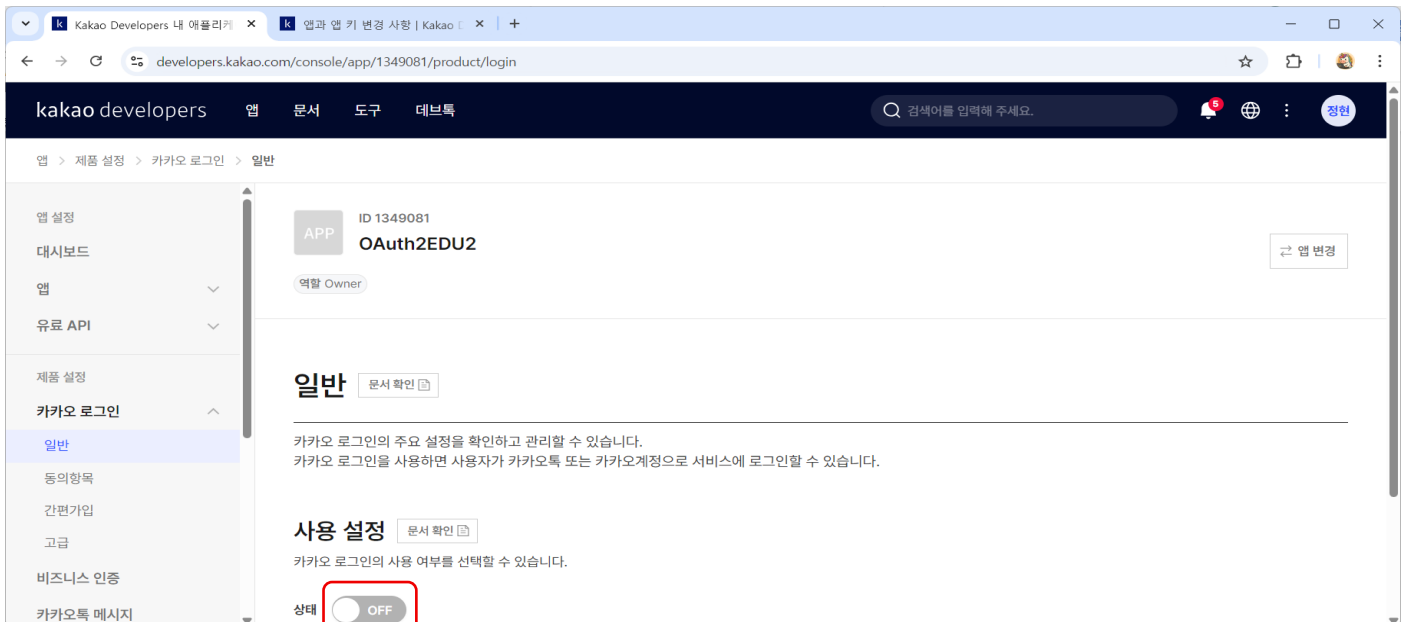
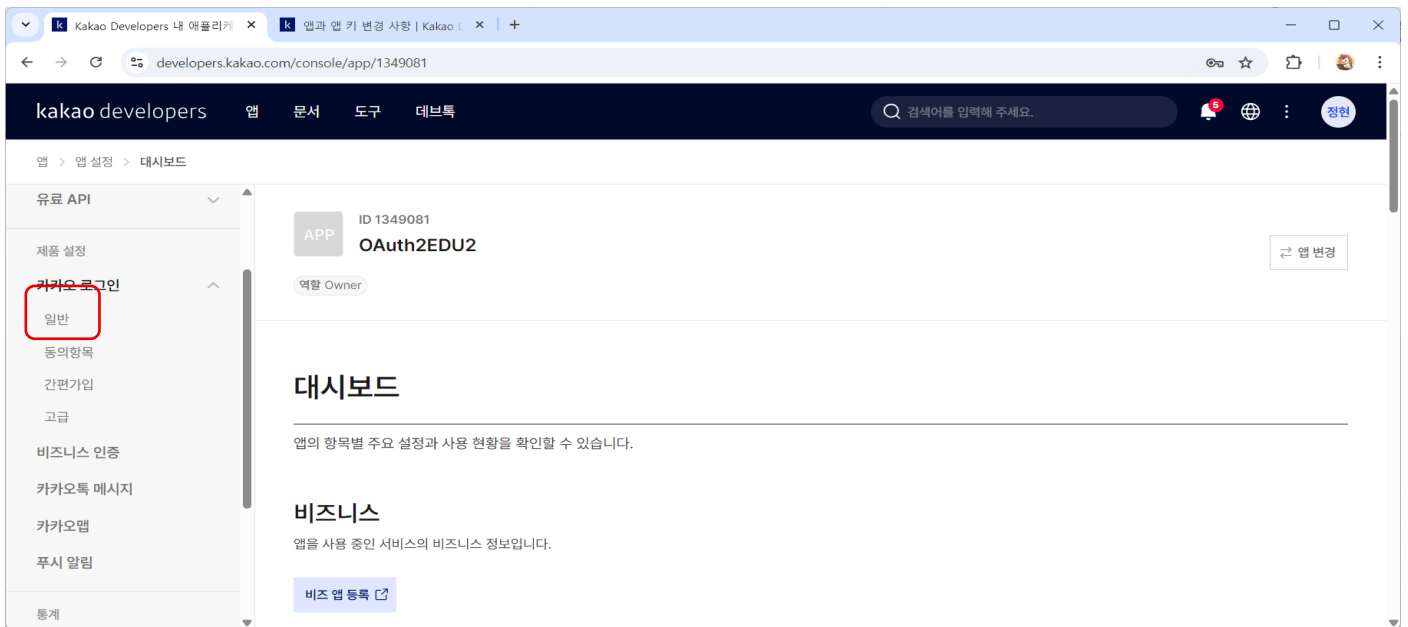


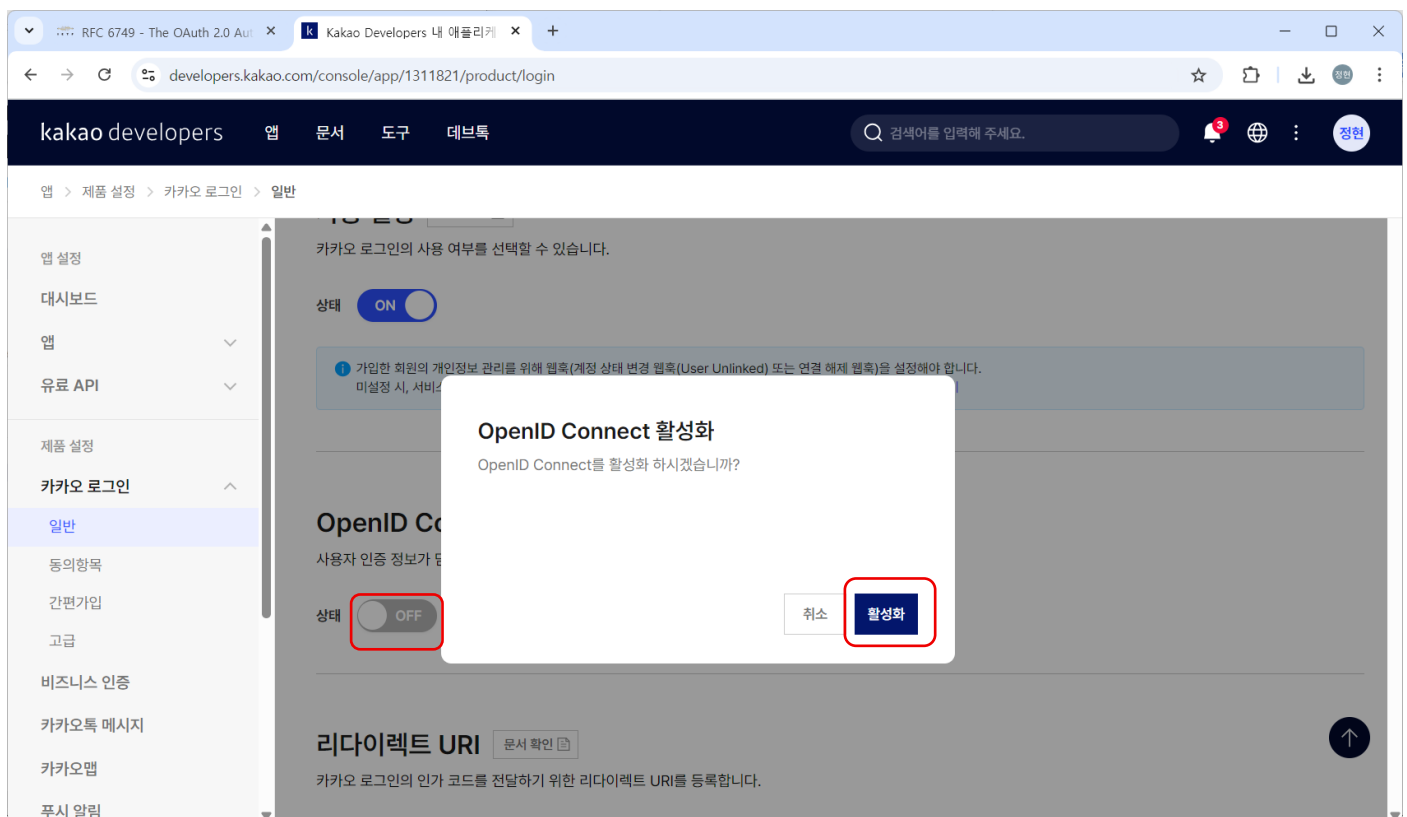
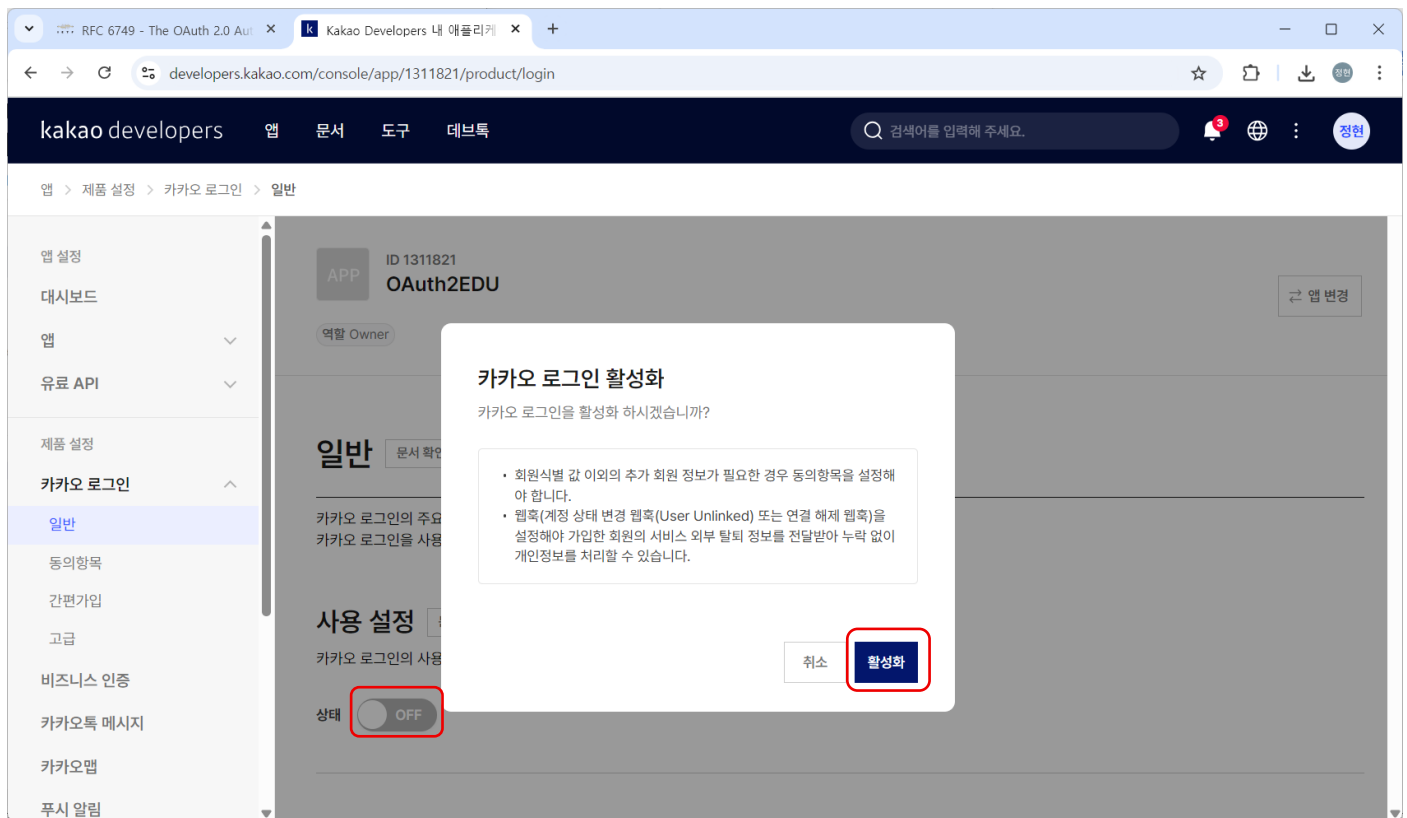


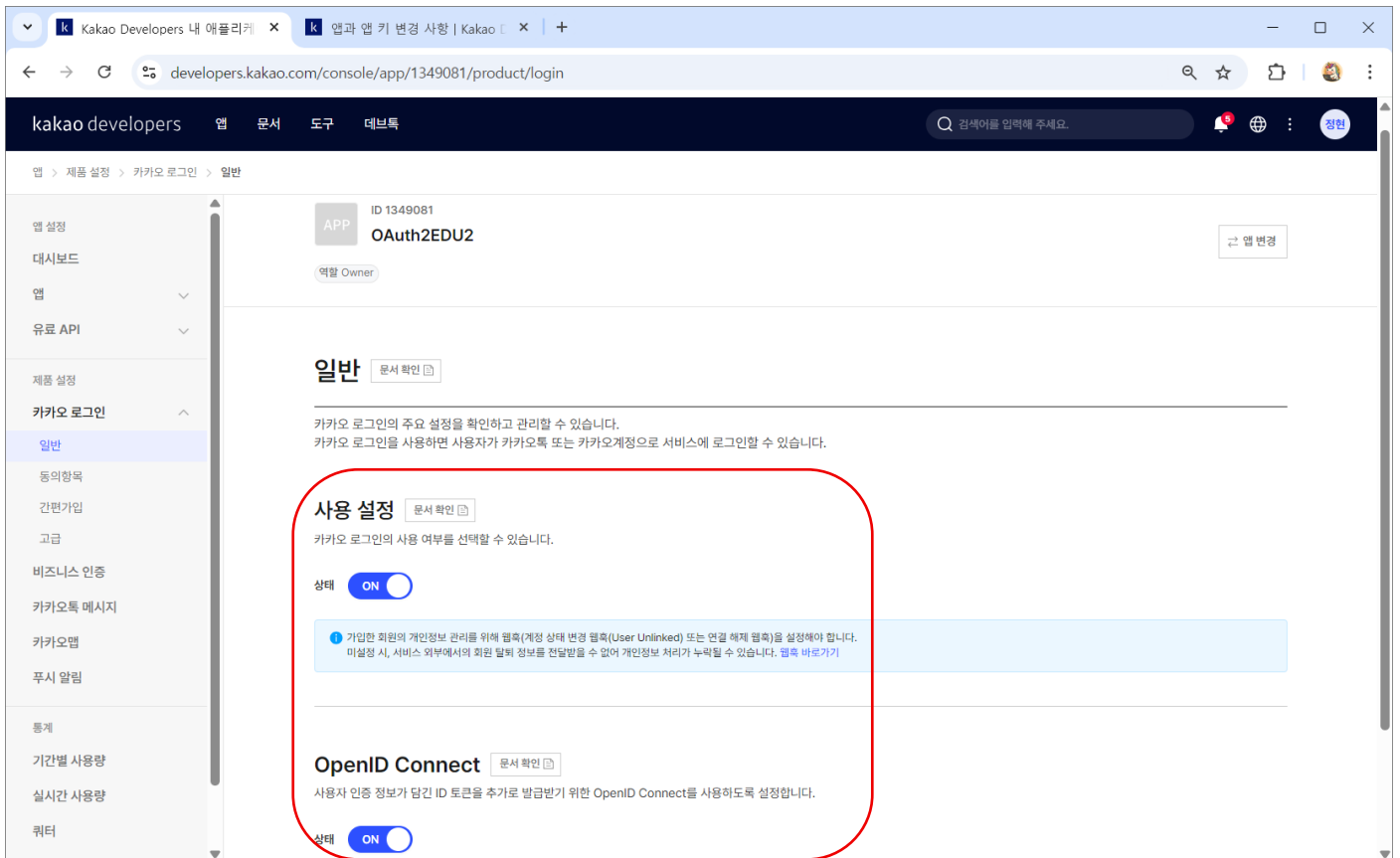
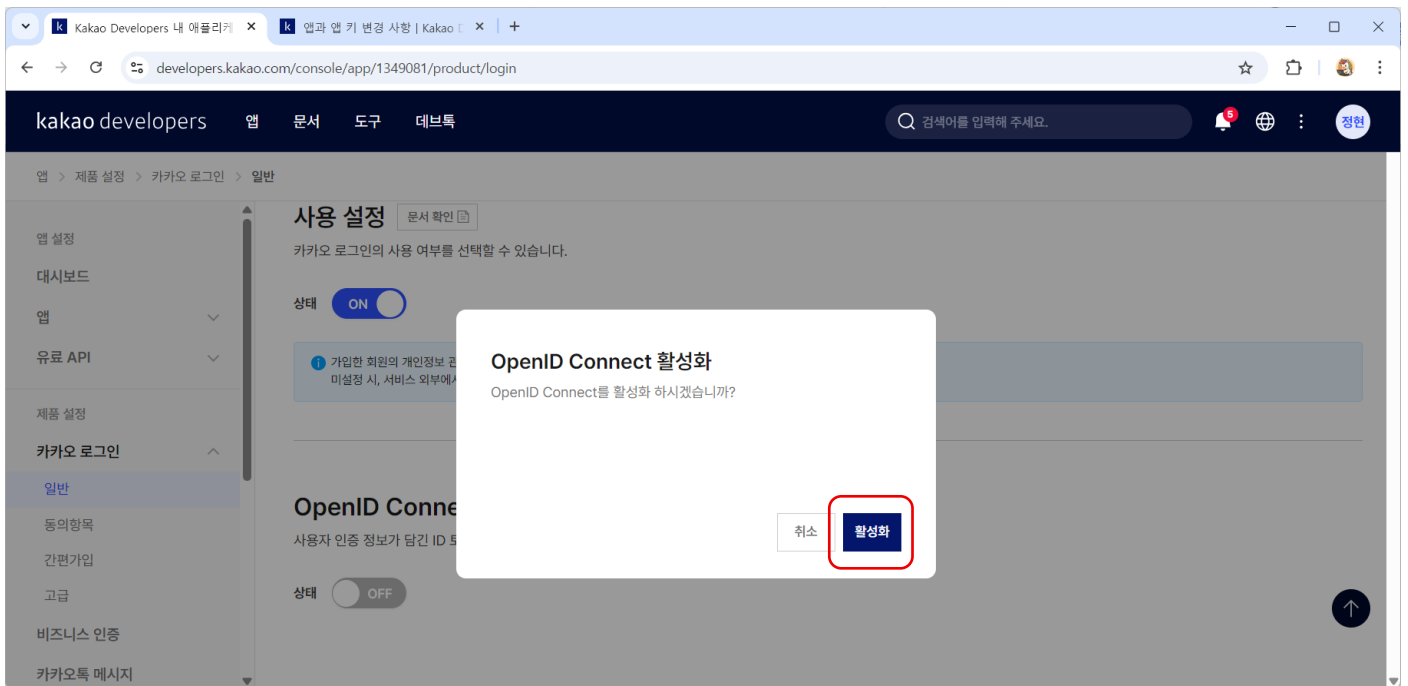


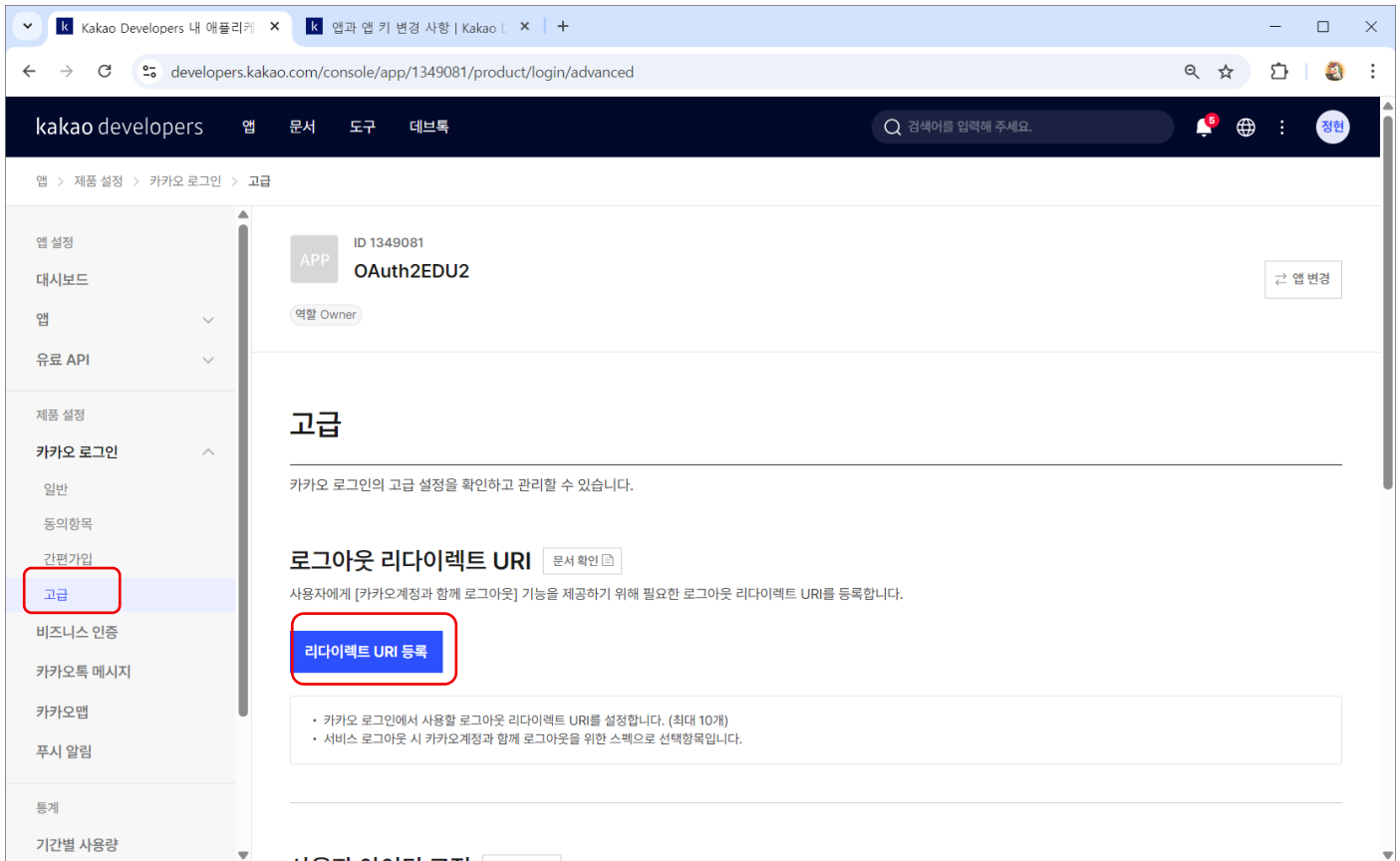




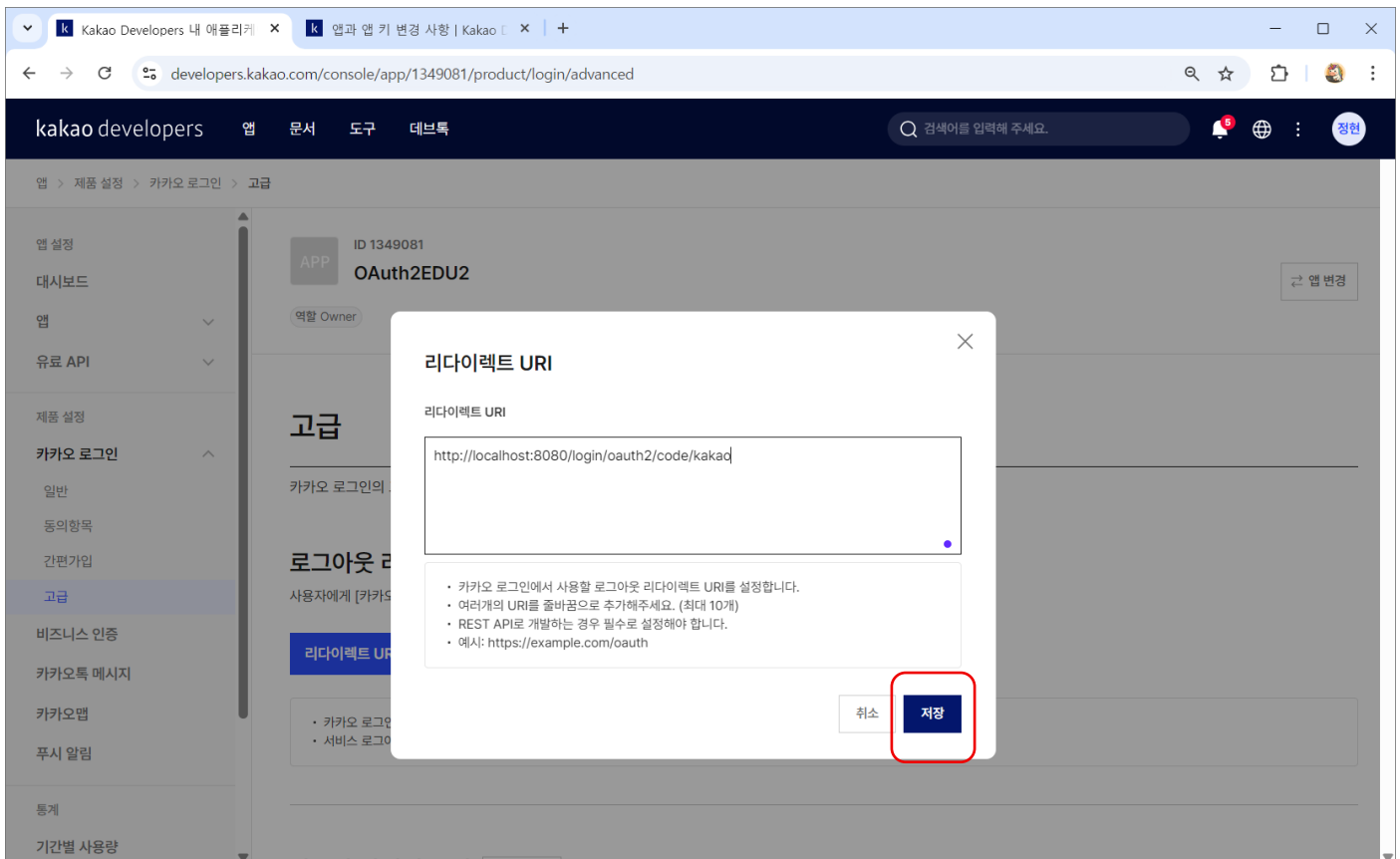








http://localhost:8080/login/oauth2/code/kakao



RFC 6749 - The OAuth 2.0 Au

Kakao Developers 내 애플리케이션

+

← → ↺ developers.kakao.com/console/app/1311821/product/login/scope

kakao developers 앱 문서 도구 데브북

검색어를 입력해 주세요.

🔔 2 🌐 ⋮ 정현

앱 > 제품 설정 > 카카오톡 로그인 > 동의항목

앱 설정
대시보드
앱
유료 API
제품 설정
카카오톡 로그인
일반
동의항목
간편가입
고급
비즈니스 인증
카카오톡 메시지
카카오맵
푸시 알림
통계

APP ID 1311821
OAuth2EDU

역할 Owner

↔ 앱 변경

동의항목

문서 확인

사용자 동의가 필요한 정보 또는 기능에 대응하는 동의항목을 카카오톡 로그인 동의 화면에 포함하도록 설정합니다.

추가 기능 신청

문서 확인

동의항목 설정 권한을 제공받기 위해 추가 기능을 신청합니다.

추가 기능 신청

개인정보

문서 확인

동의 화면 미리보기

Kakao Developers 내 애플리케이션

앱과 앱 키 변경 사항 | Kakao

+

← → ↺ developers.kakao.com/console/app/1349081/product/login/scope

kakao developers 앱 문서 도구 데브북

검색어를 입력해 주세요.

🔔 5 🌐 ⋮ 정현

앱 > 제품 설정 > 카카오톡 로그인 > 동의항목

앱 설정
대시보드
앱
유료 API
제품 설정
카카오톡 로그인
일반
동의항목
간편가입
고급
비즈니스 인증
카카오톡 메시지
카카오맵
푸시 알림

APP ID 1349081
OAuth2EDU2

역할 Owner

↔ 앱 변경

동의항목

문서 확인

사용자 동의가 필요한 정보 또는 기능에 대응하는 동의항목을 카카오톡 로그인 동의 화면에 포함하도록 설정합니다.

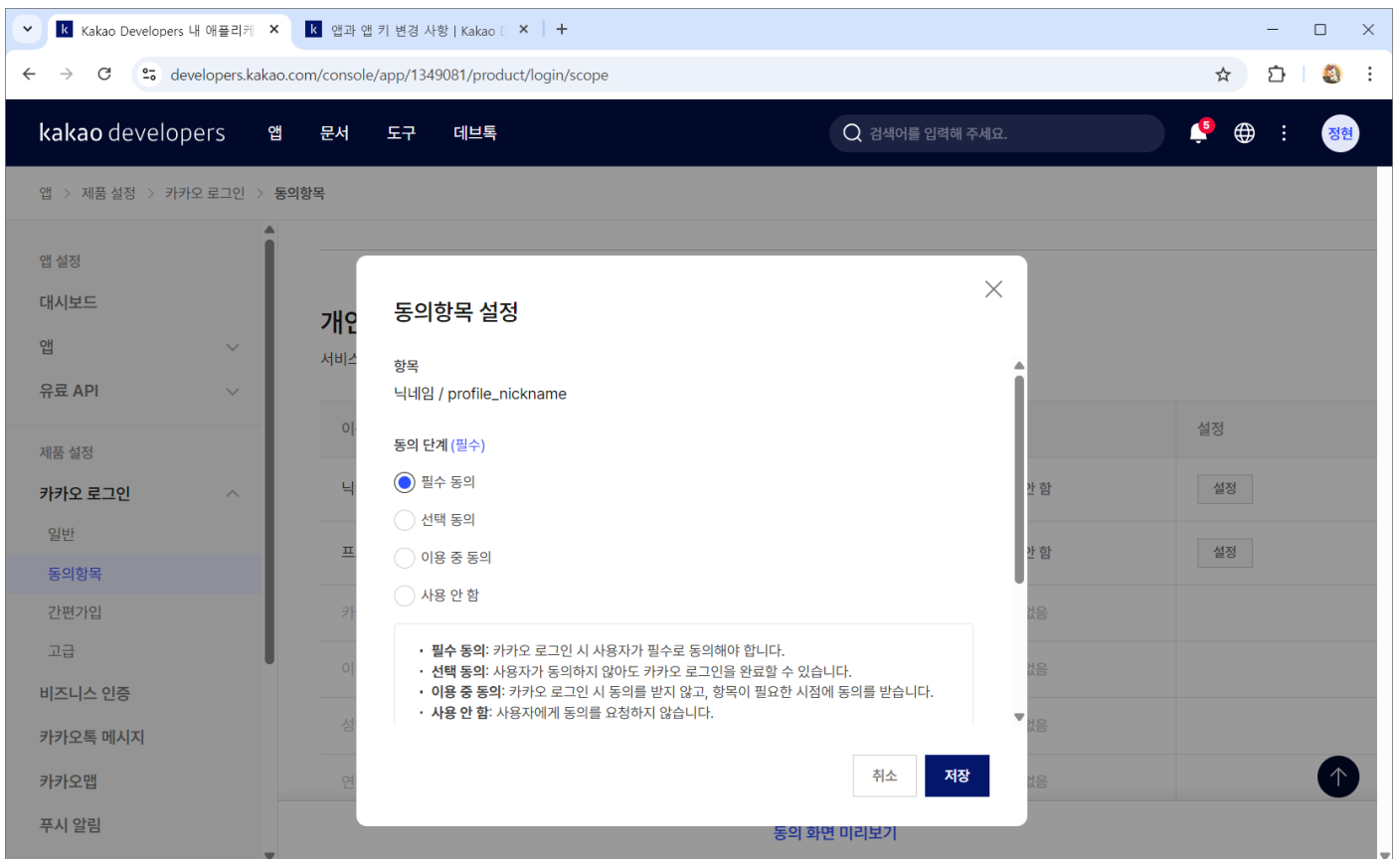
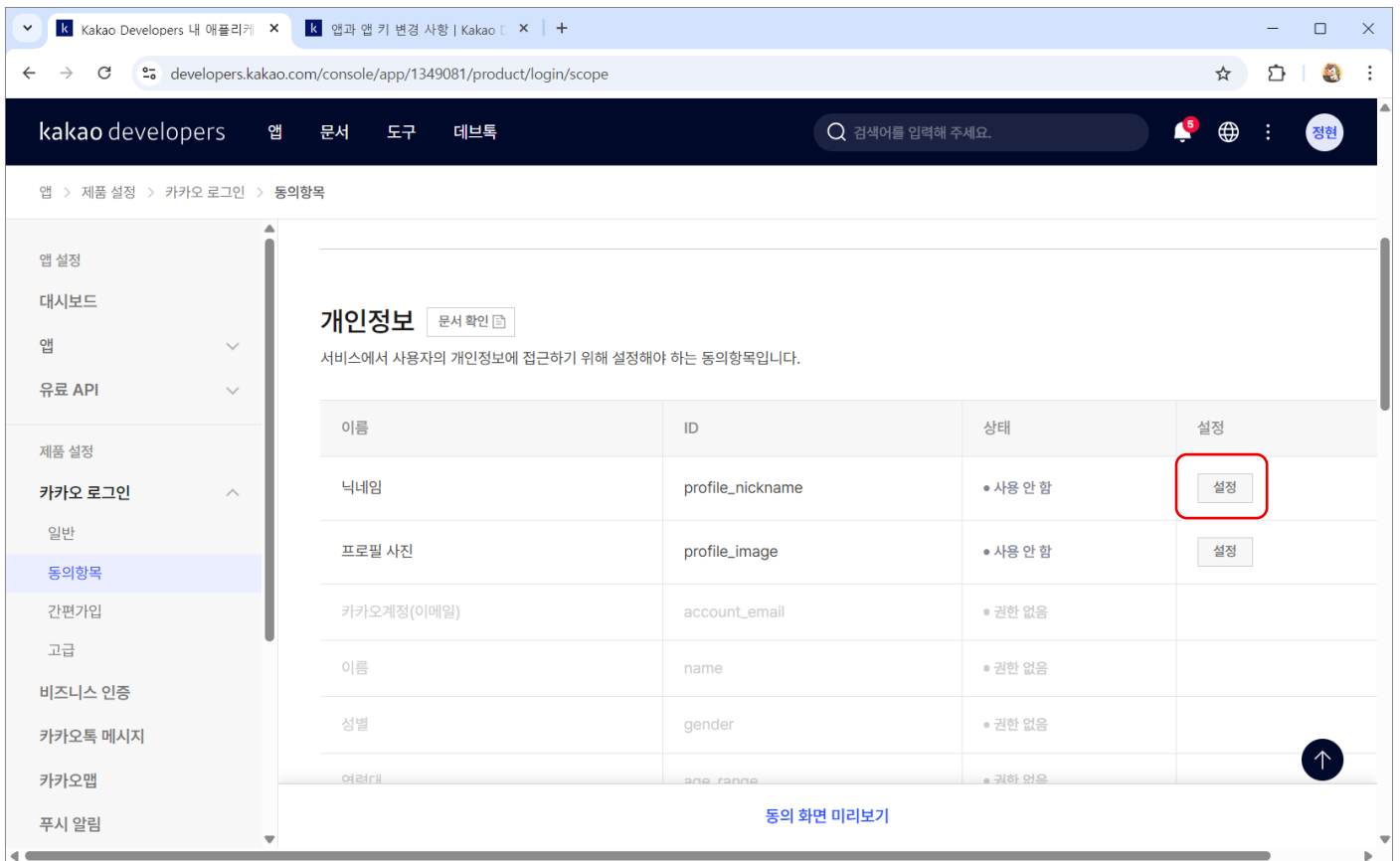
추가 기능 신청

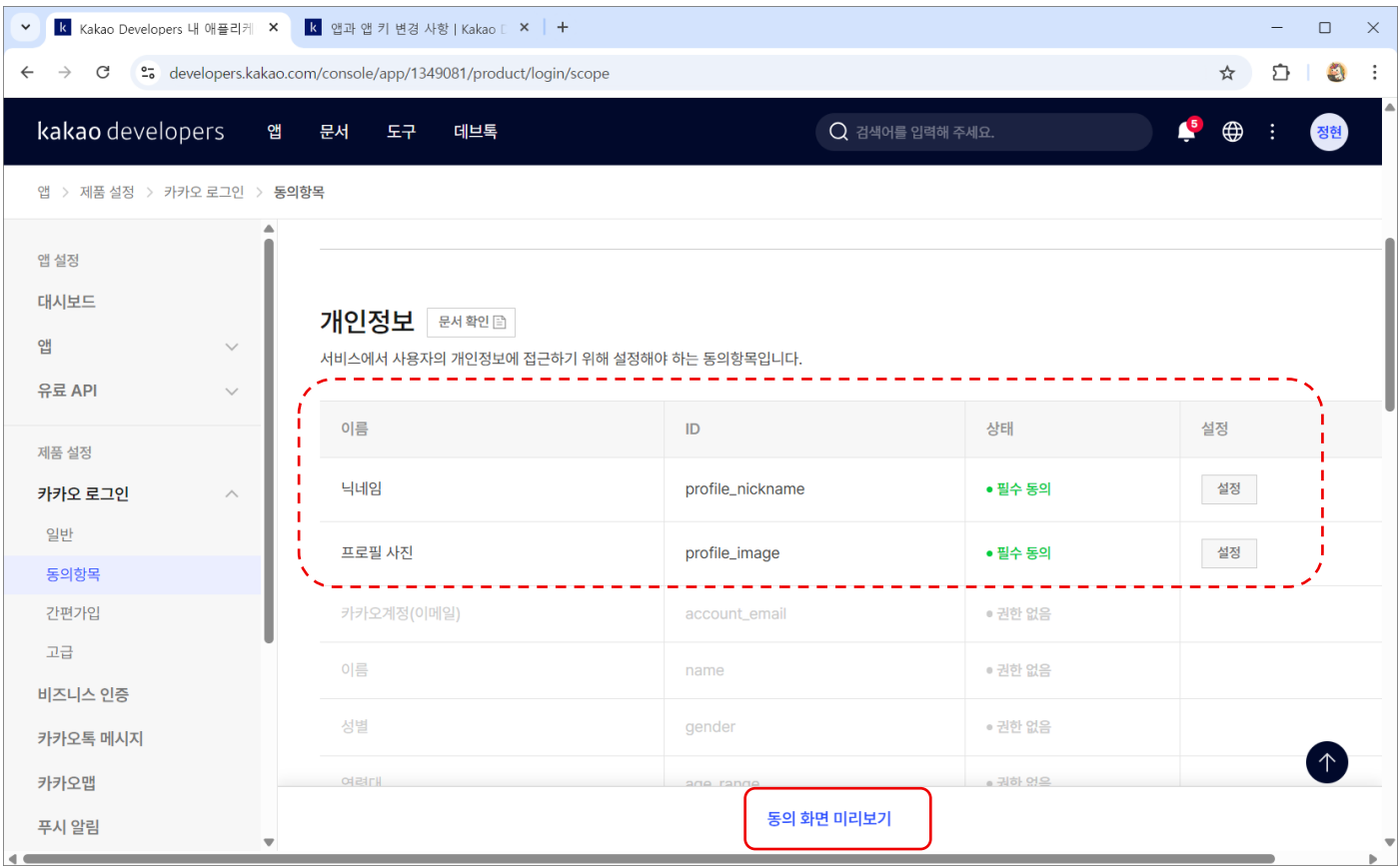
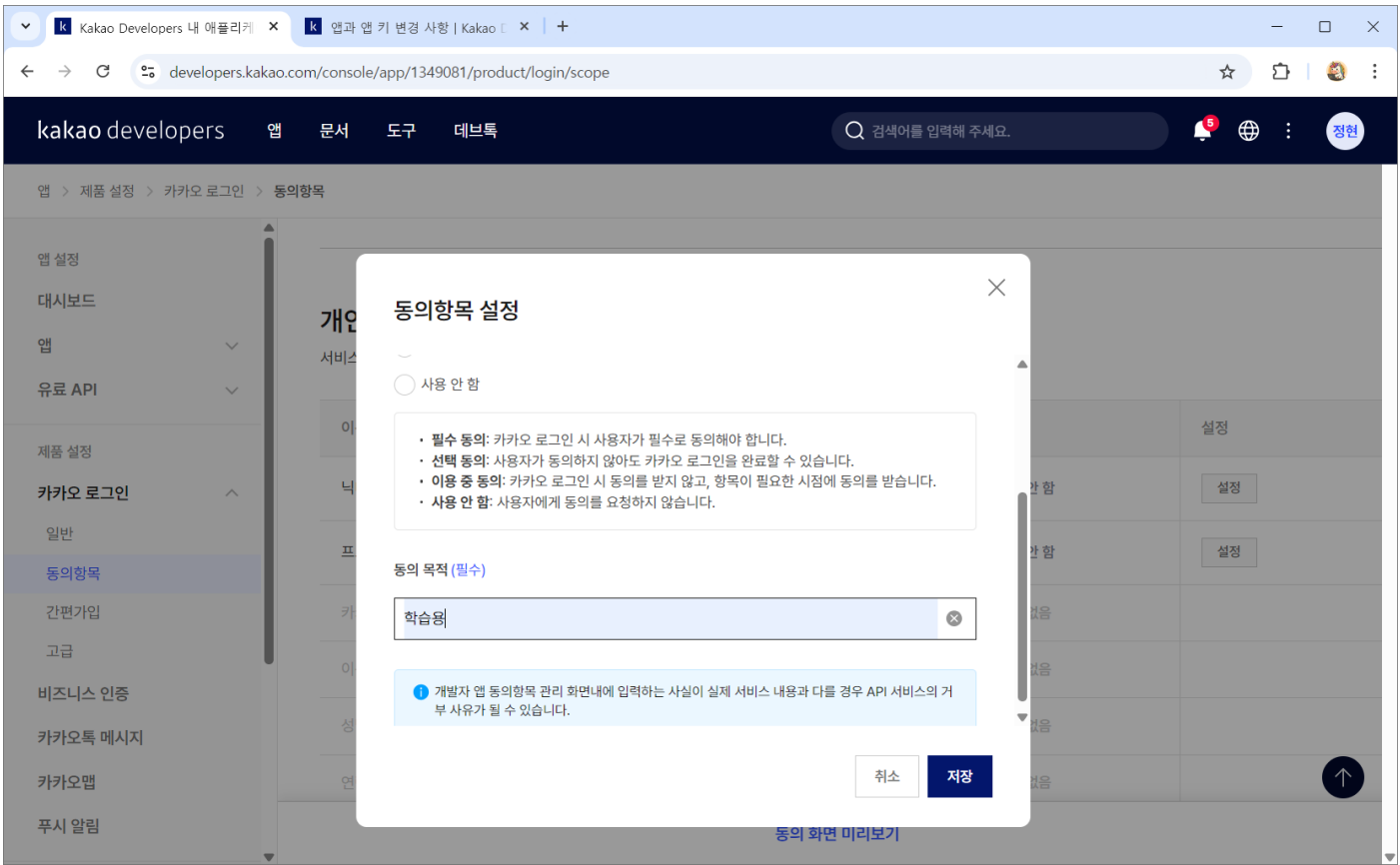
문서 확인

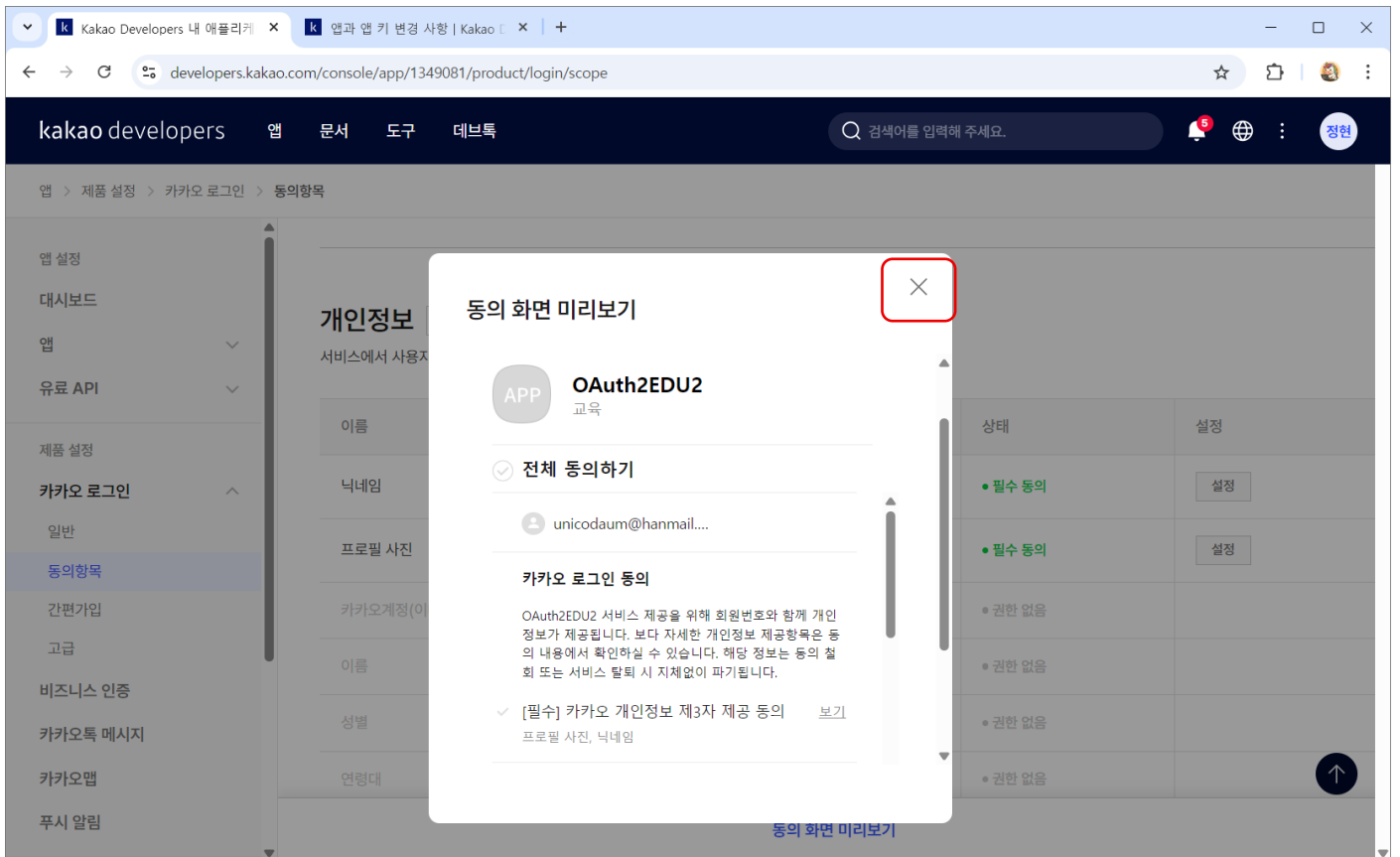
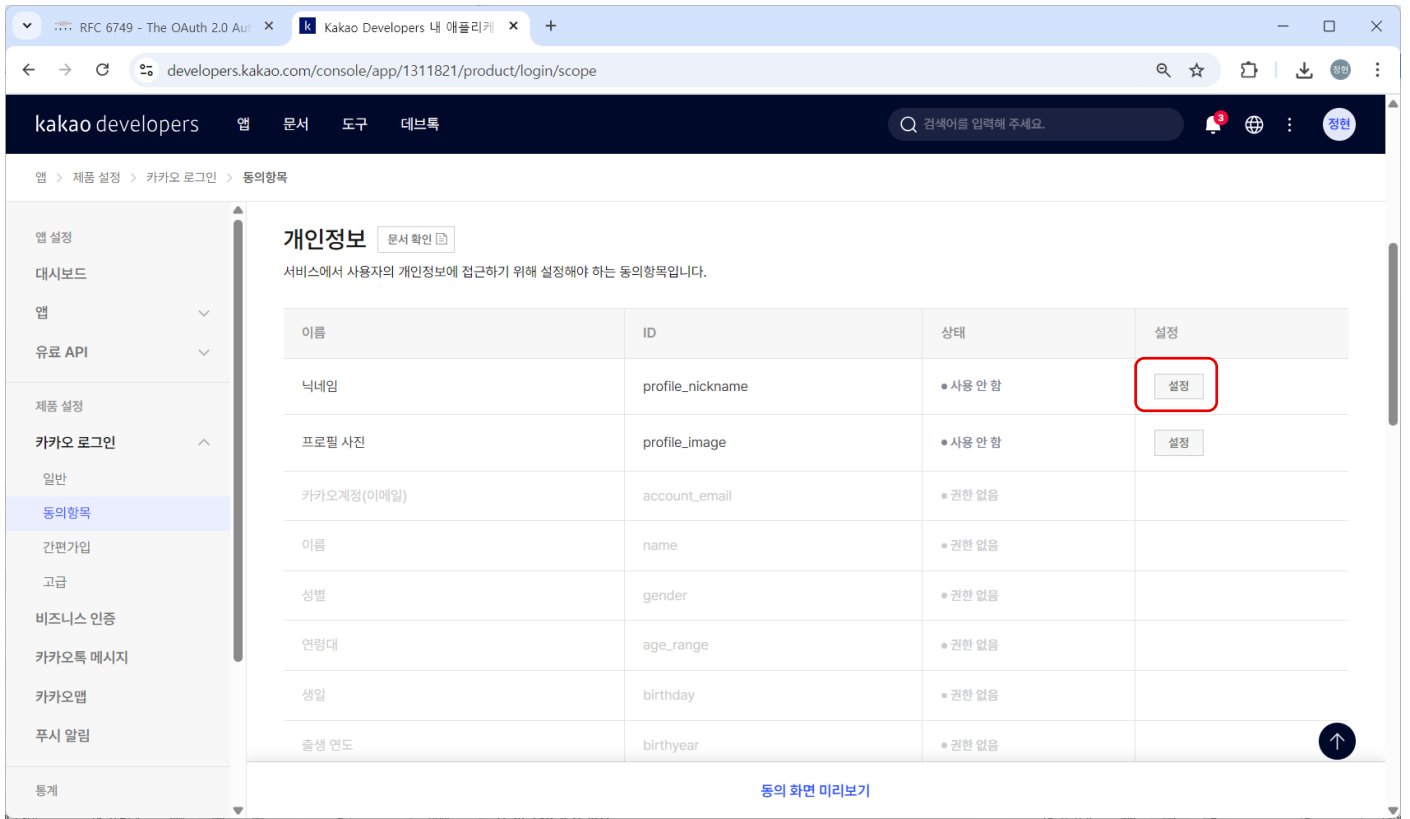
동의항목 설정 권한을 제공받기 위해 추가 기능을 신청합니다.

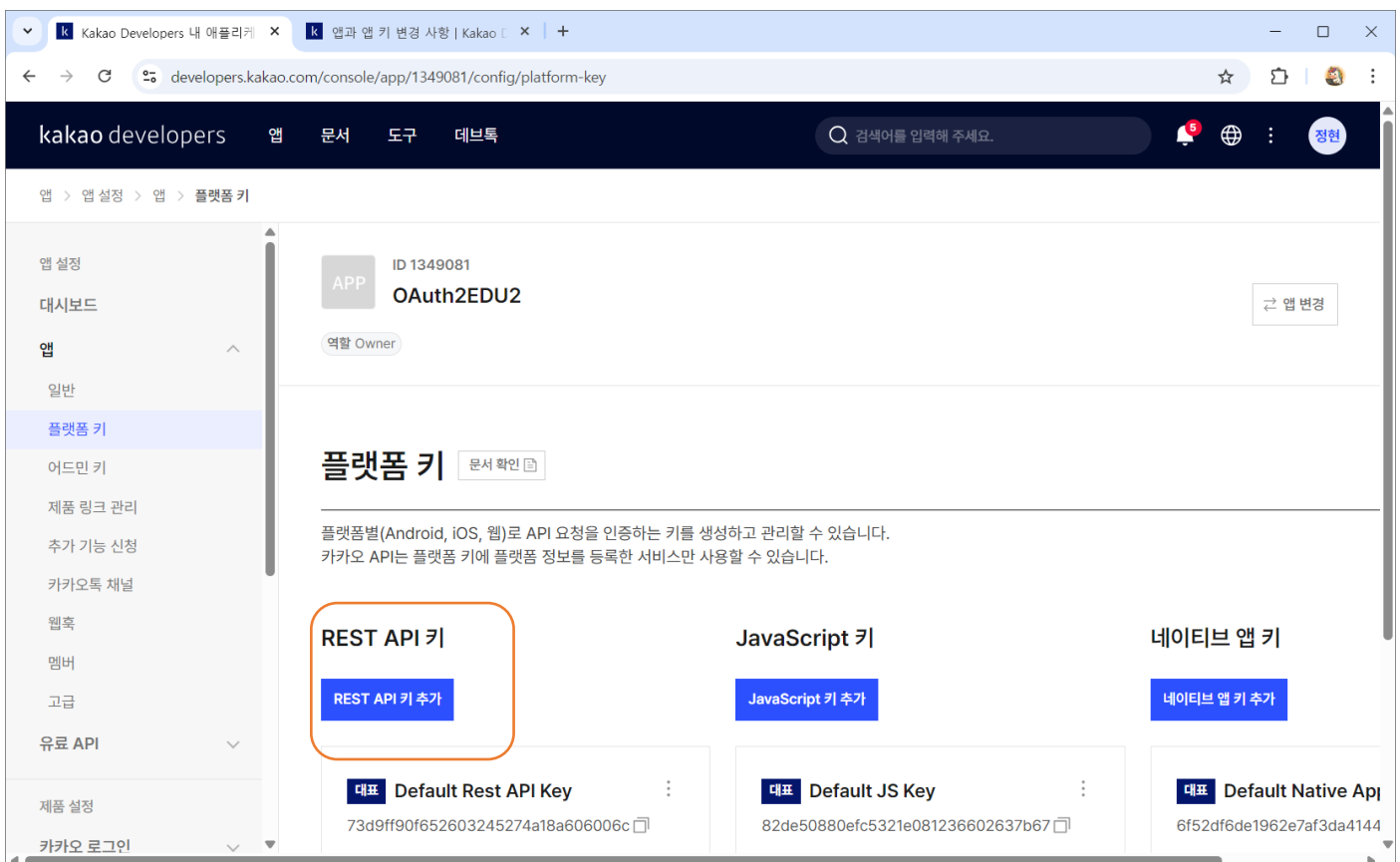
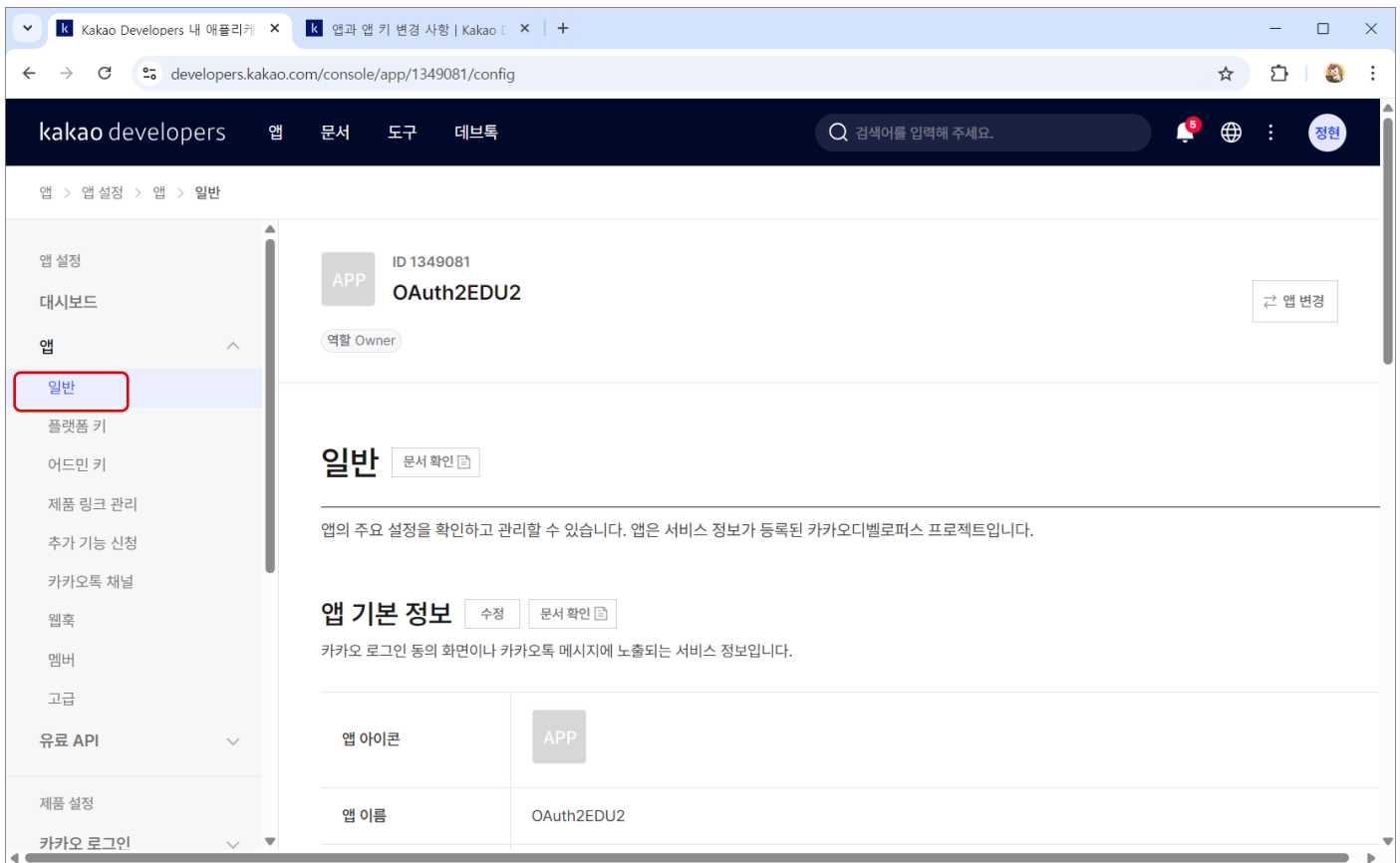
추가 기능 신청

동의 화면 미리보기









http://localhost:8080/login/oauth2/code/kakao

Kakao Developers 내 애플리케이션

앱과 앱 키 변경 사항 | Kakao

developers.kakao.com/console/app/1349081/config/platform-key/rest/create

kakao developers

앱 문서 도구 데브톡

검색어를 입력해 주세요.

5

정현

앱 > 앱 설정 > 앱 > 플랫폼 키 > 플랫폼 키 추가

앱 설정
대시보드
앱
일반
플랫폼 키
어드민 키
제품 링크 관리
추가 기능 신청
카카오톡 채널
웹훅
멤버
고급
유료 API
제품 설정
카카오 로그인
비즈니스 인증
카카오톡 메시지
카카오맵
푸시 알림

REST API 키 추가

1 REST API 키

저장 후 신규 키가 발급되며, 아래에 표시됩니다.

키 이름 (필수)

REST API 키를 식별하기 위한 이름을 지정합니다.

edu

호출 허용 IP 주소

호출 허용 IP 주소를 설정하면, 설정된 IP 외 다른 IP 주소에서는 해당 API 키로 API를 호출할 수 없습니다.
등록은 IP(IPv4), CIDR(IP/CIDR), Subnet Mask(IP/Mask) 방식으로 가능합니다.

(예: 123.1.1.1)

+

카카오 로그인 리다이렉트 URI

카카오 로그인을 사용할 때 필요한 OAuth 리다이렉트 URI를 등록합니다.

http://localhost:8080/login/oauth2/code/kakao

+

비즈니스 인증 리다이렉트 URI

비즈니스 인증을 사용할 때 필요한 OAuth 리다이렉트 URI를 등록합니다.

↑

Kakao Developers 내 애플리케이션

앱과 앱 키 변경 사항 | Kakao

developers.kakao.com/console/app/1349081/config/platform-key/rest/create

kakao developers

앱 문서 도구 데브톡

검색어를 입력해 주세요.

5

정현

앱 > 앱 설정 > 앱 > 플랫폼 키 > 플랫폼 키 추가

앱 설정
대시보드
앱
일반
플랫폼 키
어드민 키
제품 링크 관리
추가 기능 신청
카카오톡 채널
웹훅
멤버
고급
유료 API
제품 설정
카카오 로그인

비즈니스 인증 리다이렉트 URI

비즈니스 인증을 사용할 때 필요한 OAuth 리다이렉트 URI를 등록합니다.

(예: https://example.com/oauth)

+

클라이언트 시크릿

클라이언트 시크릿은 카카오에서 발급한 아래 특정 코드를 포함한 요청만, 카카오 로그인과 비즈니스 인증의 토큰을 발급받을 수 있도록 설정하는 보안 기능입니다.
클라이언트 시크릿의 사용 여부를 설정하거나 코드를 확인, 복사, 재발급받을 수 있습니다.

1 키 발급 시 클라이언트 시크릿 기본 활성화

서비스의 보안 강화를 위해 클라이언트 시크릿 기능은 REST API 키 발급 시 기본 활성화 상태로 설정되며, 사용을 원하지 않는 경우 직접 비활성화 상태로 변경해야 합니다.
클라이언트 시크릿 코드 값은 [REST API 키 수정] 페이지에서 확인할 수 있습니다.

취소

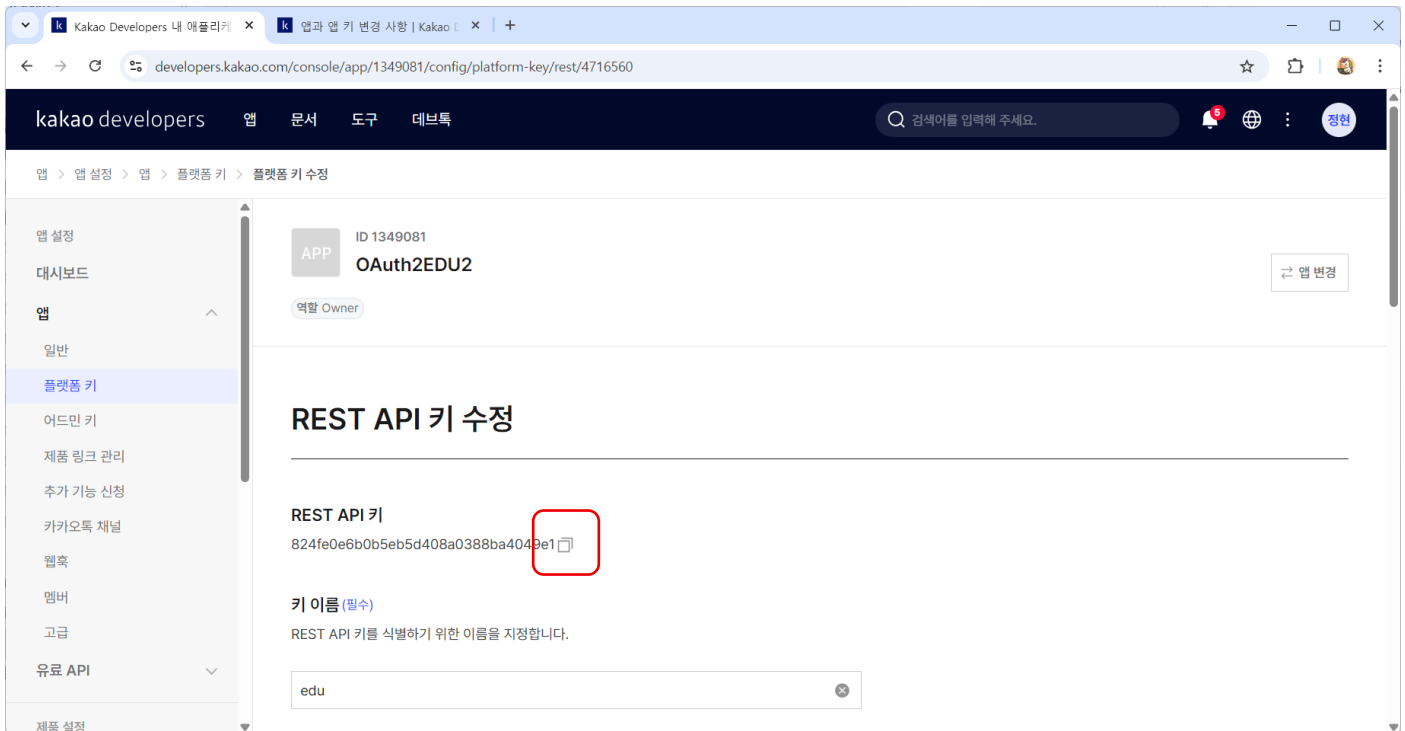
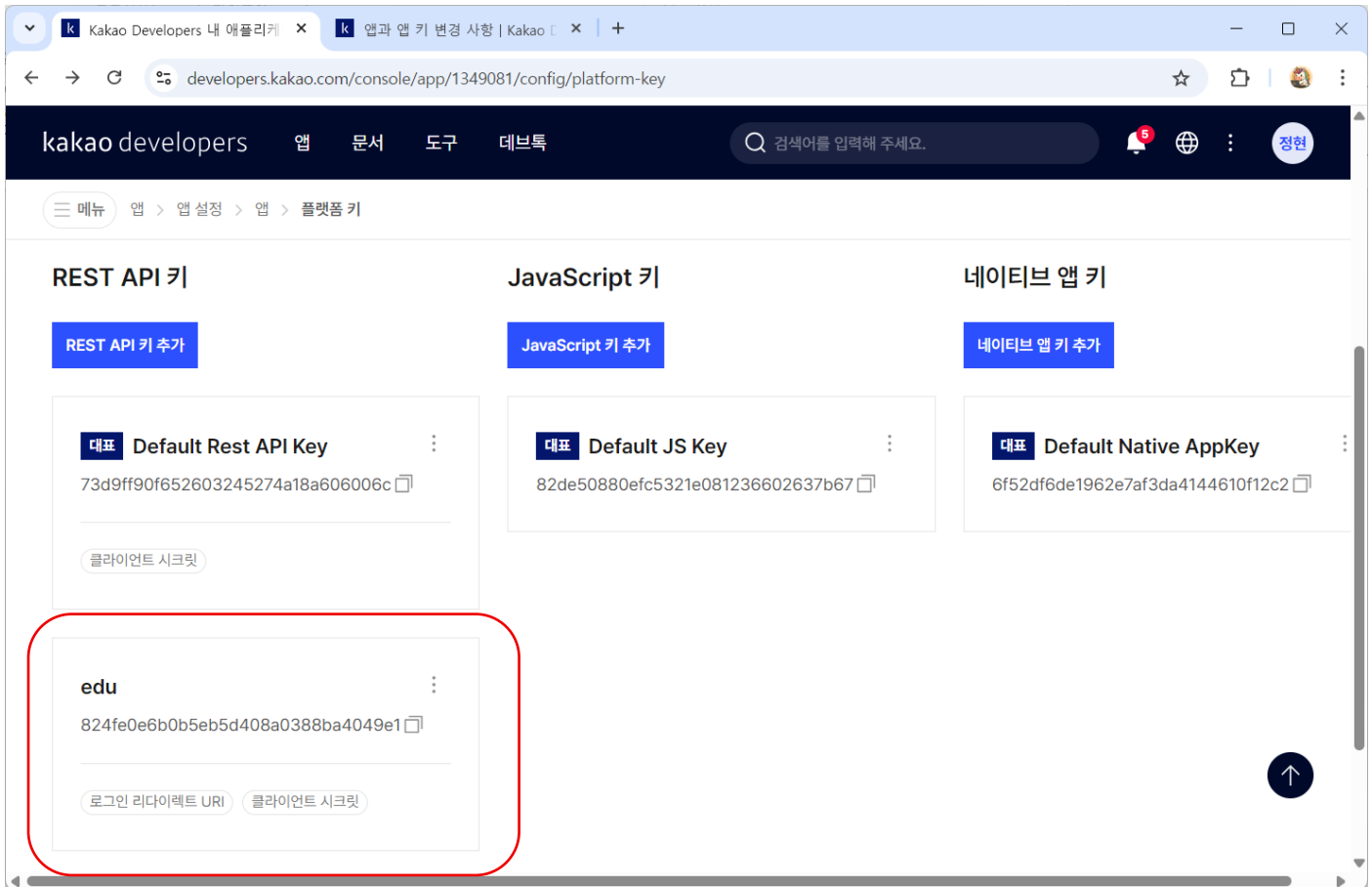
저장

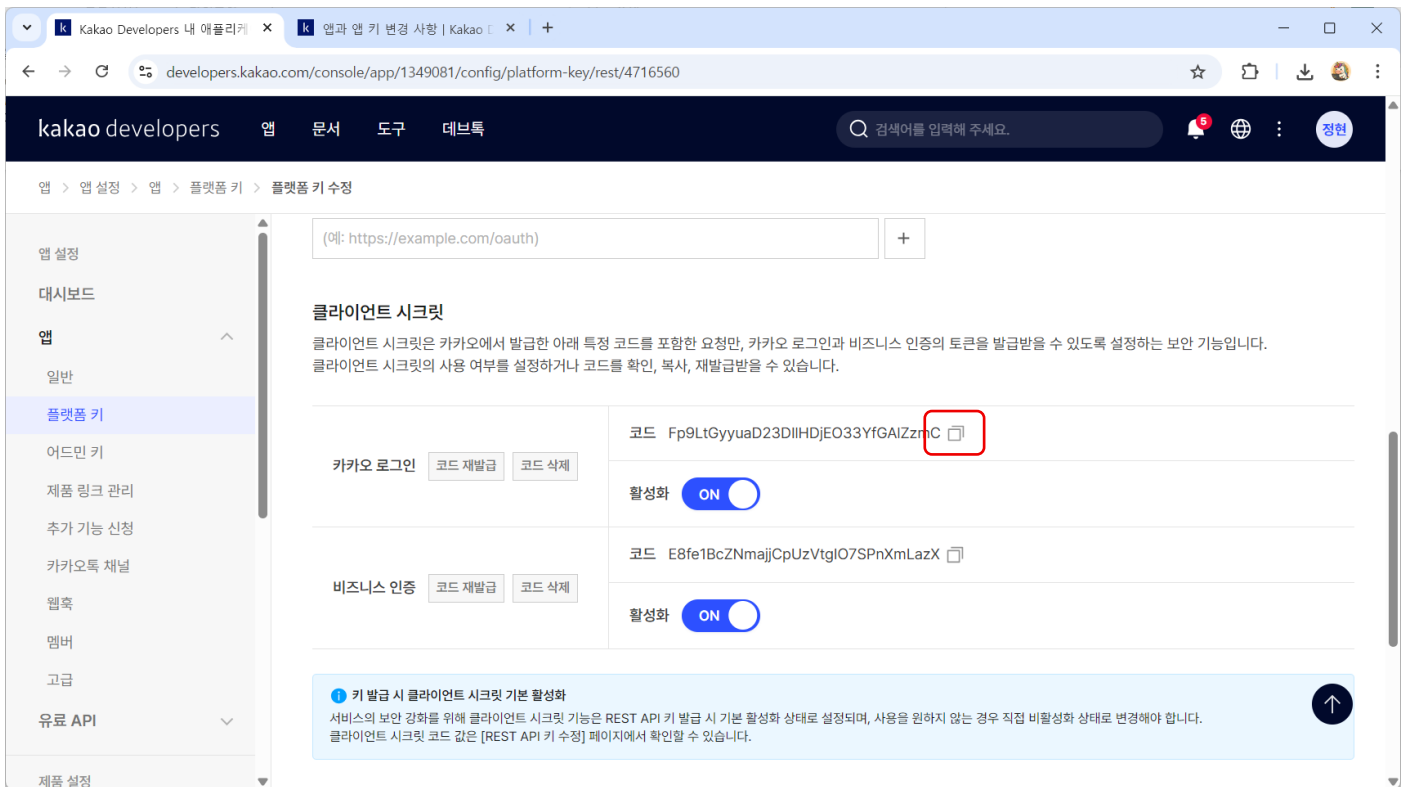
↑

카카오비즈니스 이용약관 | 서비스 약관 | 운영 정책 | 쿠키 | 개인정보 처리방침

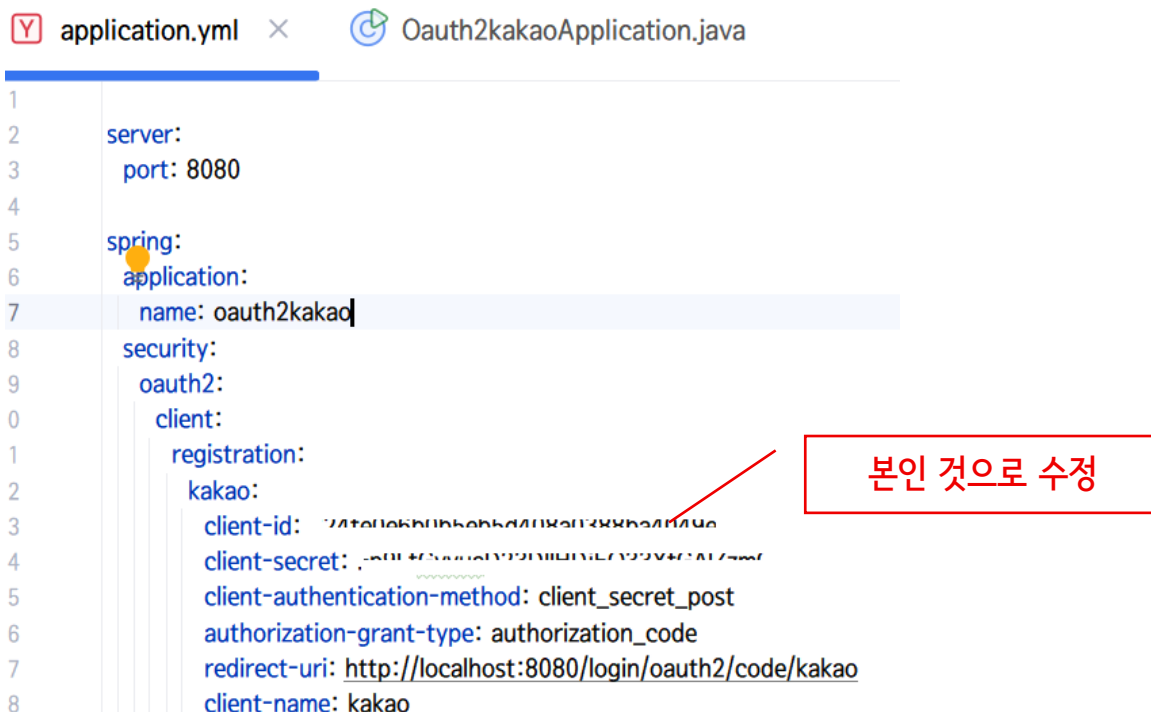
Family Site

+

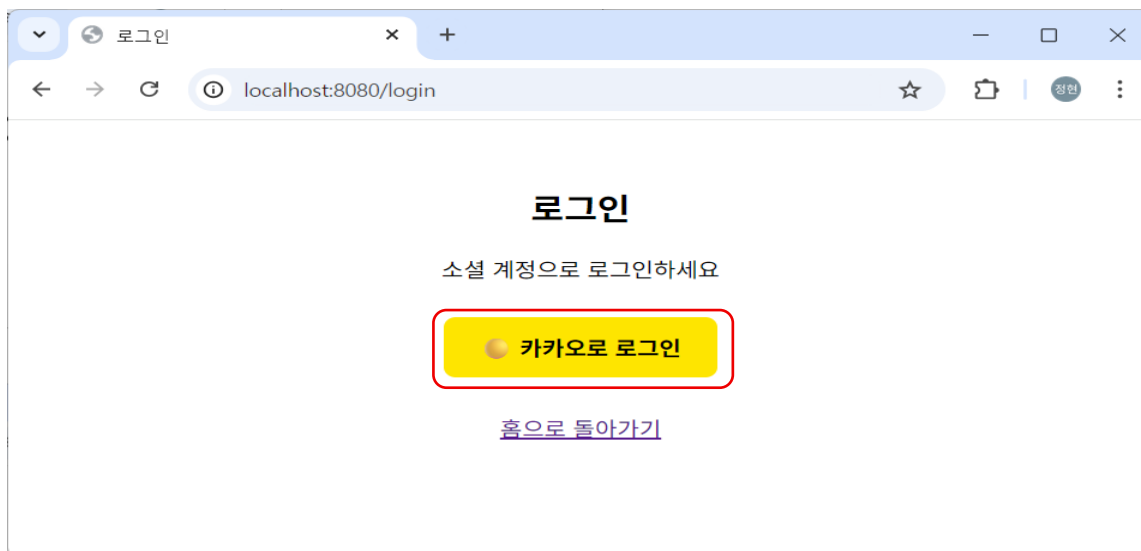
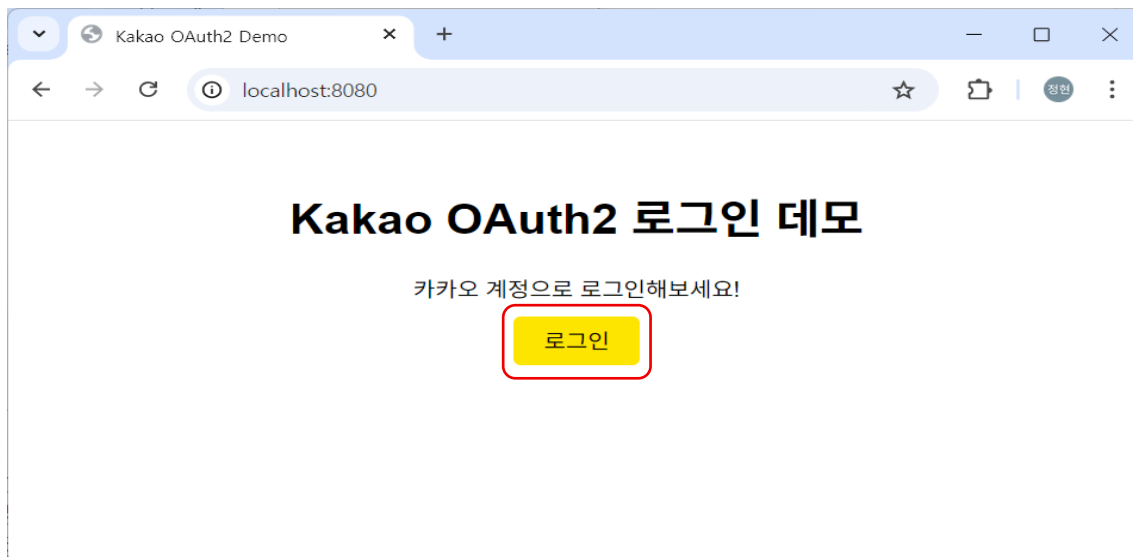


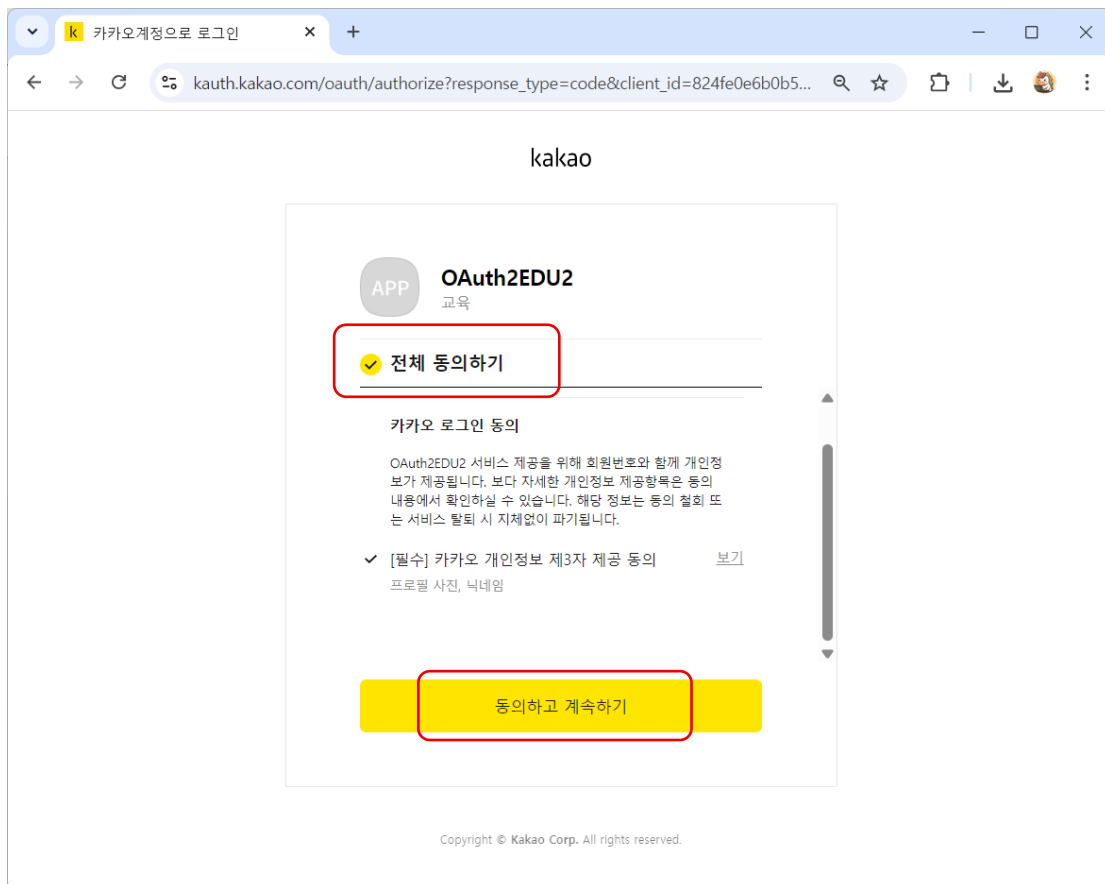
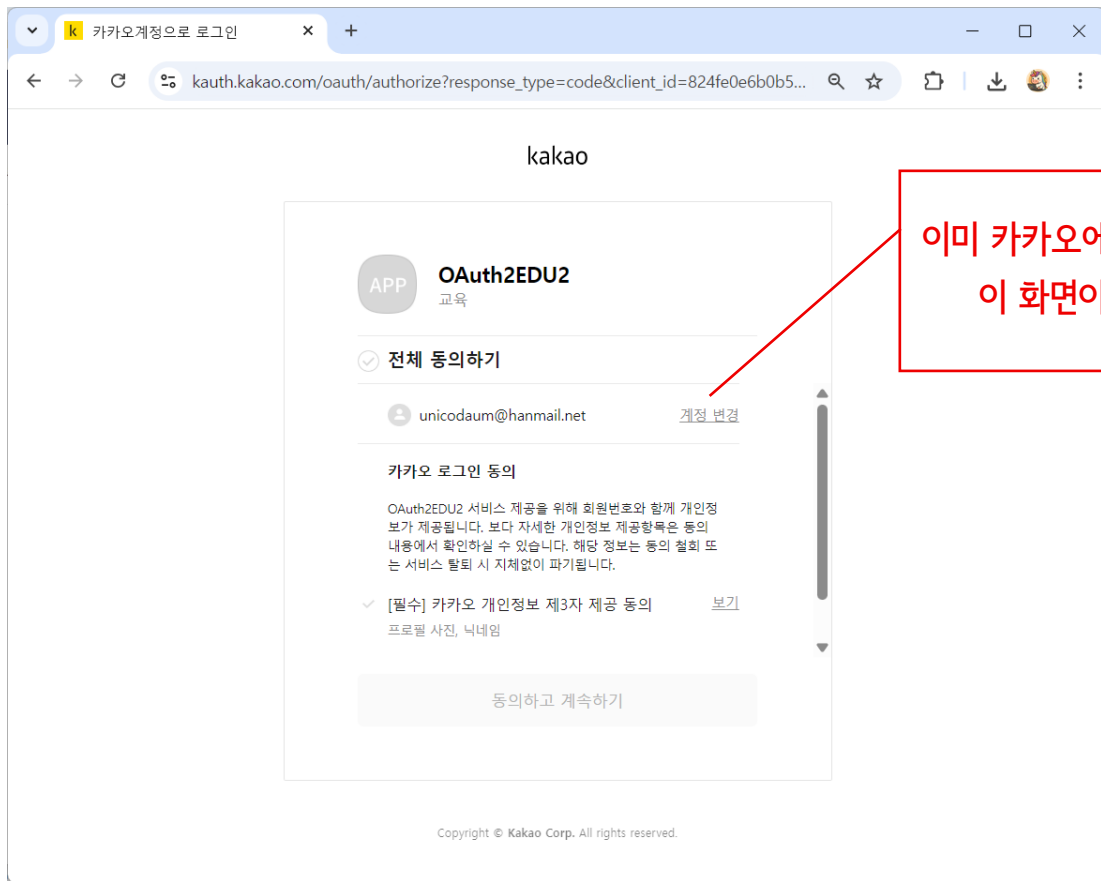


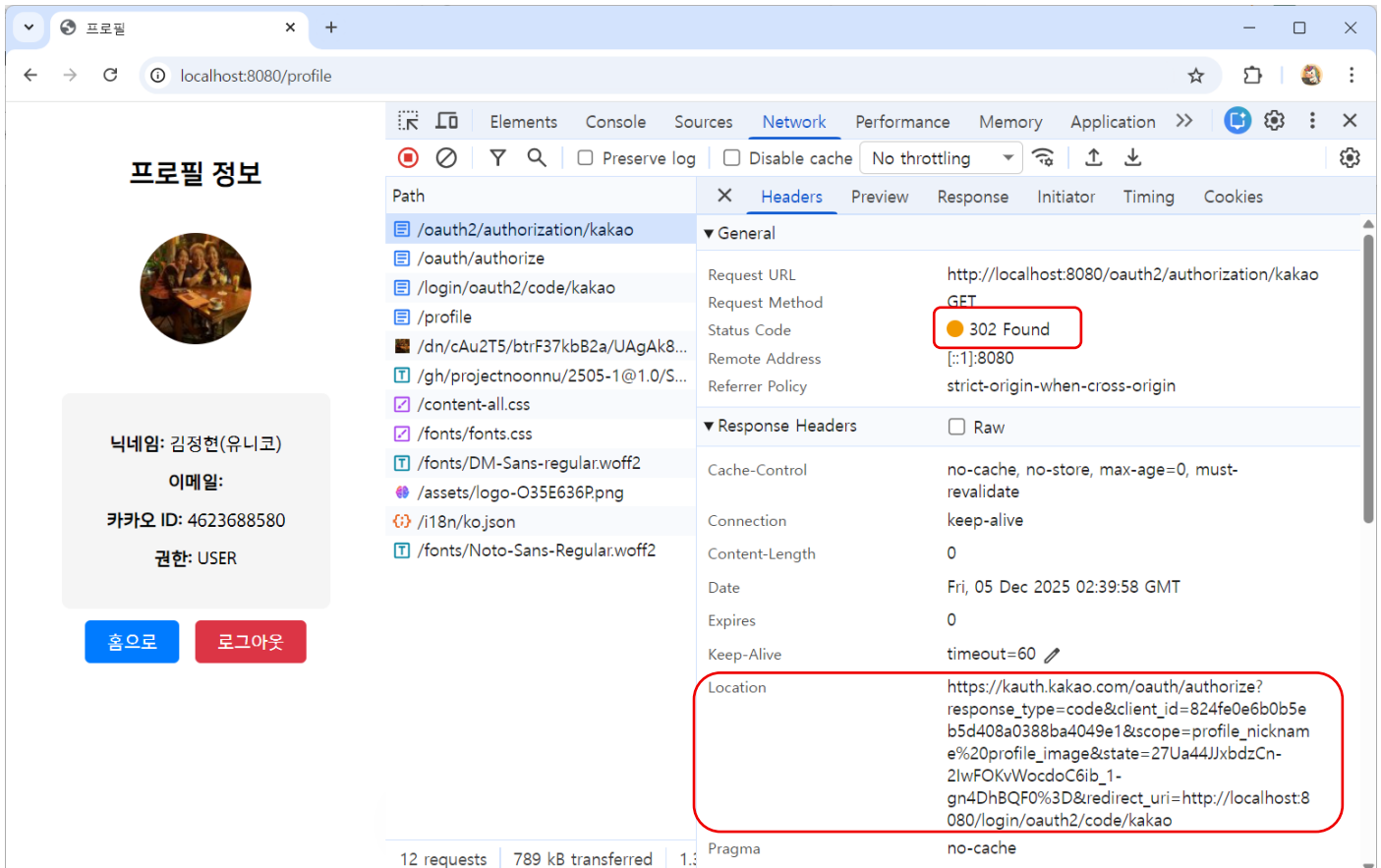
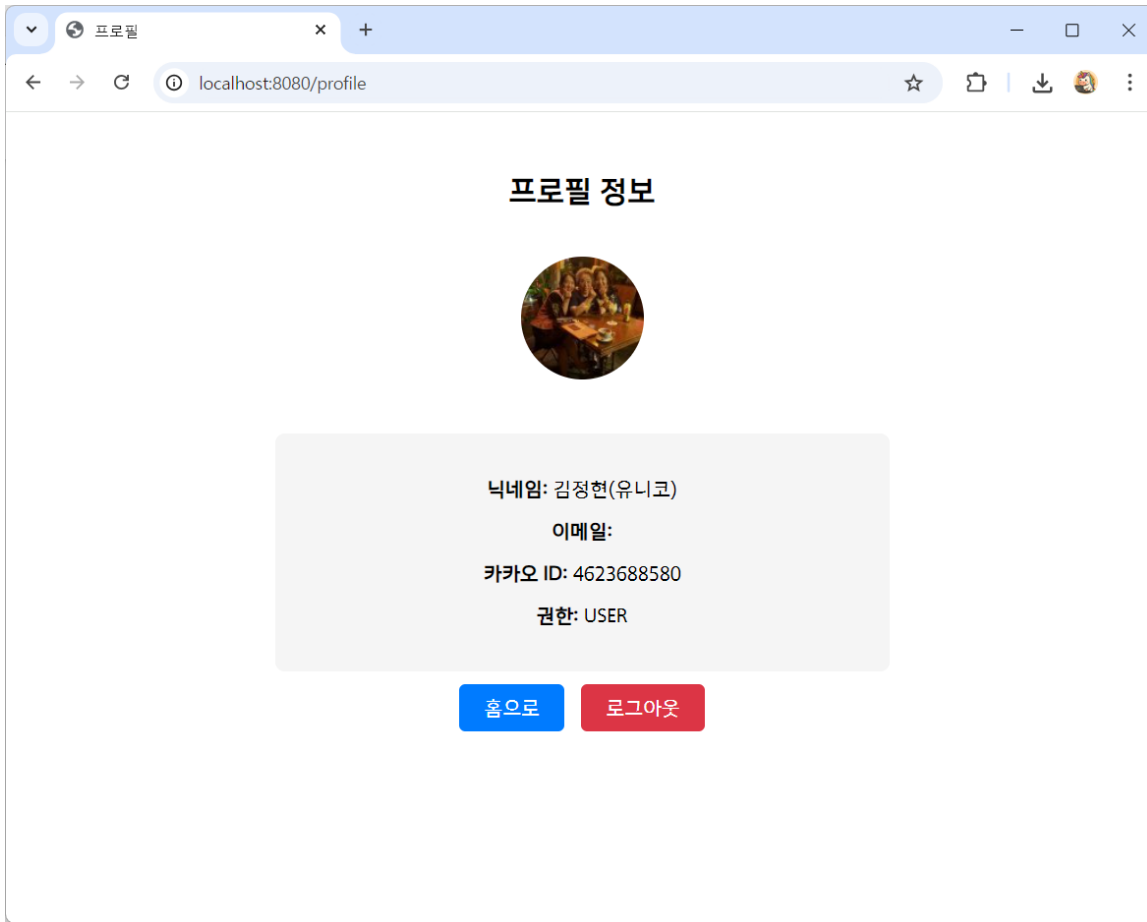
제공된 소스 중에서 oauth2kakao 를 인텔리제이로 오픈한 다음 내용을 본인의 client-id 와 client-secret 으로 수정한다.



oauth2kakao 를 기동시킨 후 다음과 같이 <http://localhost:8080/> 을 요청해 본다.







프로필 정보



닉네임: 김정현(유니코)

이메일:

카카오 ID: 4623688580

권한: USER

홈으로

로그아웃

Network

Path

/oauth2/authorization/kakao

/oauth/authorize

/login/oauth2/code/kakao

/profile

/dn/cAu2T5/btrF37kbB2a/UAgAk8...

/gh/projectnoonnu/2505-1@1.0/S...

/content-all.css

/fonts/fonts.css

/fonts/DM-Sans-regular.woff2

/assets/logo-O35E636P.png

/i18n/ko.json

/fonts/Noto-Sans-Regular.woff2

Headers

Status Code

302 Found

Remote Address

121.53.90.6:443

Referrer Policy

strict-origin-when-cross-origin

Response Headers

Access-Control-Allow-Headers

Authorization, KA, Origin, X-Requested-With, Content-Type, Accept

Access-Control-Allow-Methods

GET, POST, OPTIONS

Access-Control-Allow-Origin

*

Cache-Control

no-cache, no-store, max-age=0, must-revalidate

Connection

keep-alive

Content-Language

ko-KR

Content-Length

0

Date

Fri, 05 Dec 2025 02:39:58 GMT

Expires

0

Kakao

Talk

Location

http://localhost:8080/login/oauth2/code/kakao?code=4qGZdTtFjDolAb139raYrM0gkmV_2j7iv9opNZtyb5Jd_ba7bw1glgAAAAQKDCbAAABmuxhalAe0jm_MNo9Pw&state=27Ua44JjxbdzCn-2lwFOkVWocdoC6ib_1-gn4DhBQF0%3D

Pragma

no-cache

12 requests | 789 kB transferred | 1...

프로필 정보



닉네임: 김정현(유니코)

이메일:

카카오 ID: 4623688580

권한: USER

홈으로

로그아웃

Network

Path

/oauth2/authorization/kakao

/oauth/authorize

/login/oauth2/code/kakao

/profile

/dn/cAu2T5/btrF37kbB2a/UAgAk8...

/gh/projectnoonnu/2505-1@1.0/S...

/content-all.css

/fonts/fonts.css

/fonts/DM-Sans-regular.woff2

/assets/logo-O35E636P.png

/i18n/ko.json

/fonts/Noto-Sans-Regular.woff2

General

Request URL

http://localhost:8080/login/oauth2/code/kakao?code=4qGZdTtFjDolAb139raYrM0gkmV_2j7iv9opNZtyb5Jd_ba7bw1glgAAAAQKDCbAAABmuxhalAe0jm_MNo9Pw&state=27Ua44JjxbdzCn-2lwFOkVWocdoC6ib_1-gn4DhBQF0%3D

Request Method

GET

Status Code

302 Found

Remote Address

[::1]:8080

Referrer Policy

strict-origin-when-cross-origin

Response Headers

Cache-Control

no-cache, no-store, max-age=0, must-revalidate

Connection

keep-alive

Content-Length

0

Date

Fri, 05 Dec 2025 02:39:59 GMT

Expires

0

Keep-Alive

timeout=60

Location

http://localhost:8080/profile

Pragma

no-cache

Set-Cookie

JSESSIONID=8A737224FEFF95C1BE706FBD67C14CE; Path=/; HttpOnly

12 requests | 789 kB transferred | 1...

프로필

localhost:8080/profile

프로필 정보



닉네임: 김정현(유니코)

이메일:

카카오 ID: 4623688580

권한: USER

홈으로

로그아웃

Network

Path

/oauth2/authorization/kakao

/oauth/authorize

/login/oauth2/code/kakao

/profile

/dn/cAu2T5/btrF37kbB2a/UAgAk8...

/gh/projectnoonnu/2505-1@1.0/S...

/content-all.css

/fonts/fonts.css

/fonts/DM-Sans-regular.woff2

/assets/logo-Q35E636P.png

/i18n/ko.json

/fonts/Noto-Sans-Regular.woff2

General

Request URL

Request Method

Status Code

Remote Address

Referrer Policy

http://localhost:8080/profile

GET

200 OK

[::1]:8080

strict-origin-when-cross-origin

Response Headers

Cache-Control

Connection

Content-Language

Content-Type

Date

Expires

Keep-Alive

Pragma

Transfer-Encoding

X-Content-Type-Options

X-Frame-Options

X-Xss-Protection

no-cache, no-store, max-age=0, must-revalidate

keep-alive

ko-KR

text/html; charset=UTF-8

Fri, 05 Dec 2025 02:39:59 GMT

0

timeout=60

no-cache

chunked

nosniff

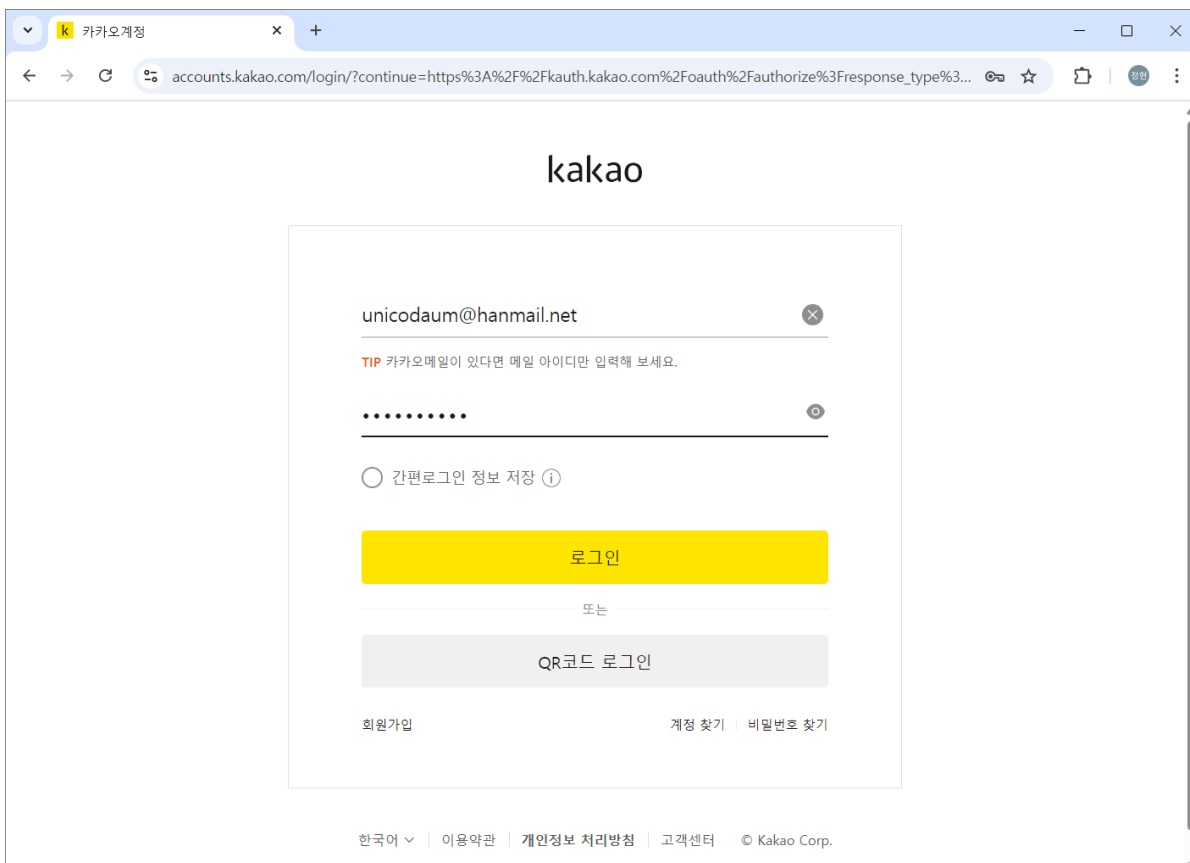
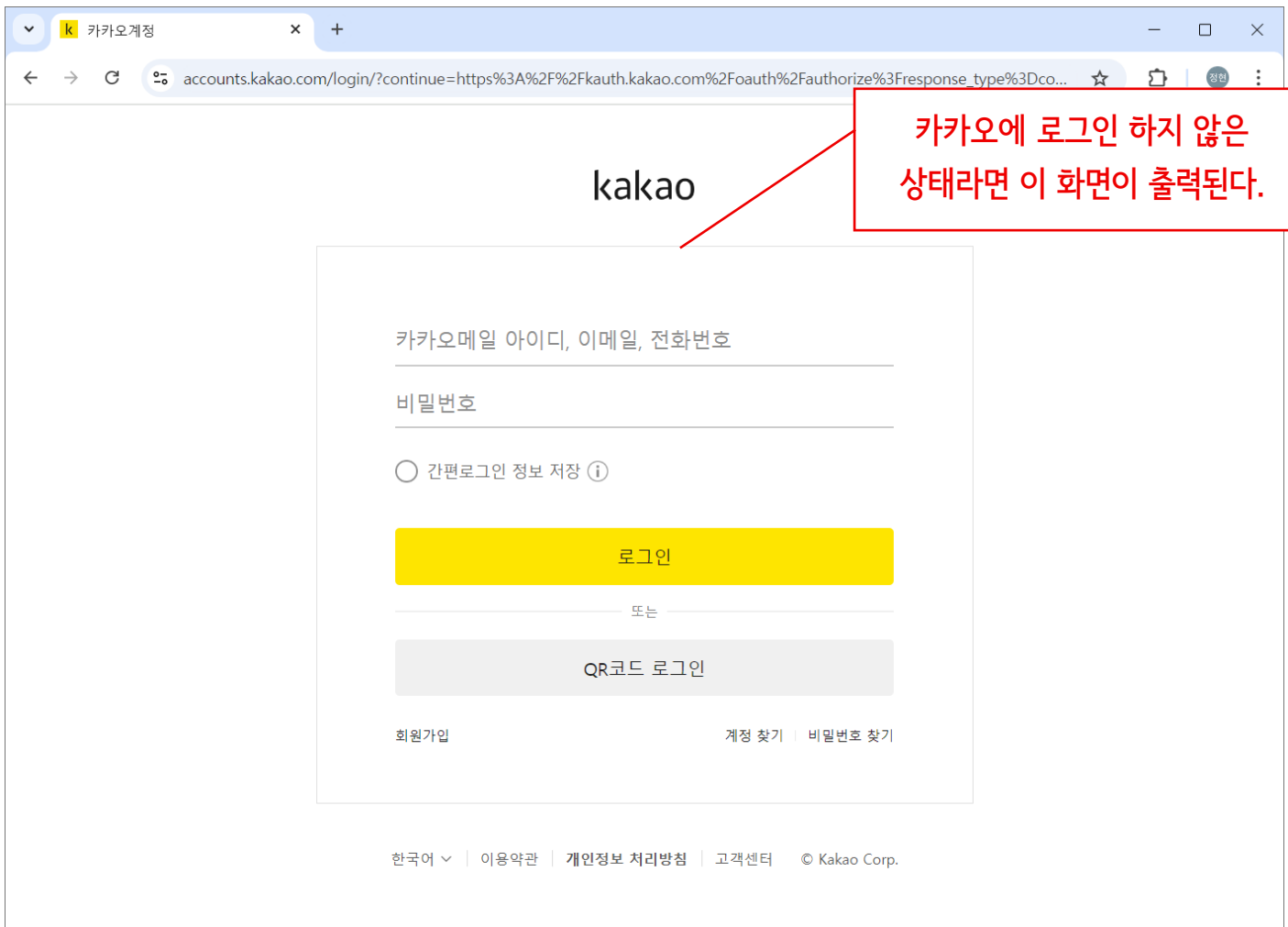
SAMEORIGIN

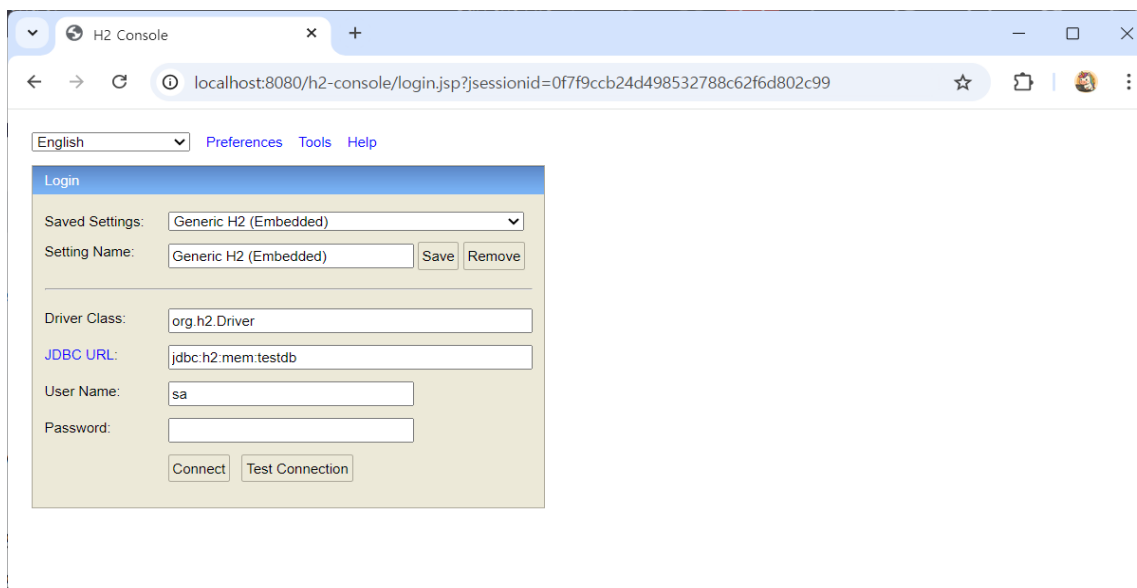
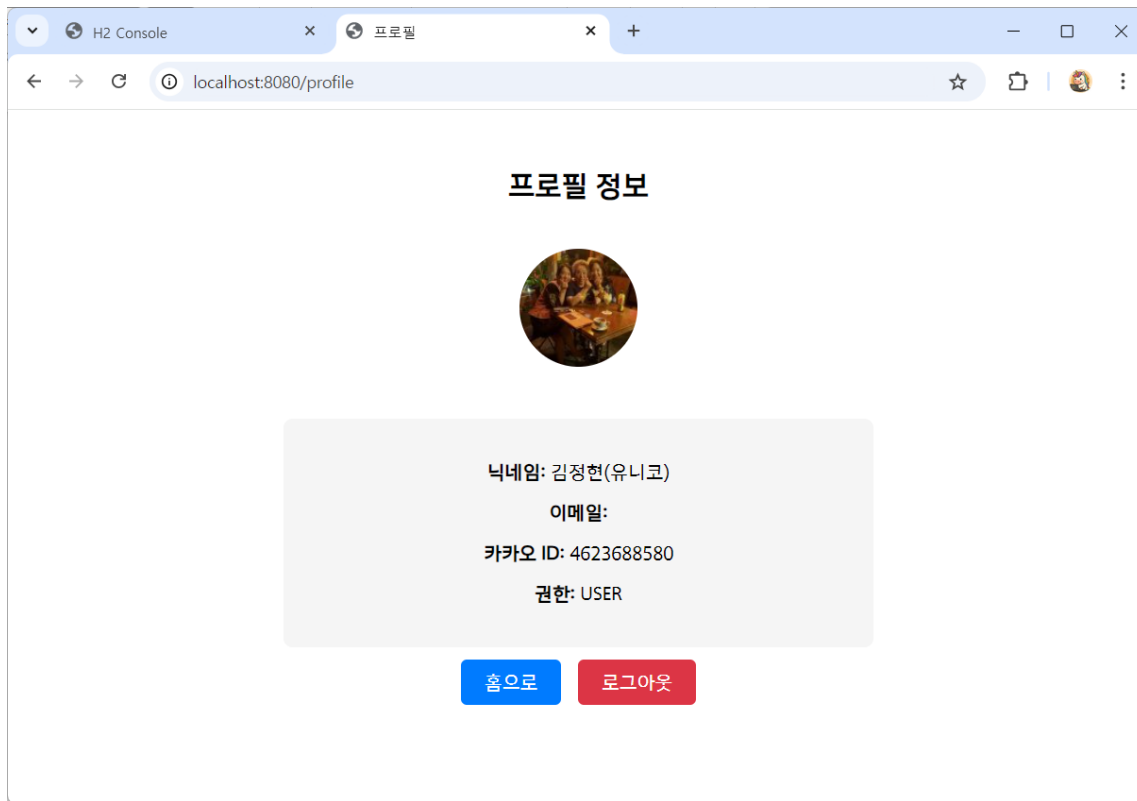
0

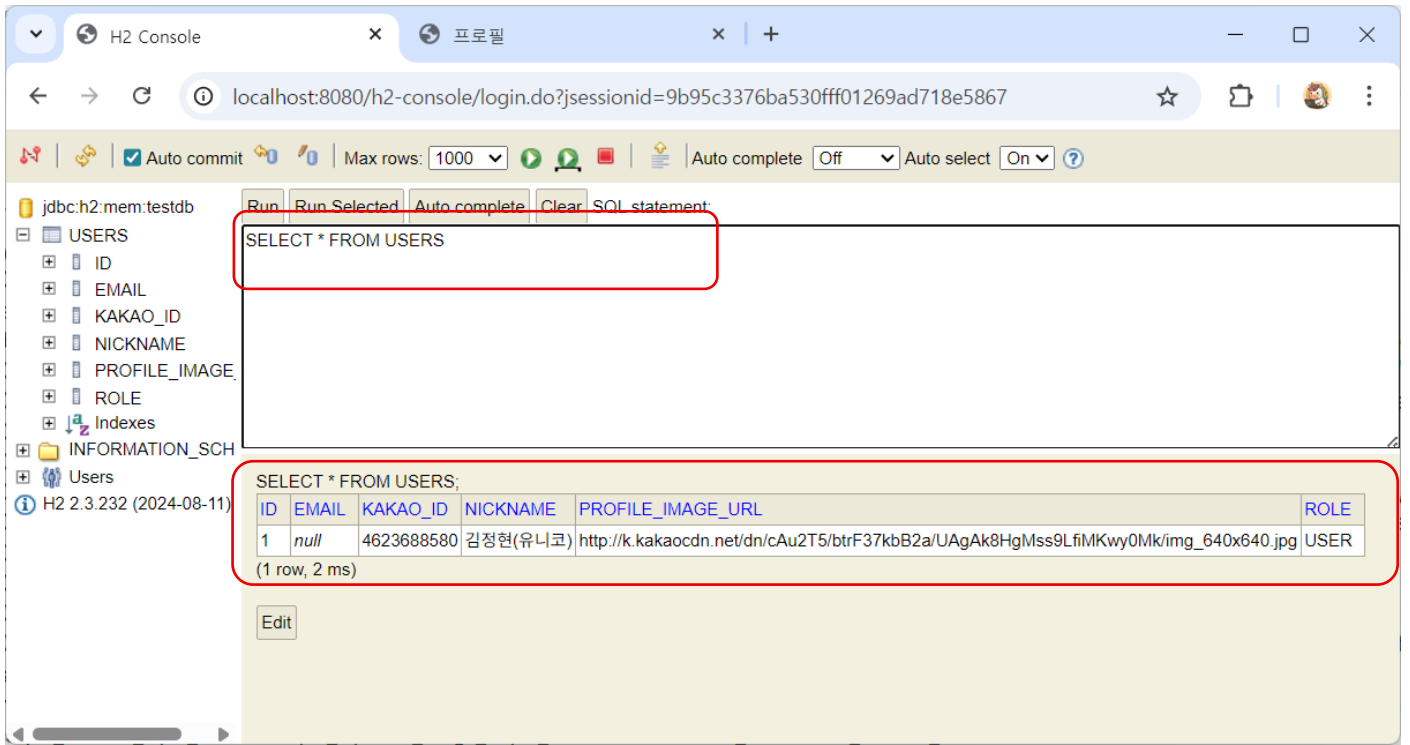
12 requests

789 kB transferred

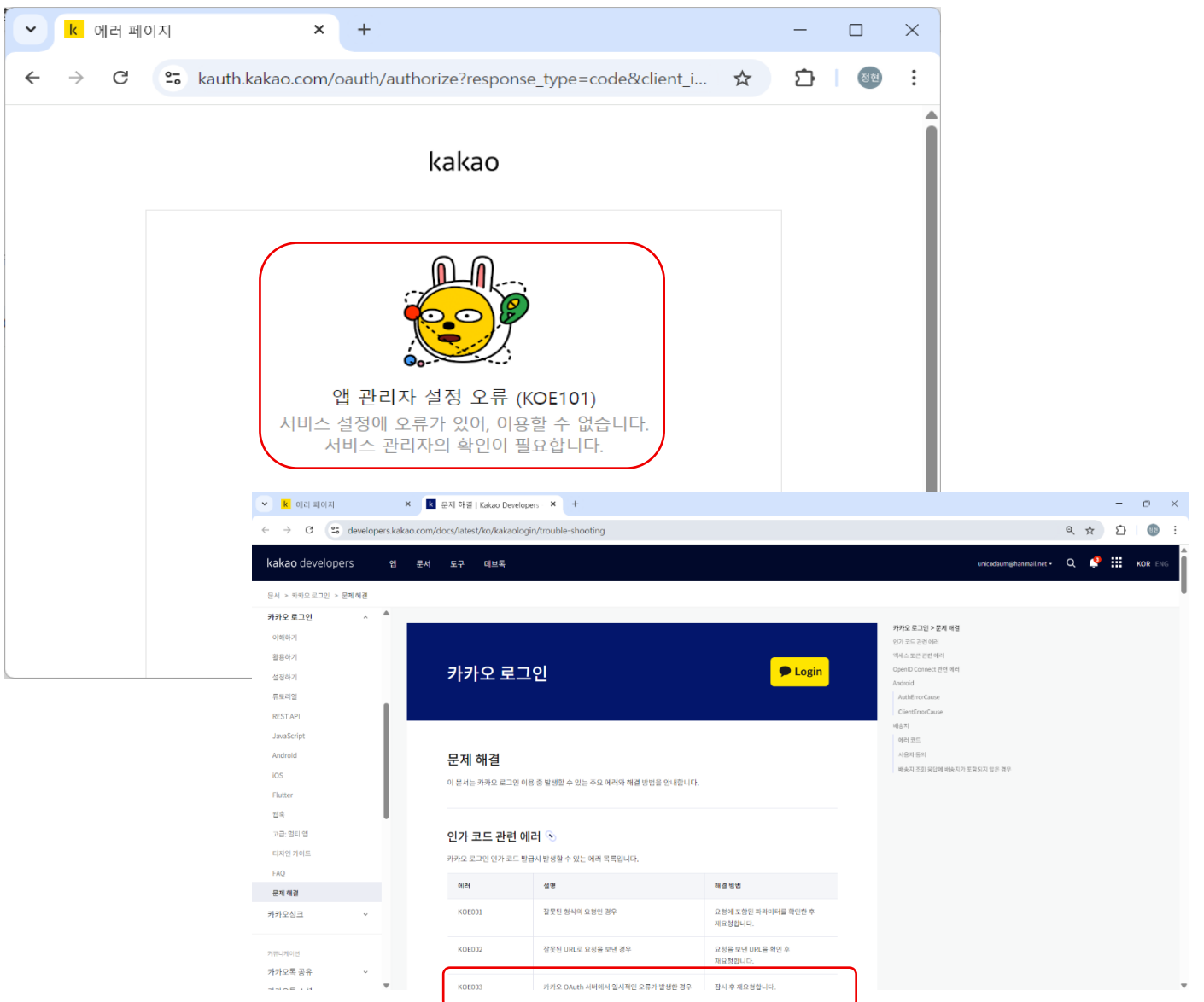
1.3







다음은 오류가 발생한 하나의 예이다.





설로인

설로인 주식회사

✓ 전체 동의하기

전체동의를 카카오 및 설로인 주식회사의 서비스 동의를 포함하고 있습니다. 전체동의를 선택목적에 대한 동의를 포함하고 있으며, 선택목적에 대한 동의를 거부해도 서비스 이용이 가능합니다.



unicodaum@hanmail.net

[계정 변경](#)

카카오 로그인 동의

설로인 서비스 제공을 위해 회원번호와 함께 개인정보가 제공됩니다. 보다 자세한 개인정보 제공항목은 동의 내용에서 확인하실 수 있습니다. 해당 정보는 동의 철회나 서비스 탈

동의하고 계속하기



설로인

설로인 주식회사

✓ 전체 동의하기

✓ [필수] 카카오 개인정보 제3자 제공 동의 [보기](#)

카카오계정(이메일), CI(연계정보), 카카오톡 채널 추가 상태 및 내역, 이름, 카카오계정(전화번호)

[선택] 카카오 개인정보 제3자 제공 동의 [보기](#)

- ✓ 성별
- ✓ 출생 연도
- ✓ 생일

동의하고 계속하기

설로인 서비스 동의

설로인 서비스 이용을 위해 해당 서비스에서 설정한 동의항목입니다. 보다 자세한 내용은 동의 내용에서 확인하실 수 있습니다.

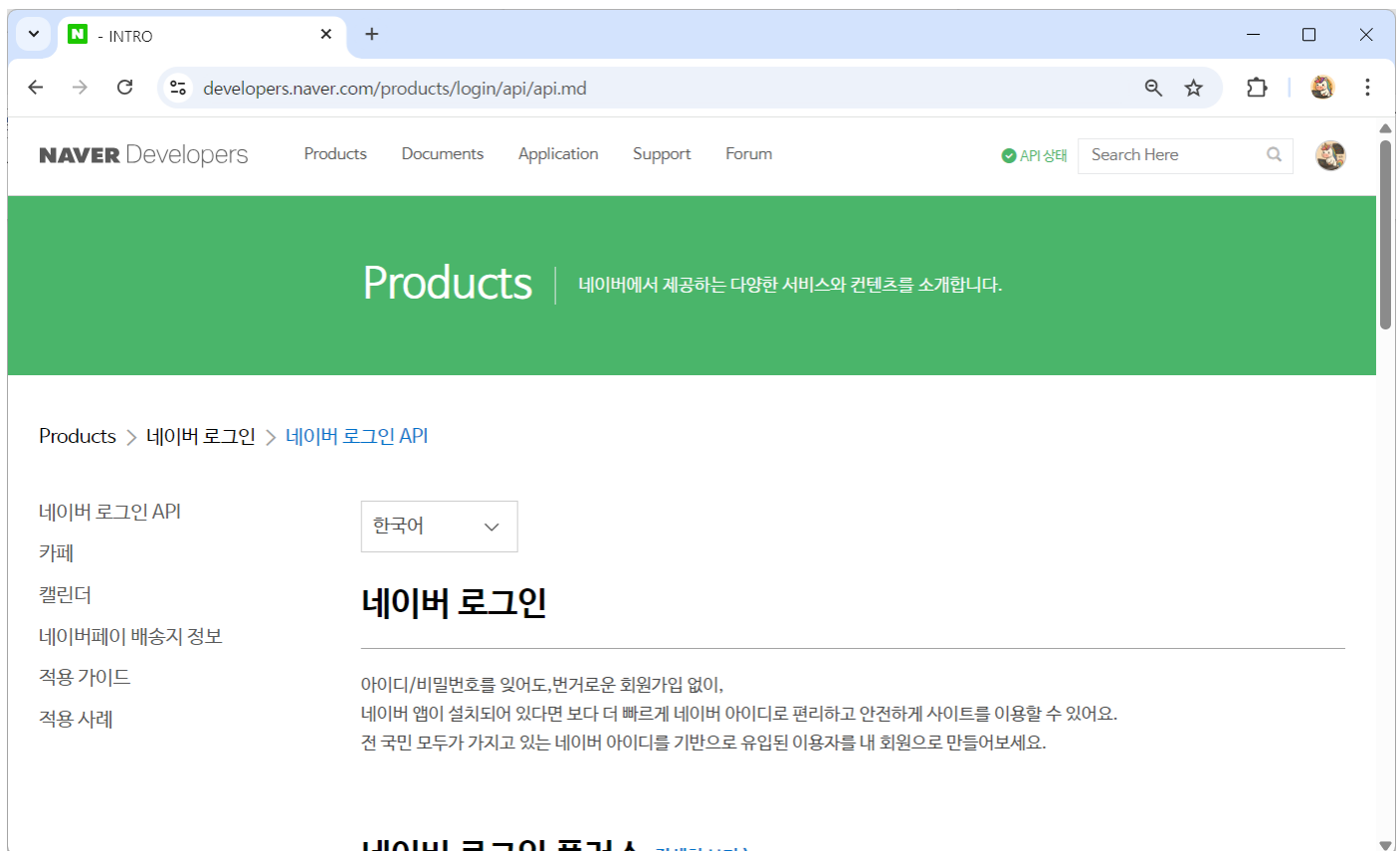
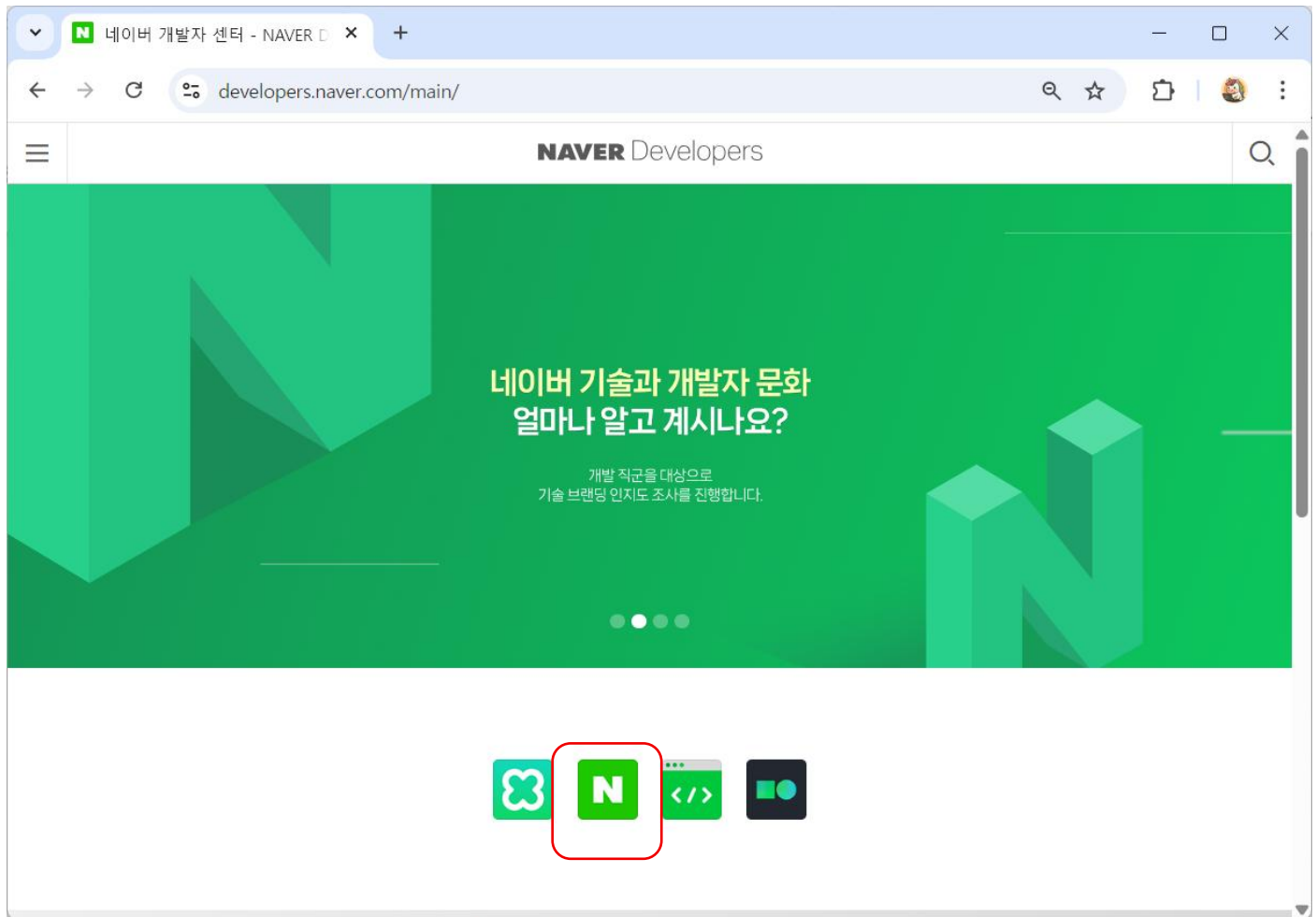
✓ [필수] 이용 약관 [보기](#)✓ [필수] 개인 정보 수집 [보기](#)✓ [선택] 마케팅 수신 동의 [보기](#)

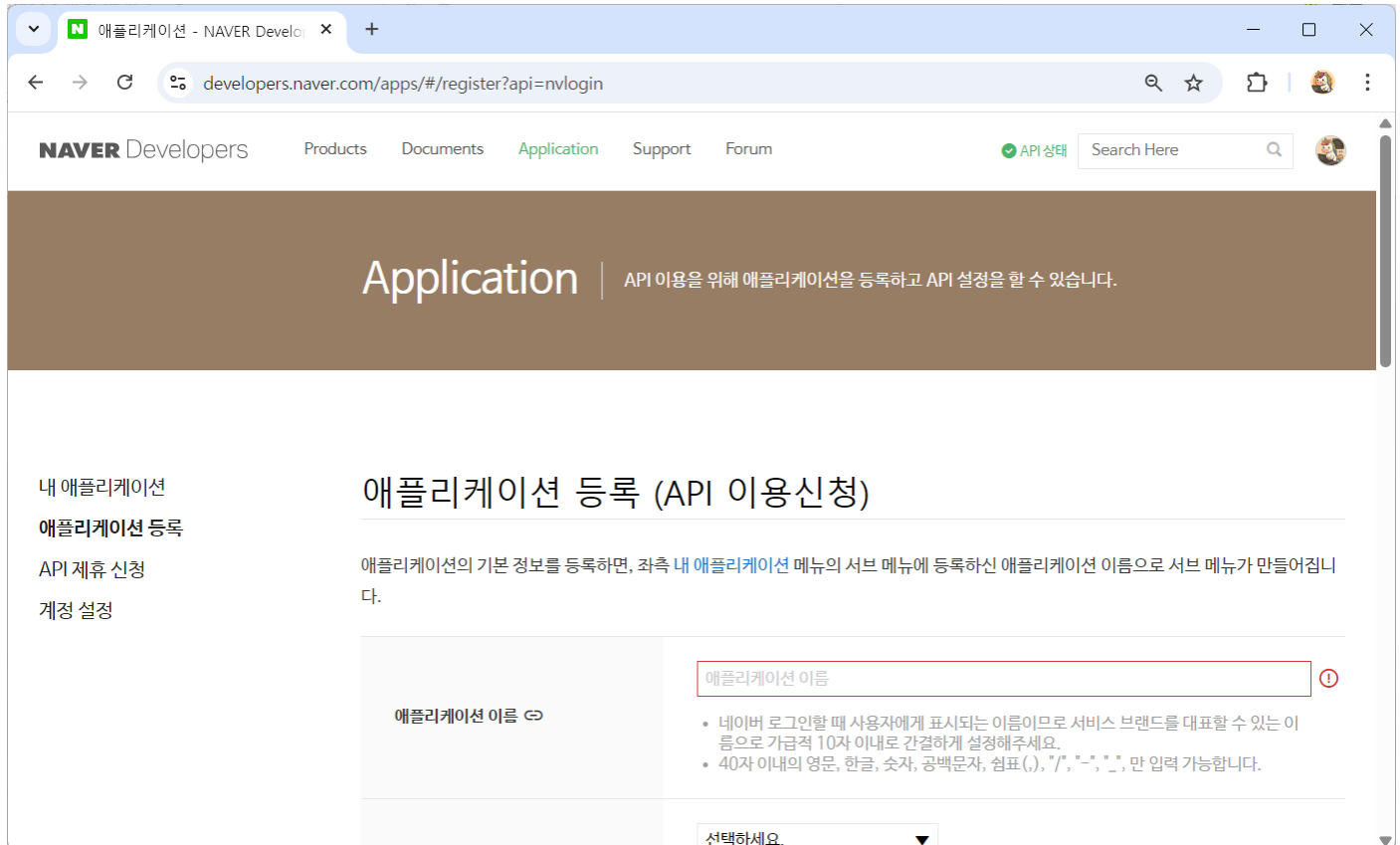
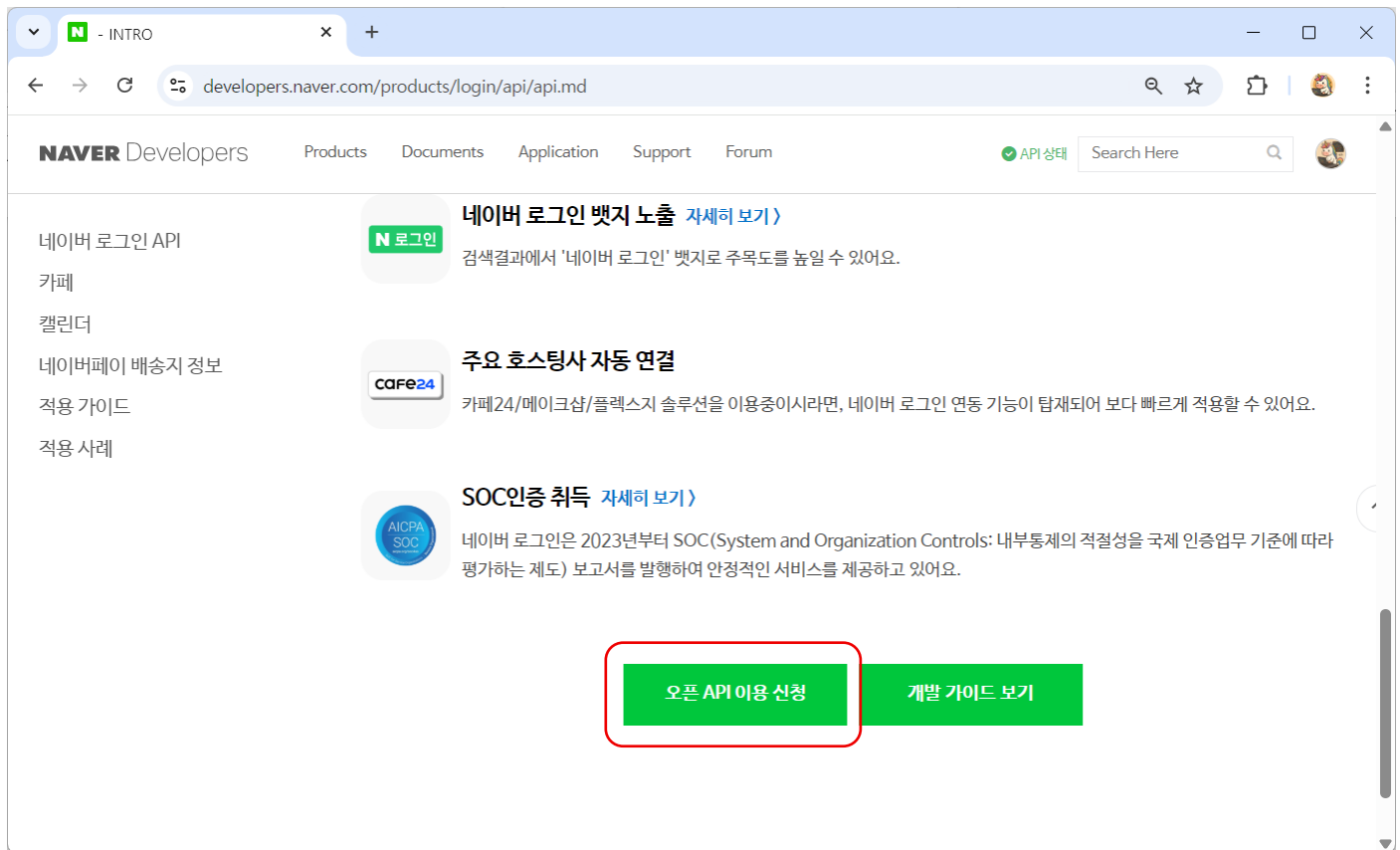
✓ [필수] 만 14세 이상입니다.

✓ [선택] 설로인의 광고와 마케팅 메시지를 카카오톡으로 받습니다

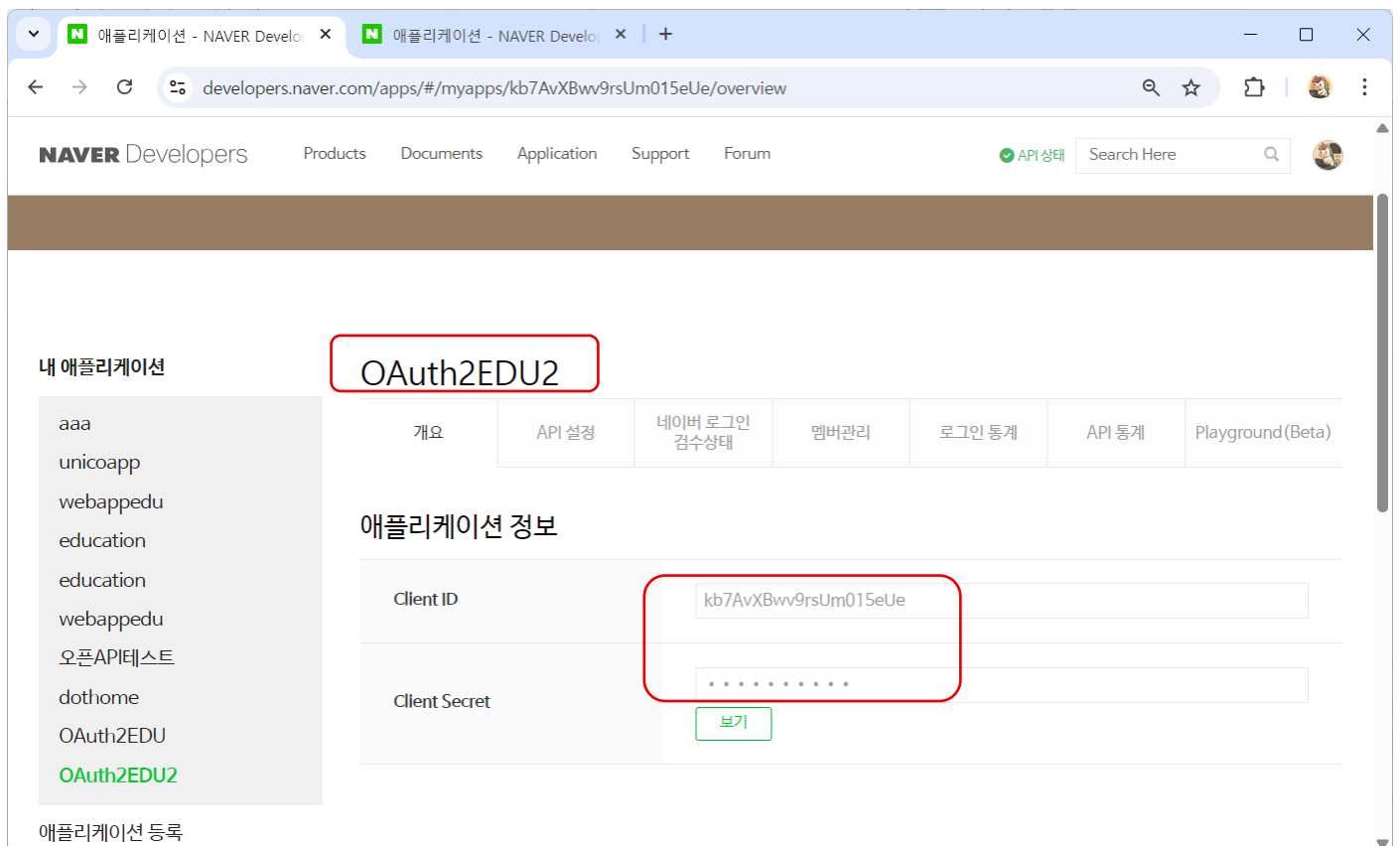
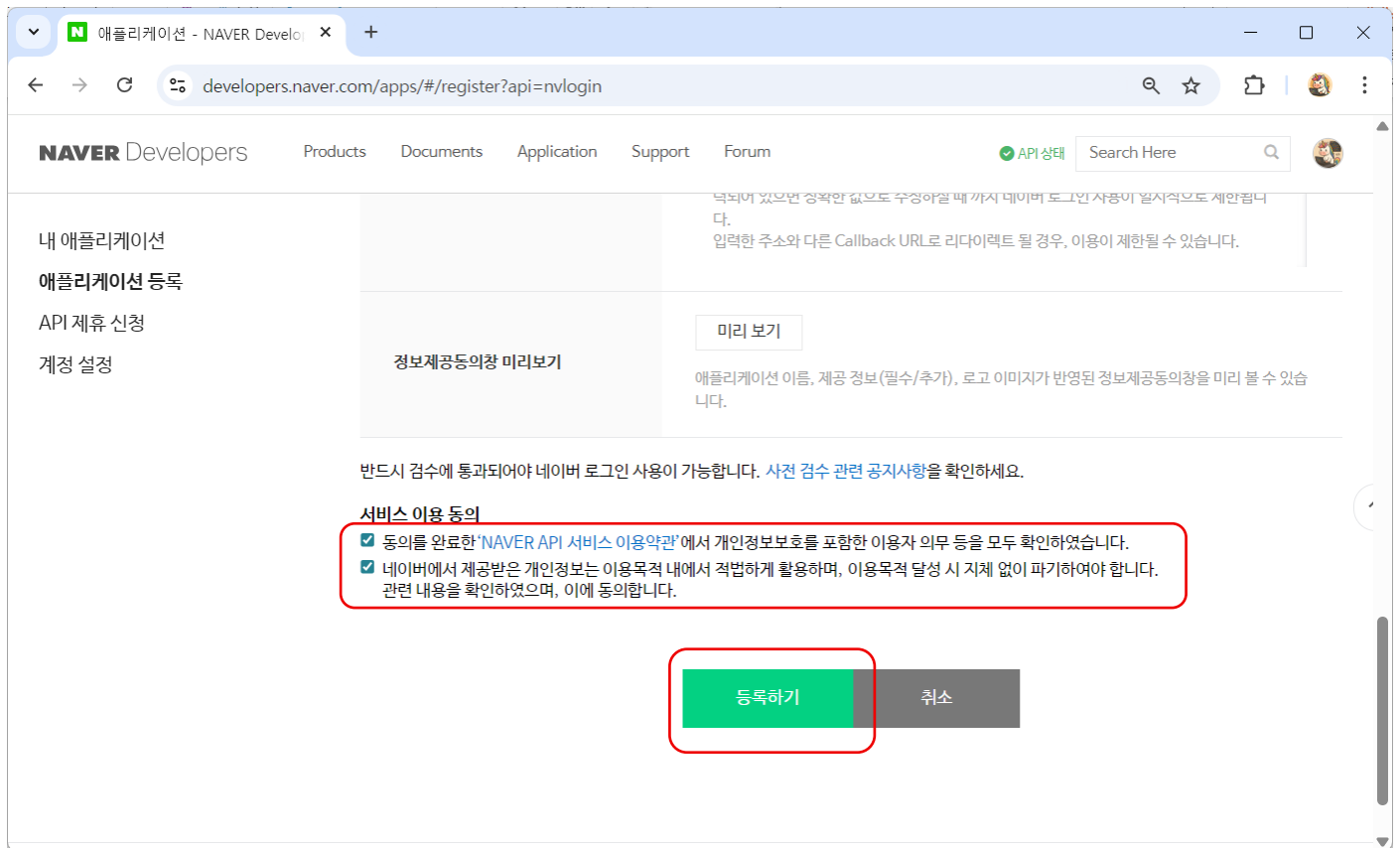
동의하고 계속하기

[네이버 개발자 사이트에서 네이버로그인 설정 과정]





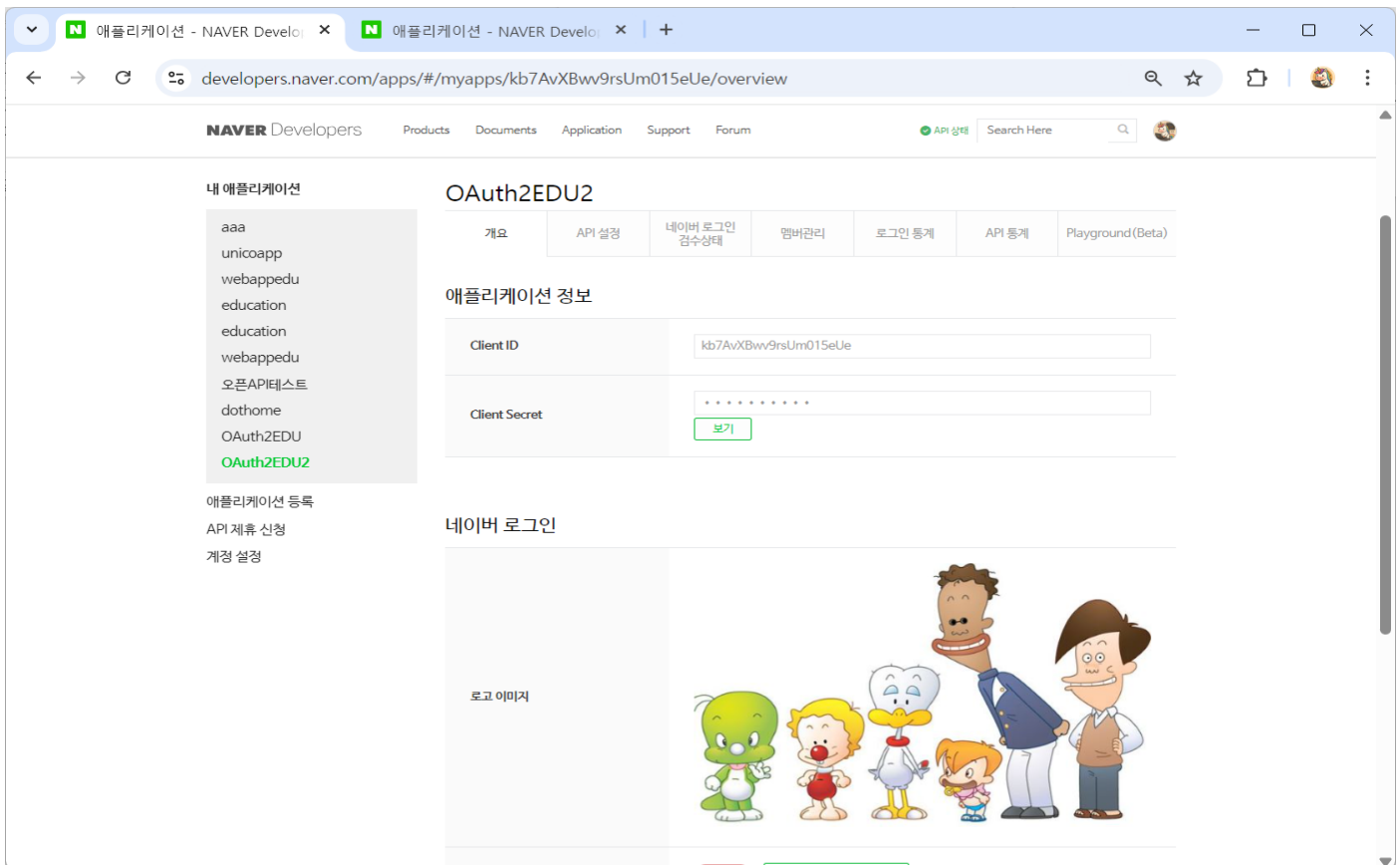
The screenshot shows the Naver Developers API registration page. The 'OAuth2EDU2' grant type is selected. In the 'scopes' section, 'openid', 'email', and 'profile' are checked. The 'user name attribute' is set to 'id'. The 'scopes' section is highlighted with a red box.



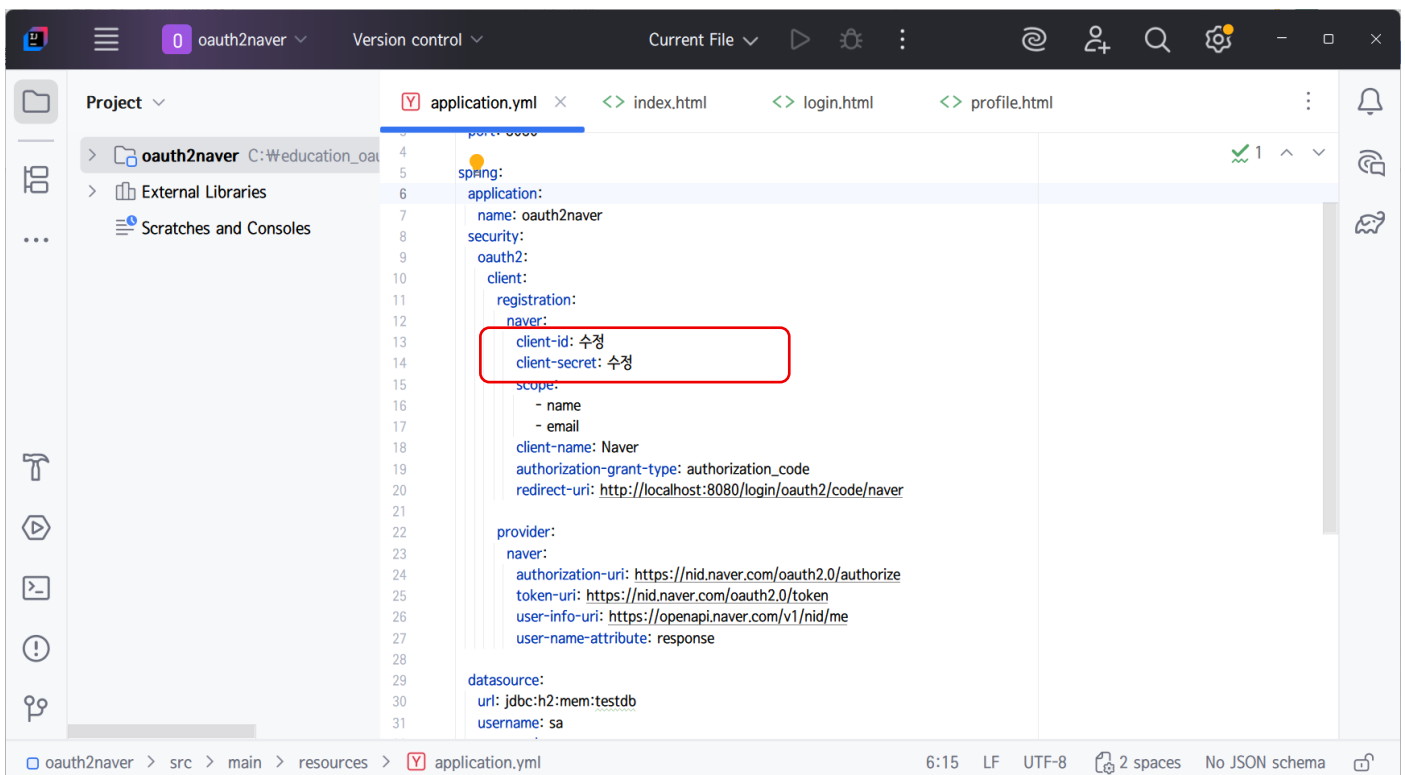
The screenshot shows the Naver Developers 'My Apps' configuration page. The left sidebar lists several applications, with 'education' highlighted. The main content area displays a table with application details and actions. The 'education' application is in the '개발 중' (In Development) state. The '수정' (Edit) button is circled in red.

애플리케이션 개발 상태	애플리케이션을 삭제합니다
애플리케이션이 '개발 중' 상태이면 [멤버관리] 탭에서 등록된 아이디만 네이버 로그인을 이용할 수 있습니다. 개발이 완료되어 실 서비스에 적용하고자 하신다면 검수를 요청해 주세요. 검수가 승인되면 모든 아이디로 네이버 로그인을 이용할 수 있습니다.	애플리케이션을 삭제하면 애플리케이션에 설정된 정보들과 API 설정들은 복구되지 않으므로 신중하게 삭제해 주시기 바랍니다.

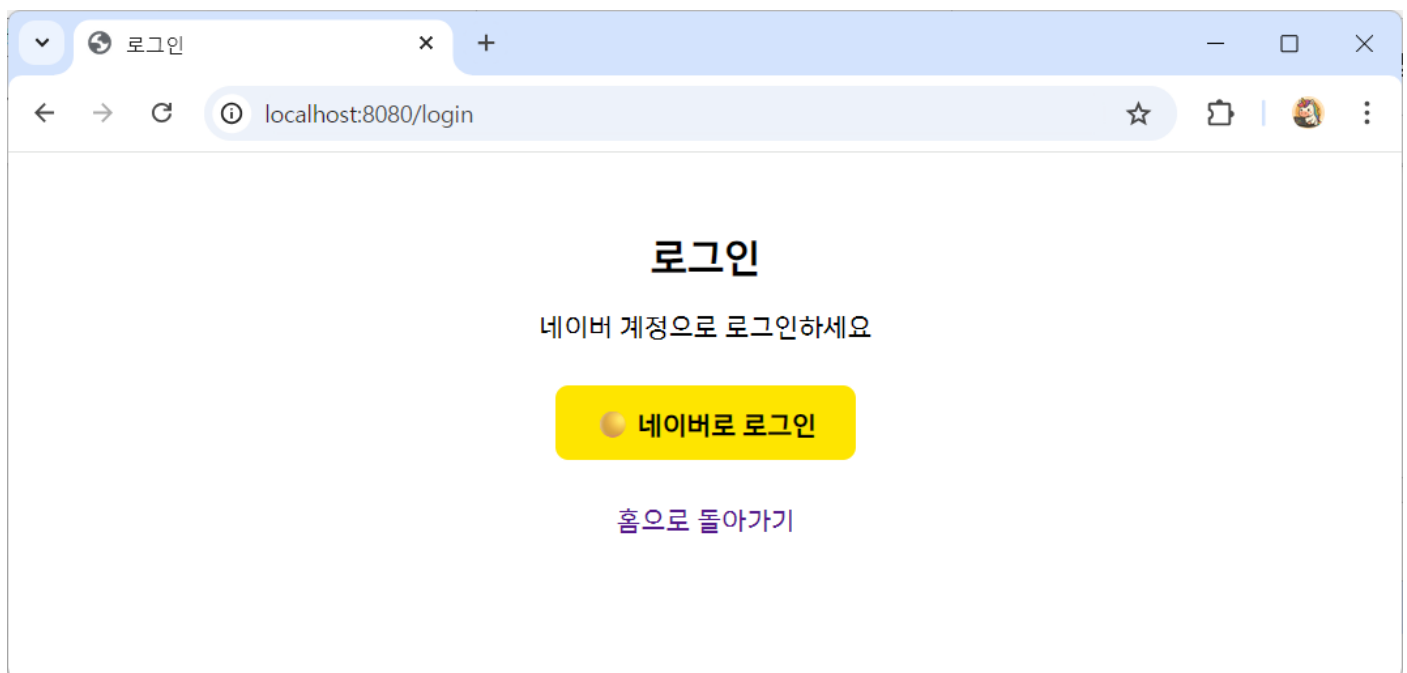
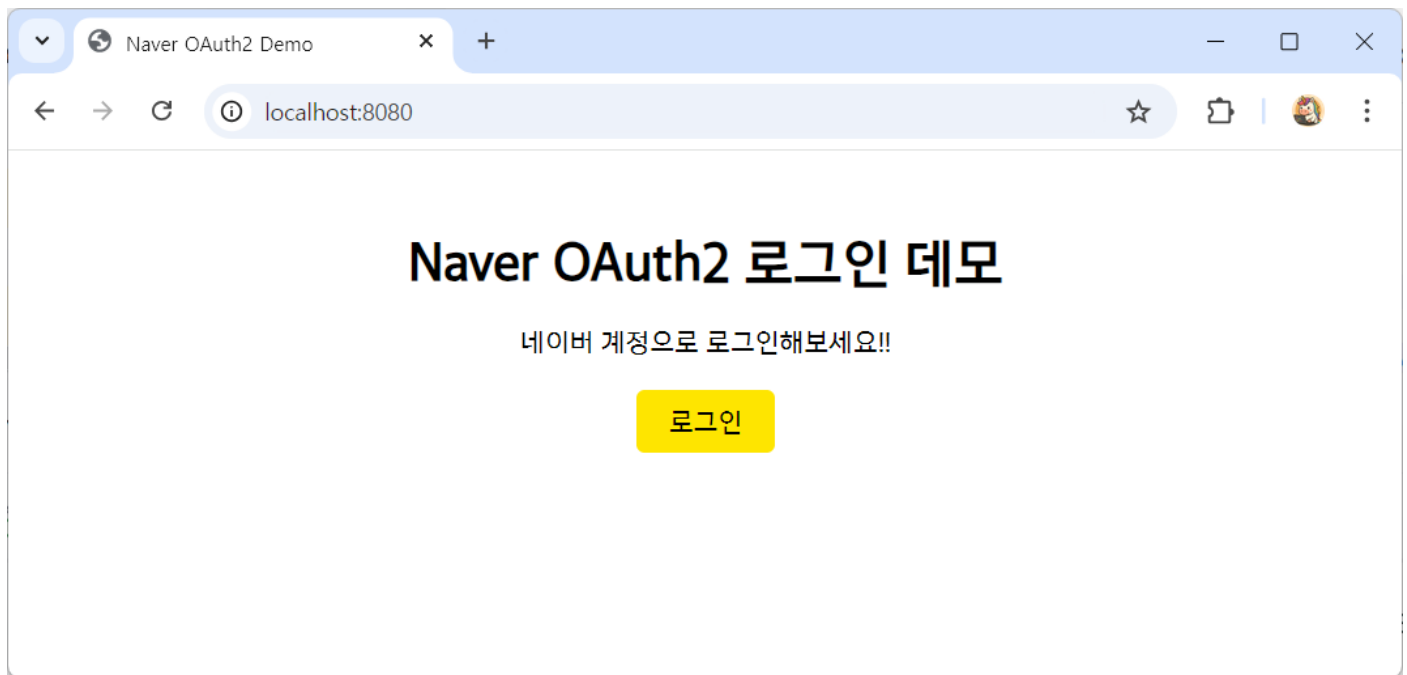
수정 취소

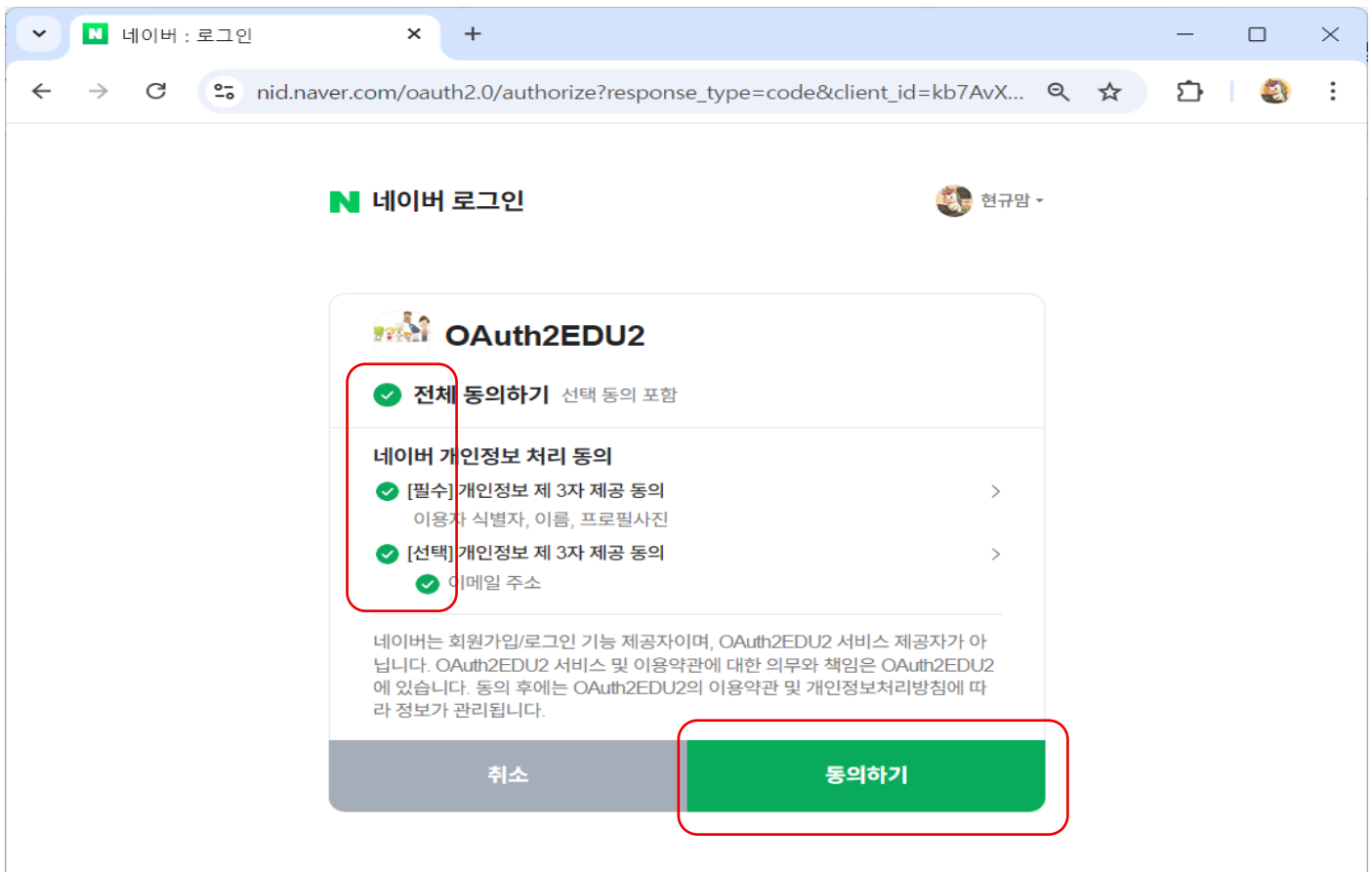
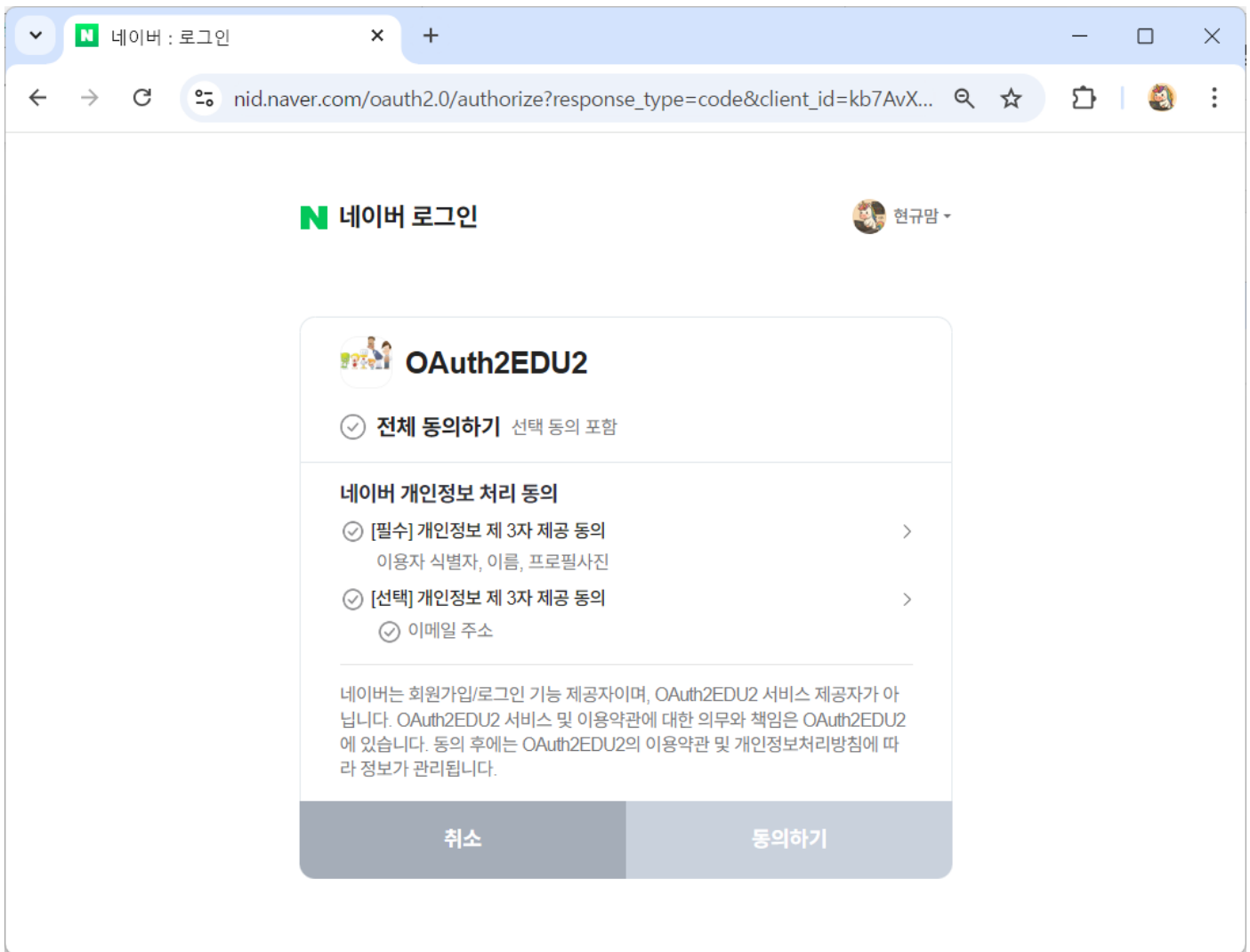


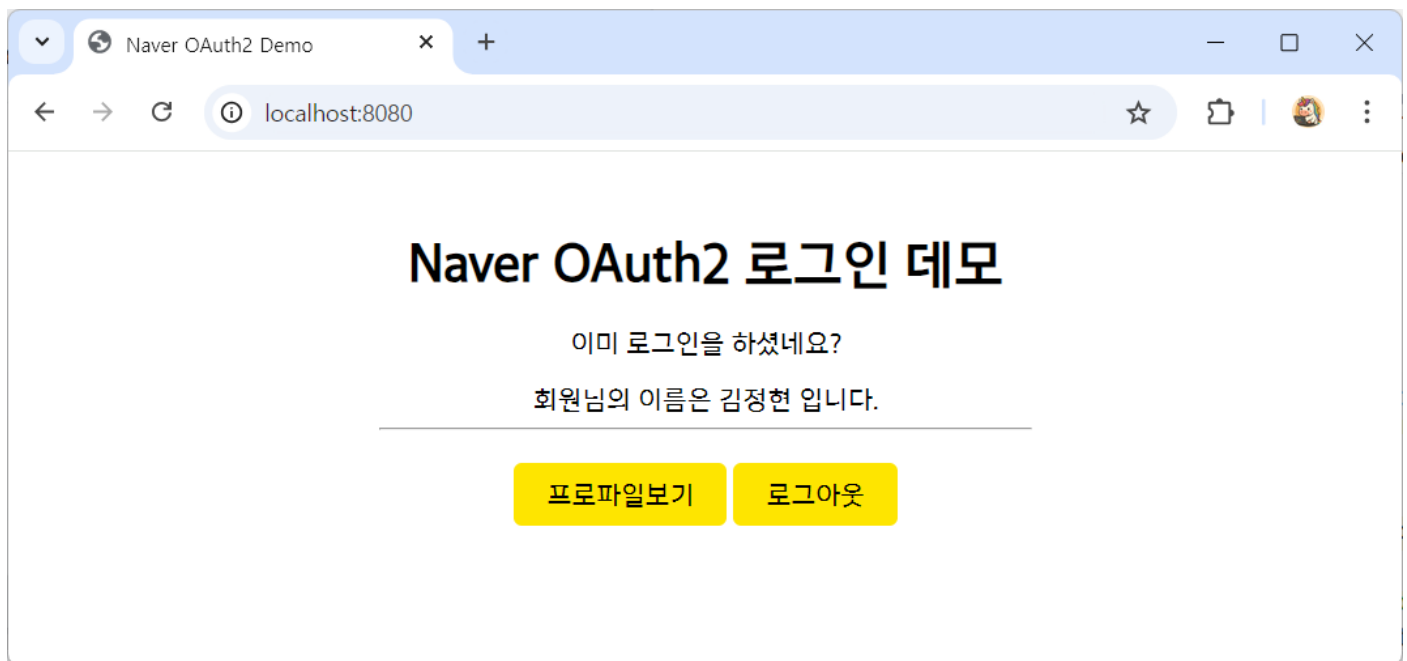
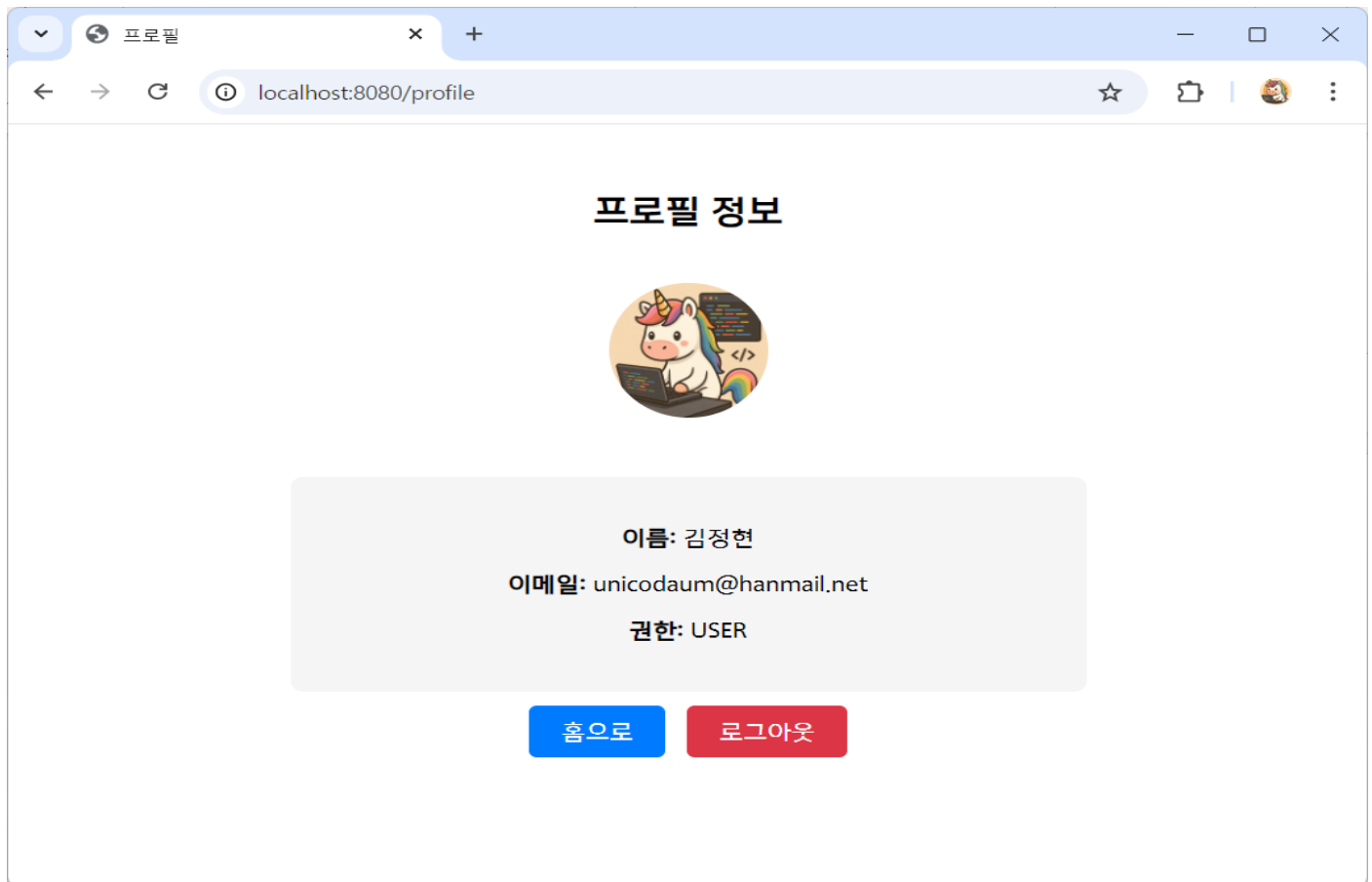
제공된 소스 중에서 oauth2naver 를 인텔리제이로 오픈한 다음 내용을 본인의 client-id 와 client-secret 으로 수정한다.

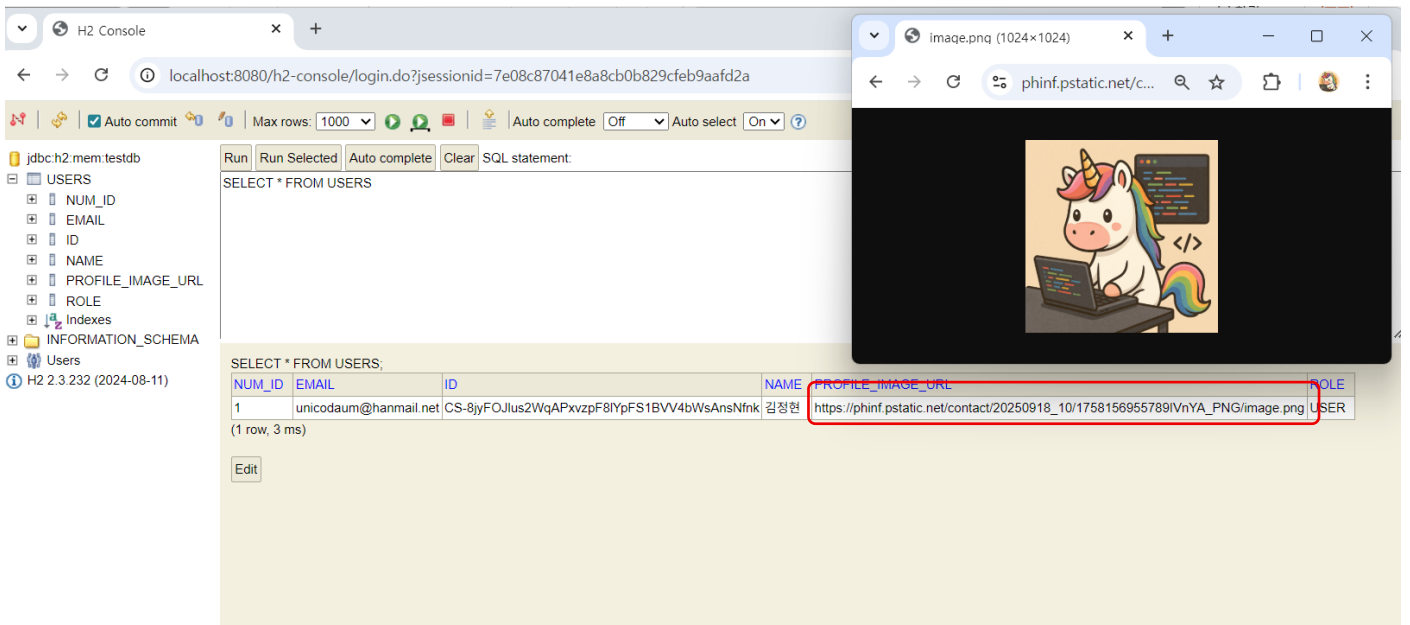
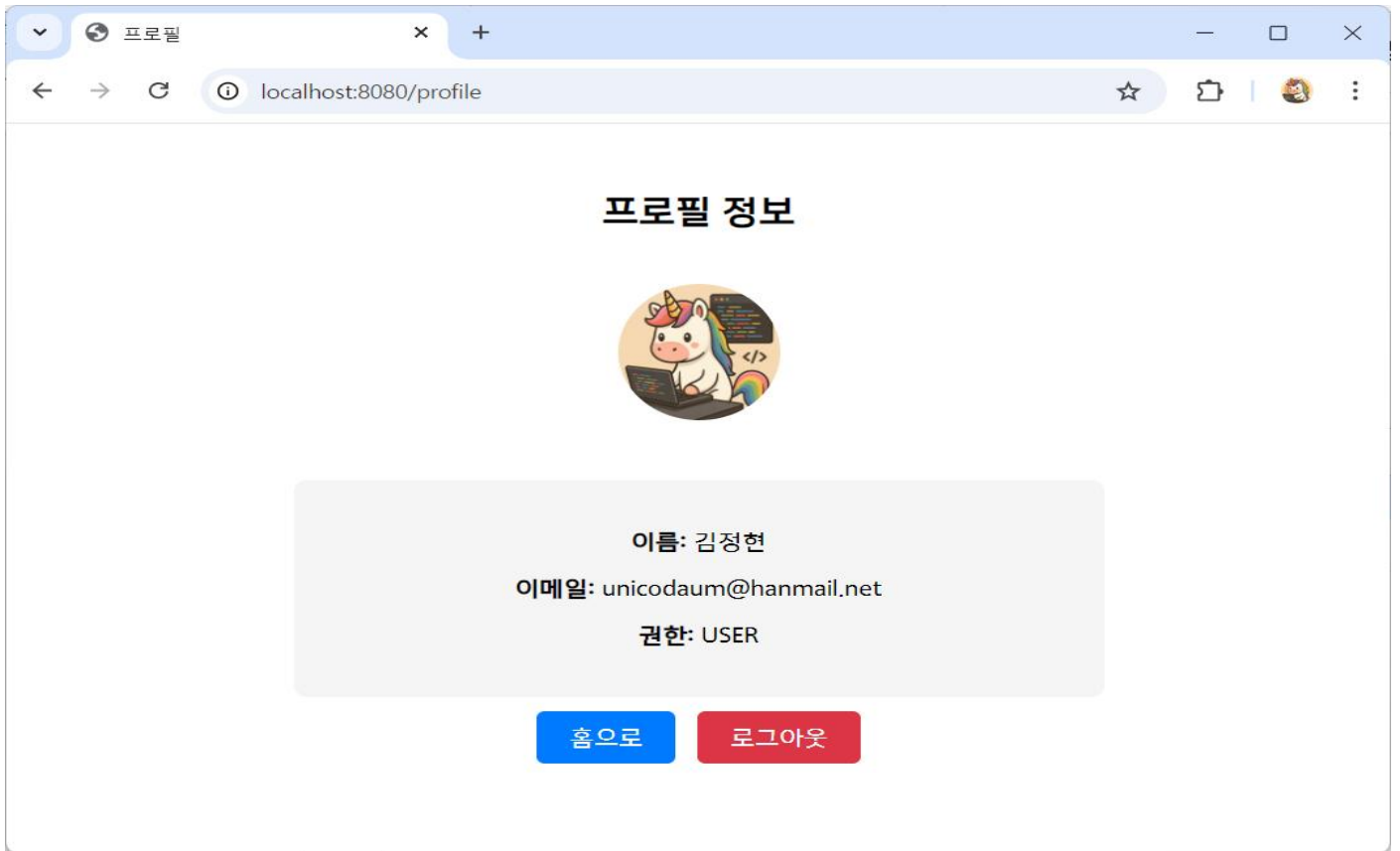


oauth2naver 를 기동시킨 후 다음과 같이 http://localhost:8080/ 을 요청해 본다.










```
{
  "resultCode": "00",
  "message": "success",
  "response": {
    "id": "사용자_고유_ID",
    "email": "사용자_이메일@naver.com",
    "name": "사용자_이름",
    "profile_image": "프로필_이미지_URL"
    // ... 기타 사용자 정보
  }
}
```

[구글 개발자 사이트에서 구글로그인 설정 과정]

<https://console.cloud.google.com/projectcreate>

새 프로젝트 – Google Cloud

리소스, 문서, 제품 등 검색(/) [검색](#)

새 프로젝트

projects 할당량이 16개 남았습니다. 할당량 증가를 요청하거나 프로젝트를 삭제하세요. [자세히 알아보기](#)

[Manage Quotas](#)

프로젝트 이름 *
OAuth2EDU2

프로젝트 ID: oauth2edu2. 나중에 변경할 수 없습니다. [수정](#)

위치 *
조직 없음 [찾아보기](#)

상위 조직 또는 폴더

[만들기](#) [취소](#)

대시보드 - Gemini API - Google Cloud

console.cloud.google.com/home/dashboard?project=gen-lang-client-0296210811

Google Cloud Gemini API 리소스, 문서, 제품 등 검색(/) 검색

대시보드 활동 추천 알림

프로젝트 정보

- 프로젝트 이름: Gemini API
- 프로젝트 번호: 909912849186
- 프로젝트 ID: gen-lang-client-0296210811

이 프로젝트에 사용자 추가

프로젝트 설정으로 이동

API API

요청(요청/초)

모든 활동 보기

이벤트 OAuth2EDU2 입니다.

Cloud 상태 대시보드로 이동

결제

- 예상 요금: KRW ₩0.00
- 결제 기간: 2025. 9. 1. ~ 2025. 9. 17.

결제 살펴보기

청구 세부정보 보기

대시보드 - OAuth2EDU2 - Google Cloud

console.cloud.google.com/home/dashboard?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

대시보드 활동 추천 맞춤설정

프로젝트 정보

- 프로젝트 이름: OAuth2EDU2
- 프로젝트 번호: 396841048854
- 프로젝트 ID: oauth2edu2

이 프로젝트에 사용자 추가

프로젝트 설정으로 이동

리소스

- BigQuery: 데이터 웨어하우스/분석
- SQL: 관리형 MySQL, PostgreSQL, SQL Server
- Compute Engine: VM, GPU, TPU 등

API API

요청(요청/초)

API 개요로 이동

Google Cloud Platform 상태

모든 서비스 정상

Cloud 상태 대시보드로 이동

결제

- 예상 요금: KRW ₩0.00
- 결제 기간: 2025. 12. 1. ~ 2025. 12. 5.

결제 살펴보기

청구 세부정보 보기

모니터링

- 내 대시보드 만들기
- 알림 정책 설정
- 업타임 체크 만들기

https://console.cloud.google.com/home/dashboard?project=oauth2edu2

API API 및 서비스 - API 및 서비스 x +

console.cloud.google.com/apis/dashboard?project=oauth2edu2&show=all

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 🔍 검색

API API 및 서비스 API 및 서비스 + API 및 서비스 사용 설정

사용 설정된 API 및 서비스

라이브러리

사용자 인증 정보

OAuth 동의 화면

페이지 사용 동의

1시간 6시간 12시간 ✓ 1일 2일 4일 7일 14일 30일

트래픽

선택한 기간에는 데이터가 없습니다.

OAuth 개요 - Google 인증 플랫폼 x +

console.cloud.google.com/auth/overview?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 🔍 검색

Google 인증 플랫폼 / 개요

개요

OAuth 개요

브랜딩

대상

클라이언트

데이터 액세스

인증 센터

설정

Google 인증 플랫폼이 아직 구성되지 않음

시작하여 애플리케이션의 ID를 구성하고 Google API 호출 및 Google 계정으로 로그인에 사용할 사용자 인증 정보를 관리하세요. 자세히 알아보기

시작하기

프로젝트 구성 - Google 인증

console.cloud.google.com/auth/overview/create?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 개요 / 브랜딩 만들기

개요

브랜딩

대상

클라이언트

데이터 액세스

인증 센터

설정

프로젝트 구성

1 앱 정보

앱 이름 *
동의를 요청하는 앱의 이름

사용자 지원 이메일 *
사용자가 동의에 대해 문의하는 용도입니다. 자세히 알아보기

다음

2 대상

3 연락처 정보

4 완료

< 만들기 취소

프로젝트 구성 - Google 인증

console.cloud.google.com/auth/overview/create?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 개요 / 브랜딩 만들기

개요 브랜딩 대상 클라이언트 데이터 액세스 인증 센터 설정

프로젝트 구성

1 앱 정보

앱 이름 *
oauth2study
중의를 요청하는 앱의 이름

사용자 지원 이메일 *
ggunico@gmail.com
사용자가 동의에 대해 문의하는 용도입니다. 자세히 알아보기

다음

프로젝트 구성 - Google 인증

console.cloud.google.com/auth/overview/create?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 개요 / 브랜딩 만들기

개요 브랜딩 대상 클라이언트 데이터 액세스 인증 센터 설정

프로젝트 구성

✓ 앱 정보

2 대상

☐ 내부 ②
조직 내 사용자만 사용할 수 있습니다. 인증을 위해 앱을 제출할 필요는 없습니다. 사용자 유형 자세히 알아보기

☒ 외부 ②
Google 계정이 있는 모든 테스트 사용자가 사용할 수 있습니다. 앱이 테스트 모드로 시작되고 테스트 사용자 목록에 추가된 사용자에게만 제공됩니다. 앱을 프로덕션에 푸시할 준비가 되면 앱을 인증해야 할 수도 있습니다. 사용자 유형 자세히 알아보기

다음

프로젝트 구성 - Google 인증

console.cloud.google.com/auth/overview/create?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 개요 / 브랜딩 만들기

개요

브랜딩

대상

클라이언트

데이터 액세스

인증 센터

설정

프로젝트 구성

✓ 앱 정보

✓ 대상

3 연락처 정보

이메일 주소 *
ggunico@gmail.com

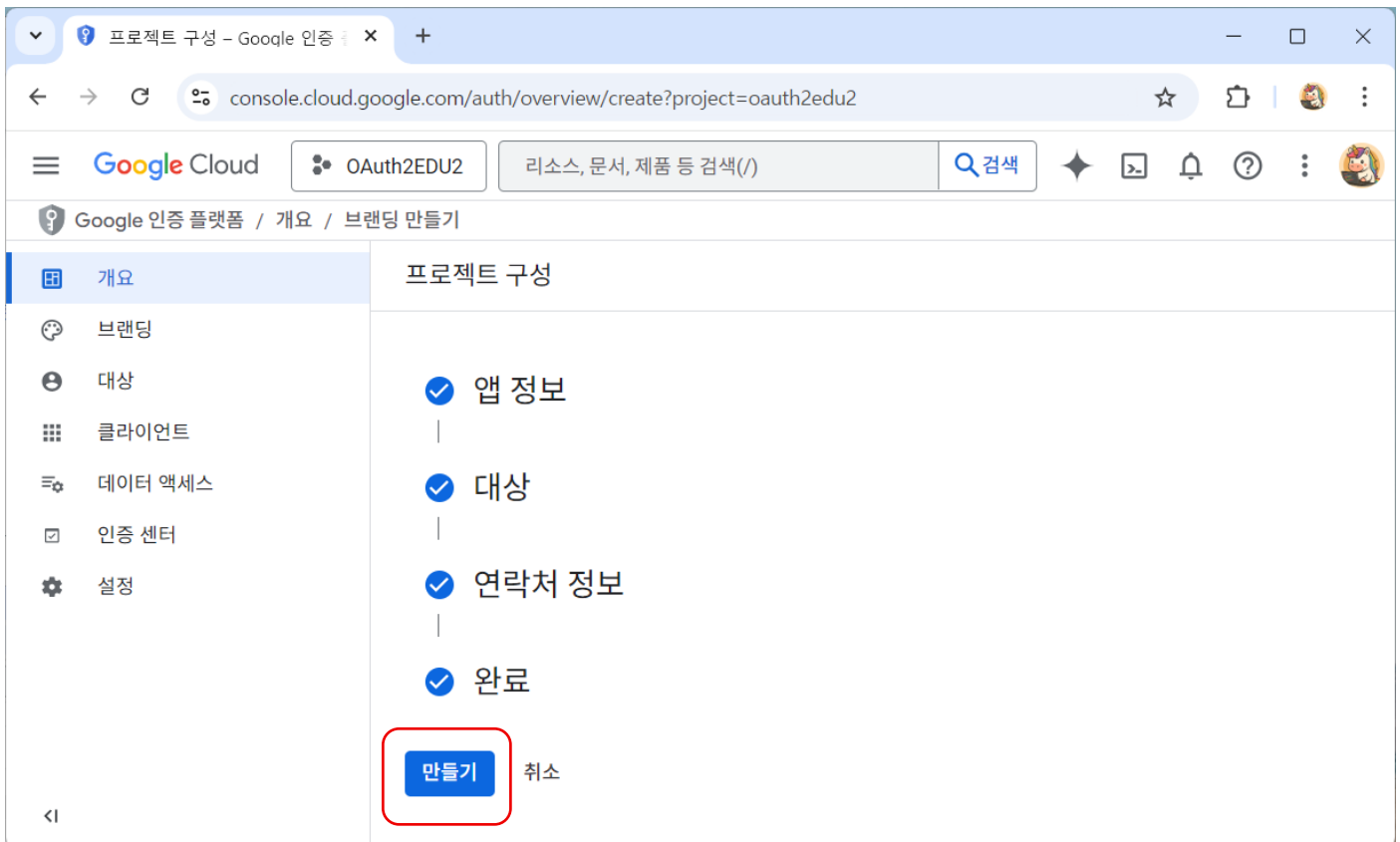
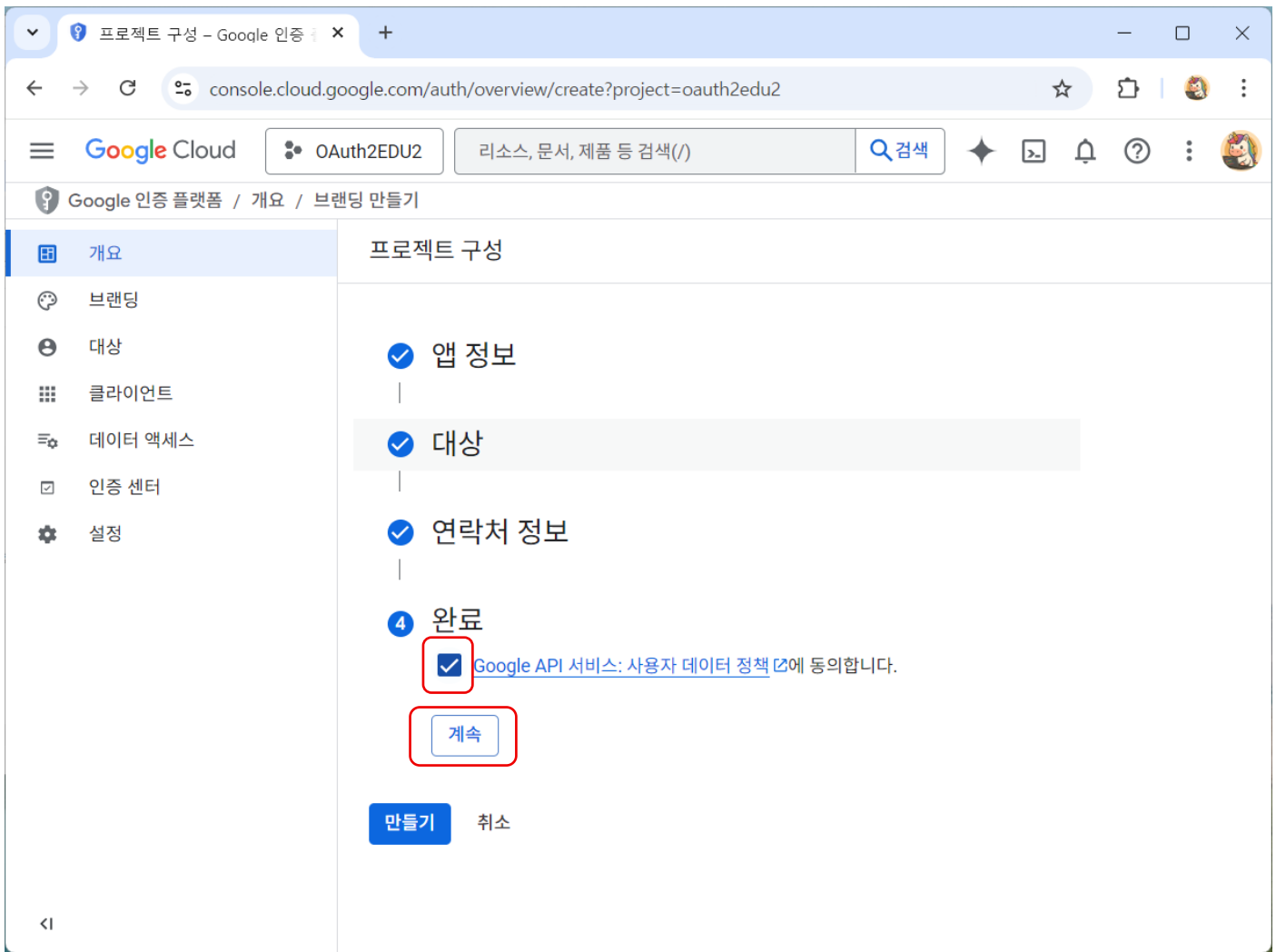
이 이메일 주소는 Google에서 프로젝트 변경사항에 대해 알림을 보내기 위한 용도입니다.

다음

4 완료

만들기 취소

<



OAuth 개요 - Google 인증 플랫

console.cloud.google.com/auth/overview?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 개요

개요

OAuth 개요

측정항목

아직 이 프로젝트에 OAuth 클라이언트를 구성하지 않았습니다. OAuth 클라이언트 만들기

프로젝트 진단

프로젝트에 대한 프로젝트 상태 추천이 없습니다. OAuth 2.0 정책 자세히 알아보기

OAuth 구성 생성됨

OAuth 클라이언트 ID 만들기

console.cloud.google.com/auth/clients/create?project=oauth2edu2

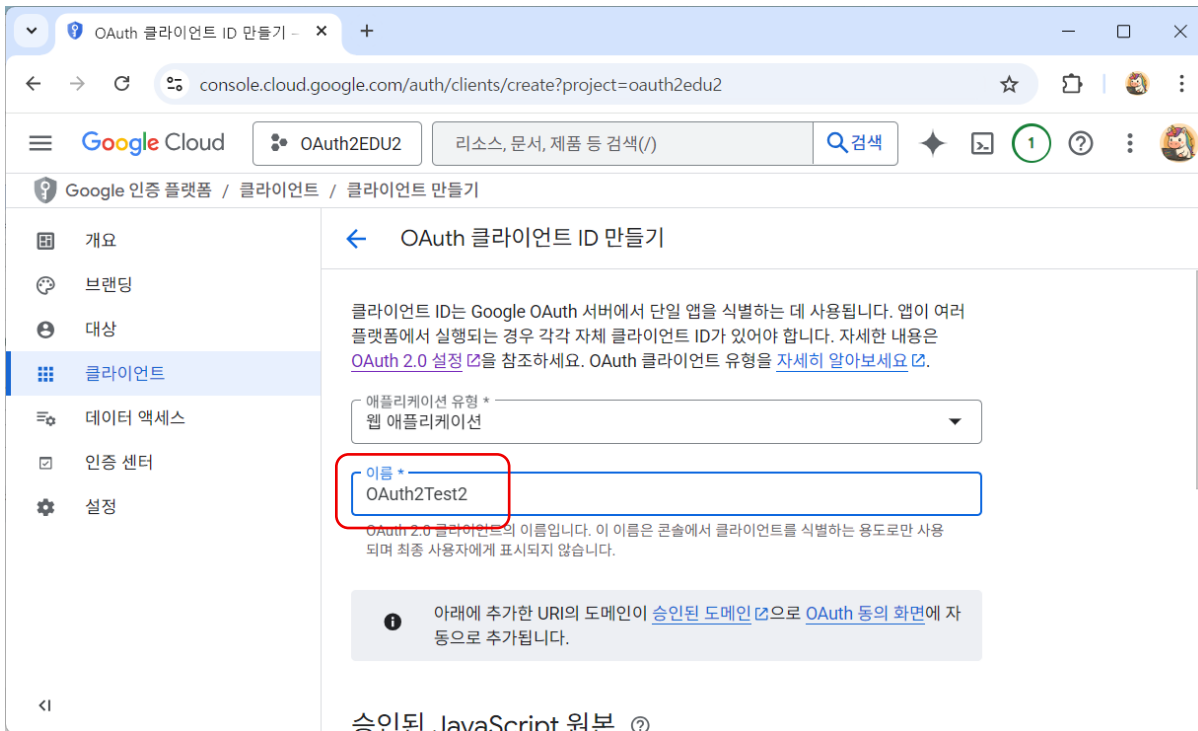
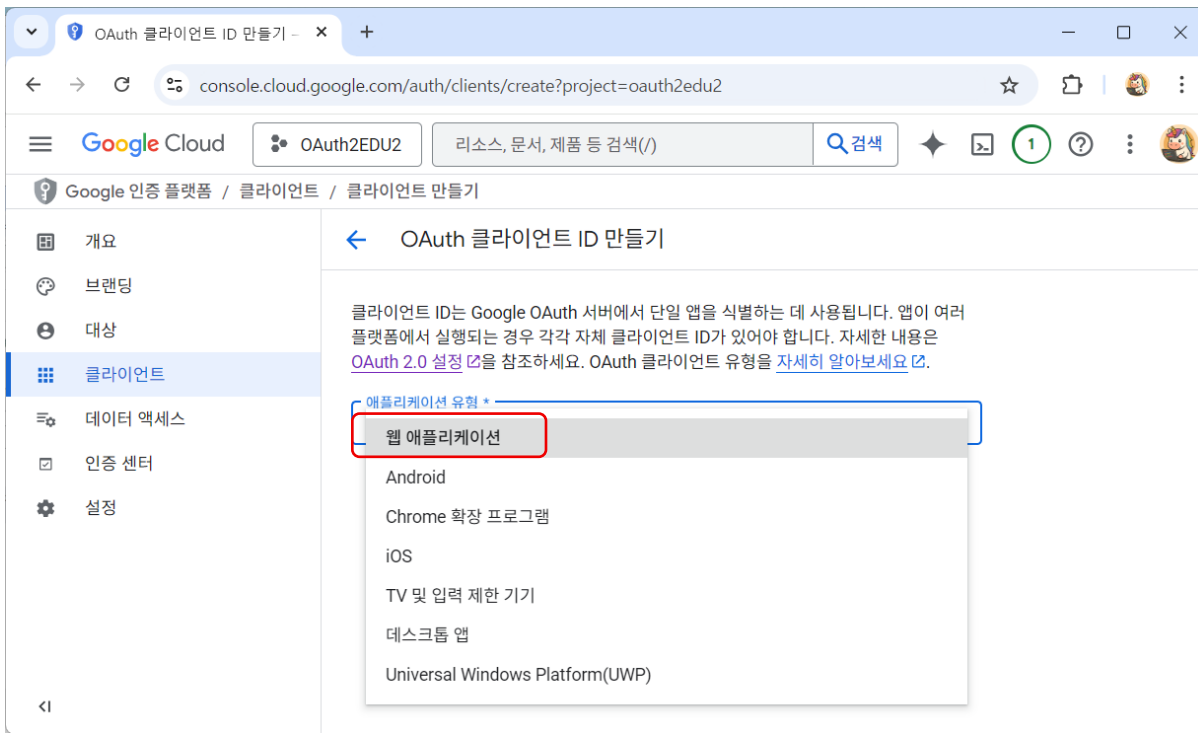
Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

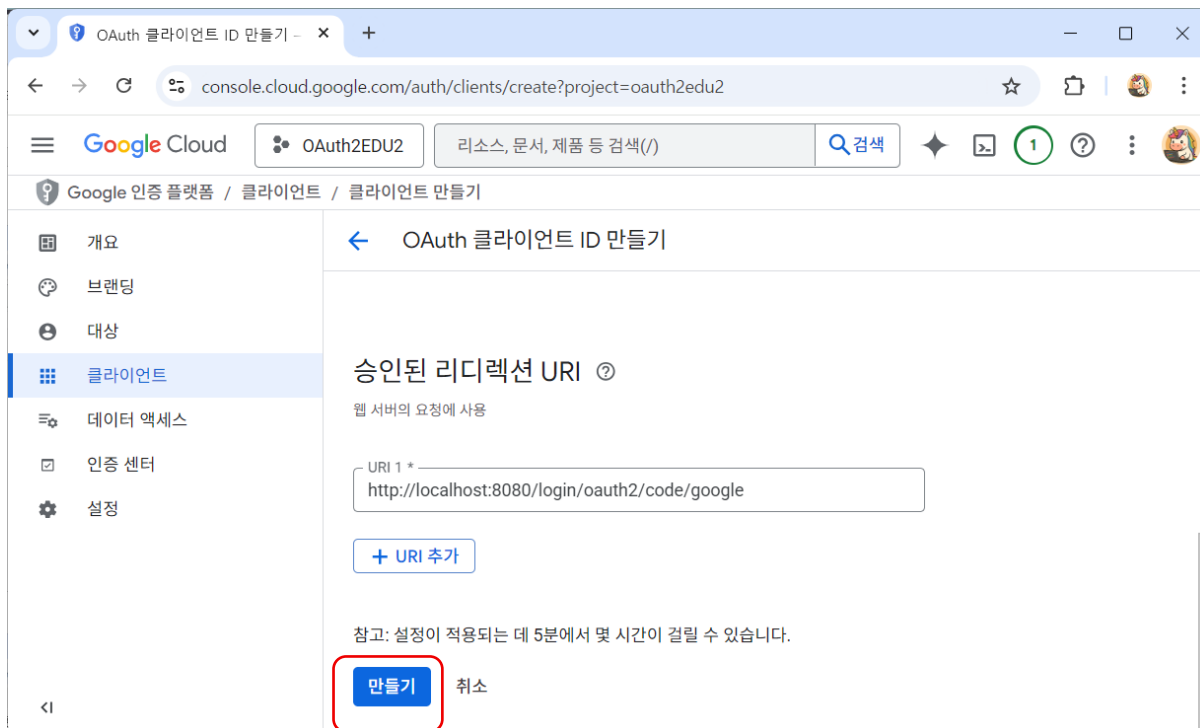
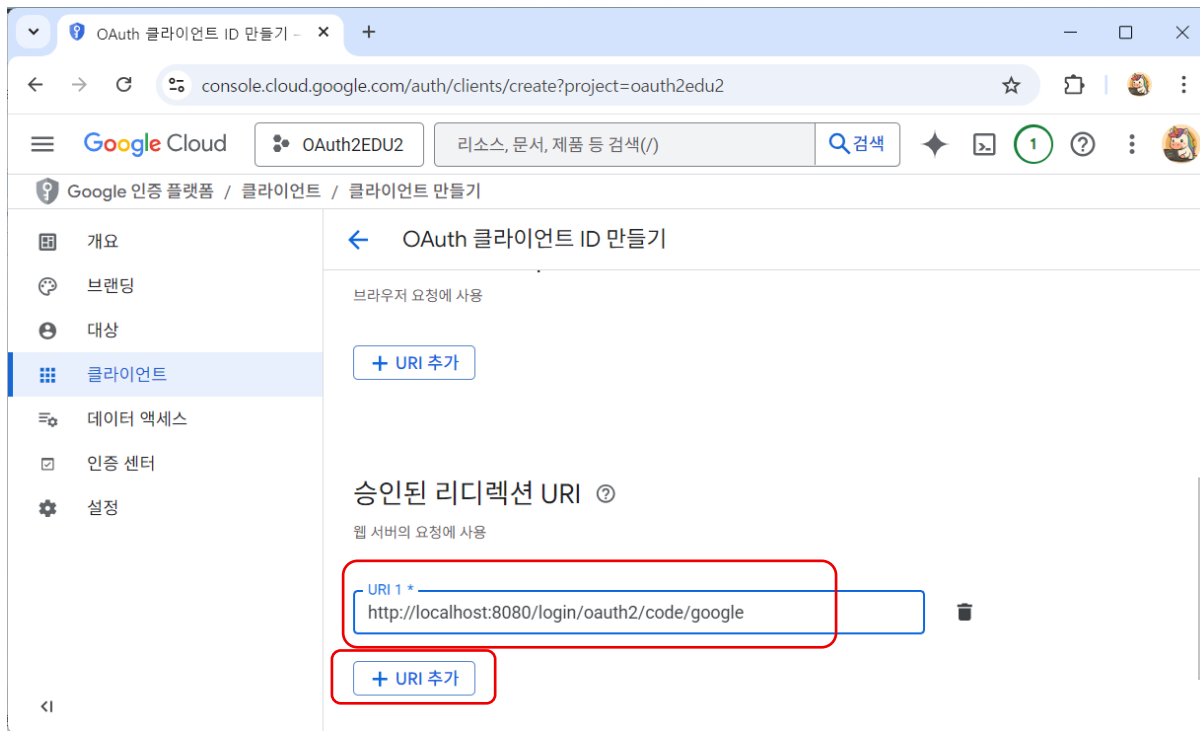
Google 인증 플랫폼 / 클라이언트 / 클라이언트 만들기

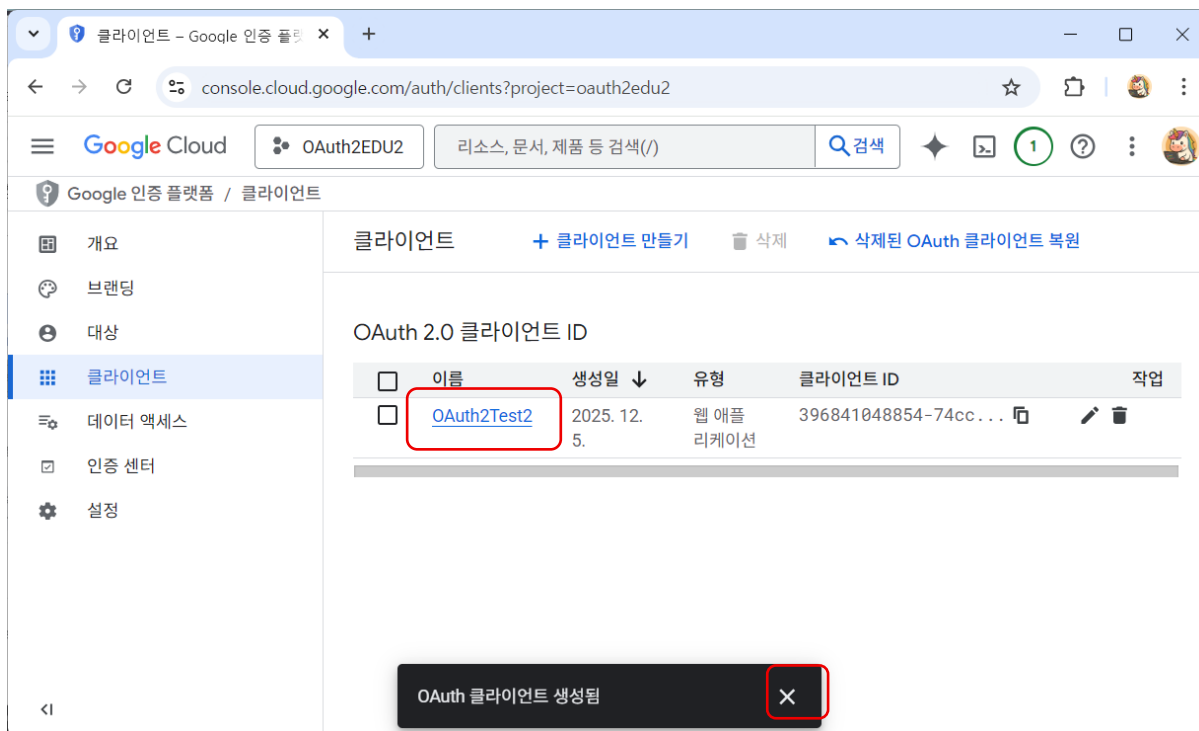
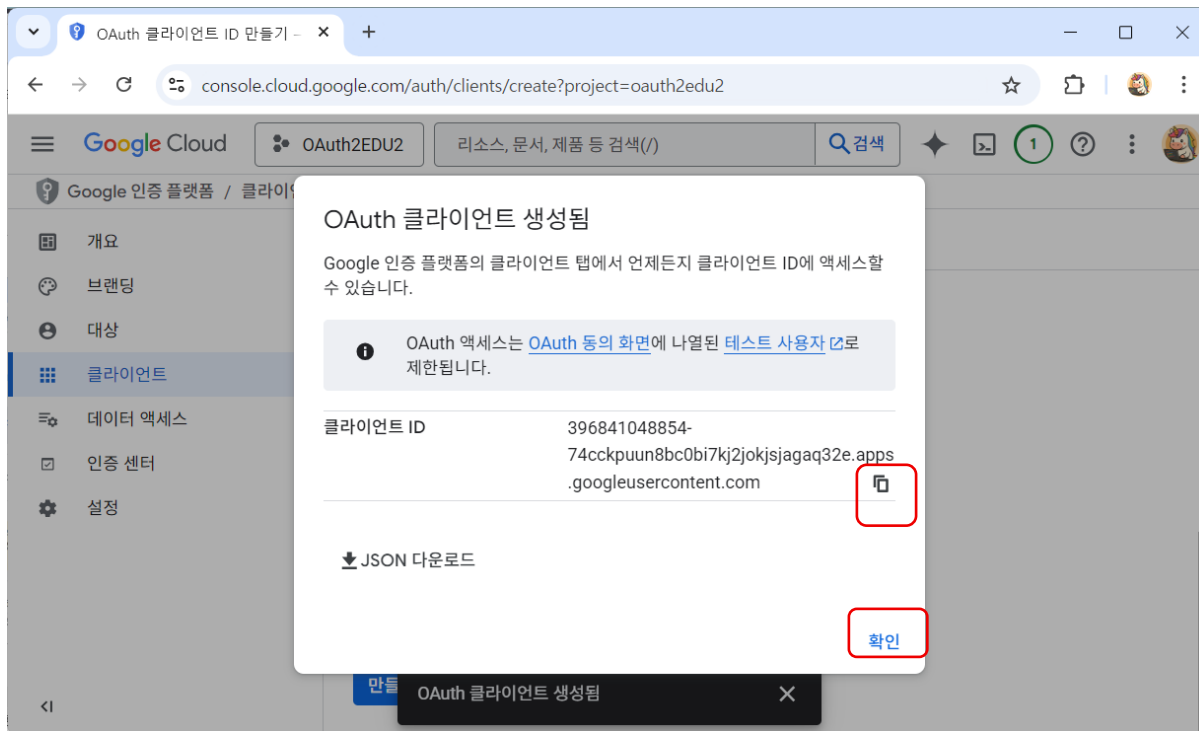
← OAuth 클라이언트 ID 만들기

클라이언트 ID는 Google OAuth 서버에서 단일 앱을 식별하는 데 사용됩니다. 앱이 여러 플랫폼에서 실행되는 경우 각각 자체 클라이언트 ID가 있어야 합니다. 자세한 내용은 [OAuth 2.0 설정](#)을 참조하세요. OAuth 클라이언트 유형을 [자세히 알아보세요](#).

애플리케이션 유형 *







웹 애플리케이션의 클라이언트

console.cloud.google.com/auth/clients/396841048854-74cckpuun8bc0bi7kj2jokjsjagaq32e.apps.googleusercontent.com?project=oauth2edu2

Google Cloud OAuth2EDU2 리소스, 문서, 제품 등 검색(/) 검색

Google 인증 플랫폼 / 클라이언트: 396841048854-74cckpuun8bc0bi7kj2jokjsjagaq32e.apps.googleusercontent.com

개요 브랜딩 대상 클라이언트 데이터 액세스 인증 센터 설정

웹 애플리케이션의 클라이언트 ID 삭제

이름 * OAuth2Test2

OAuth 2.0 클라이언트의 이름입니다. 이 이름은 콘솔에서 클라이언트를 식별하는 용도로만 사용되며 최종 사용자에게 표시되지 않습니다.

아래에 추가한 URI의 도메인이 승인된 도메인으로 OAuth 동의 화면에 자동으로 추가됩니다.

승인된 JavaScript 원본

브라우저 요청에 사용

+ URI 추가

승인된 리디렉션 URI

웹 서버의 요청에 사용

URI 1 * http://localhost:8080/login/oauth2/code/google

+ URI 추가

참고: 설정이 적용되는 데 5분에서 몇 시간이 걸릴 수 있습니다.

저장 취소

Additional information

클라이언트 ID 396841048854-74cckpuun8bc0bi7kj2jokjsjagaq32e.apps.googleusercontent.com

생성일 2025년 12월 5일 PM 7시 2분 40초 GMT+9

마지막 사용일 2025년 12월 5일 (Note: this data could be delayed by a day or more.)

비활성 OAuth 클라이언트는 6개월 동안 사용되지 않으면 삭제됩니다. 비활성으로 인한 삭제 알림이 전송되며 삭제 후 최대 30일 동안 클라이언트를 복원할 수 있습니다. [Learn more](#)

클라이언트 보안 비밀번호

클라이언트 보안 비밀번호를 변경하는 중인 경우 다운로드 없이 수동으로 순환할 수 있습니다. [자세히 알아보기](#)

2025년 11월부터는 클라이언트 보안 비밀번호를 더 이상 보거나 다운로드할 수 없습니다. 아래 보안 비밀번호를 분실한 경우 새 보안 비밀번호를 추가하세요. 클라이언트 보안 비밀번호는 안전하게 저장하고 최대한 주의하여 다루어야 합니다. 보안 비밀번호를 코드 저장소에 보관해서는 안 됩니다. [자세히 알아보기](#)

클라이언트 보안 비밀번호 ****doVY

생성일 2025년 12월 5일 PM 7시 2분 40초 GMT+9

상태 사용 설정됨

+ Add secret

[도큐먼트]

데이터 액세스 - Google 인증 x Google Developer Program x google oauth2 login - Google x

google.com/search?q=google+oauth2+login&oq=google+oauth2+login&gs_lcrp=EgZjaHJvWUyBggAEUYOTIHCA...

Google

google oauth2 login

전체 이미지 동영상 쇼핑 뉴스 도서 더보기 도구

Google for Developers

https://developers.google.com > ... > Authorization

Using OAuth 2.0 to Access Google APIs | Authorization

Google APIs use the OAuth 2.0 protocol for authentication and authorization. Google supports common OAuth 2.0 scenarios such as those for web server, client- ...

For JavaScript Web Apps

For example, an application can use OAuth 2.0 to obtain ...

웹 서버 애플리케이션용 OAuth ...

중요: JavaScript용 Google API 클라이언트 라이브러리와 Google ...

Google API의 OAuth 2.0 범위

Cloud OS Login API, v1. 범위, 설명. <https://www.googleapis.com> ...

OAuth2 서비스 계정

예를 들면 다음과 같습니다. `import com.google.auth.oauth2` ...

모바일 및 데스크톱 애플리케이션용 OAuth ...

OAuth 2.0 액세스 토큰 가져오기. 다음 단계에서는 애플리케이션이 ...

Google Identity Authentication 승인 교차 플랫폼 계정 간 보안 (RISC) 검색 / 한국어 정현

Authorization

Google 계정 승인 Google Authorization API를 사용하기 위한 앱 인증 Google 계정 연결 리소스

필터

Google 계정 승인

개요

교차 클라이언트 ID

OAuth2 범위

OAuth 2.0 정책

앱 유형별 승인 고려사항

서버 측 웹 앱용 (자바스크립트 웹 앱용)

Android 앱

iOS 및 데스크톱 앱

TV 및 기기 앱용 (서비스 계정)

승인 코드 샘플

translated by Google 이 페이지는 Cloud Translation API를 통해 번역되었습니다. Switch to English

홈 > 제품 > Google Identity > 승인 > Authorization 도움이 되었나요? 의견 보내기

OAuth 2.0을 사용하여 Google API에 액세스하기

이 페이지의 내용 기본 단계

1. Google API Console에서 OAuth 2.0 사용자 인증 정보를 가져옵니다.
2. Google 승인 서버에서 액세스 토큰을 가져옵니다.
3. 사용자가 부여한 액세스 범위를 검사합니다.
4. 액세스 토큰을 API에 전송합니다.
5. 필요한 경우 액세스 토큰을 새로고침합니다.

시나리오

웹 서버 애플리케이션

Google Identity Authentication 승인 교차 플랫폼 계정 간 보안 (RISC) 검색 / 한국어 정현

필터

Google 계정 승인

개요

교차 클라이언트 ID

OAuth2 범위

OAuth 2.0 정책

앱 유형별 승인 고려사항

서버 측 웹 앱용 (자바스크립트 웹 앱용)

Android 앱

iOS 및 데스크톱 앱

TV 및 기기 앱용 (서비스 계정)

승인 코드 샘플

Windows

★ 참고: Google의 OAuth 2.0 구현 사용은 OAuth 2.0 정책의 적용을 받습니다.

Google API는 인증 및 승인에 OAuth 2.0 프로토콜을 사용합니다. Google은 웹 서버, 클라이언트 측, 설치된 애플리케이션, 제한된 입력 기기 애플리케이션 등 일반적인 OAuth 2.0 시나리오를 지원합니다.

먼저 Google API Console에서 OAuth 2.0 클라이언트 사용자 인증 정보를 가져옵니다. 그런 다음 클라이언트 애플리케이션이 Google 승인 서버에서 액세스 토큰을 요청하고, 응답에서 토큰을 추출하고, 액세스하려는 Google API에 토큰을 전송합니다. 자체 클라이언트 사용자 인증 정보를 사용하는 옵션을 포함하여 Google에 OAuth 2.0을 사용하는 대화형 데모를 보려면 OAuth 2.0 Playground를 사용해 보세요.

이 페이지에서는 Google에서 지원하는 OAuth 2.0 승인 시나리오를 간략히 설명하고 자세한 콘텐츠로 연결되는 링크를 제공합니다. 인증에 OAuth 2.0을 사용하는 방법에 관한 자세한 내용은 OpenID Connect를 참고하세요.

★ 참고: 구현을 올바르게 수행하는 것의 보안 관련 영향을 고려할 때 Google의 OAuth 2.0 엔드포인트와 상호작용할 때는 OAuth 2.0 라이브러리를 사용하는 것이 좋습니다. 다른 사용자가 제공하는 디버깅된 코드를 사용하는 것이 좋으며, 이는 개발자와 사용자를 보호하는 데 도움이 됩니다. 자세한 내용은 클라이언트 라이브러리를 참고하세요.

기본 단계

모든 애플리케이션은 OAuth 2.0을 사용하여 Google API에 액세스할 때 기본 패턴을 따릅니다. 대략적으로 다음 다섯 단계를 따릅니다.

데이터 액세스 – Google 인증 | Google Developer Program | OAuth 2.0을 사용하여 Google

developers.google.com/identity/protocols/oauth2?hl=ko&gl=1*czlb3l*_up*MQ.*_ga*MTU2OTY1NzYxNS4xNzU4MDk3MjA4*_ga_SM8HXJ53K2*...

Google Identity Authentication 승인 교차 플랫폼 계정 간 보안 (RISC) 검색 / 한국어 정현

Google 계정 승인 Google Authorization API를 사용하기 위한 앱 인증 Google 계정 연결 리소스

필터

Google 계정 승인
개요
교차 클라이언트 ID
OAuth2 범위
OAuth 2.0 정책

앱 유형별 승인 고려사항
서버 측 웹 응용
(자바스크립트 웹 응용)
Android 앱
iOS 및 데스크톱 앱
TV 및 기기 응용
(서비스 계정)

승인 코드 샘플
Windows

기본 단계

모든 애플리케이션은 OAuth 2.0을 사용하여 Google API에 액세스할 때 기본 패턴을 따릅니다. 대략적으로 다음 다섯 단계를 따릅니다.

1. Google API Console에서 OAuth 2.0 사용자 인증 정보를 가져옵니다.

[Google API Console](#)을 방문하여 Google과 애플리케이션 모두에 알려진 클라이언트 ID 및 클라이언트 보안 비밀번호와 같은 OAuth 2.0 사용자 인증 정보를 획득합니다. 값 집합은 빌드 중인 애플리케이션의 유형에 따라 다릅니다. 예를 들어 JavaScript 애플리케이션에는 보안 비밀번호가 필요하지 않지만 웹 서버 애플리케이션에는 필요합니다.

앱이 실행될 플랫폼에 적합한 OAuth 클라이언트를 만들어야 합니다.

- <> **서버 측 또는 JavaScript 웹 앱**의 경우 'web' 클라이언트 유형을 사용합니다. 네이티브 앱이나 모바일 앱과 같은 다른 애플리케이션에는 이 클라이언트 유형을 사용하지 마세요.
- Android 앱**의 경우 'Android' 클라이언트 유형을 사용합니다.
- iOS 및 macOS 앱**의 경우 'iOS' 클라이언트 유형을 사용합니다.
- 유니버설 Windows 플랫폼 앱**의 경우 '유니버설 Windows 플랫폼' 클라이언트 유형을 사용합니다.
- Chrome 확장 프로그램**의 경우 **Chrome 확장 프로그램** 클라이언트 유형을 사용합니다.
- TV 또는 삽입된 기기와 같은 입력 제한 기기**의 경우 'TV 및 입력 제한 기기' 클라이언트 유형을 사용합니다.

호스팅 간 상호작용에는 서비스 계정을 사용합니다.

정보
채팅
API

데이터 액세스 – Google 인증 | Google Developer Program | OAuth 2.0을 사용하여 Google

developers.google.com/identity/protocols/oauth2?hl=ko&gl=1*czlb3l*_up*MQ.*_ga*MTU2OTY1NzYxNS4xNzU4MDk3MjA4*_ga_SM8HXJ53K2*...

Google Identity Authentication 승인 교차 플랫폼 계정 간 보안 (RISC) 검색 / 한국어 정현

Google 계정 승인 Google Authorization API를 사용하기 위한 앱 인증 Google 계정 연결 리소스

필터

Google 계정 승인
개요
교차 클라이언트 ID
OAuth2 범위
OAuth 2.0 정책

앱 유형별 승인 고려사항
서버 측 웹 응용
(자바스크립트 웹 응용)
Android 앱
iOS 및 데스크톱 앱
TV 및 기기 응용
(서비스 계정)

승인 코드 샘플
Windows

2. Google 승인 서버에서 액세스 토큰을 가져옵니다.

애플리케이션이 Google API를 사용하여 비공개 데이터에 액세스하려면 해당 API에 대한 액세스 권한을 부여하는 액세스 토큰을 획득해야 합니다. 단일 액세스 토큰은 여러 API에 다양한 수준의 액세스 권한을 부여할 수 있습니다. `scope` 라는 변수 매개변수는 액세스 토큰이 허용하는 리소스 및 작업 집합을 제어합니다. 액세스 토큰 요청 중에 애플리케이션은 `scope` 매개변수에 하나 이상의 값을 전송합니다.

이 요청을 하는 방법은 여러 가지가 있으며, 빌드하는 애플리케이션의 유형에 따라 다릅니다. 예를 들어 JavaScript 애플리케이션은 브라우저 리디렉션 사용하여 Google에 액세스 토큰을 요청하는 반면 브라우저가 없는 기기에 설치된 애플리케이션은 웹 서비스 요청을 사용합니다. 요청하는 방법에 관한 자세한 내용은 [Scenarios](#) 및 각 앱 유형의 자세한 구현 가이드를 참고하세요.

일부 요청에는 사용자가 Google 계정으로 로그인하는 인증 단계가 필요합니다. 로그인한 후 사용자에게 애플리케이션에서 요청하는 하나 이상의 권한을 부여할지 묻습니다. 이 프로세스를 *사용자 동의*라고 합니다.

사용자가 하나 이상의 권한을 부여하면 Google 승인 서버는 애플리케이션에 액세스 토큰 (또는 애플리케이션이 액세스 토큰을 획득하는 데 사용할 수 있는 승인 코드)과 해당 토큰으로 부여된 액세스 범위 목록을 전송합니다. 사용자가 권한을 부여하지 않으면 서버에서 오류를 반환합니다.

일반적으로 액세스가 필요한 시점에 미리 요청하는 것보다 점진적으로 범위를 요청하는 것이 좋습니다. 예를 들어 캘린더에 일정을 저장하는 기능을 지원하려는 앱은 사용자가 '캘린더에 추가' 버튼을 누를 때까지 Google Calendar 액세스를 요청해서는 안 됩니다. [점진적 승인을](#) 참고하세요.

정보
채팅
API

The screenshot shows the Google Identity Developer Console interface. The left sidebar contains navigation links: 필터, Google 계정 승인, 개요 (highlighted), 교차 클라이언트 ID, OAuth2 범위, OAuth 2.0 정책, 앱 유형별 승인 고려사항, 서버 측 웹 응용 (with sub-link for 자바스크립트 웹 응용), Android 앱, iOS 및 데스크톱 앱, TV 및 기기 응용, (서비스 계정), 승인 코드 샘플, and Windows. The main content area displays two steps:

- 4. 액세스 토큰을 API에 전송합니다.**
애플리케이션이 액세스 토큰을 획득한 후에는 HTTP 승인 요청 헤더에서 토큰을 Google API로 전송합니다. 토큰을 URI 쿼리 문자열 매개변수로 전송할 수 있지만 URI 매개변수가 완전히 안전하지 않은 로그 파일에 저장될 수 있으므로 권장하지 않습니다. 또한 불필요한 URI 매개변수 이름을 만들지 않는 것이 좋은 REST 사례입니다.

액세스 토큰은 토큰 요청의 scope에 설명된 작업 및 리소스 집합에 대해서만 유효합니다. 예를 들어 Google Calendar API용 액세스 토큰이 발급된 경우 Google Contacts API에 대한 액세스 권한이 부여되지 않습니다. 하지만 유사한 작업을 위해 액세스 토큰을 Google Calendar API에 여러 번 전송할 수 있습니다.
- 5. 필요한 경우 액세스 토큰을 새로고침합니다.**
액세스 토큰의 수명은 제한되어 있습니다. 애플리케이션이 단일 액세스 토큰의 수명을 초과하는 Google API에 액세스해야 하는 경우 갱신 토큰을 획득할 수 있습니다. 갱신 토큰을 사용하면 애플리케이션에서 새 액세스 토큰을 획득할 수 있습니다.

A blue callout box contains a star icon and the following text:
★ 참고: 갱신 토큰을 안전한 장기 저장소에 저장하고 유효한 동안 계속 사용하세요. 제한은 클라이언트-사용자 조합별로 발급되는 갱신 토큰 수와 모든 클라이언트에서 사용자별로 발급되는 갱신 토큰 수에 적용되며 이러한 제한은 서로 다릅니다. 애플리케이션에서 한도를 초과하는 갱신 토큰을 요청하면 이전 갱신 토큰이 작동하지 않습니다.

At the bottom, the heading "시나리오" is visible.

Google Identity

- Authentication
- 승인
- 교차 플랫폼
- 더보기
- 검색
- 한국어
- 정현

웹 서버 애플리케이션

Google OAuth 2.0 엔드포인트는 PHP, Java, Go, Python, Ruby, ASP.NET과 같은 언어와 프레임워크를 사용하는 웹 서버 애플리케이션을 지원합니다.

승인 시퀀스는 애플리케이션이 브라우저를 Google URL로 리디렉션할 때 시작됩니다. URL에는 요청된 액세스 유형을 나타내는 쿼리 매개변수가 포함됩니다. Google은 사용자 인증, 세션 선택, 사용자 동의를 처리합니다. 결과는 승인 코드이며, 애플리케이션은 이를 액세스 토큰 및 갱신 토큰으로 교환할 수 있습니다.

애플리케이션은 나중에 사용할 수 있도록 갱신 토큰을 저장하고 액세스 토큰을 사용하여 Google API에 액세스해야 합니다. 액세스 토큰이 만료되면 애플리케이션은 갱신 토큰을 사용하여 새 토큰을 가져옵니다.

```

sequenceDiagram
    participant User
    participant App as Your App
    participant Servers as Google Servers

    App->>Servers: Request token
    Note over User: User login & consent
    Servers-->>App: Authorization code
    App->>Servers: Exchange code for token
    Servers-->>App: Token response
    App->>Servers: Use token to call Google API
  
```

The screenshot shows the Google Identity Developer Console interface. The left sidebar contains navigation links: 필터 (Filter), Google 계정 승인 (Google Account Approval), 개요 (Overview) - which is highlighted, 교차 클라이언트 ID (Cross-client ID), OAuth2 범위 (OAuth2 Scopes), OAuth 2.0 정책 (OAuth 2.0 Policy), 앱 유형별 승인 고려사항 (Approval considerations by app type), 서버 측 웹 응용 (Server-side web application), (자바스크립트 웹 응용) (JavaScript web application), Android 앱 (Android app), iOS 및 데스크톱 앱 (iOS and desktop apps), TV 및 기기 응용 (TV and device applications), (서비스 계정) (Service account), 승인 코드 샘플 (Approval code sample), and Windows.

The main content area displays the '설치된 애플리케이션' (Installed Applications) section under the '개요' (Overview) tab. It explains that Google OAuth 2.0 endpoints support computers, mobile devices, and other devices where applications are installed. It instructs users to use the [Google API Console](#) to create a client ID and select an application type (Android, Chrome app, iOS, Universal Windows Platform (UWP), or desktop app). It notes that the process generates a client ID and security key based on the application type and scope, which are then used to generate access tokens. A diagram illustrates the OAuth flow: a user logs in and consents to the app's request for an authorization code; the app then exchanges this code with Google servers to obtain an access token.

```

sequenceDiagram
    participant User
    participant App as Your App
    participant Servers as Google Servers
    Note over App: Request token
    App->>Servers: Request token
    Note over User: User login & consent
    User->>App: Authorization code
    Note over App: Exchange code for token
    App->>Servers: Exchange code for token
  
```


데이터 액세스 - Google 인증 | Google Developer Program | 웹 서버 애플리케이션용 OAuth

developers.google.com/identity/protocols/oauth2/web-server?hl=ko&gl=1*1spoo5v*_up*MQ*_ga*MTU2OTY1NzYxNS4xNzU4M...

Google Identity

Authentication | 승인 | 교차 플랫폼 | 더보기 | 검색 | 한국어 | 정현

필터

Google 계정 승인
개요
교차 클라이언트 ID
OAuth2 범위
OAuth 2.0 정책

앱 유형별 승인 고려사항
서버 측 웹 응용
(자바스크립트 웹 응용)
Android 앱
iOS 및 데스크톱 앱
TV 및 기기 응용
(서비스 계정)

승인 코드 샘플
Windows

참고: 구현을 올바르게 수행하는 것의 보안상 영향을 고려할 때 Google의 OAuth 2.0 엔드포인트와 상호작용할 때는 OAuth 2.0 라이브러리를 사용하는 것이 좋습니다. 다른 사용자가 제공하는 디버깅된 코드를 사용하는 것이 좋으며, 이는 개발자와 사용자를 보호하는 데 도움이 됩니다. 자세한 내용은 [클라이언트 라이브러리](#)를 참고하세요.

클라이언트 라이브러리

이 페이지의 언어별 예에서는 [Google API 클라이언트 라이브러리](#)를 사용하여 OAuth 2.0 승인을 구현합니다. 코드 샘플을 실행하려면 먼저 해당 언어용 클라이언트 라이브러리를 설치해야 합니다.

Google API 클라이언트 라이브러리를 사용하여 애플리케이션의 OAuth 2.0 흐름을 처리하면 클라이언트 라이브러리에서 애플리케이션이 직접 처리해야 하는 많은 작업을 실행합니다. 예를 들어 애플리케이션이 저장된 액세스 토큰을 사용하거나 새로고침할 수 있는 시점과 애플리케이션이 동의를 다시 획득해야 하는 시점을 결정합니다. 클라이언트 라이브러리는 올바른 리디렉션 URL을 생성하고 액세스 토큰에 대한 승인 코드를 교환하는 리디렉션 핸들러를 구현하는 데도 도움이 됩니다.

서버 측 애플리케이션용 Google API 클라이언트 라이브러리는 다음 언어로 제공됩니다.

- Go
- Java
- .NET
- Node.js
- Dart
- PHP

데이터 액세스 - Google 인증 | Google Developer Program | 교차 클라이언트 ID | Authori

developers.google.com/identity/protocols/oauth2/cross-client-identity?hl=ko&gl=1*1zds3jx*_up*MQ*_ga*MTU2OTY1NzYxNS4x...

Google Identity

Authentication | 승인 | 교차 플랫폼 | 더보기 | 검색 | 한국어 | 정현

Authorization

Google 계정 승인 | Google API 인증 | Google 계정 연결 | 리소스

Google 계정으로 로그인
패스키
사용자 인증 정보 공유
샘플

이 페이지는 [Cloud Translation API](#)를 통해 번역되었습니다. [Switch to English](#)

홈 > 제품 > Google Identity > 승인 > Authorization

도움이 되었나요? | 의견 보내기

교차 클라이언트 ID

이 페이지의 내용
교차 클라이언트 승인 목표
크로스 클라이언트 액세스 토큰

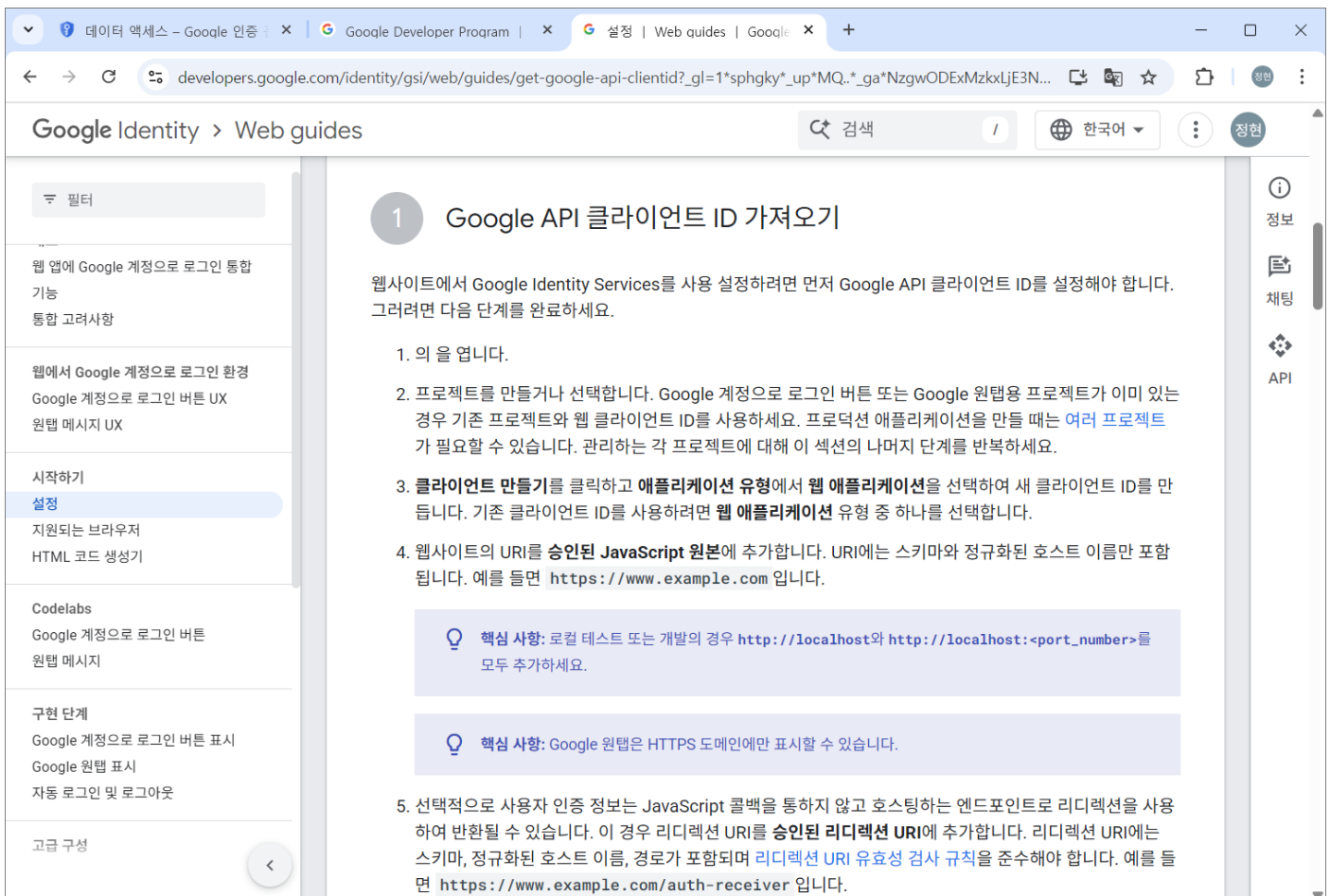
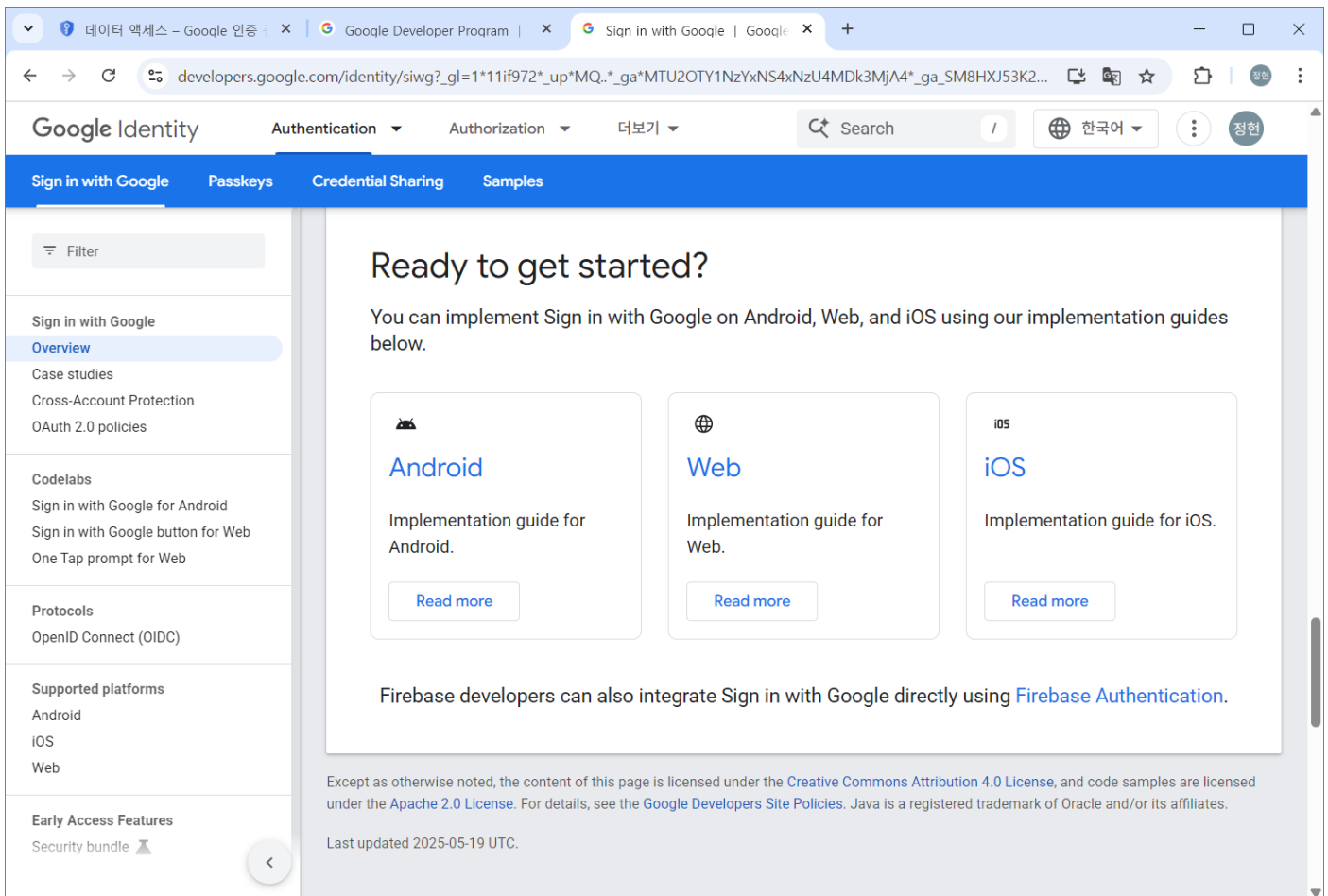
개발자가 소프트웨어를 빌드할 때 웹 서버에서 실행되는 모듈, 브라우저에서 실행되는 다른 모듈, Android 또는 iOS 모바일 앱으로 실행되는 모듈이 포함되는 경우가 많습니다. 개발자와 소프트웨어를 사용하는 사람은 일반적으로 이러한 모든 모듈을 단일 앱의 일부로 생각합니다.

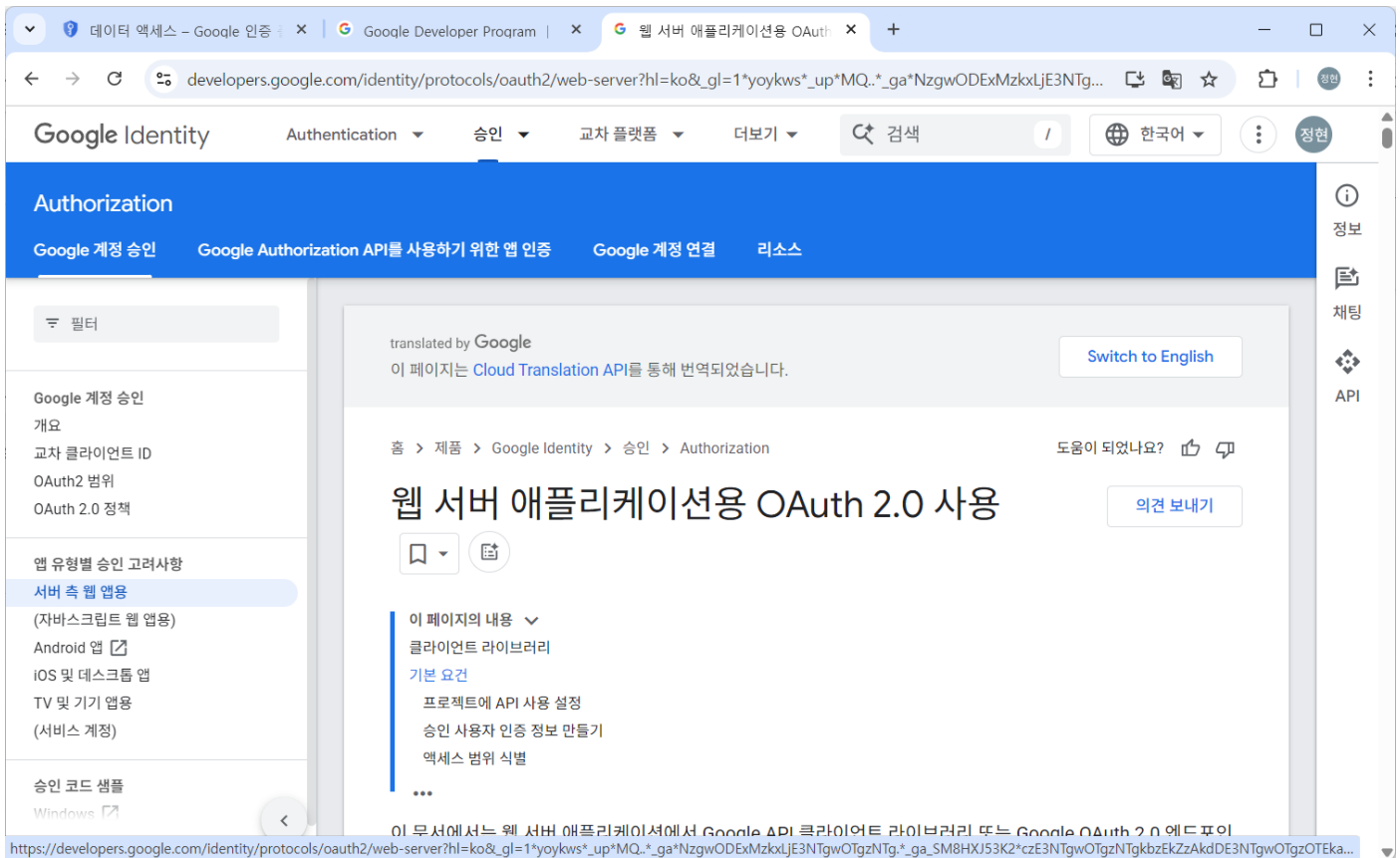
Google의 OAuth 2.0 구현은 이러한 관점을 지원합니다. OAuth2.0 기반 서비스를 사용하려면 [Google API Console](#)에서 소프트웨어를 설정해야 합니다. API Console의 조직 단위는 다중 구성요소 앱에 해당하는 '프로젝트'입니다. 각 프로젝트에 브랜딩 정보를 제공할 수 있으며 앱이 액세스할 API를 지정해야 합니다. 다중 구성요소 앱의 각 구성요소는 API Console에서 생성되는 고유한 문자열인 **클라이언트 ID**로 식별됩니다.

https://developers.google.com/identity/passkeys?hl=ko

The screenshot shows the Google Developers page for OAuth 2.0 and OpenID Connect (OIDC). The page is in Korean. The sidebar on the left contains a search bar and several menu items: 'Google로 로그인' (Sign in with Google), '개요' (Overview), '사례 연구' (Case Studies), '교차 계정 보호' (Cross-account protection), 'OAuth 2.0 정책' (OAuth 2.0 policy), '코덱스' (Code snippets), 'Android용 Google 로그인' (Google Sign-in for Android), '웹용 Google 버튼으로 로그인' (Sign in with Google button for web), '웹용 원터치 프로그래밍' (Web one-tap programming), '프로토콜' (Protocols), '오픈 ID 커넥트(OIDC)' (OpenID Connect), '지원되는 플랫폼' (Supported platforms), '기계적 인증 인자' (Machine authentication factors), '아이 오에스' (iOS), '핀들' (FIDO), '조기 액세스 기능' (Early access features), and '보안 번들' (Security bundles). The main content area has a heading '사용자들이 좋아하는 이유' (Reasons users love it) and four bullet points:

- 원터치 온보딩**: Users can quickly and easily sign in with their Google account without entering a password, as the onboarding process is simplified.
- 강화된 보안**: Users can protect their accounts by enabling two-step verification and using secure passwords.
- 향상된 개인정보 보호 및 신뢰**: Users can control their privacy settings and see what information is shared with third parties. Google also provides transparency about its data handling practices.
- 보편적인 가용성**: Google Sign-in works across all devices and platforms, making it easy for users to sign in wherever they are.





OAuth 2.0 액세스 토큰 가져오기

다음 단계에서는 애플리케이션이 Google의 OAuth 2.0 서버와 상호작용하여 사용자를 대신하여 API 요청을 실행하기 위한 사용자의 동의를 얻는 방법을 보여줍니다. 사용자 승인이 필요한 Google API 요청을 실행하려면 애플리케이션에 해당 동의가 있어야 합니다.

아래 목록은 이러한 단계를 간략하게 요약한 것입니다.

1. 애플리케이션은 필요한 권한을 식별합니다.
2. 애플리케이션은 요청된 권한 목록과 함께 사용자를 Google로 리디렉션합니다.
3. 사용자가 애플리케이션에 권한을 부여할지 결정합니다.
4. 애플리케이션은 사용자가 결정한 내용을 파악합니다.
5. 사용자가 요청된 권한을 부여한 경우 애플리케이션은 사용자를 대신하여 API 요청을 하는 데 필요한 토큰을 가져옵니다.

1단계: 승인 매개변수 설정

첫 번째 단계는 승인 요청을 만드는 것입니다. 이 요청은 애플리케이션을 식별하고 사용자에게 애플리케이션에 부여하도록 요청할 권한을 정의하는 매개변수를 설정합니다.

- OAuth 2.0 인증 및 승인에 Google 클라이언트 라이브러리를 사용하는 경우 이러한 매개변수를 정의하는 객체를 만들고 구성합니다.
- Google OAuth 2.0 엔드포인트를 직접 호출하면 URL이 생성되고 해당 URL에 매개변수가 설정

됩니다

PHP Python Ruby Node.js HTTP/REST

Google의 OAuth 2.0 엔드포인트는 `https://accounts.google.com/o/oauth2/v2/auth`에 있습니다. 이 엔드포인트는 HTTPS를 통해서만 액세스할 수 있습니다. 일반 HTTP 연결은 거부됩니다.

Google 승인 서버는 웹 서버 애플리케이션에 대해 다음 쿼리 문자열 매개변수를 지원합니다.

매개변수	
client_id	필수 애플리케이션의 클라이언트 ID입니다. 이 값은 Cloud Console Clients page에서 확인할 수 있습니다.
redirect_uri	필수 사용자가 승인 흐름을 완료한 후 API 서버가 사용자를 리디렉션하는 위치를 결정합니다. 이 값은 클라이언트의 Cloud Console Clients page에 구성된 OAuth 2.0 클라이언트의 승인된 리디렉션 URI 중 하나와 정확히 일치해야 합니다. 이 값이 제공된 client_id의 승인된 리디렉션 URI와 일치하지 않으면 <code>redirect_uri_mismatch</code> 오류가 발생합니다. <code>http</code> 또는 <code>https</code> 스키마, 케이스, 후행 슬래시('/')는 모두 일치해야 합니다.
response_type	필수 Google OAuth 2.0 엔드포인트가 승인 코드를 반환하는지 여부를 결정합니다. 웹 서버 애플리케이션의 경우 매개변수 값을 <code>code</code>로 설정합니다.
scope	필수 애플리케이션이 사용자를 대신하여 액세스할 수 있는 리소스를 식별하는 공백으로 구분된 범위 목록입니다. 이러한 값은 Google이 사용자에게 표시하는 동의 화면에 정보를 제공합니다. 범위를 사용 설정하면 애플리케이션은 필요한 리소스에 대한 액세스만 요청하고 사용자는 애플리케이션에 부여하는 액세스 양을 제어할 수 있습니다. 따라서 요청된 범위의 수와 사용자 동의를 얻을 가능성 사이에는 역관계가 있습니다. 가능한 경우 애플리케이션이 컨텍스트에서 승인 범위에 대한 액세스를 요청하는 것이 좋습니다. 점진적 승인을 통해 관련 맥락 안에서 사용자 데이터에 대한 액세스를 요청하면 사용자가 애플리케이션에서 요청하는 액세스가 필요한 이유를 더 쉽게 이해할 수 있습니다.

access_type	권장 사용자가 브라우저에 없을 때 애플리케이션이 액세스 토큰을 갱신할 수 있는지 여부를 나타냅니다. 유효한 매개변수 값은 online (기본값) 및 offline 입니다. 사용자가 브라우저에 없을 때 애플리케이션이 액세스 토큰을 새로고침해야 하는 경우 값을 offline 로 설정합니다. 이는 이 문서의 뒷부분에 설명된 액세스 토큰을 갱신하는 방법입니다. 이 값은 애플리케이션이 처음으로 승인 코드를 토큰으로 교환할 때 Google 승인 서버가 갱신 토큰과 액세스 토큰을 반환하도록 지시합니다.
state	권장 애플리케이션이 승인 요청과 승인 서버의 응답 간에 상태를 유지하는 데 사용하는 문자열 값을 지정합니다. 사용자가 애플리케이션의 액세스 요청에 동의하거나 거부하면 서버는 redirect_uri 의 URL 쿼리 구성요소 (?)에서 name=value 쌍으로 전송한 정확한 값을 반환합니다. 이 매개변수는 사용자를 애플리케이션의 올바른 리소스로 안내하고, nonce를 전송하고, 크로스 사이트 요청 위조를 완화하는 등 여러 용도로 사용할 수 있습니다. redirect_uri 는 추측할 수 있으므로 state 값을 사용하면 수신 연결이 인증 요청의 결과임을 더 확실하게 알 수 있습니다. 무작위 문자열을 생성하거나 쿠키 또는 클라이언트의 상태를 캡처하는 다른 값의 해시를 인코딩하는 경우 응답을 검증하여 요청과 응답이 동일한 브라우저에서 시작되었는지 추가로 확인할 수 있으므로 크로스 사이트 요청 위조 와 같은 공격으로부터 보호할 수 있습니다. state 토큰을 만들고 확인하는 방법의 예는 OpenID Connect 문서를 참고하세요. ★ 중요: OAuth 클라이언트는 OAuth2 사양에 명시된 대로 CSRF를 방지해야 합니다. 이를 달성하는 한 가지 방법은 state 매개변수를 사용하여 승인 요청과 승인 서버의 응답 간에 상태를 유지하는 것입니다.
include_granted_scopes	선택사항 애플리케이션이 단계적 승인을 사용하여 컨텍스트에서 추가 범위에 대한 액세스를 요청할 수 있도록 지원합니다. 이 매개변수의 값을 true 로 설정하고 승인 요청이 승인되면 새 액세스 토큰은 사용자가 이전에 애플리케이션 액세스를 승인한 범위에 포함합니다. 예시는 중분 승인 섹션을 참고하세요.

2단계: Google의 OAuth 2.0 서버로 리디렉션

사용자를 Google의 OAuth 2.0 서버로 리디렉션하여 인증 및 승인 프로세스를 시작합니다. 일반적으로 이 문제는 애플리케이션에서 사용자의 데이터에 처음 액세스해야 할 때 발생합니다. **중분 승인**의 경우 이 단계는 애플리케이션이 아직 액세스 권한이 없는 추가 리소스에 처음 액세스해야 할 때도 발생합니다.

PHP
Python
Ruby
Node.js
HTTP/REST

Google 승인 서버로 리디렉션하는 샘플

아래에는 가독성을 위해 줄바꿈과 공백이 있는 URL의 예가 나와 있습니다.

https://accounts.google.com/o/oauth2/v2/auth?scope=https%3A//www.googleapis.com/auth/drive.metadata.readonly%20https%3A//www.googleapis.com/auth/calendar.readonly&access_type=offline&include_granted_scopes=true&response_type=code&state=state_parameter_passthrough_value&redirect_uri=https%3A//oauth2.example.com/code&client_id=client_id

요청 URL을 만든 후 사용자를 해당 URL로 리디렉션합니다.

Google의 OAuth 2.0 서버는 사용자를 인증하고 애플리케이션이 요청된 범위에 액세스할 수 있도록 사용자 동의를 얻습니다. 지정한 리디렉션 URL을 사용하여 응답이 애플리케이션으로 다시 전송됩니다.

3단계: Google에서 사용자에게 동의를 요청함

이 단계에서 사용자는 애플리케이션에 요청된 액세스 권한을 부여할지 결정합니다. 이 단계에서 Google은 애플리케이션의 이름과 사용자의 승인 사용자 인증 정보에 대한 액세스 권한을 요청하는 Google API 서비스, 부여될 액세스 범위의 요약을 표시하는 동의 창을 표시합니다. 그러면 사용자는 애플리케이션에서 요청한 하나 이상의 범위에 대한 액세스 권한을 부여하는 데 동의하거나 요청을 거부할 수 있습니다.

4단계: OAuth 2.0 서버 응답 처리

중요: 서버에서 OAuth 2.0 응답을 처리하기 전에 Google에서 수신한 **state**가 승인 요청에서 전송된 **state**와 일치하는지 확인해야 합니다. 이 확인은 악의적인 스크립트가 아닌 사용자가 요청을 하고 있는지 확인하고 [CSRF 공격](#)의 위험을 줄이는 데 도움이 됩니다.

OAuth 2.0 서버는 요청에 지정된 URL을 사용하여 애플리케이션의 액세스 요청에 응답합니다.

승인 코드 응답:

```
https://oauth2.example.com/auth?code=4/P7q7W91a-oMsCeLvIaQm6bTrgtp7
```

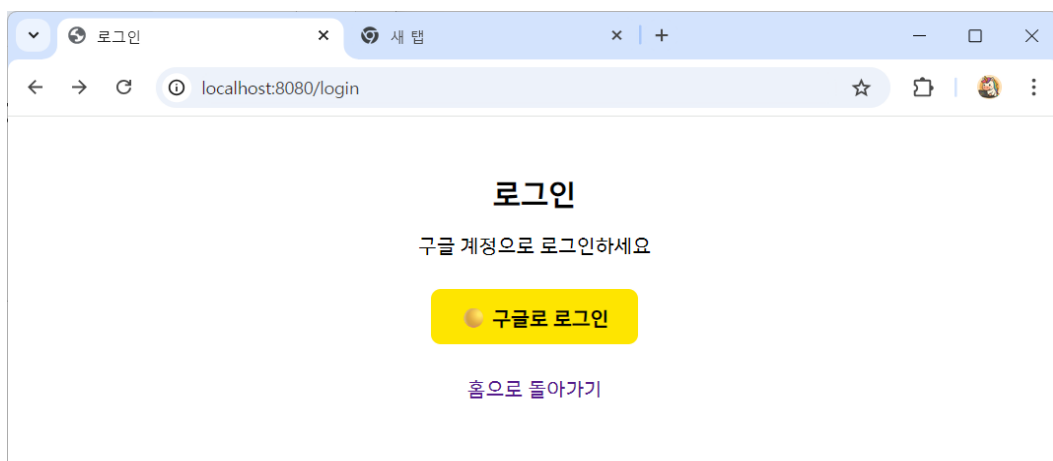
5단계: 갱신 및 액세스 토큰의 승인 코드 교환

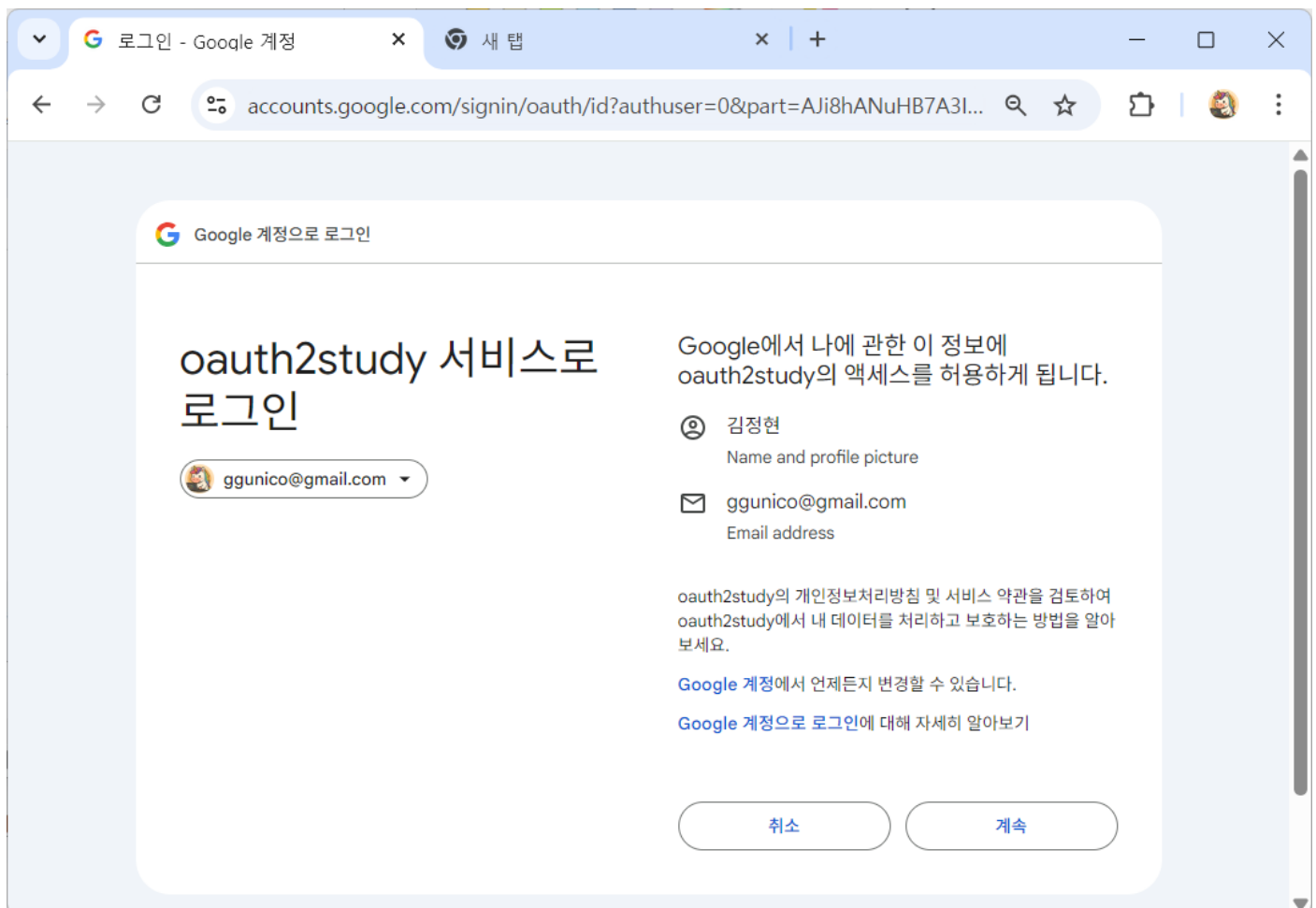
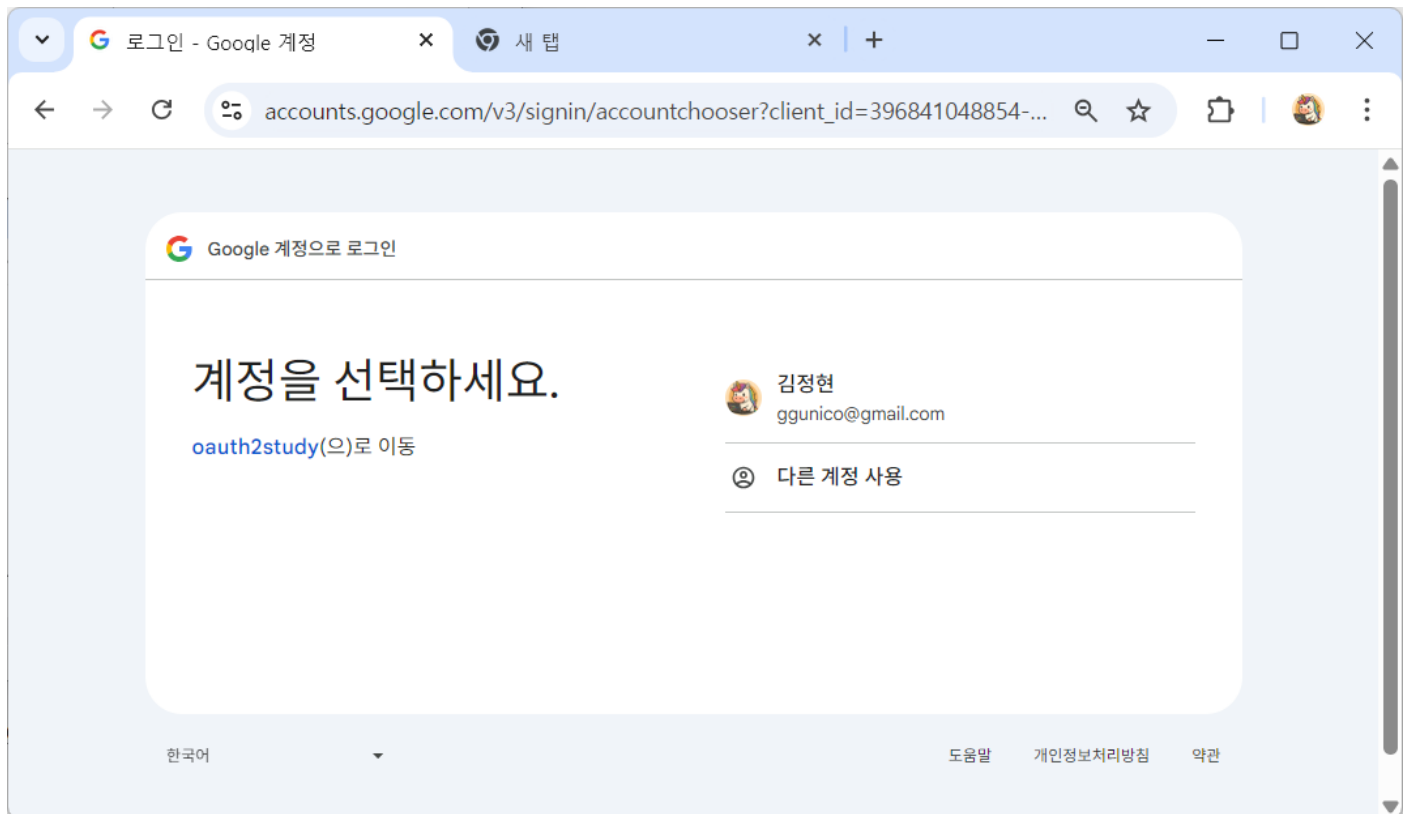
웹 서버는 승인 코드를 받은 후 승인 코드를 액세스 토큰으로 교환할 수 있습니다.

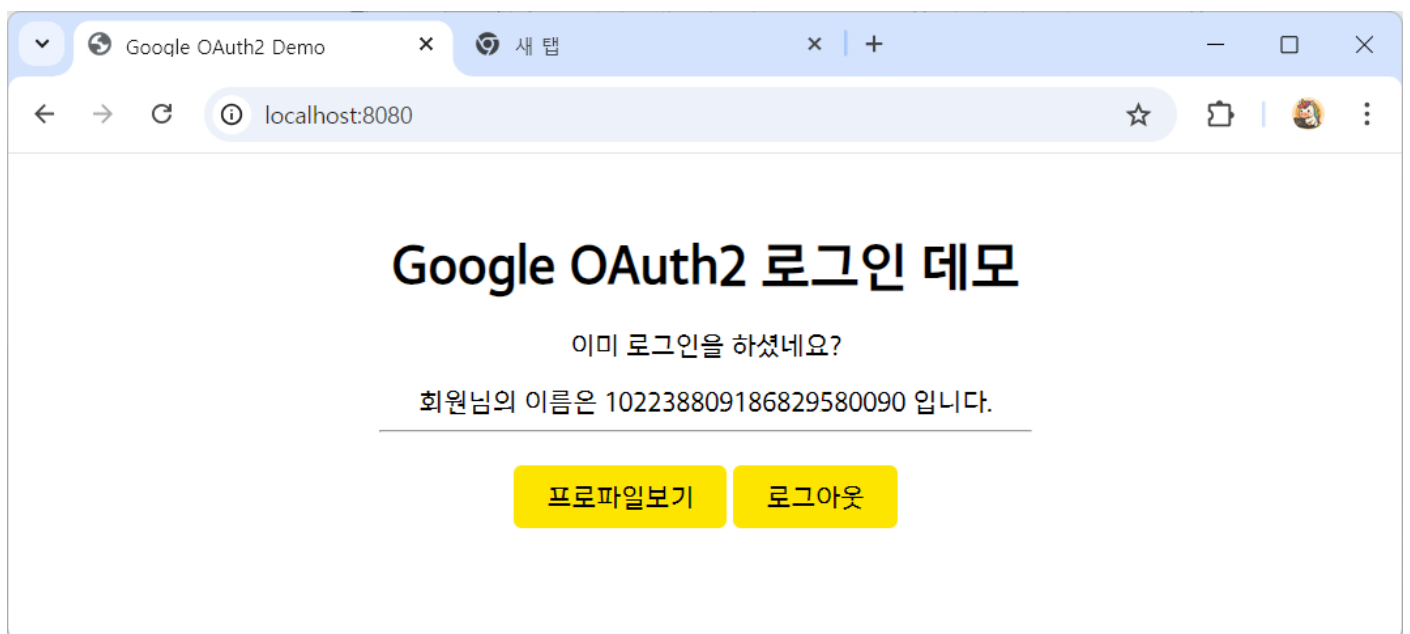
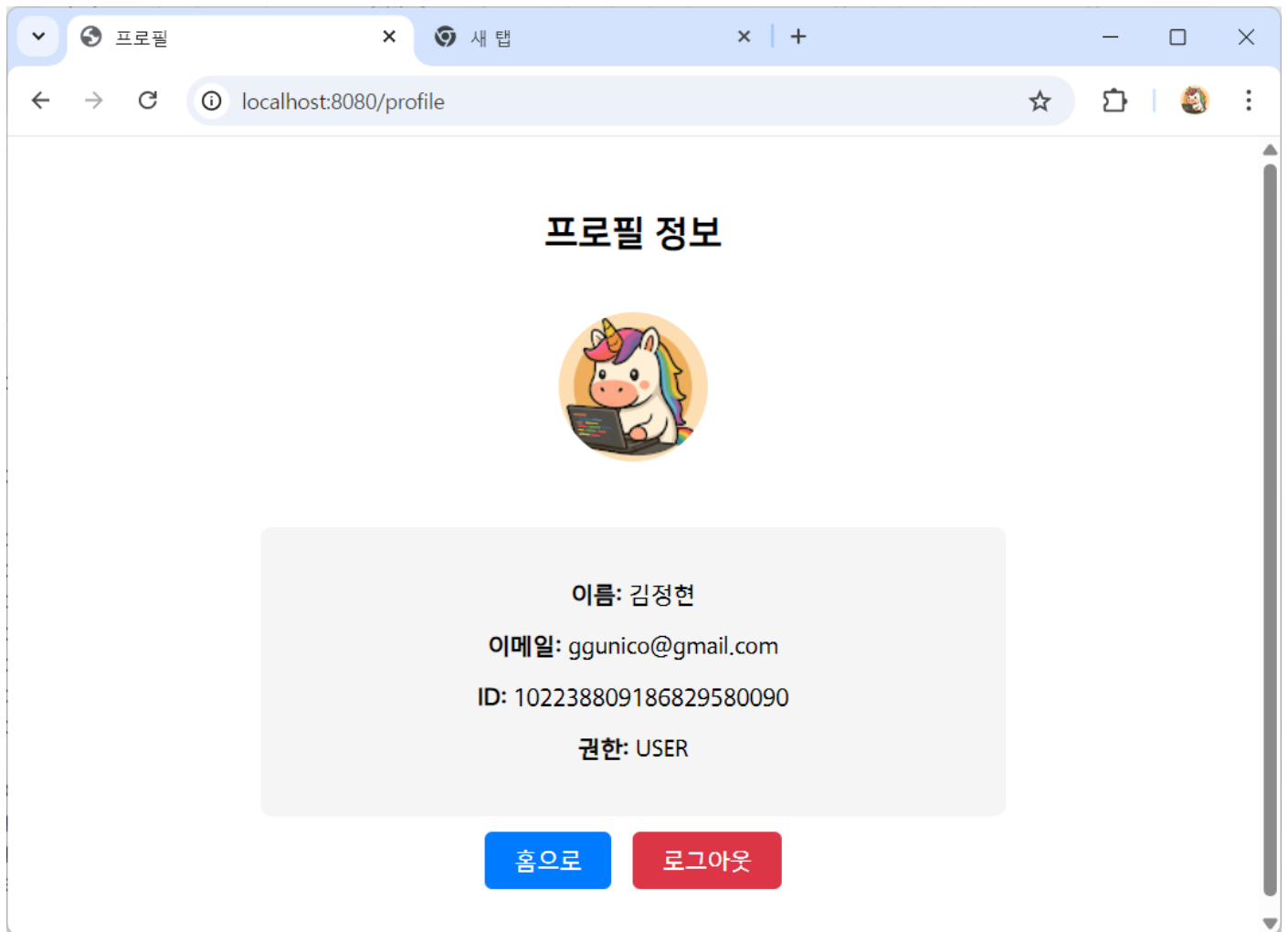
제공된 소스 중에서 oauth2google 를 인텔리제이로 오픈한 다음 내용을 본인의 client-id 와 client-secret 으로 수정한다.

```
application.yml  Oauth2googleApplication.java  index.html
1
2 server:
3   port: 8080
4
5 spring:
6   application:
7     name: oauth2google
8   security:
9     oauth2:
10      client:
11        registration:
12          google:
13            client-id: [REDACTED]
14            client-secret: [REDACTED]
15            scope:
16              - profile
17              - email
18            redirect-uri: http://localhost:8080/login/oauth2/code/google
19
20 datasource:
21   url: jdbc:h2:mem:testdb
```

oauth2google 를 기동시킨 후 다음과 같이 http://localhost:8080/ 을 요청해 본다.







The screenshot shows the H2 Console interface. On the left, a tree view displays the database structure: jdbc:h2:mem:testdb, USERS, EMAIL, ID, NAME, PROFILE_IMAGE_URL, ROLE, Indexes, INFORMATION_SCHEMA, and Users. The main area shows the SQL statement `SELECT * FROM USERS;` and its result. The result is a single row with the following data:

EMAIL	ID	NAME	PROFILE_IMAGE_URL	ROLE
ggunico@gmail.com	102238909186829580090	김정현	https://lh3.googleusercontent.com/a/ACg8oclxgRmb0Biz99CijGjfgX0mMSYOML2OA4IK3CLEIGcAChfAikIB=s96-c	USER

Below the table, it indicates "(1 row, 5 ms)".

[카카오, 네이버 그리고 구글을 모두 사용하는 소셜 로그인]

제공된 소스 중에서 oauth2sociallogin 을 인텔리제이로 오픈한 각 프로바이더별 client-id 와 client-secret 을 본인 것으로 수정한다.

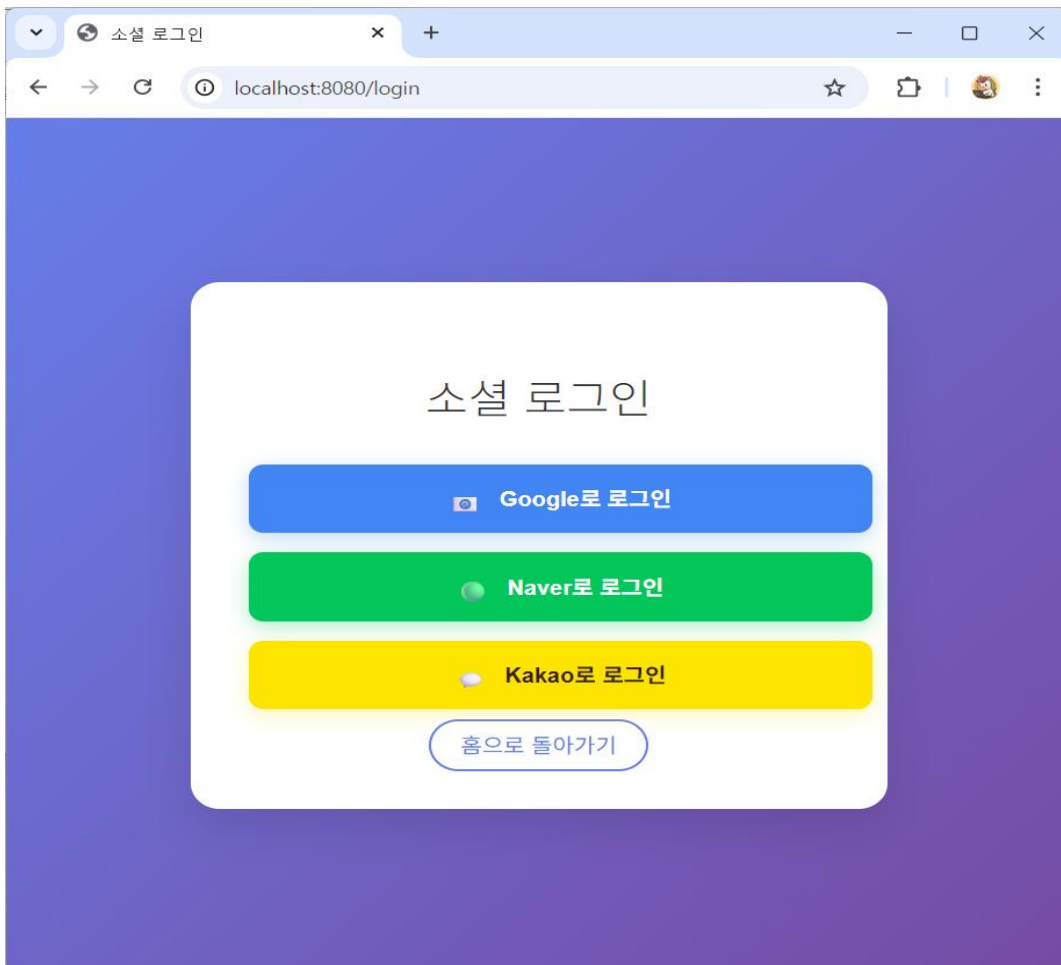
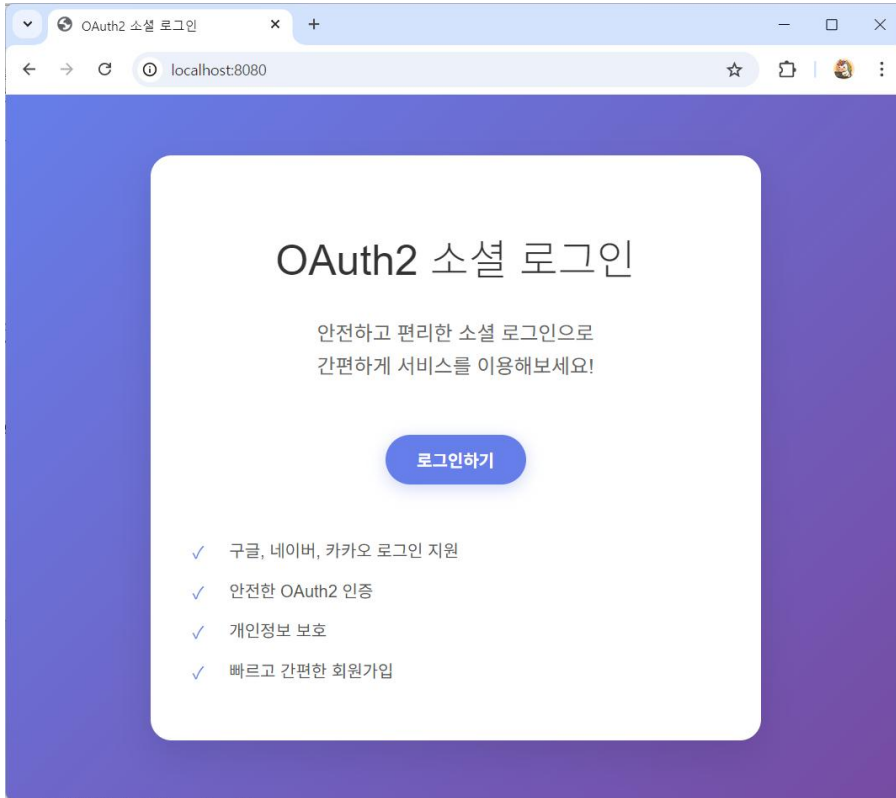
application.yml

```

1      spring:
2
3
4      security:
5
6      oauth2:
7
8      client:
9
10     registration:
11
12     google:
13
14     client-id: 102238909186829580090.apps.googleusercontent.com
15     client-secret: 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VY
16     scope:
17       - profile
18       - email
19     redirect-uri: http://localhost:8080/login/oauth2/code/google
20
21     naver:
22
23     client-id: K577VX8WV8VS0M510300
24     client-secret: 2h017g7070
25     scope:
26       - name
27       - email
28     client-name: Naver
29     authorization-grant-type: authorization_code
30     redirect-uri: http://localhost:8080/login/oauth2/code/naver
31
32     kakao:
33
34     client-id: 174781980101038030418211338024114901
35     client-secret: 2PVL7E3Y7G81231WMLJ5U166Y7E1M2ZML
36     scope:
37       - profile_nickname
38       - profile_image
39     client-name: kakao

```

oauth2sociallogin 을 기동시킨 후 다음과 같이 http://localhost:8080/ 을 요청해 본다.



[kakao, naver, google 의 OAuth2 기반의 소셜 로그인 정리]

1. 공통 구현 흐름 (OAuth2 Login standard flow)

OAuth2 Authorization Code Flow를 사용한다.

[전체 흐름 요약]

- (1) 사용자가 /oauth2/authorization/{provider} 로 접근
- (2) Spring Security가 Provider의 인증 페이지로 리디렉션
- (3) 로그인 성공 → Authorization Code 반환
- (4) 서버가 Code로 Token 요청 → Access Token 받음
- (5) Access Token으로 UserInfo API 호출
- (6) UserDetails 객체 생성 후 로그인 완료

Spring Security가 1~5 과정 대부분을 자동 처리해주기 때문에 Provider 설정만 잘 넣으면 된다.

2. Spring Boot 기본 소셜 로그인 URL

소셜 로그인	기본 로그인 트리거 URL
Google	/oauth2/authorization/google
Naver	/oauth2/authorization/naver
Kakao	/oauth2/authorization/kakao

브라우저에서 이 URL을 호출하면 각 서비스 로그인 페이지로 이동한다.

3. application.yml 설정

```
security:
  oauth2:
    client:
      registration:
        google:
          client-id: 수정
          client-secret: 수정
      scope:
        - profile
```

- email

redirect-uri: <http://localhost:8080/login/oauth2/code/google>

naver:

client-id: 수정

client-secret: 수정

scope:

- name

- email

client-name: Naver

authorization-grant-type: authorization_code

redirect-uri: <http://localhost:8080/login/oauth2/code/naver>

kakao:

client-id: 수정

client-secret: 수정

scope:

- profile_nickname

- profile_image

client-name: kakao

authorization-grant-type: authorization_code

redirect-uri: <http://localhost:8080/login/oauth2/code/kakao>

client-authentication-method: client_secret_post

provider:

naver:

authorization-uri: <https://nid.naver.com/oauth2.0/authorize>

token-uri: <https://nid.naver.com/oauth2.0/token>

user-info-uri: <https://openapi.naver.com/v1/nid/me>

user-name-attribute: response

kakao:

authorization-uri: <https://kauth.kakao.com/oauth/authorize>

token-uri: <https://kauth.kakao.com/oauth/token>

user-info-uri: <https://kapi.kakao.com/v2/user/me>

user-name-attribute: id

4. 주요 Endpoints 비교

Kakao

<https://kauth.kakao.com/oauth/authorize>

<https://kauth.kakao.com/oauth/token>

<https://kapi.kakao.com/v2/user/me>

Google

<https://accounts.google.com/o/oauth2/v2/auth>

<https://oauth2.googleapis.com/token>

<https://www.googleapis.com/oauth2/v3/userinfo>

Naver

<https://nid.naver.com/oauth2.0/authorize>

<https://nid.naver.com/oauth2.0/token>

<https://openapi.naver.com/v1/nid/me>

[OAuth2 기반의 소셜 로그인의 로그아웃 구현]

OAuth2 기반의 소셜로그인을 수행한 상태에서 로그아웃을 했을 때 **해당 SNS 사이트에서의 전역적 로그아웃을 할지** 아니면 **이 페이지(지역적)에서만 로그아웃을 할지**는 선택이며 가급적 지역적 로그아웃을 권장하고 있고 그 때는

```
.logout(logout -> logout
    .logoutUrl("/logout")    // 생략가능
    .logoutSuccessUrl("/")
    .invalidateHttpSession(true)
    .deleteCookies("JSESSIONID")
)
```

와 같이 구현(수업때 제공된 소스)해서 해당 페이지에서만 지역적으로 로그아웃 하도록 구현하는 것이 일반적이다. 그런데 전역적 로그아웃을 구현하고자 한다면 소셜 사이트마다 그 구현 방법이 다르며 2025년 9월에 기준으로 카카오, 네이버 그리고 구글에 올려진 문서를 참고하여 그 방법을 정리한다. 로그인과 로그아웃은 보안적으로도 중요한 기능이라서 구현 방법의 변경이 좀 일어나는 기능이라는 것을 고려하여 적용해야 한다.

(1) kakao

kakao 의 경우에는 먼저 카카오 개발자 사이트에 가서 로그아웃 리다이렉트 URI 를 등록해야 한다. 로그아웃후에 옮겨가게될 페이지의 URL 을 지정한다.

developers.kakao.com/console/app/1311821/product/login/advanced

kakao developers 앱 문서 도구 데보트

앱 > 제품 설정 > 카카오 로그인 > 고급

앱 설정
대시보드
앱
유료 API
제품 설정
카카오 로그인
일반
동의항목
간편가입
고급
비즈니스 인증
카카오톡 메시지
카카오맵
푸시 알림

APP ID 1311821 OAuth2EDU
역할 Owner

고급

카카오 로그인의 고급 설정을 확인하고 관리할 수 있습니다.

로그아웃 리다이렉트 URI [문서 확인](#)

사용자에게 [카카오계정과 함께 로그아웃] 기능을 제공하기 위해 필요한 로그아웃 리다이렉트 URI를 등록합니다.

리다이렉트 URI 등록

- 카카오 로그인에서 사용할 로그아웃 리다이렉트 URI를 설정합니다. (최대 10개)
- 서비스 로그아웃 시 카카오계정과 함께 로그아웃을 위한 스택으로 선택항목입니다.

리다이렉트 URI

리다이렉트 URI

http://localhost:8080/

로그아웃이 성공적으로 수행된 후에 리다이렉트 되는 페이지의 URL

- 카카오 로그인에서 사용할 로그아웃 리다이렉트 URI를 설정합니다.
- 여러개의 URI를 줄바꿈으로 추가해주세요. (최대 10개)
- REST API로 개발하는 경우 필수로 설정해야 합니다.
- 예시: https://example.com/oauth

취소 저장

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```

@Autowired
private CustomOAuth2UserService customOAuth2UserService;

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login", "/h2-console/**", "/favicon.ico").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(oauth2 -> oauth2
            .loginPage("/login")
            .defaultSuccessUrl("/profile", true)
            .userInfoEndpoint(userInfo -> userInfo
                .userService(customOAuth2UserService)
            )
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessHandler(logoutSuccessHandler())
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
        )
        .headers(headers -> headers
            .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)); // H2 콘솔
    return http.build();
}

@Bean
public LogoutSuccessHandler logoutSuccessHandler() {
    return (request, response, authentication) -> {
        String kakaoLogoutRedirectUri = "http://localhost:8080/";
        String kakaoLogoutUrl = "https://kauth.kakao.com/oauth/logout?client_id=수정" +
            "&logout_redirect_uri=" + URLEncoder.encode(kakaoLogoutRedirectUri, "UTF-8");
        response.sendRedirect(kakaoLogoutUrl);
    };
}
}

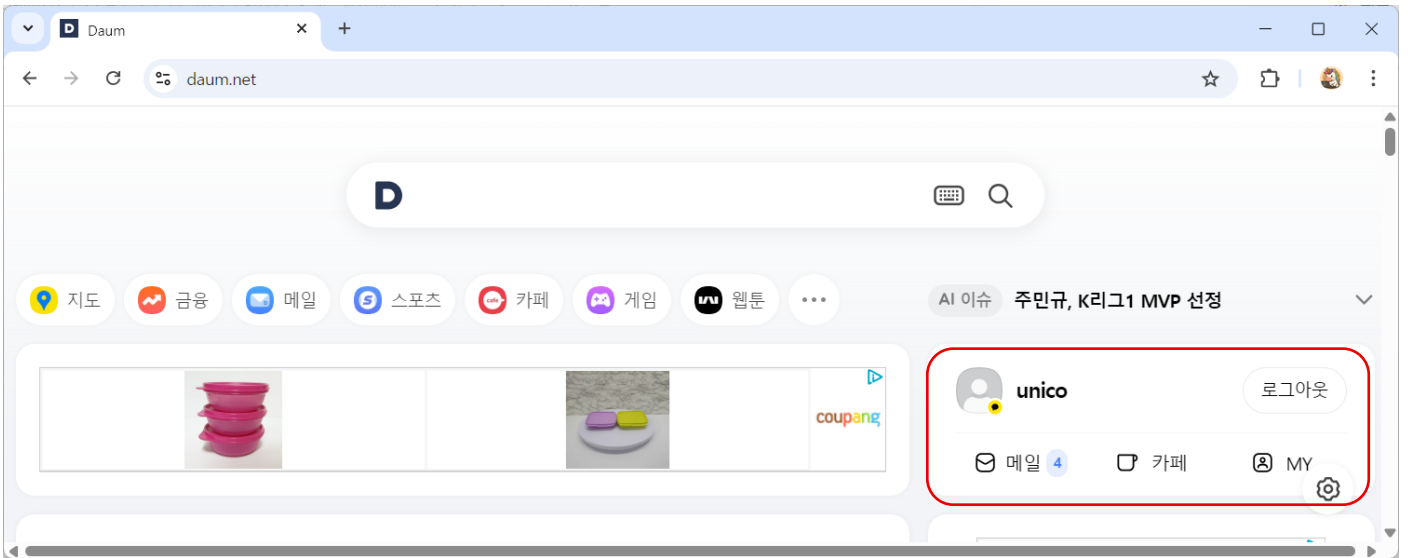
```

로그아웃이 성공적으로 수행된 후에
리다이렉트
되는 페이지의 URL

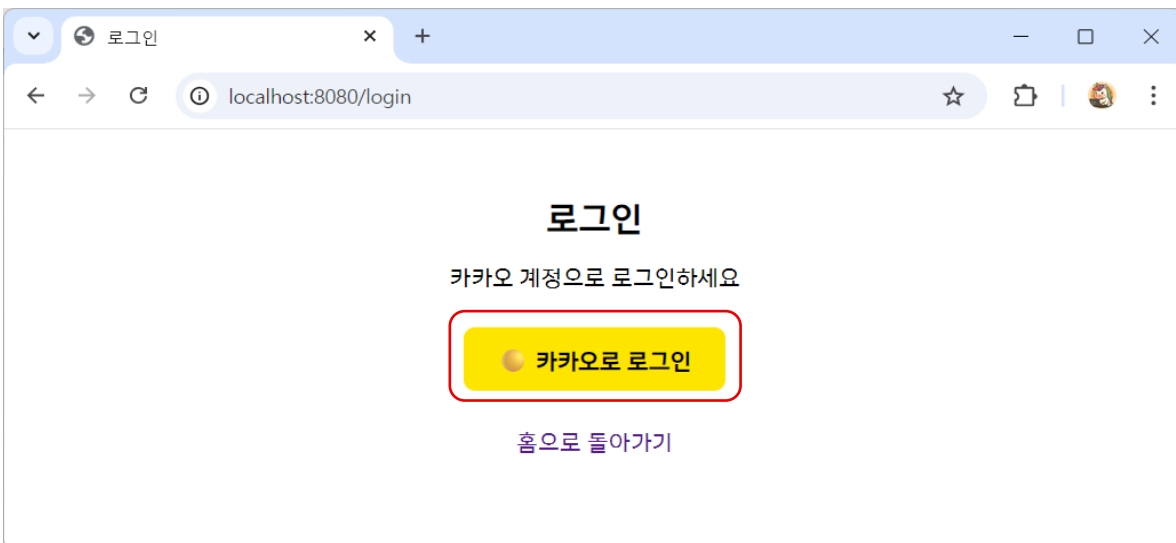
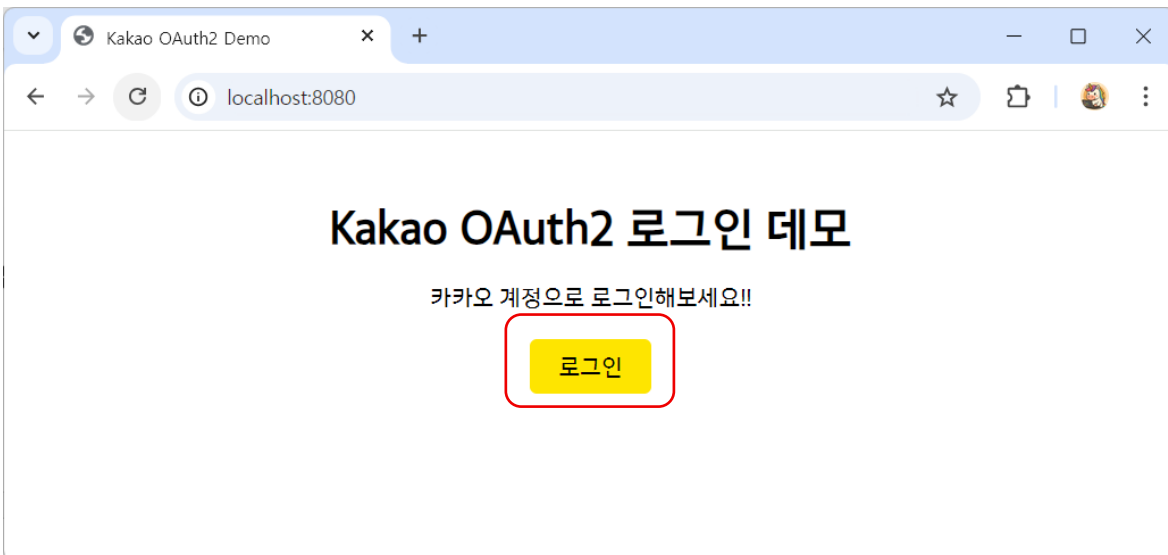


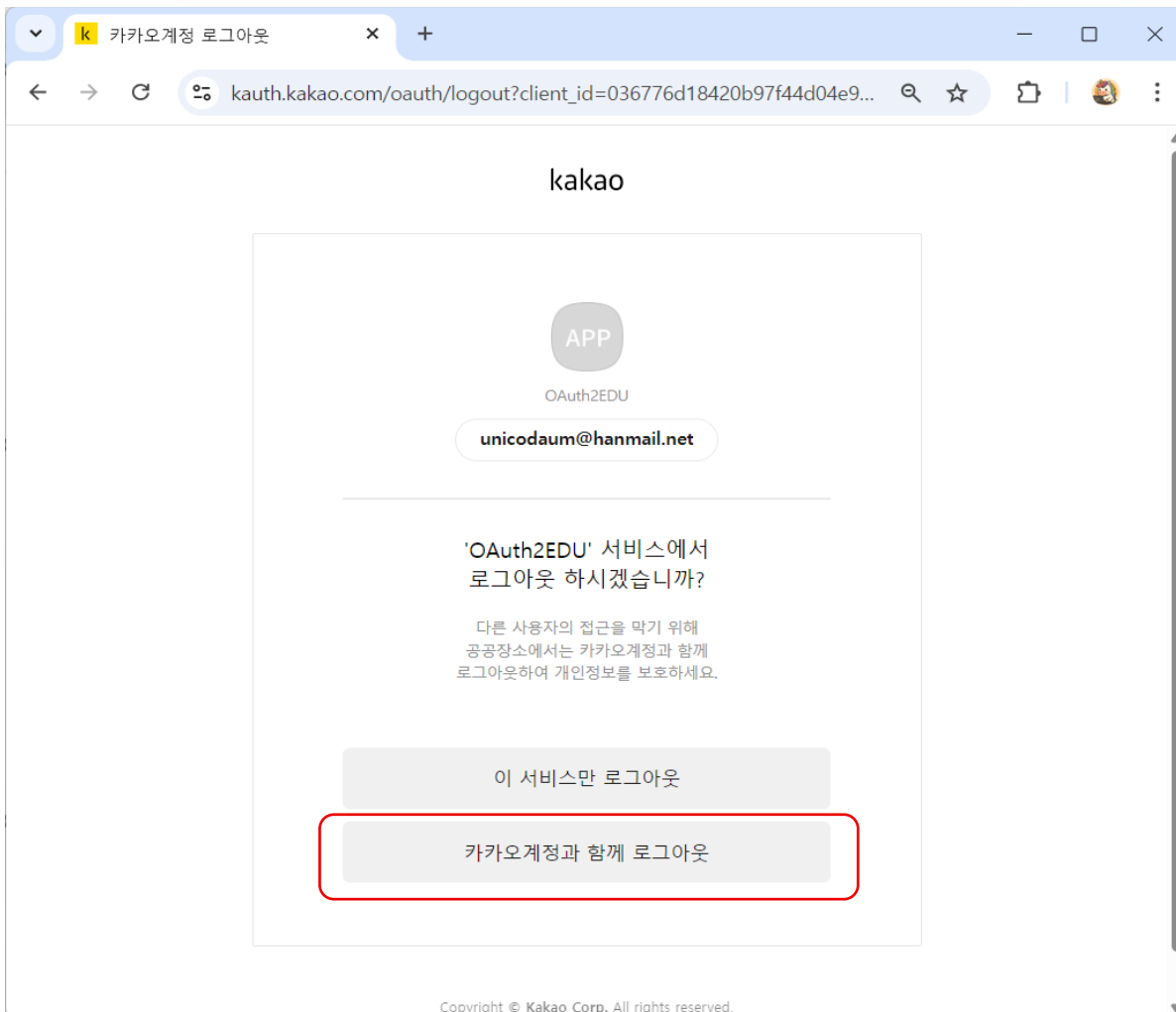
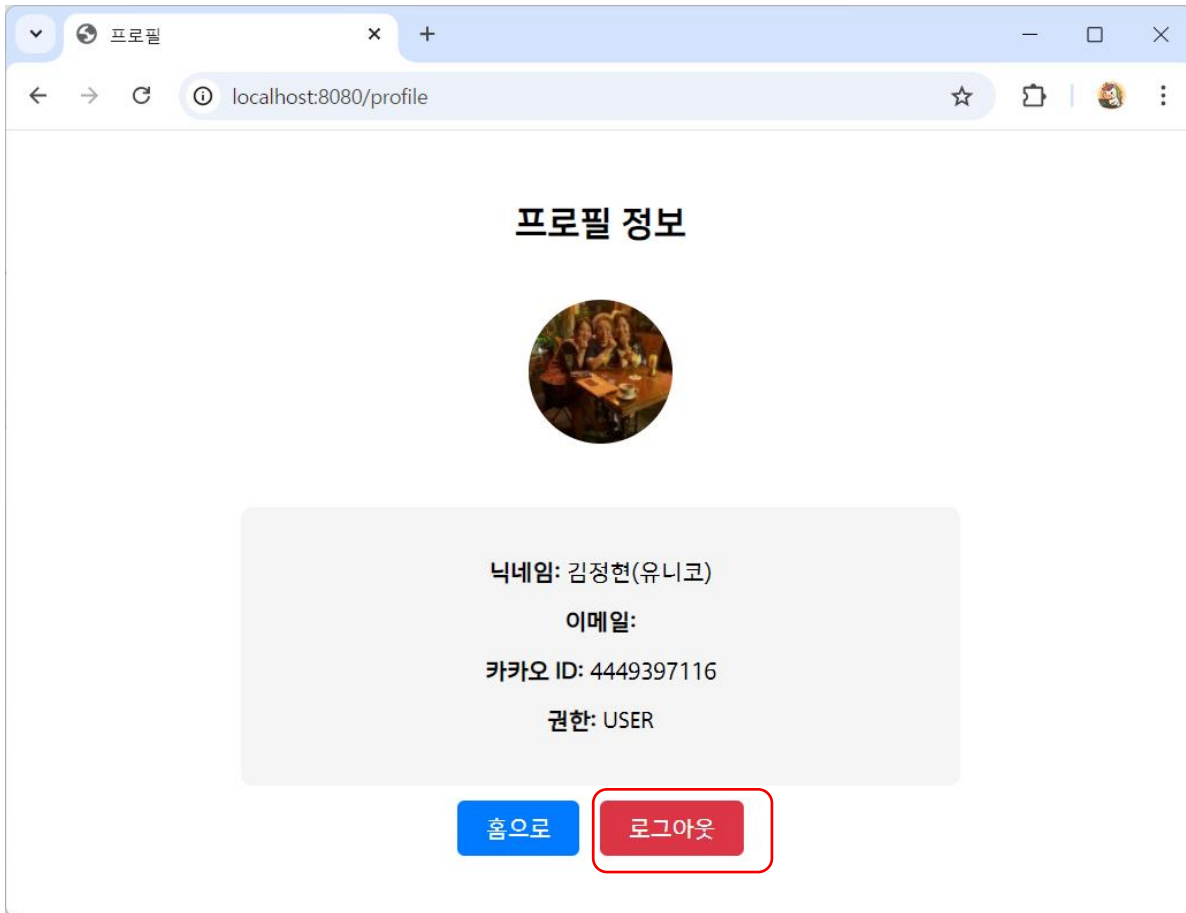
[테스트]

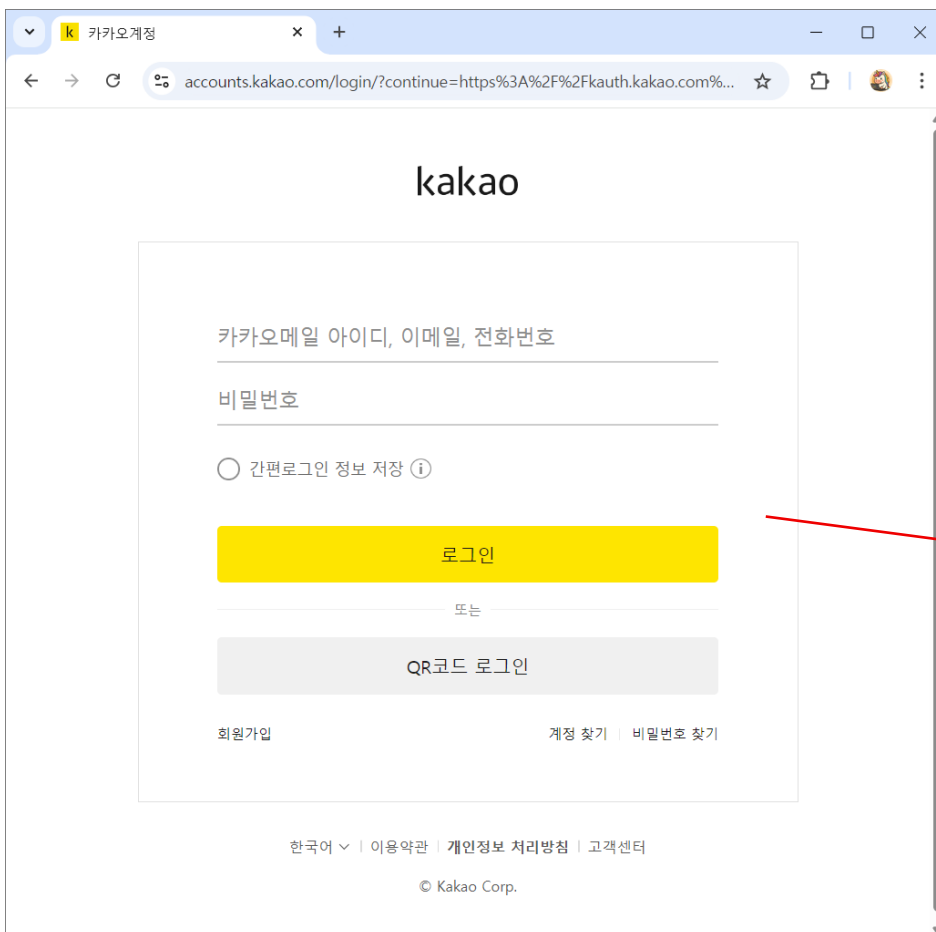
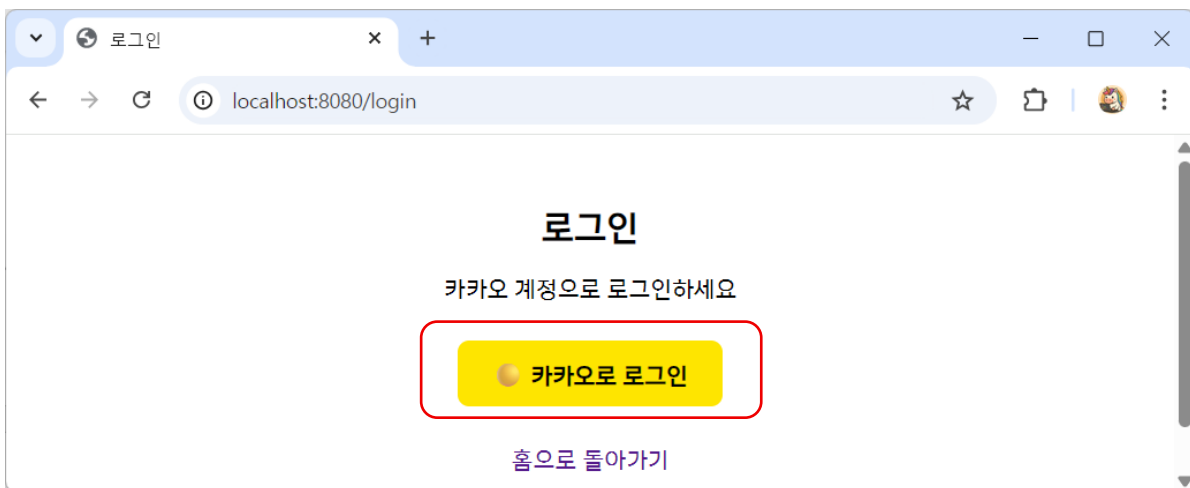
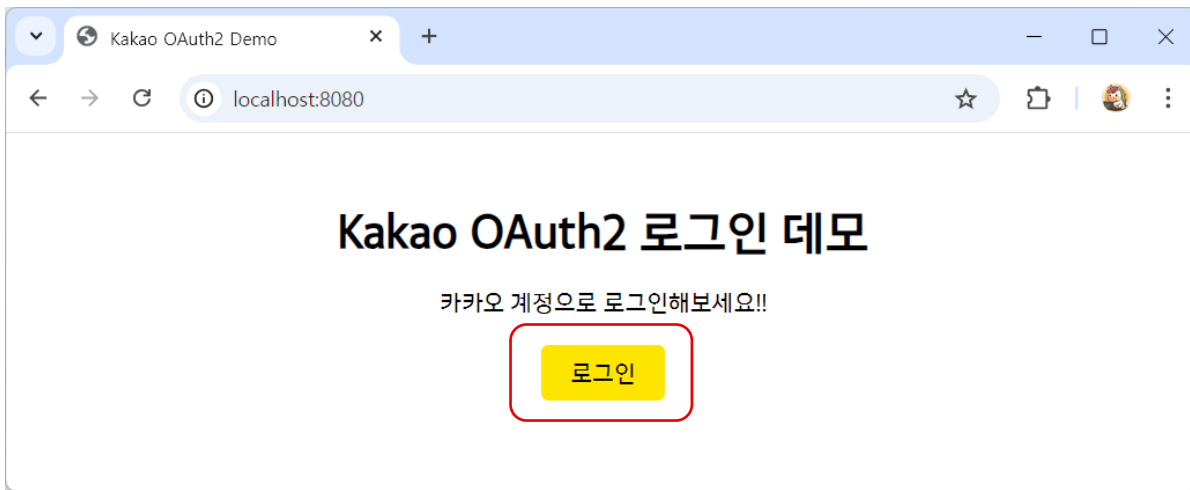
미리 다음(카카오)에 로그인된 상태



또 다른 탭에서 `http://localhost:8080/` 을 요청해 본다.

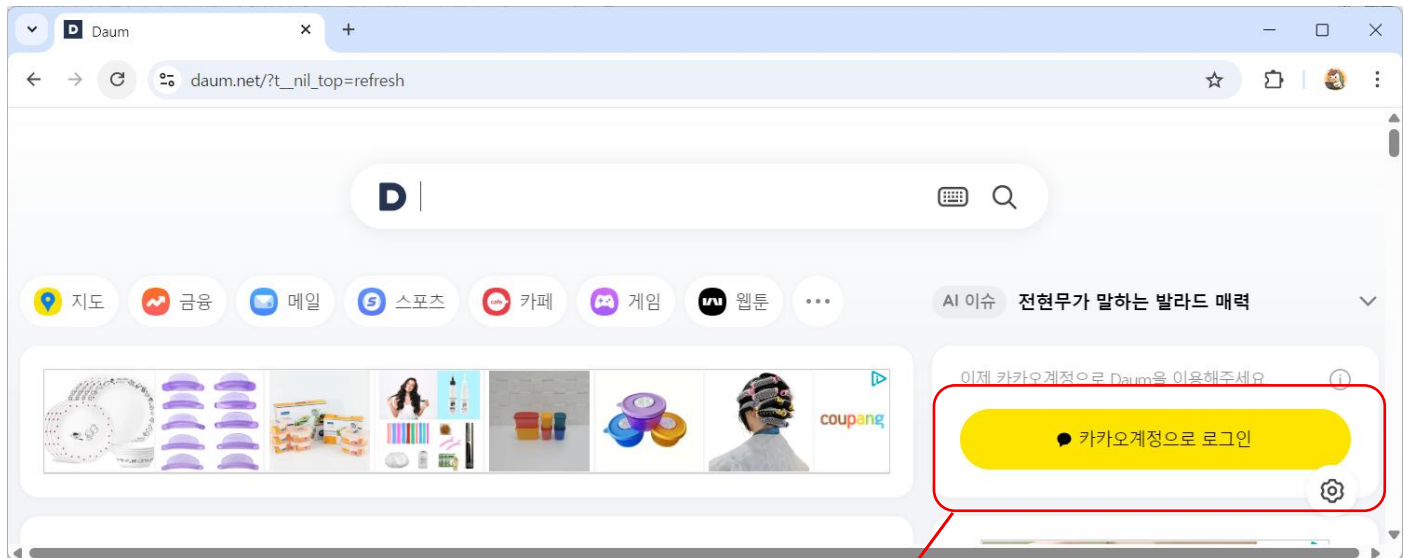






사용자 정보를 다시 입력받는
입력폼이 출력되는 것을
볼 수 있다.

다음에 로그인되어 있던 페이지에 가서 새로고침을 해보면 자동으로 로그아웃된 것을 확인할 수 있다.



다음에 로그인되어 있던 다른 페이지도 자동으로 로그아웃된 것을 확인할 수 있다.

(2) naver

네이버 OAuth2 로그아웃은 **access token** 삭제 API를 활용해 토큰을 무효화함으로써 완성된다. 추가적으로 세션과 쿠키도 같이 삭제하여, Spring Boot와 안전하게 연동할 수 있다. Spring Boot에서 네이버 OAuth2 로그아웃을 구현할 때는 네이버가 공식적으로 "로그아웃 URL"을 제공하지 않기 때문에, 네이버 access token을 삭제(무효화)하는 방식으로 로그아웃을 처리해야 한다.

네이버 토큰 삭제(로그아웃) API

엔드포인트: **<https://nid.naver.com/oauth2.0/token>**

주요 파라미터:

grant_type=delete (고정)

client_id=YOUR_CLIENT_ID

client_secret=YOUR_CLIENT_SECRET

access_token=USER_ACCESS_TOKEN

service_provider=NAVER (고정)

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Autowired

 private CustomOAuth2UserService customOAuth2UserService;

 @Bean

 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

 http

 .csrf(csrf -> csrf.disable())

 .authorizeHttpRequests(auth -> auth

 .requestMatchers("/", "/login", "/h2-console/**", "/favicon.ico").permitAll()

 .anyRequest().authenticated()

)

 .oauth2Login(oauth2 -> oauth2

 .loginPage("/login")

 .defaultSuccessUrl("/profile", true)

 .userInfoEndpoint(userInfo -> userInfo

 .userService(customOAuth2UserService)

)

)

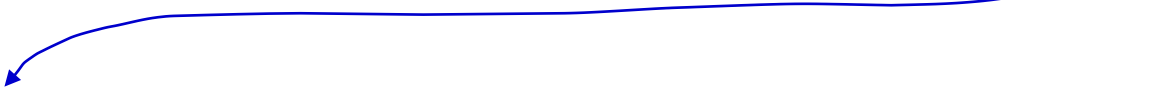
 .logout(logout -> logout

 .logoutUrl("/logout")

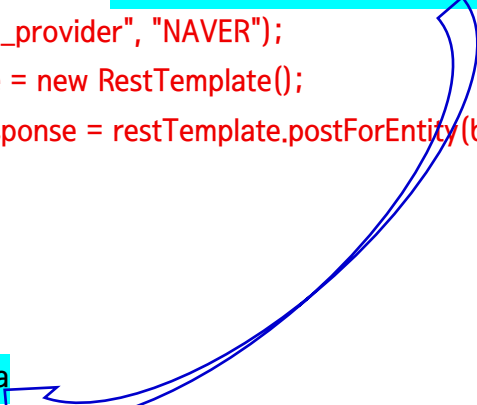
```
.logoutSuccessHandler((request, response, authentication) -> {
    naverLogout();
    response.sendRedirect("/"); // 로그아웃 직후 이동할 페이지
})
.invalidateHttpSession(true)
.deleteCookies("JSESSIONID")

)
.headers(headers -> headers
    .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)); // H2 콘솔

return http.build();
}
```



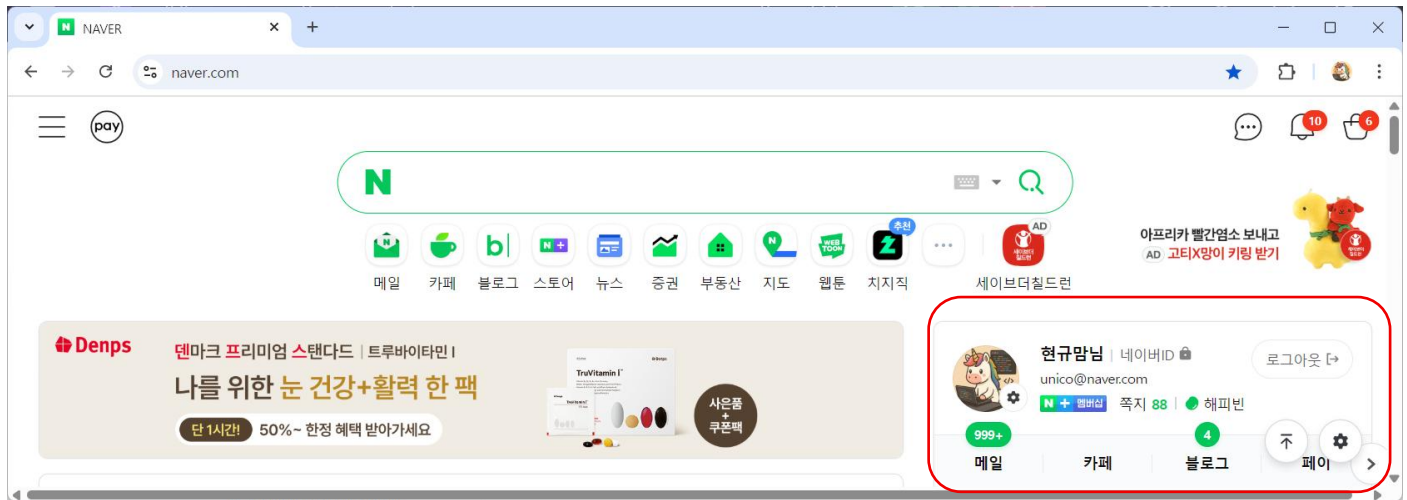
```
public void naverLogout() {
    String url = "https://nid.naver.com/oauth2.0/token";
    UriComponentsBuilder builder = UriComponentsBuilder.fromHttpUrl(url)
        .queryParams("grant_type", "delete")
        .queryParams("client_id", "수정")
        .queryParams("client_secret", "수정")
        .queryParams("access_token", customOAuth2UserService.getAccessToken())
        .queryParams("service_provider", "NAVER");
    RestTemplate restTemplate = new RestTemplate();
    ResponseEntity<String> response = restTemplate.postForEntity(builder.toUriString(), null,
String.class);
}
}
```



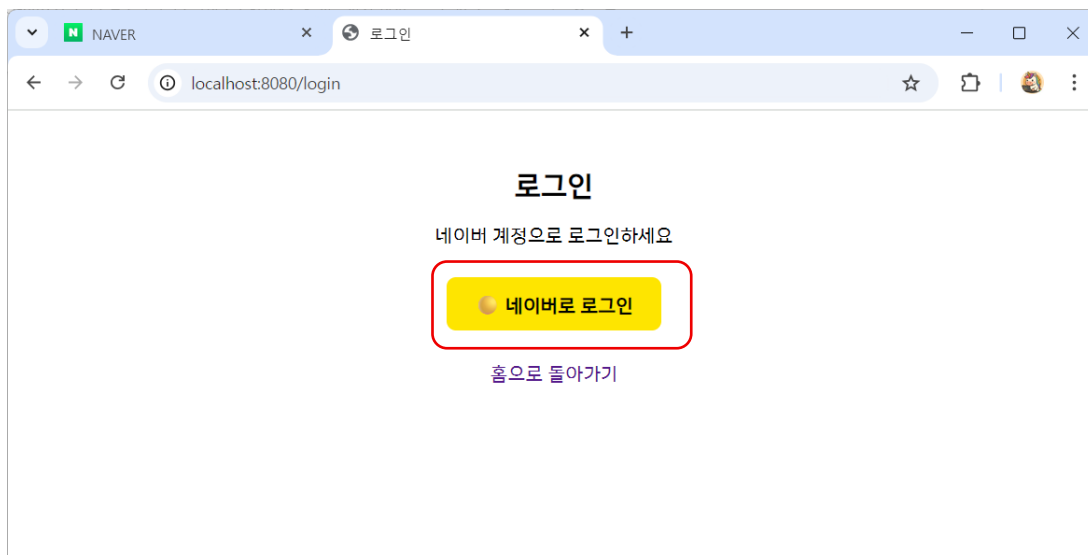
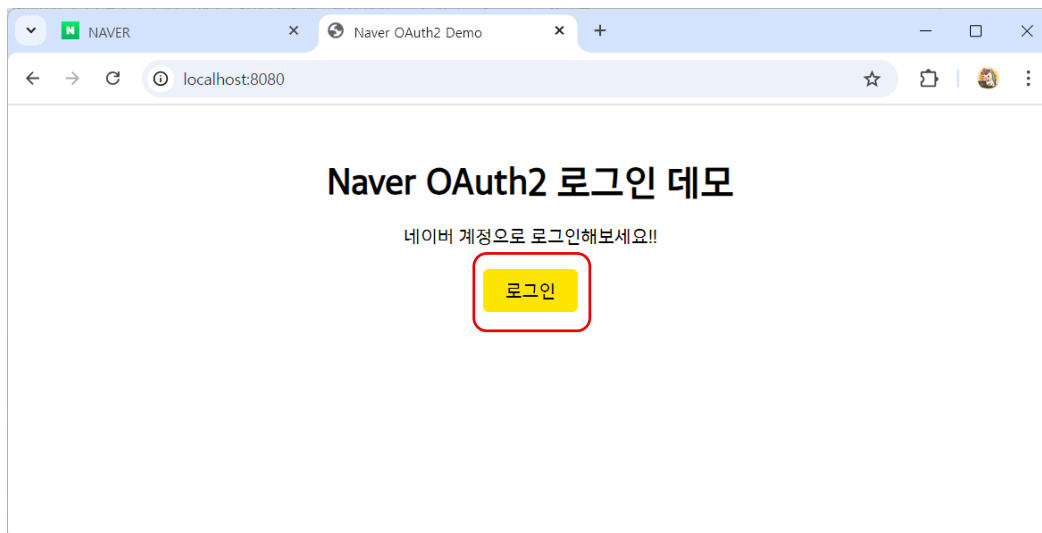
```
// CustomOAuth2UserService.java
public String getAccessToken() {
    return accessToken;
}
```

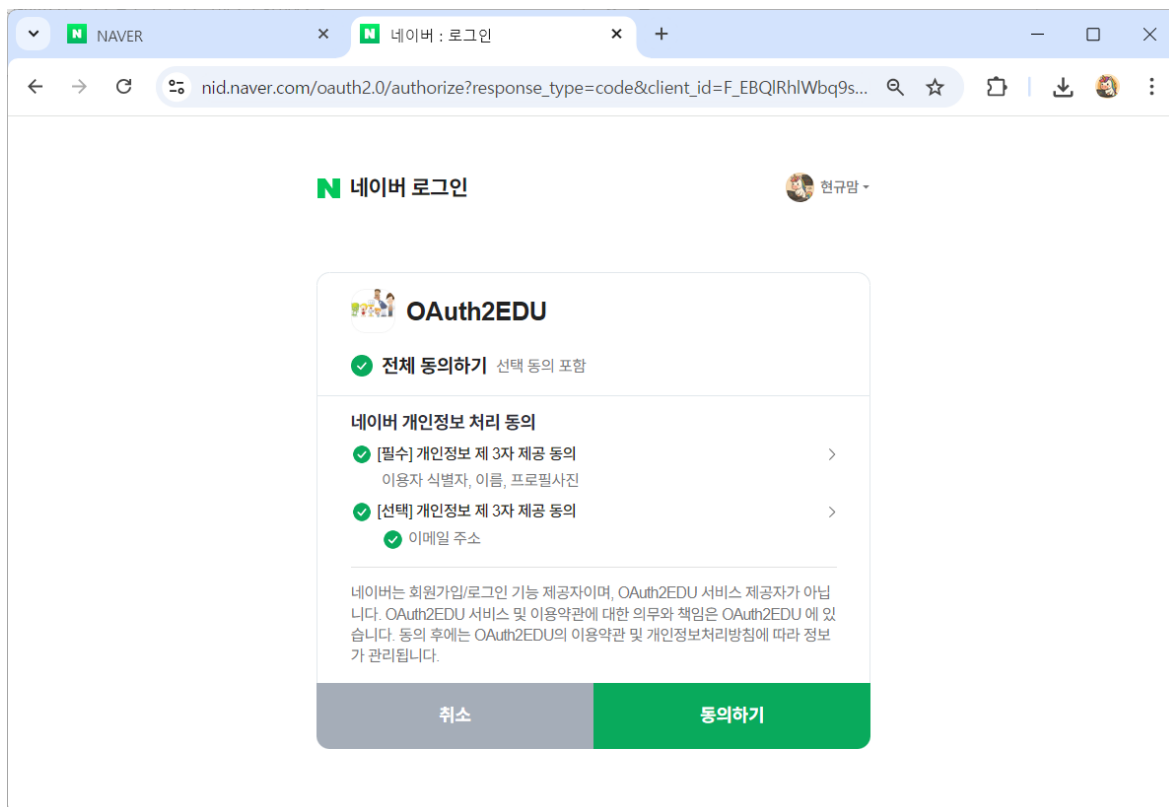
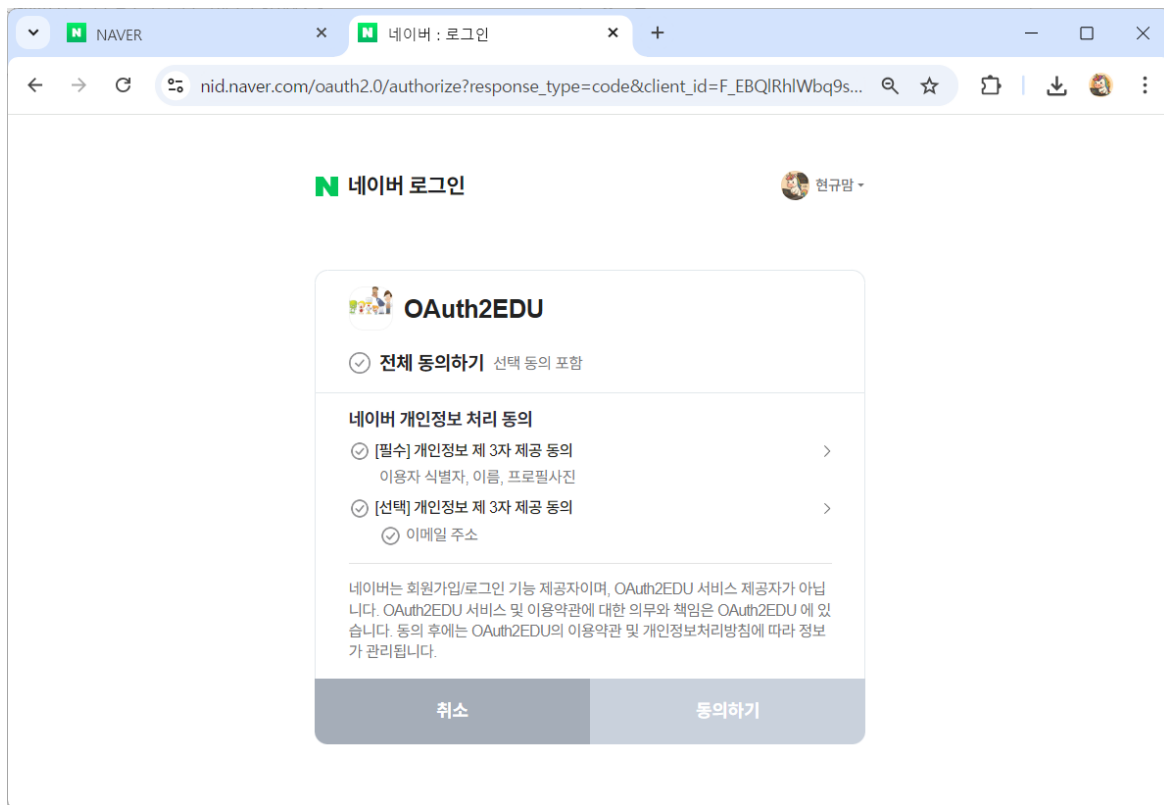

[테스트]

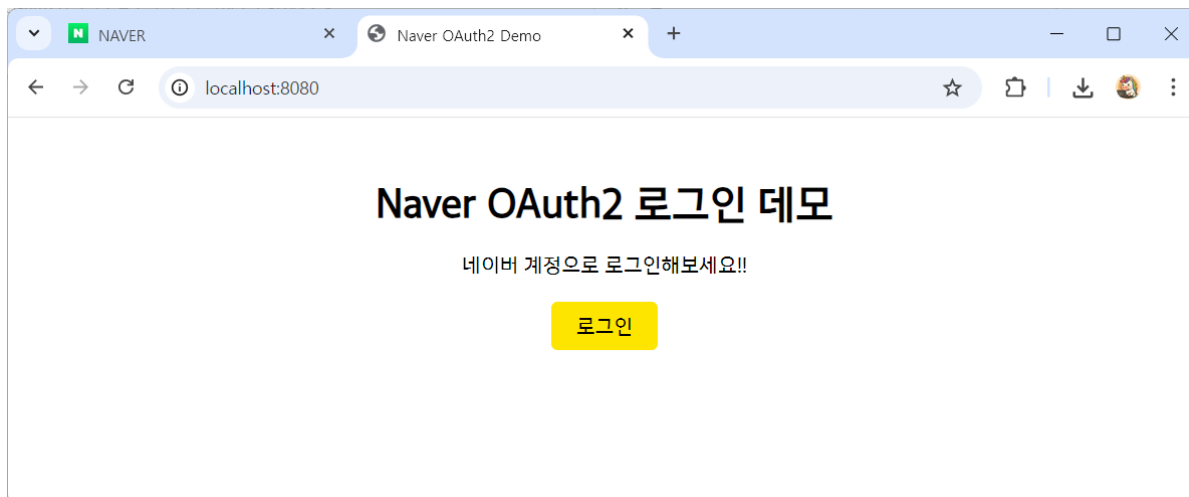
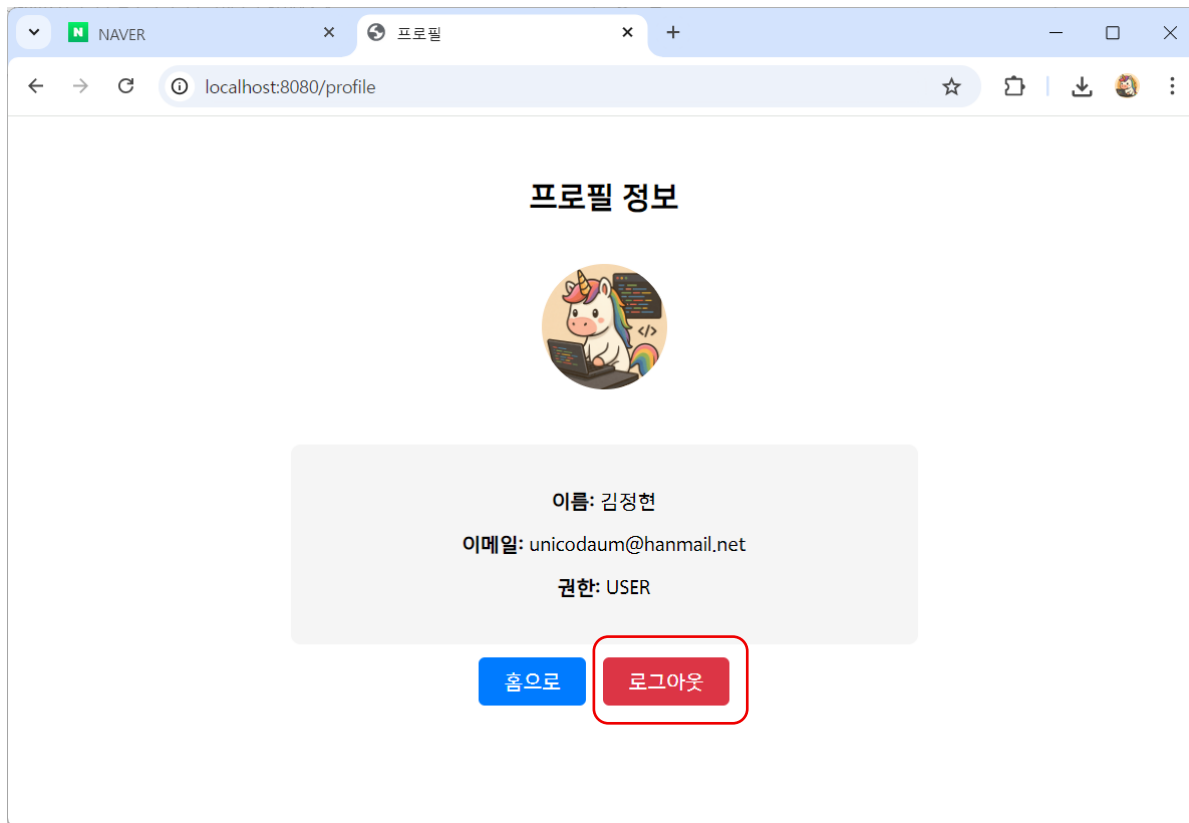
미리 로그인되어 있는 네이버 페이지



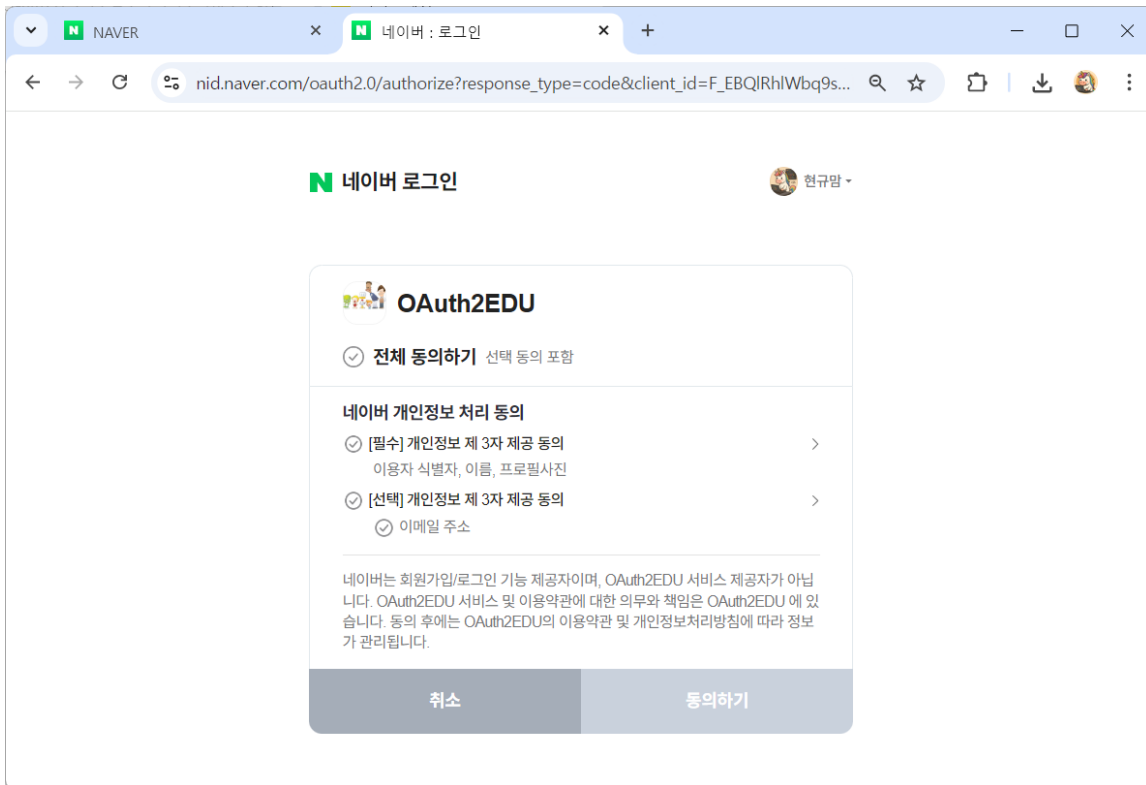
또 다른 탭에서 http://localhost:8080/ 을 요청해 본다.



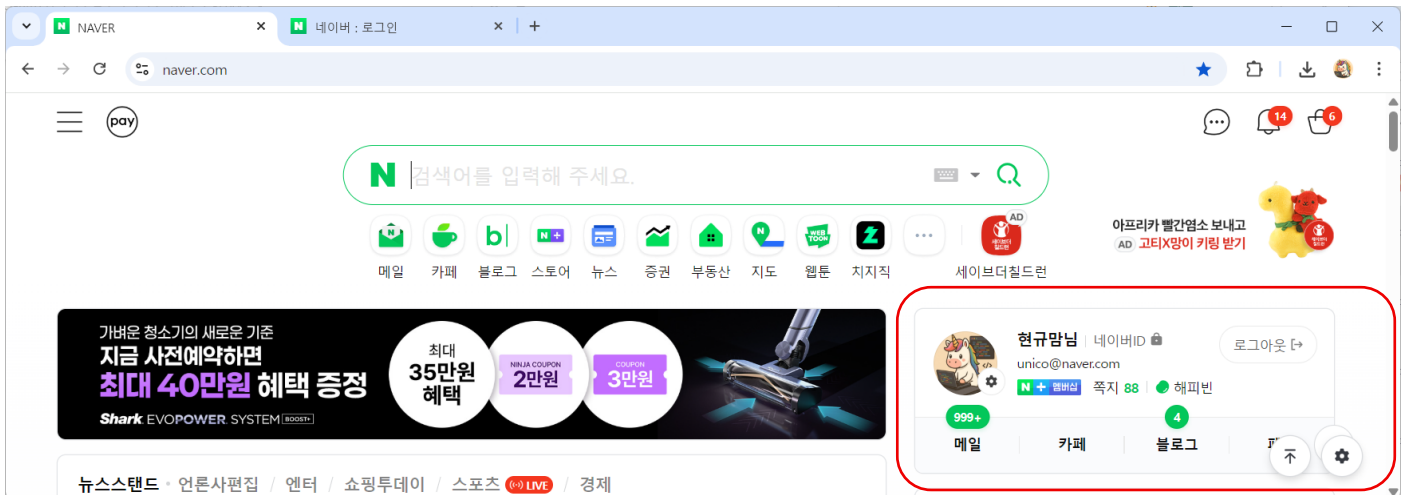




다시 네이버로그인을 선택하면 다음 화면이 출력되는 것을 볼 수 있다.



그러나 다음 카카오처럼 네이버에 로그인되어 있던 다른 페이지까지 자동으로 로그 아웃되는 것은 아니다.



(3) Google

Google OAuth2 로그아웃은 사용자가 Google 계정에서 로그아웃한 후 애플리케이션에서도 로그아웃하는 방식으로 구현해야 한다. 이를 위해 사용자를 Google의 로그아웃 URL로 리디렉션하여 Google 세션을 종료시킨 후, 애플리케이션의 자체적인 세션과 쿠키를 무효화하는 과정이 필요하다.

- Google 로그아웃 URL로 리디렉션

사용자가 애플리케이션에서 로그아웃할 때, Google의 공통 OAuth2 로그아웃을 위해 URL(<https://accounts.google.com/Logout>)로 리디렉션한다.

이때, 애플리케이션의 특정 URL로 리디렉션하도록 `post_logout_redirect_uri` 파라미터를 함께 전달할 수 있다. 이 파라미터는 Google 로그아웃이 완료된 후 사용자가 다시 돌아갈 애플리케이션의 페이지를 지정한다.

- 애플리케이션 세션/쿠키 무효화

사용자가 Google 로그아웃 페이지에 도달하기 전에, 또는 Google 로그아웃이 완료된 후 애플리케이션의 서버에서 해당 사용자의 세션 정보와 관련된 쿠키를 삭제하거나 무효화한다.

이렇게 하면 사용자가 다른 브라우저나 애플리케이션에서 Google 계정에 다시 로그인하더라도, 애플리케이션에서는 로그인이 되어 있지 않은 상태가 된다.

@Configuration

@EnableWebSecurity

```
public class SecurityConfig {
```

```
    @Autowired
```

```
    private CustomOAuth2UserService customOAuth2UserService;
```

```
    @Bean
```

```
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/", "/login", "/h2-console/**", "/favicon.ico").permitAll()
```

```
                .anyRequest().authenticated()
```

```
            )
```

```
            .oauth2Login(oauth2 -> oauth2
```

```
                .loginPage("/login")
```

```
                .defaultSuccessUrl("/profile", true)
```

```
                .userInfoEndpoint(userInfo -> userInfo
```

```

        .userService(customOAuth2UserService)
    )
)
.logout(logout -> logout
    .logoutUrl("/logout")
    .logoutSuccessHandler(logoutSuccessHandler())
    .invalidateHttpSession(true)
    .deleteCookies("JSESSIONID")
)
.headers(headers -> headers
    .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)); // H2 콘솔
return http.build();
}

```

@Bean

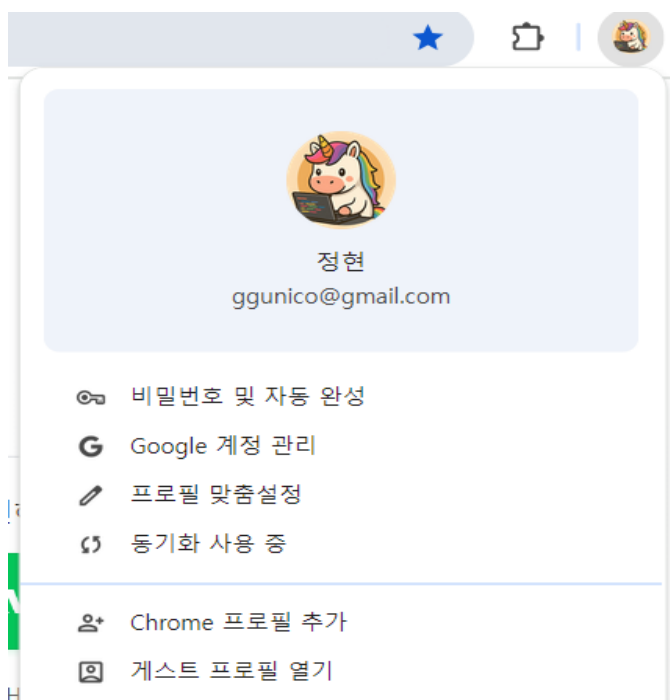
```

public LogoutSuccessHandler logoutSuccessHandler() {
    return (request, response, authentication) -> {
        String googleLogoutUrl =
            "https://accounts.google.com/Logout?post_logout_redirect_uri=http://localhost:8080/";
        response.sendRedirect(googleLogoutUrl);
    };
}
}
}

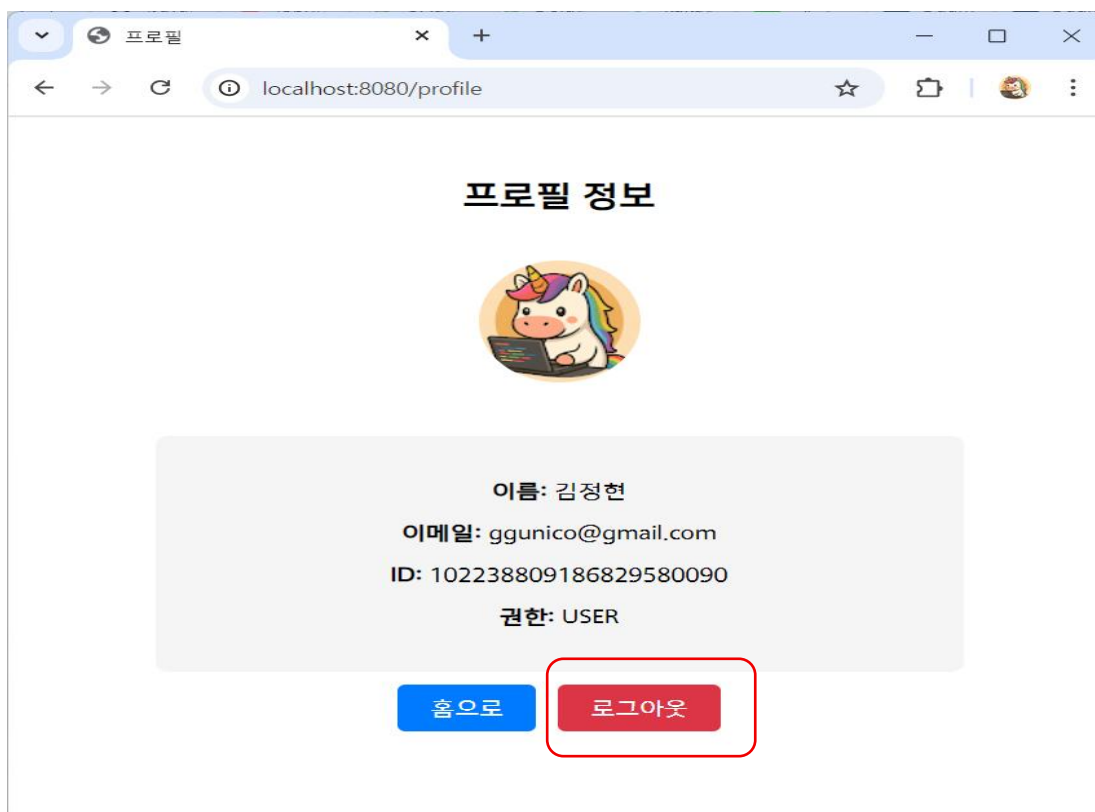
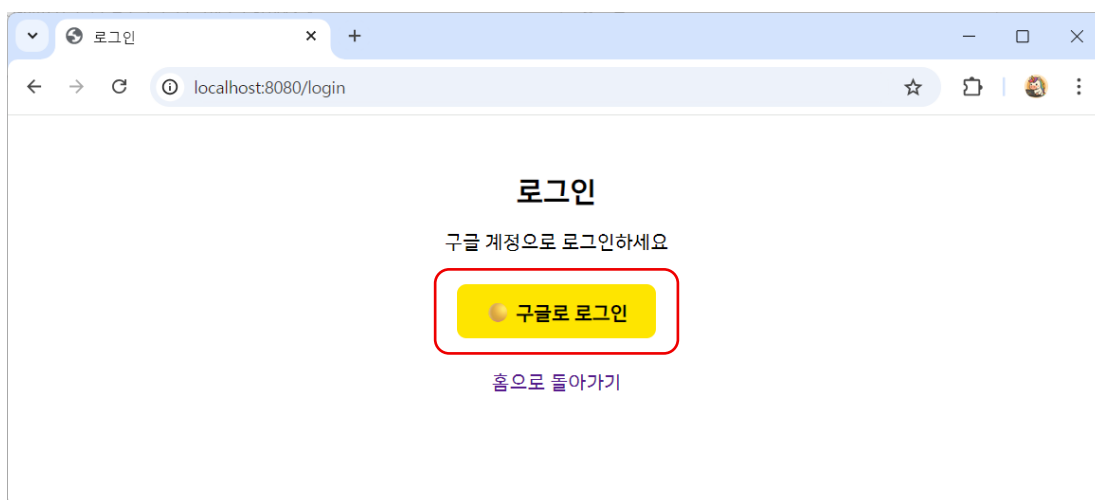
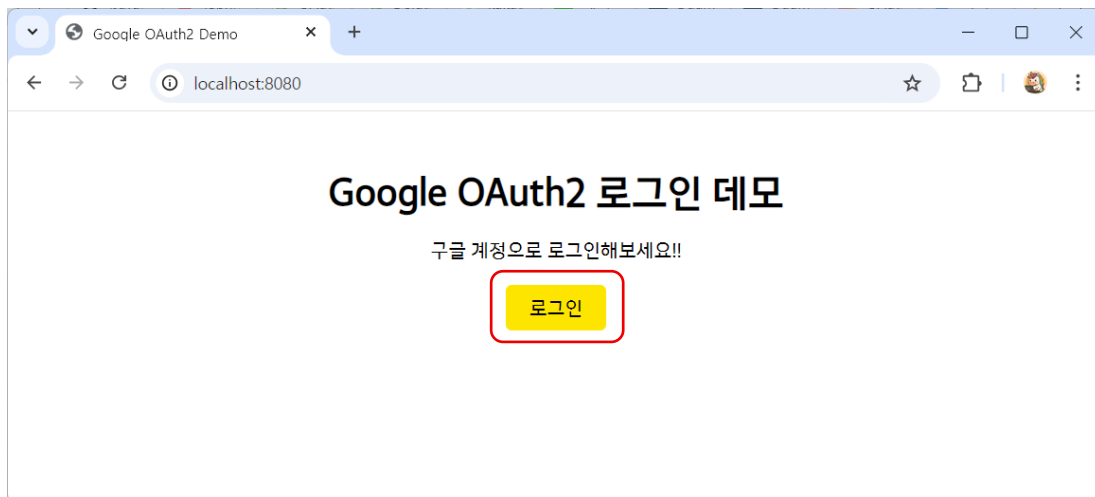
```

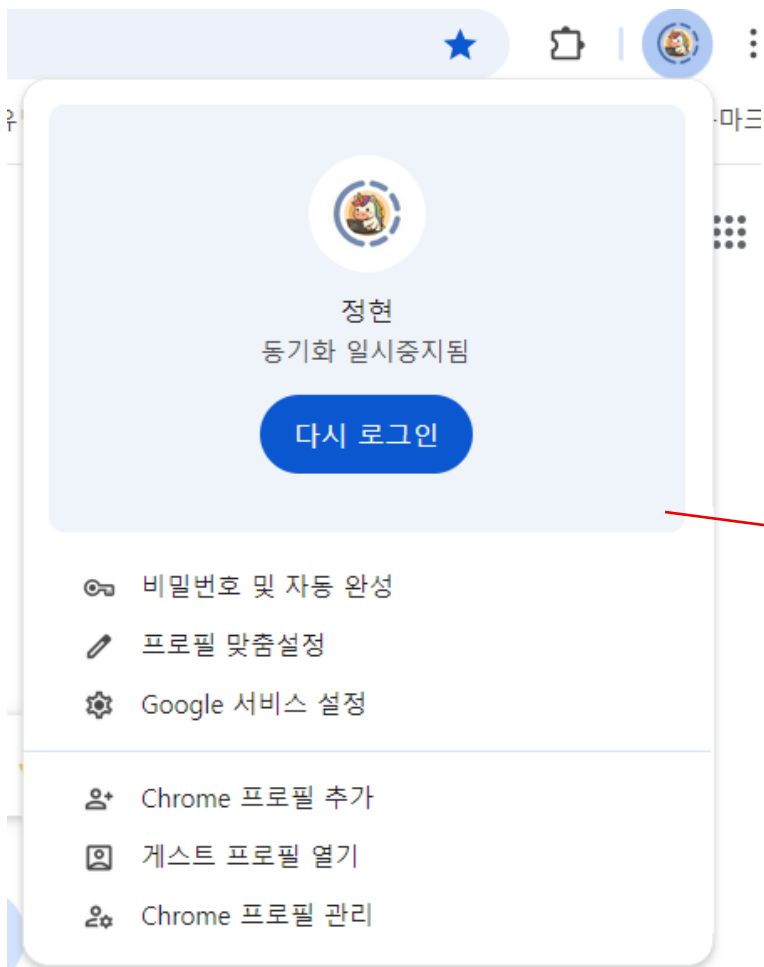
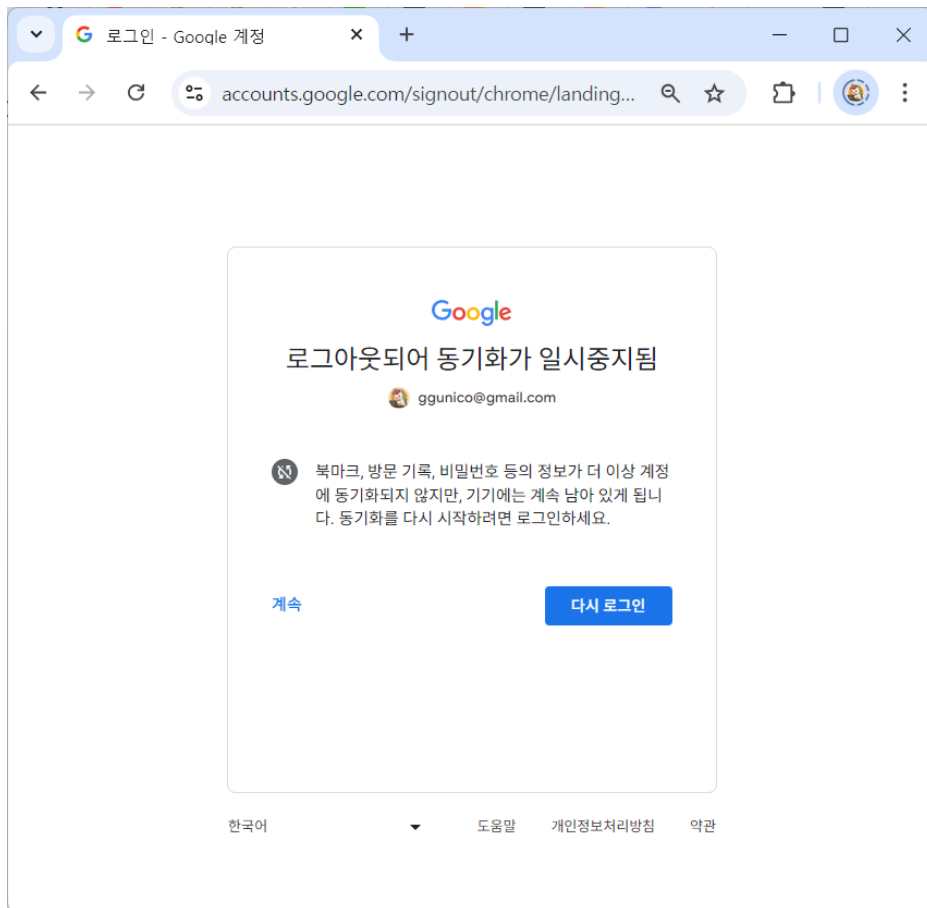
[테스트]

미리 구글에 로그인된 상태

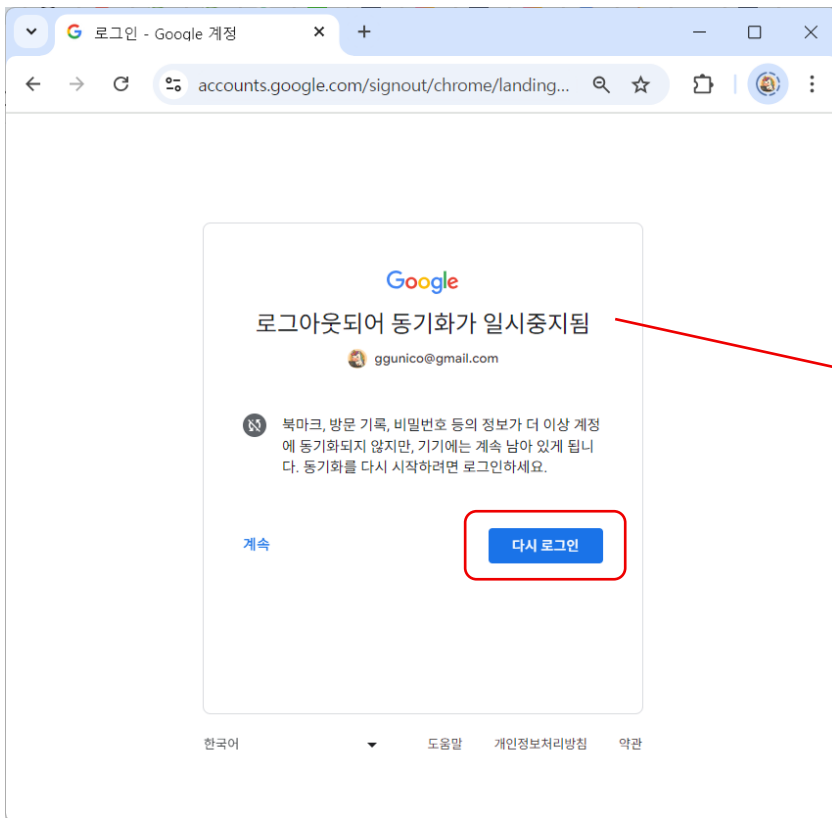


또 다른 탭에서 <http://localhost:8080/> 을 요청해 본다.

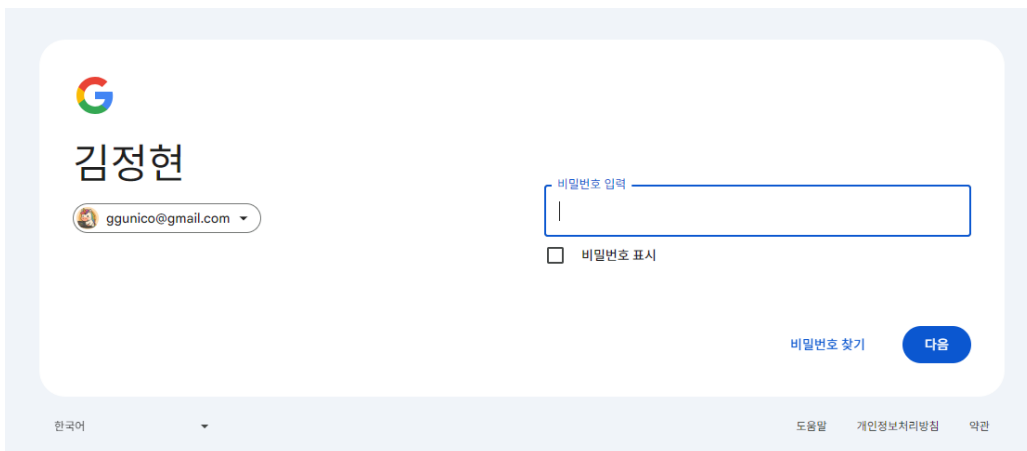
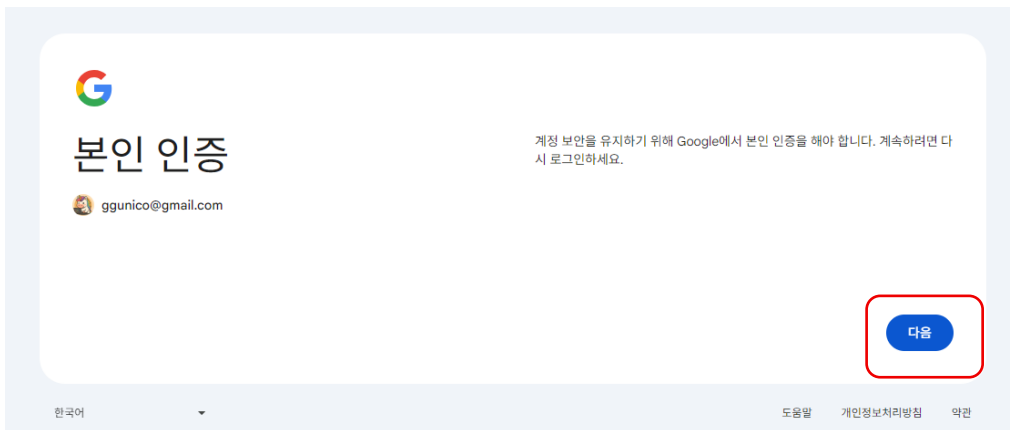




미리 로그인 해놔던 구글도
함께 로그아웃된 것을
볼 수 있다.



우리의 앱에서 로그아웃하여 출력된 페이지에서 다시 로그인 과정을 수행한다. 카카오나 네이버와 다르게 재로그인과정이 좀 여러 단계로 나뉘어 진행된다.





김정현

 ggunico@gmail.com

비밀번호 입력

.....|

☐ 비밀번호 표시

[비밀번호 찾기](#)

다음

[한국어](#) [도움말](#) [개인정보처리방침](#) [약관](#)



빠르게 로그인

 ggunico@gmail.com



패스키를 사용하면 기기에서 Windows PIN 또는 생체 인식 데이터를 요청하여 로그인하는 사용자가 본인임을 Google에 알립니다.

본인 기기인 경우에만 패스키를 만드세요. [자세히 알아보기](#)

[나중에](#)

계속

[한국어](#) [도움말](#) [개인정보처리방침](#) [약관](#)

myaccount.google.com/?utm_source=sign_in_no_continue

! 정보 보호

계정

개인 정보 보호 및 보안 설정을 관리하여 나에게 맞는 방식으로 Google을 사용할 수 있습니다.

Google 계정 검색

내 비밀번호 기기 비밀번호 관리자 내 활동 이메일

개인 정보 보호 및 맞춤설정

Google 계정에 저장된 데이터를 확인하고

보안 도움말 있음

보안 진단에서 보안 도움말을 확인하세요

계정

드라이브

Gmail

YouTube

Gemini

지도

검색

Calendar

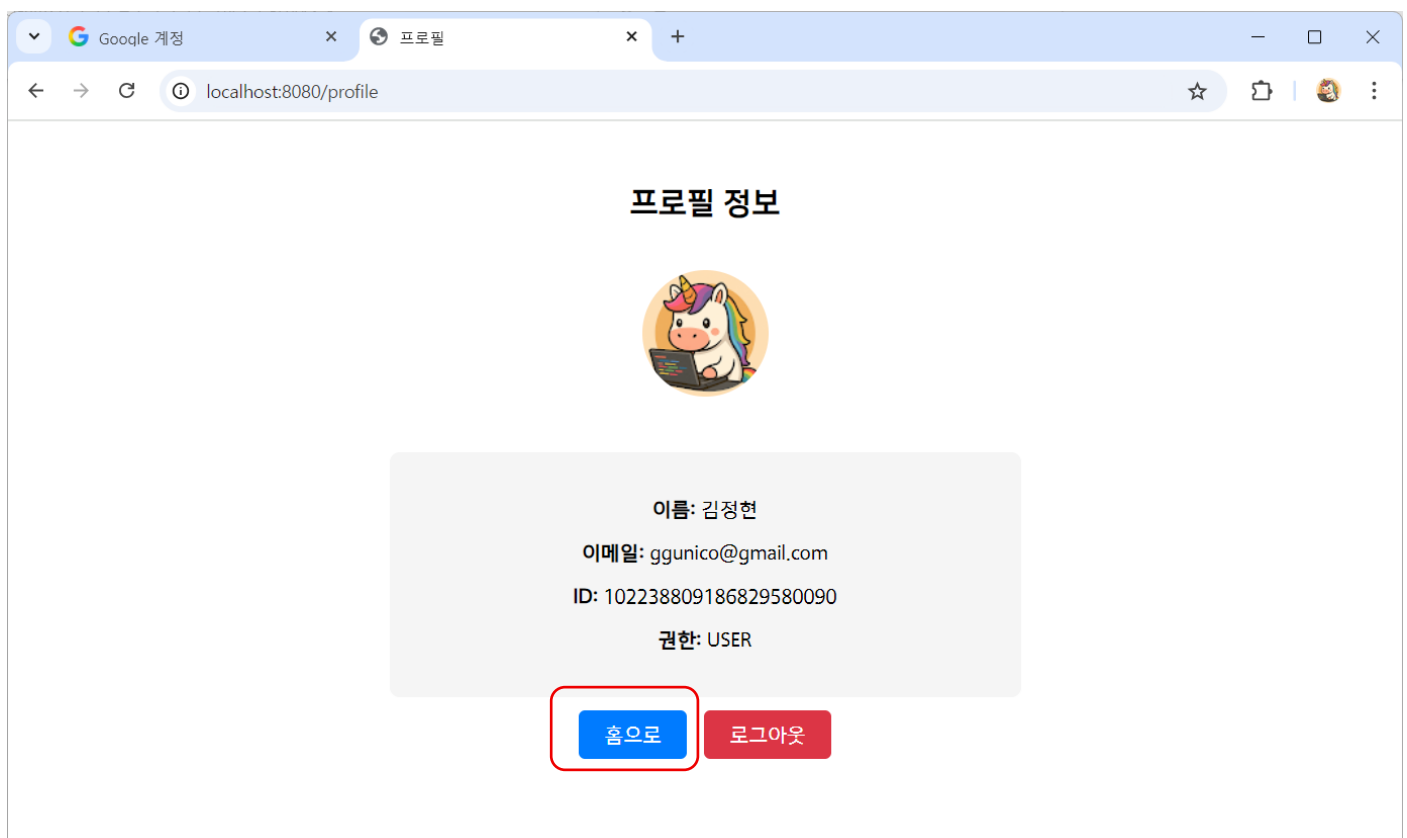
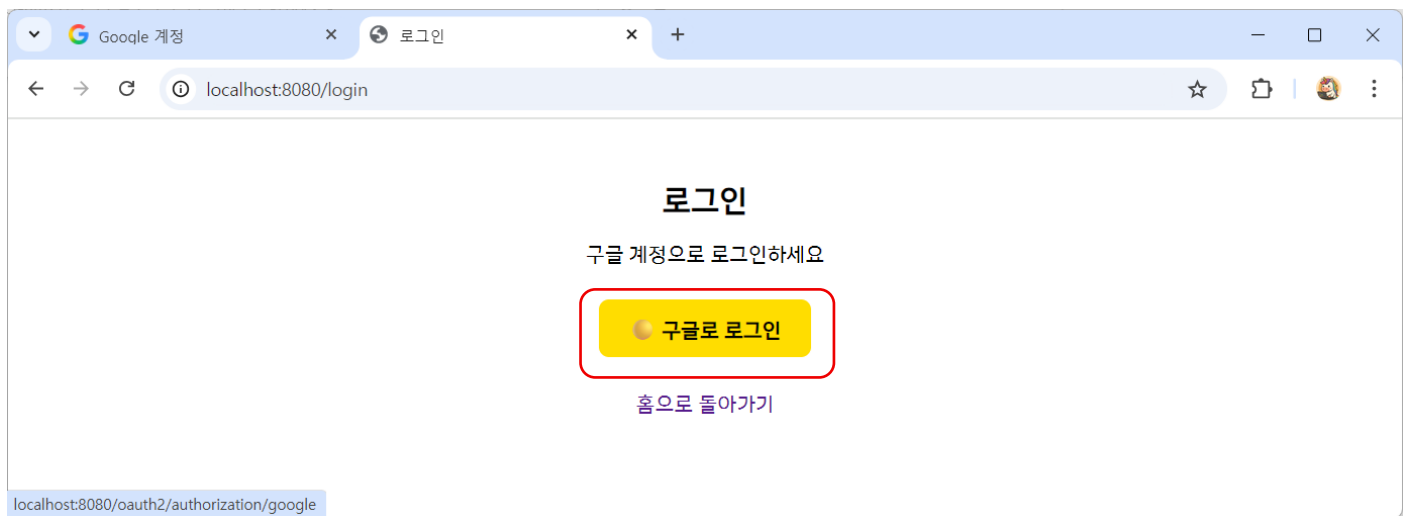
뉴스

사진

Meet

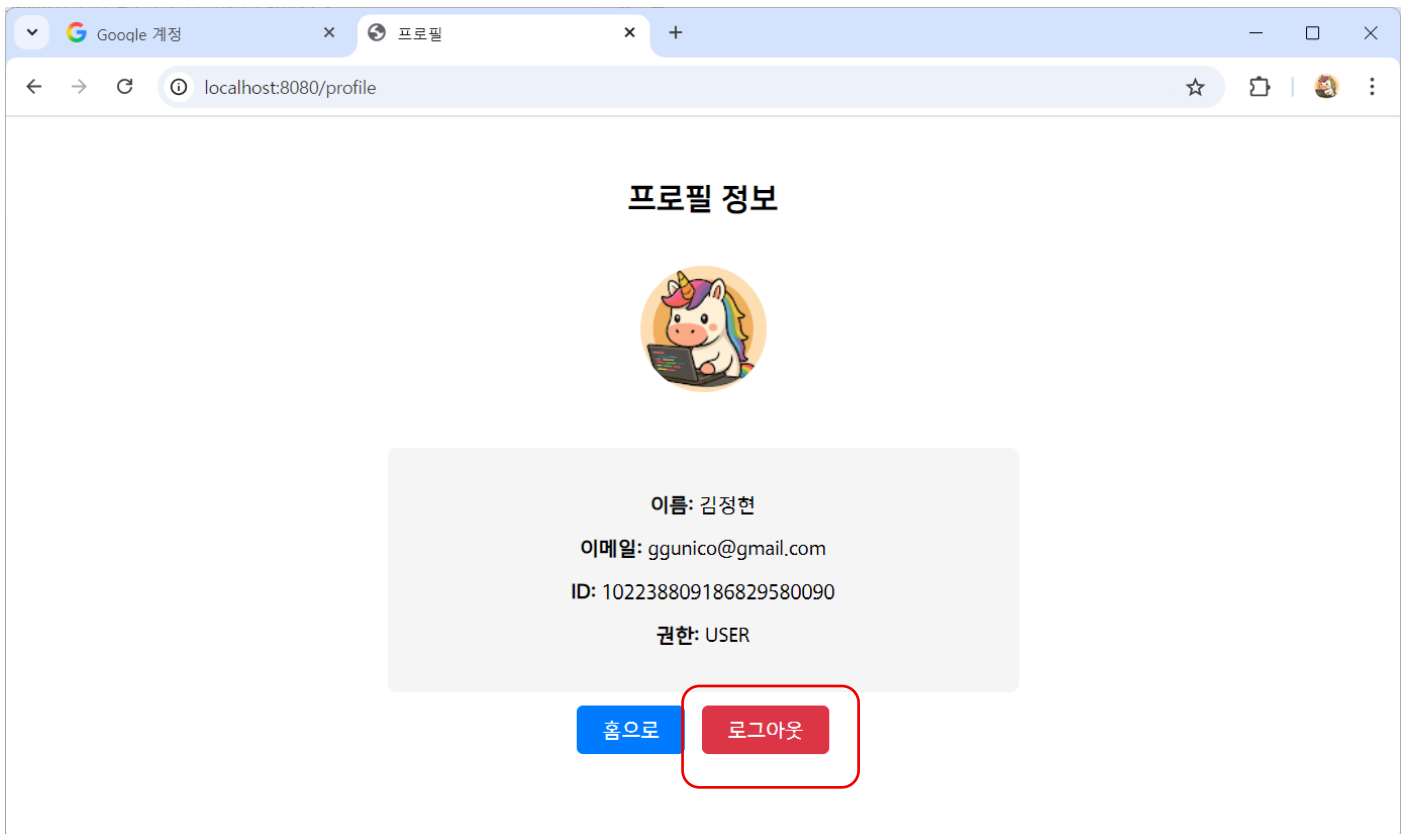
번역

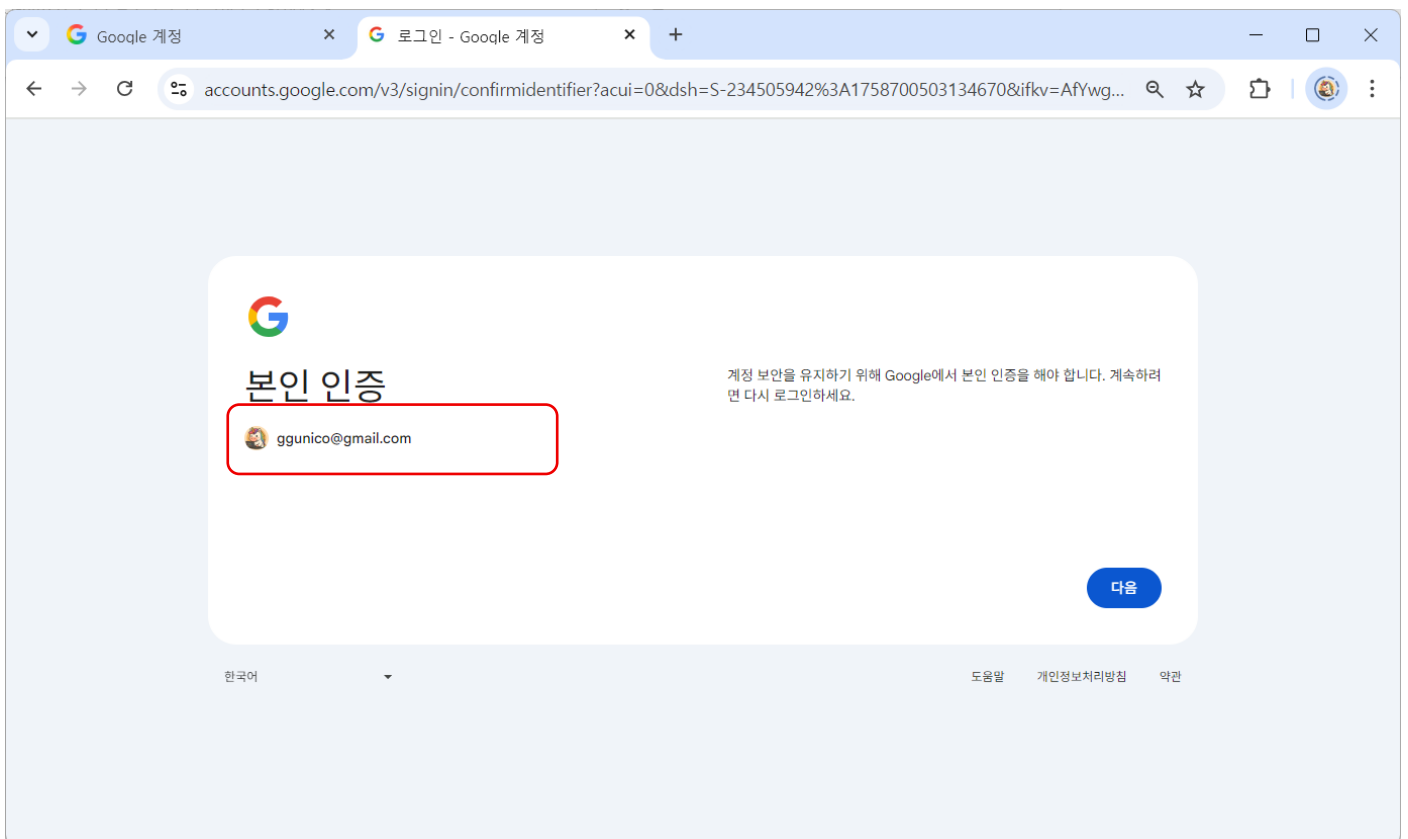
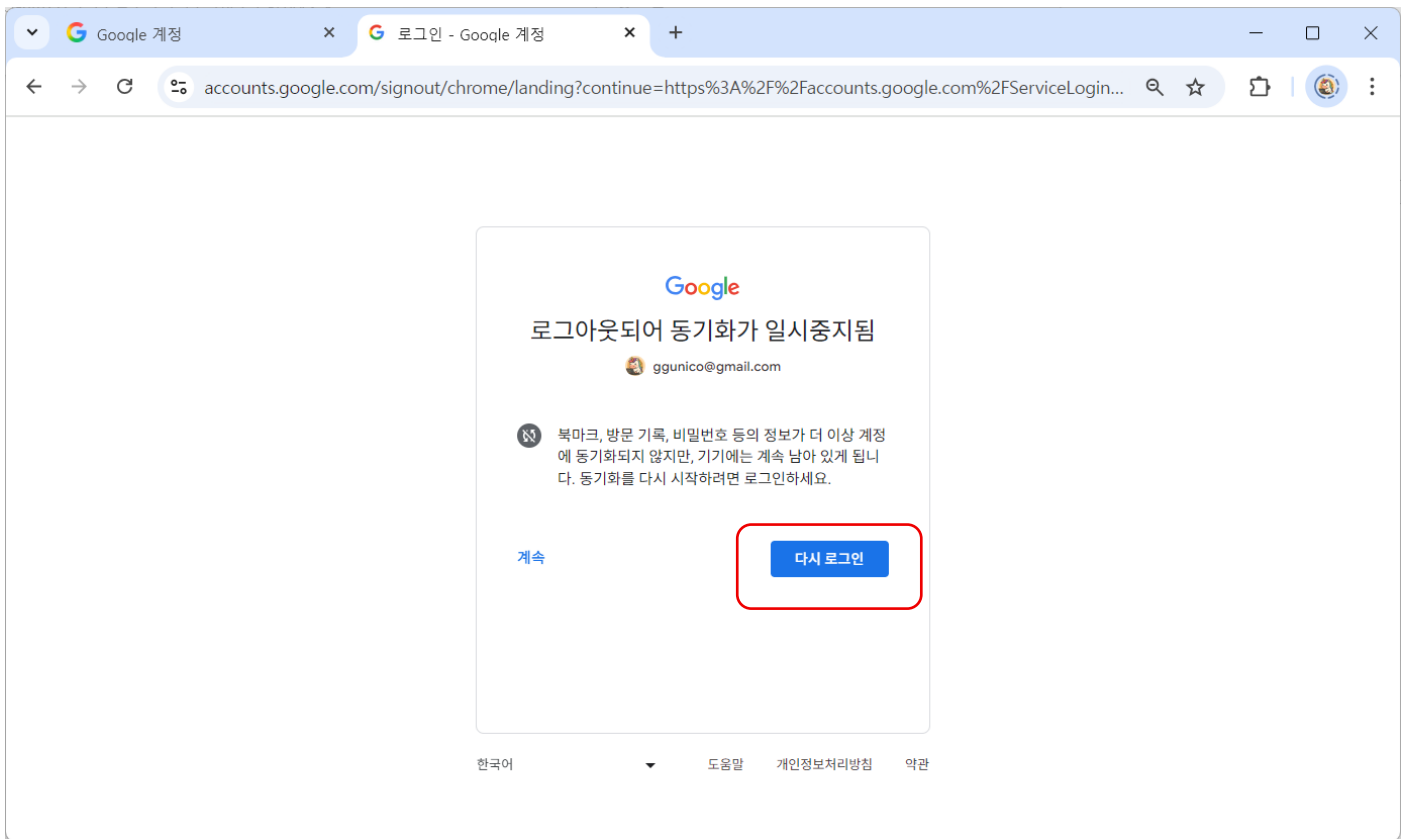
다시 우리의 앱을 요청해 본다.

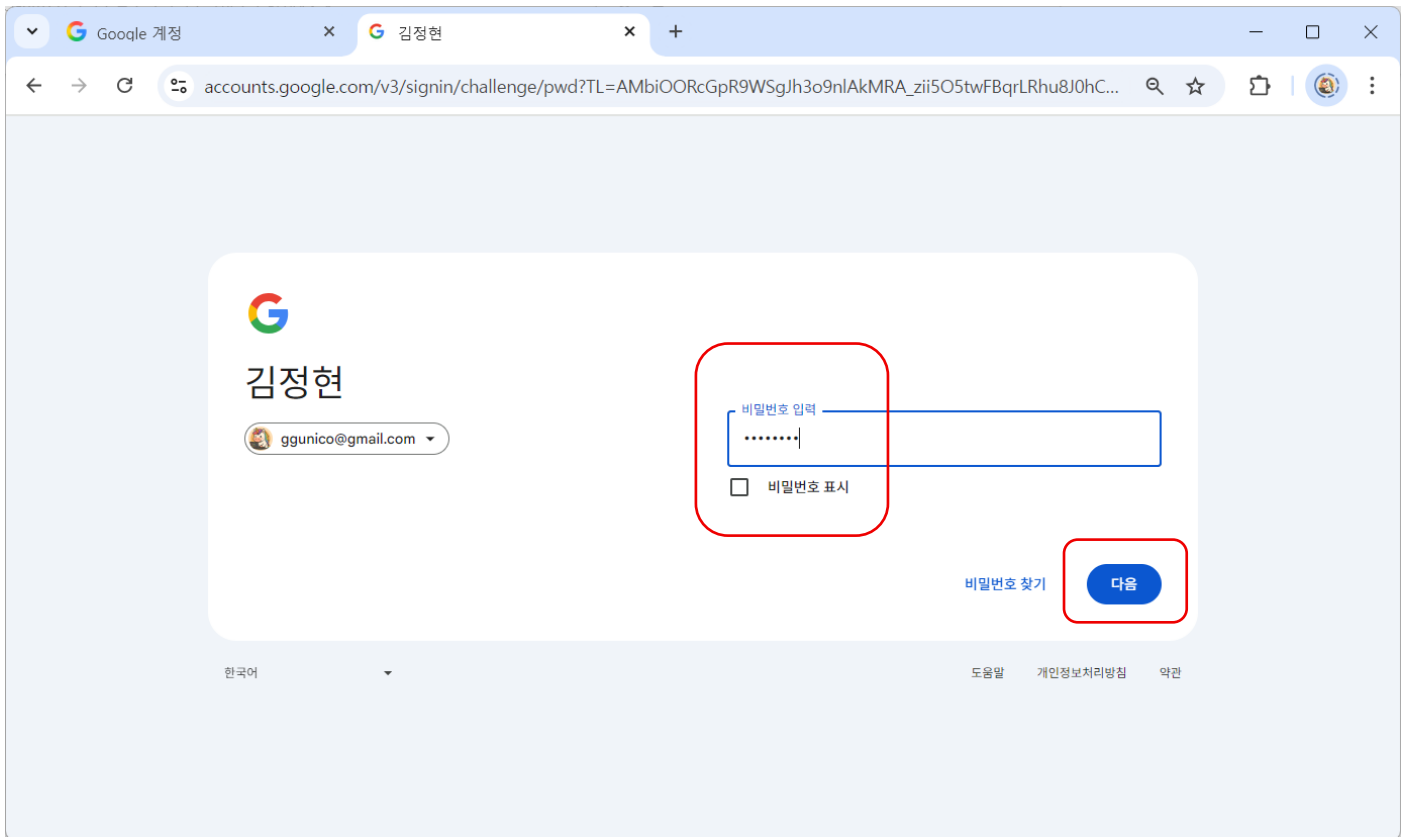
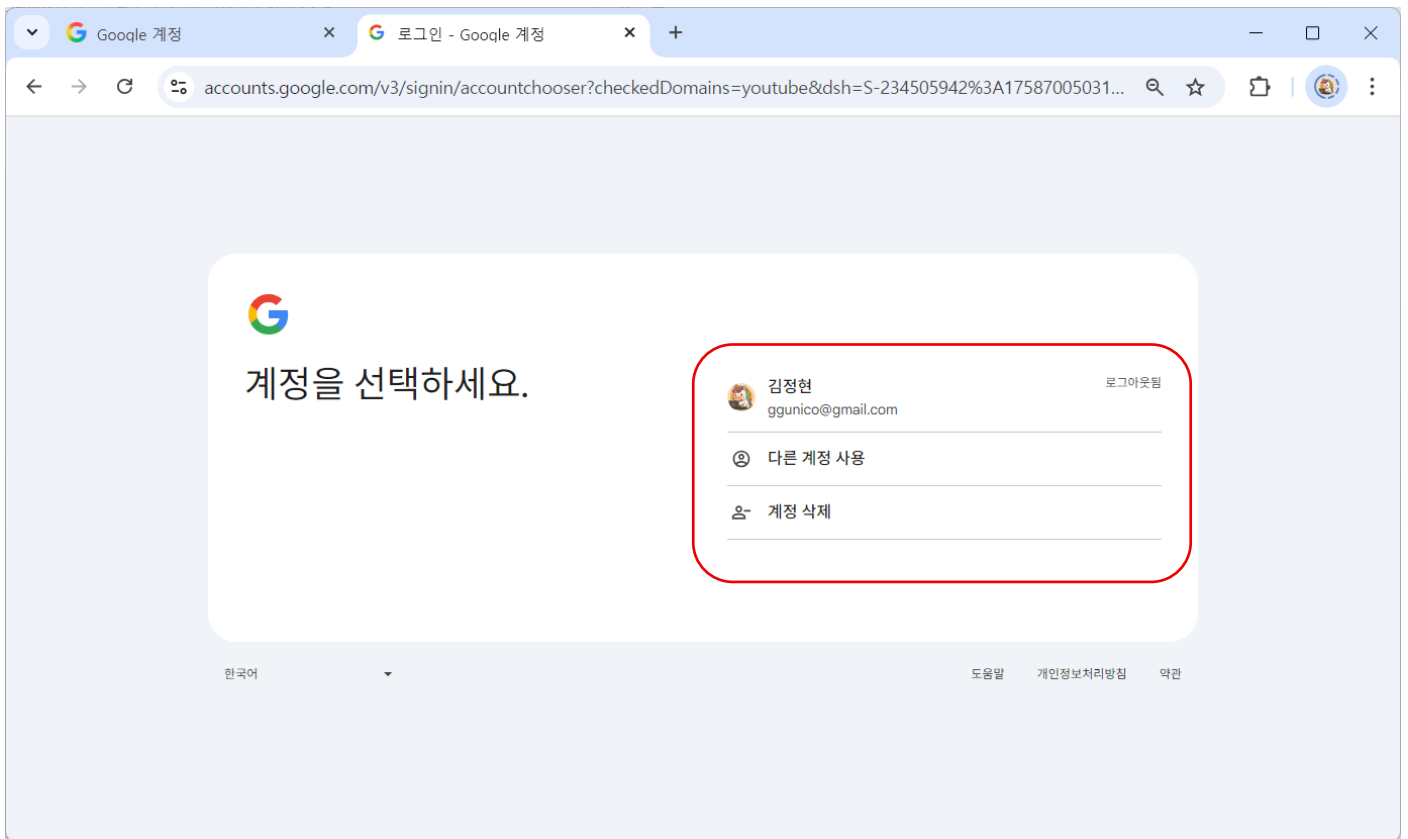


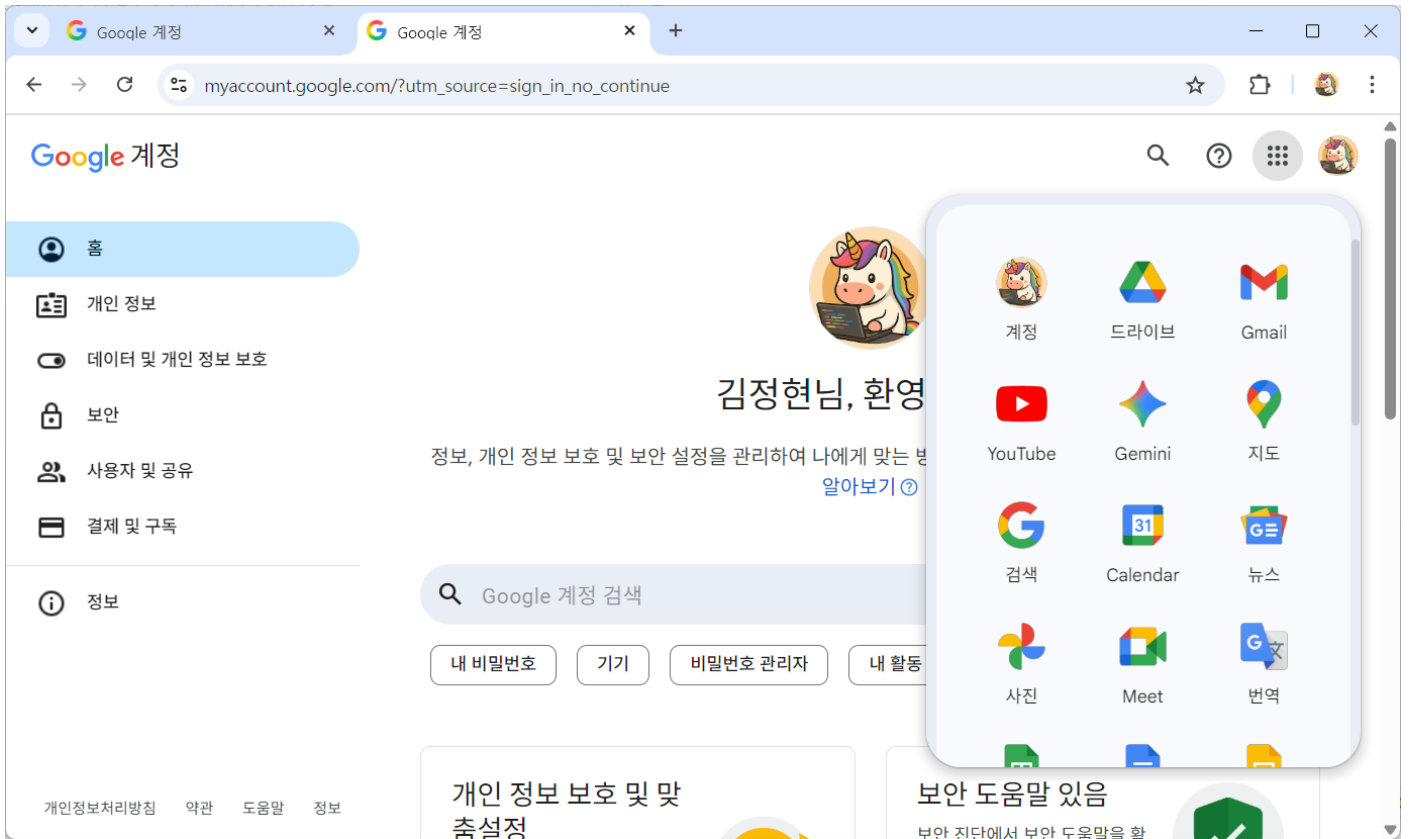


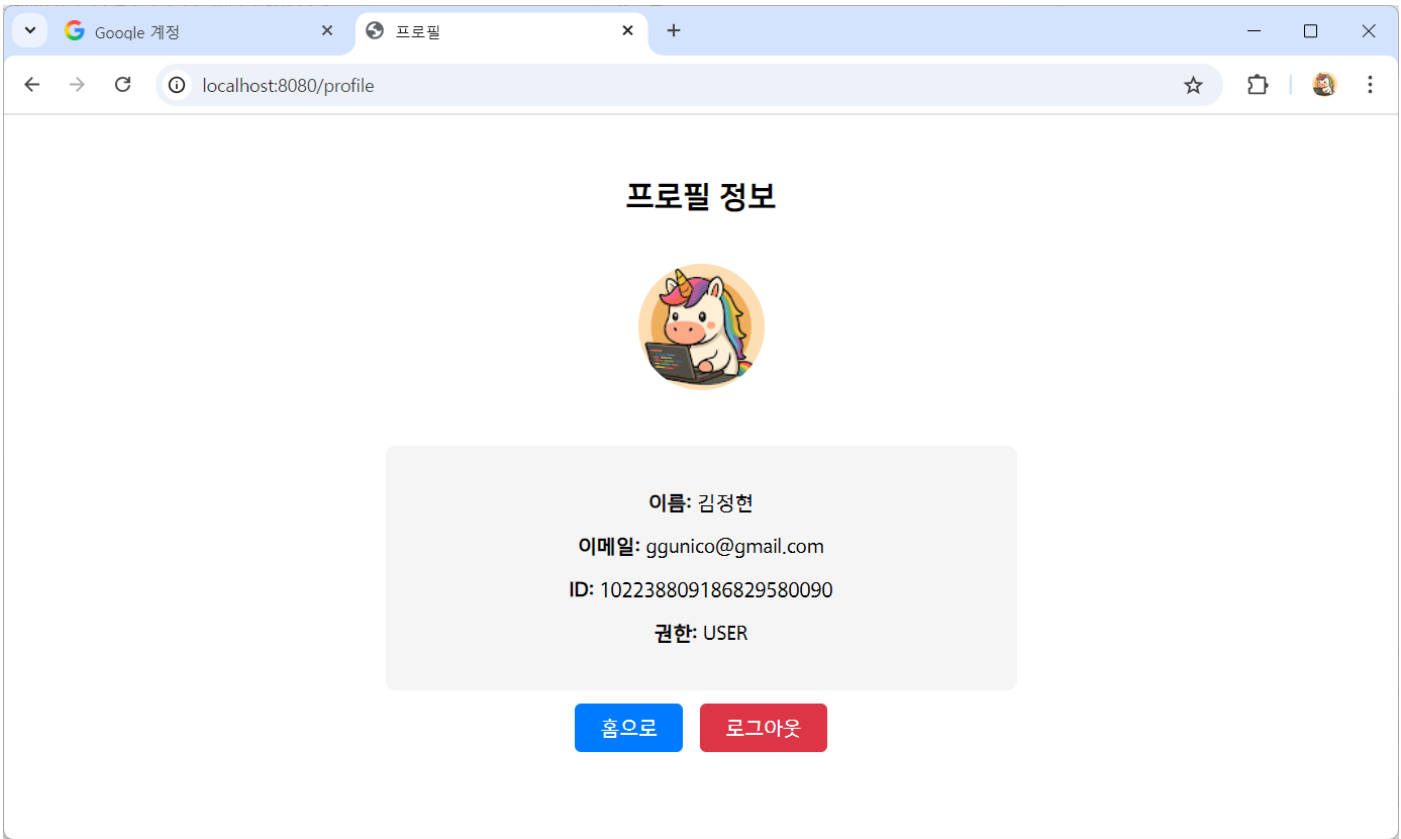
다시 한번 더 로그아웃 과정을 반복해 본다.











kakao



알라딘

알라딘커뮤니케이션

✓ 전체 동의하기

전체동의를 선택목적에 대한 동의를 포함하고 있으며, 선택목적에 대한 동의를 거부해도 서비스 이용이 가능합니다.



unicodaum@hanmail.net

[계정 변경](#)

카카오 로그인 동의

알라딘 서비스 제공을 위해 회원번호와 함께 개인정보가 제공됩니다. 보다 자세한 개인정보 제공항목은 동의 내용에서 확인하실 수 있습니다. 해당 정보는 동의 철회나 서비스 탈퇴 시 또는 제공 목적 달성 시 지체없이 파기됩니다.

✓ [필수] 카카오 개인정보 제3자 제공 동의 [보기](#)

카카오계정(이메일), CI(연계정보), 프로필 정보(닉네임/프로필 사진)

[선택] 카카오 개인정보 제3자 제공 동의 [보기](#)

✓ 성별

✓ 출생 연도

✓ 생일

동의하고 계속하기

